UNIVERSIDAD MAYOR DE SAN ANDRÉS

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA MECÁNICA Y ELECTROMECÁNICA

Programas: Ingeniería Mecatrónica - Ingeniería Automotriz

PROYECTO DE GRADO PARA OBTENER EL TÍTULO DE INGENIERO MECATRÓNICO

"Sistema embebido de estimación y monitoreo del consumo de combustible mediante el protocolo OBD-II en vehículos a gasolina con inyección electrónica"

Postulante: Univ. Cristian Erick Calle Ali

Tutor: Ing. Eloy Cristian Sarabia Yapuchura

LA PAZ - BOLIVIA

Índice general

GLOSARIO Y LISTA DE ABREVIATURAS 15

RESUMEN **18**

1 GENERALIDADES **19**
1.1 Introducción 19
1.2 Antecedentes 20
1.3 Planteamiento del problema 20
1.4 Justificación 22
1.5 Objetivos del proyecto 23
1.5.1 Objetivo General 23
1.5.2 Objetivos específicos 23
1.6 Alcances y limites 24
1.6.1 Alcances 24
1.6.2 Limites 24

2 MARCO TEÓRICO **27**
2.1 Unidad de control electrónica ECU 27
2.1.1 Tipos de modulo de control 28
2.1.2 Estructura interna de la ECU 28
2.1.3 Importancia de la ECU en el consumo de combustible 34
2.2 Protocolos de comunicación en vehículos 36
2.2.1 CAN 36
2.2.2 LIN 37
2.2.3 FlexRay 37
2.2.4 MOST 37
2.3 Sistema OBD-II 37
2.3.1 Evolución del diagnóstico a bordo 37
2.3.2 Estructura del sistema OBD-II 38
2.3.3 Conector OBD-II y asignación de pines 39
2.3.4 Protocolos de comunicación 41
2.3.5 Parámetros de diagnóstico (PIDs) 42
2.3.6 Importancia del OBD-II en el monitoreo del consumo de combustible 44
2.4 Red CAN 45
2.4.1 Arquitectura de la red CAN 45
2.4.2 Arquitectura física del bus CAN 46

2.5 Arquitectura física del bus CAN 47
2.5.1 Topología del bus CAN 48
2.5.2 Tramas del protocolo CAN 49
2.5.3 Errores en trama de datos 49
2.5.4 Red CAN en vehículos 51
2.5.5 Archivo DBC CAN 52
2.6 Sistema de inyección electrónica 53
2.6.1 Evolución de los sistemas de alimentación de combustible 53
2.6.2 Tipos de sistemas de inyección electrónica 55
2.6.3 Componentes principales del sistema de inyección 58
2.6.4 Control de la mezcla aire-combustible 61
2.6.5 Relación del sistema de inyección con el consumo de combustible 62
2.7 Modelos matemáticos para el consumo del combustible 63
2.7.1 Concepto de consumo de combustible 63
2.7.2 Métodos de estimación del consumo 64
2.7.3 Relación aire-combustible (AFR) 65
2.7.4 Modelo basado en el sensor MAF 67
2.7.5 Ejemplo de Cálculo Instantáneo 69
2.7.6 El Método Speed-Density: Origen y Evolución 70
2.7.7 Densidad del combustible 71
2.7.8 Efecto de la altura en el consumo de gasolina 72
2.8 Vehículos bi-fuel 73
2.8.1 Principio de funcionamiento 73
2.8.2 Tipos de combustibles utilizados 74
2.8.3 Ventajas y desventajas 74
2.8.4 Impacto en el consumo de combustible 75
2.9 Microcontroladores y sistemas embebidos 77
2.9.1 Microprocesador vs microcontrolador 77
2.9.2 Sistema embebido 78
2.9.3 Arquitectura de microcontroladores 78
2.9.4 Programación y desarrollo de firmware 82
2.9.5 Comunicación y redes embebidas en automoción 84
2.9.6 Requisitos eléctricos y ambientales 85
2.9.7 Aplicaciones en la industria automotriz 87
2.9.8 ECUs de powertrain 87
2.9.9 Registrador de datos vehiculares vía CAN/OBD-II con ESP32 . 89
2.9.10 Validación, verificación y certificación 90
2.9.11 Buenas prácticas de firmware y mantenimiento 92
2.9.12 Documentación y soporte a largo plazo 94
2.9.13 Futuro y tendencias 95
2.10 Manufactura aditiva: impresión 3D 98
2.10.1 Modelado por deposición fundida (FDM/FFF) 98
2.10.2 Materiales termoplásticos comunes en FDM 98

3 INGENIERÍA DEL PROYECTO 101

3.1 Descripción general del sistema . . . 101
3.1.1 Configuración del entorno de captura . . . 103
3.1.2 Esquema de conexión ESP32 – OBD-II . . . 103
3.1.3 Interfaz con el puerto OBD-II . . . 104
3.1.4 Configuración del transceptor SN65HVD230 . . . 104
3.1.5 Enlace lógico con el ESP32 . . . 105
3.1.6 Consideraciones de montaje y seguridad . . . 105
3.1.7 Configuración SLCAN y SavvyCAN para el análisis de datos . . . 106
3.1.8 Análisis de tramas capturadas . . . 111
3.1.9 Selección final de PIDs para monitoreo de consumo . . . 115
3.2 Selección y justificación de componentes . . . 119
3.2.1 Interfaz OBD-II y controlador CAN . . . 119
3.2.2 Microcontrolador del Sistema . . . 120
3.2.3 Periféricos de visualización . . . 120
3.2.4 Lenguaje de programación . . . 120
3.3 Diseño de hardware . . . 121
3.3.1 Módulo de Interfaz CAN Bus (SN65HVD230 + ESP32) . . . 121
3.3.2 Módulo de Visualización (OLED I2C) . . . 122
3.3.3 Módulo de Registro en Memoria (SSD) . . . 123
3.3.4 Módulo de Comunicación Inalámbrica y Servidor Web . . . 125
3.3.5 Placa preliminar (Altium Designer) . . . 127
3.3.6 Diseño de la carcasa del prototipo (impresión 3D) . . . 130
3.3.7 Diagrama esquemático general . . . 133
3.3.8 Diseño de la PCB (Altium Designer) . . . 148
3.3.9 Parámetros de diseño del PCB según la norma IPC-2221B . . . 152
3.3.10 Lista de materiales (BOM) . . . 157
3.3.11 Capacidades de fabricación de JLCPCB . . . 159
3.4 Modelo matemático de estimación de consumo de combustible . . . 161
3.4.1 Método por sensor MAF . . . 162
3.4.2 Método Speed-Density . . . 163
3.4.3 Eficiencia volumétrica y corrección por altitud . . . 164
3.4.4 Comparativa de métodos y criterio de selección . . . 166
3.4.5 Consulta de soporte de PIDs (PID 0x00) . . . 166
3.4.6 Algoritmo de conmutación automática . . . 168
3.5 Sistema de estimación con retroalimentación en lazo cerrado . . . 170
3.5.1 Justificación del enfoque en lazo cerrado . . . 171
3.5.2 Arquitectura del lazo cerrado . . . 172
3.5.3 Modelo base Speed-Density (lazo abierto) . . . 172
3.5.4 Señales de error del ECU . . . 174
3.5.5 Corrector multiplicativo . . . 174
3.5.6 Aprendizaje adaptativo — Factor K . . . 175
3.5.7 Análisis de estabilidad del filtro EMA . . . 176
3.5.8 Criterios de sintonización de α . . . 177
3.5.9 Identificación de la curva de eficiencia volumétrica . . . 177
3.5.10 Modelo matemático final y variables de entrada/salida . . . 181

3.6 Diseño de software . . . 183
3.6.1 Herramientas y entorno de desarrollo . . . 183
3.6.2 Interfaz con el hardware . . . 185
3.6.3 Arquitectura de tareas . . . 186
3.6.4 Protocolo CAN / OBD2 . . . 188
3.6.5 Método de cálculo: Speed-Density . . . 189
3.6.6 Registro en microSD . . . 195
3.6.7 Modo de bajo consumo ("sin contacto") . . . 197
3.6.8 Panel web . . . 203
3.6.9 Compilación y programación . . . 204
3.6.10 Solución de problemas comunes . . . 205

4 IMPLEMENTACIÓN Y PRUEBAS 207
4.1 Montaje del sistema . . . 207
4.1.1 Prototipado en protoboard . . . 207
4.1.2 Integración del módulo ESP32-S3 y periféricos en protoboard . . . 210
4.1.3 Placa preliminar en Altium Designer . . . 213
4.1.4 Conexión al bus CAN del vehículo . . . 214
4.1.5 Instalación física en el vehículo de pruebas . . . 215
4.2 Configuración del entorno de pruebas . . . 216
4.2.1 Vehículos utilizados para la validación . . . 216
4.2.2 Configuración del adaptador ELM327 como referencia de validación 217
4.2.3 Puesta en marcha del firmware y verificación inicial . . . 218
4.2.4 Herramientas y procesamiento de datos para el análisis . . . 219
4.3 Pruebas experimentales . . . 220
4.3.1 Validación de la comunicación CAN (ESP32 vs. ELM327) . . . 221
4.3.2 Verificación de comunicación OBD-II mediante PIDs . . . 222
4.3.3 Pruebas del modelo Speed-Density . . . 223
4.3.4 Validación del sensor MAP frente a la altitud . . . 228
4.3.5 Protocolo de trayectos de referencia (tanque lleno a tanque lleno) 229
4.3.6 Pruebas de registro de datos en MicroSD . . . 230
4.3.7 Pruebas del panel de control web (WebSocket/HMI) . . . 231
4.3.8 Pruebas de consumo energético y modo de bajo consumo . . . 234
4.4 Resultados obtenidos . . . 234
4.4.1 Resultados de la validación de la comunicación CAN . . . 234
4.4.2 Resultados de precisión del modelo Speed-Density . . . 235
4.4.3 Resultados de la validación del sensor MAP frente a la altitud . . . 237
4.4.4 Resultados de las pruebas del panel web (WebSocket/HMI) . . . 238
4.4.5 Comparación de consumo estimado vs. consumo real . . . 238
4.4.6 Desempeño del sistema en condiciones reales de conducción . . . 246
4.5 Validación del sistema . . . 246
4.5.1 Validación cuantitativa (error porcentual y métricas estadísticas) 246
4.5.2 Validación cualitativa (usabilidad de la interfaz y confiabilidad) 248
4.5.3 Discusión de resultados frente a los objetivos del proyecto . . . 249

5 ANÁLISIS DE COSTOS Y PRESUPUESTO 251
5.1 Costos de componentes electrónicos . . . 251
5.1.1 Costos del módulo de procesamiento (ESP32-S3) . . . 252
5.1.2 Costos de la etapa de alimentación (buck y LDOs) . . . 252
5.1.3 Costos de la interfaz CAN (transceptor y protección) . . . 253
5.1.4 Costos de periféricos (MicroSD, OLED, conectores) . . . 253
5.2 Costos de desarrollo y software . . . 254
5.3 Costos de fabricación y ensamblaje . . . 254
5.3.1 Costos de la placa preliminar (Robokit) . . . 254
5.3.2 Costo de la carcasa del prototipo . . . 256
5.3.3 Costos de manufactura de la placa final (JLCPCB) . . . 257
5.3.4 Costos de ensamblaje SMD de la placa final . . . 257
5.4 Costos de pruebas y validación . . . 258
5.5 Presupuesto total del proyecto . . . 259
5.5.1 Precio de venta estimado (utilidad e IVA) . . . 261
5.6 Análisis de viabilidad económica . . . 262
5.6.1 Comparación con soluciones comerciales existentes . . . 262
5.6.2 Análisis costo-beneficio . . . 264

6 CONCLUSIONES Y RECOMENDACIONES 267
6.1 Conclusiones . . . 267
6.1.1 Conclusiones respecto a los objetivos planteados . . . 267
6.1.2 Conclusiones sobre el diseño del hardware . . . 268
6.1.3 Conclusiones sobre el desarrollo del firmware . . . 268
6.1.4 Conclusiones sobre la interfaz web HMI . . . 269
6.1.5 Conclusiones sobre las pruebas y validación del sistema . . . 269
6.1.6 Aplicación derivada: autonomía estimada por tanque lleno . . . 270
6.2 Recomendaciones . . . 270
6.2.1 Mejoras propuestas para el hardware . . . 270
6.2.2 Mejoras propuestas para el firmware . . . 271
6.2.3 Escalabilidad y trabajo futuro . . . 271
6.2.4 Recomendaciones para la etapa de producción en PCB . . . 273
6.3 Aportes del proyecto . . . 274
6.3.1 Aporte tecnológico . . . 274
6.3.2 Aporte académico . . . 274
REFERENCIAS . . . 275
Apéndice A: PIDs OBD-II modo 01 . . . 287
Apéndice B: Hoja de datos técnicos del transceptor CAN TJA1050 . . . 289
Apéndice C: Hoja de datos transceptor SN65HVD230 . . . 291
Apéndice D: Hoja de datos técnicos del módulo ESP32-S3-WROOM-1-N16R8 293
Apéndice E: Hoja de datos técnicos del controlador de pantalla OLED SSD1306 295
Apéndice F: Hoja de datos técnicos del fusible PTC reseteable, serie BSMD1812 297
Apéndice G: Diodo Schottky SS34 . . . 298
Apéndice H: Hoja de datos técnicos del diodo supresor de tensión transitoria (TVS) SMAJ18A . . . 299

Apéndice I: Hoja de datos técnicos del convertidor buck síncrono TPS54302DDCR 300
Apéndice J: Hoja de datos técnicos del regulador lineal AMS1117-3,3 302
Apéndice K: Hoja de datos técnicos del regulador lineal AP2112K-3,3TRG1 303
Apéndice L: Hoja de datos técnicos del interruptor de carga TPS22918 . . . 304
Apéndice M: Esquemático eléctrico completo y diseño de la PCB 307
Apéndice N: Lista de materiales (BOM) — detalle de fabricación 308
Apéndice O: PIDs OBD-II soportados por los vehículos de prueba 309
Apéndice P: Hoja de datos técnicos del controlador CAN MCP2515 (arquitectura descartada) 311
Apéndice Q: Registro de un viaje 312

Índice de figuras

Figura 1.1: Incremento del precio de la gasolina en Bolivia 21

Figura 2.1: Estructura de la ECU 27
Figura 2.2: Arquitectura interna de la ECU 30
Figura 2.3: Diagrama de procesamiento de la ECU 31
Figura 2.4: Esquema de la unidad de control electrónica ECU 34
Figura 2.5: Cartografía de inyección 35
Figura 2.6: Relación de inyección entre el sensor MAP y las RPM 36
Figura 2.7: Escáner automotriz 39
Figura 2.8: Esquema general del flujo de información en el sistema OBD-II 40
Figura 2.9: Distribución de pines del conector OBD-II 41
Figura 2.10: Conexión de múltiples módulos mediante el protocolo CAN . . 45
Figura 2.11: Estructura de la trama de datos del protocolo CAN 46
Figura 2.12: Configuración física del cableado del protocolo CAN 47
Figura 2.13: Niveles de voltaje diferencial en el bus CAN 48
Figura 2.14: Campos constitutivos del formato de trama de datos CAN . . 50
Figura 2.15: Estructura de una trama de error en el protocolo CAN 50
Figura 2.16: Sintaxis básica de un mensaje y una señal en un archivo DBC 52
Figura 2.17: Línea de tiempo de la evolución en los sistemas de alimentación de combustible 55
Figura 2.18: Diferencias estructurales entre los sistemas de inyección monopunto y multipunto 57
Figura 2.19: Componentes principales y distribución del sistema de inyección electrónica 60
Figura 2.20: Relación entre el sistema de inyección y el consumo de combustible 63
Figura 2.21: Curva estequiométrica del factor lambda y respuesta del sensor de oxígeno 66
Figura 2.22: Principio de funcionamiento eléctrico y térmico del sensor MAF 68
Figura 2.23: Esquema de componentes y distribución en una adaptación de sistema Bi-Fuel (Gasolina/GLP) 74
Figura 2.24: Esquema de componentes que conforman un vehículo bi-fuel, mostrando la coexistencia de sistemas de inyección paralelos y su gestión electrónica integrada 76
Figura 2.25: Diagrama de bloques general de la arquitectura interna de un microcontrolador estándar 83

Figura 2.26: Perfiles morfológicos de los principales transitorios de tensión en un sistema eléctrico automotriz de 12 V 86
Figura 2.27: Arquitectura y distribución de dominios de ECUs en un vehículo moderno 88

Figura 3.1: Diagrama de bloques de la arquitectura funcional y flujo de señales del sistema 102
Figura 3.2: Diagrama de flujo lógico para la adquisición y procesamiento de datos 103
Figura 3.3: Esquema de conexión de los pines CAN-H y CAN-L al microcontrolador 106
Figura 3.4: Firmware bidireccional entre el driver TWAI del ESP32 y SavvyCAN. 108
Figura 3.5: Comando para los permisos de administrador a SavvyCAN. . . 109
Figura 3.6: Comando para la carga del firmware al microcontrolador ESP32 mediante PlatformIO. 109
Figura 3.7: Configuración en el software SavvyCAN 110
Figura 3.8: Conexion exitosa con el software SavvyCAN 111
Figura 3.11: Algoritmo en C++ para la detección activa y decodificación del mapa de bits de PIDs soportados en el bus CAN. 116
Figura 3.12: Captura de pantalla del monitor serie durante la ejecución del escáner de PIDs OBD-II (Changan Honor). 117
Figura 3.13: Inicialización del periférico TWAI® del ESP32 a 500 kbps. . . . 122
Figura 3.14: Configuración del filtro de aceptación para recepción exclusiva de respuestas OBD-II. 122
Figura 3.15: Flujo de datos desde el bus CAN hasta el cliente web 126
Figura 3.16: Estructura de la trama JSON transmitida por WebSocket al cliente web. 127
Figura 3.17: Esquemático de la placa preliminar en Altium Designer 128
Figura 3.18: Vista 3D de la placa preliminar en Altium Designer 128
Figura 3.19: Placa preliminar fabricada por Robokit, sin ensamblar 129
Figura 3.20: Placa preliminar ensamblada 130
Figura 3.21: Modelo STL base, tomado de MakerWorld 131
Figura 3.22: Tapa modificada en SolidWorks, con abertura para la pantalla OLED 131
Figura 3.23: Impresión 3D en la Creality Ender V3 KE 132
Figura 3.24: Carcasa impresa del prototipo 133
Figura 3.25: Diagrama de bloques del sistema de protección y regulación de energía (12V a 3,3V) 134
Figura 3.26: Esquemático del convertidor DC/DC TPS54302 en Altium Designer 138
Figura 3.27: Esquemático de los reguladores LDO AMS1117 y AP2112 en Altium Designer 138
Figura 3.28: Configuración de conexión óptima del transceptor CAN SN65HVD230 139
Figura 3.29: Esquemático del transceptor CAN SN65HVD230 en EasyEDA 140

Figura 3.30: Circuito de conexión mínima y configuración óptima del módulo ESP32-S3-WROOM-1-N16R8. 141
Figura 3.31: Esquemático final del módulo ESP32-S3-WROOM-1-N16R8 en EasyEDA 142
Figura 3.32: Esquemático de la etapa de visualización OLED en EasyEDA . 143
Figura 3.33: Esquemático del sistema de almacenamiento mediante memoria MicroSD en Altium Designer 145
Figura 3.34: Esquemático del control de alimentación de periféricos 146
Figura 3.35: Esquemático del sistema de indicadores visuales 148
Figura 3.36: Conversión del circuito esquemático al diseño de la PCB en Altium Designer 149
Figura 3.37: Anchos de pista en el ruteo del PCB 157
Figura 3.38: Verificación de fabricabilidad (DFM) del PCB en JLCPCB . . . 161
Figura 3.39: Máquina de estados del algoritmo de conmutación automática 168
Figura 3.40: Diagrama de bloques del estimador de consumo 172
Figura 3.41: Mapa de polos y ceros del filtro EMA para $\alpha = 0{,}20$. El polo en $z = 0{,}80$ se encuentra dentro del círculo unitario, lo que garantiza estabilidad. 176
Figura 3.42: Curva de eficiencia volumétrica identificada en función del régimen. Los marcadores corresponden a las medias por bin (datos ilustrativos) y la línea, al ajuste polinómico cúbico que constituye la curva $\eta_v(N)$ del modelo. 180
Figura 3.43: *Creación de tareas FreeRTOS en* `src/main.cpp`. 184
Figura 3.44: *Fusión bajo mutex al final de cada ciclo,* `src/can_obd2.cpp`. . 187
Figura 3.45: *Trama de petición Mode 01, PID genérico.* 189
Figura 3.46: *Núcleo del cálculo speed-density,* `src/fuel_calc.cpp`. 190
Figura 3.47: Inversión general de la ecuación speed-density, `ve_curve_calibrator.py`. 192
Figura 3.48: Curva de eficiencia volumétrica identificada por `ve_curve_calibrator.py` sobre datos reales del Nissan Vanette 194
Figura 3.49: *Fila de ejemplo del CSV (encabezado y fila de datos).* 196
Figura 3.50: *Interruptor de alto lado con P-MOSFET, un canal por riel.* . . 198
Figura 3.51: *Entrada a deep sleep,* `src/power_sleep.cpp`. 199
Figura 3.52: *Configuración relevante de* `platformio.ini`. 204

Figura 4.1: Componentes del prototipo y elementos principales del sistema. 209
Figura 4.2: Diagrama de interconexión de periféricos en el prototipo 212
Figura 4.3: Ensamblaje de la placa preliminar 214
Figura 4.4: Conexión del sistema propuesto mediante OBD-II 215
Figura 4.5: Identificación física del OBD2 en el vehículo antes de las pruebas 216
Figura 4.6: Vehículos de prueba utilizados en la validación experimental . 217
Figura 4.7: Adaptador comercial ELM327 Bluetooth utilizado para la validación de lecturas OBD-II. 219

Figura 4.8: Núcleo de medición de latencia en el benchmark del ESP32-S3, `can_bench_tool/src/main.cpp`. 222
Figura 4.9: Núcleo de medición y clasificación de respuestas en el benchmark del ELM327, `elm327_bench.py`. 222
Figura 4.10: Cálculo del MAF estimado en `speed_density_bench.py`, réplica en Python de la fórmula de `fuel_calc.cpp`. 224
Figura 4.11: Comparación del caudal de aire estimado por el modelo Speed-Density frente al valor de referencia de la ECU, por régimen de motor 225
Figura 4.12: Validación de la tabla VE calibrada frente al MAF de referencia, mismo conjunto de datos de calibración 227
Figura 4.13: Dispersión del MAF estimado frente al MAF de referencia durante una ruta real 228
Figura 4.14: Distribución del intervalo de actualización del panel web frente al valor nominal 233
Figura 4.15: Distribución del error porcentual del modelo Speed-Density, ruta real (Nissan Vanette) 236
Figura 4.16: Ejemplo del trayecto corto 239
Figura 4.17: Ejemplo del trayecto medio 239
Figura 4.18: Trayecto del viaje largo 240
Figura 4.19: Comparación gráfica del consumo de combustible por trayecto y vehículo según los tres métodos evaluados 243
Figura 4.20: Curva VE calibrada y su impacto en el consumo estimado, por bin de RPM (ruta real, Nissan Vanette) 245

Figura 5.1: Distribución porcentual del presupuesto total del proyecto por categoría 261
Figura 5.2: Comparación de costo inicial entre el sistema propuesto y alternativas comerciales 264
Figura 5.3: Punto de equilibrio (*breakeven*) entre el costo del dispositivo y el ahorro acumulado de combustible 266

Índice de tablas

Tabla 2.1: Comparación de los protocolos de comunicación en vehículos . 38
Tabla 2.2: Comparación de protocolos de comunicación OBD-II 43
Tabla 2.3: PIDs OBD-II más utilizados en monitoreo vehicular 44
Tabla 2.4: Relación entre longitud del bus, velocidad y tiempo máximo de transmisión 48
Tabla 2.5: Comparación entre los tipos de sistemas de inyección electrónica 58
Tabla 2.6: Comparación de métodos para la estimación del consumo de combustible 65
Tabla 2.7: Interpretación de la relación aire-combustible y el factor lambda 67
Tabla 2.8: Densidad típica de combustibles líquidos a temperatura de referencia 72
Tabla 2.9: Efecto de la altitud sobre los parámetros del motor y el consumo de gasolina 73
Tabla 2.10: Comparación general entre combustibles en vehículos bi-fuel . 77
Tabla 2.11: Comparación de arquitecturas de CPU en sistemas embebidos y computación 79
Tabla 2.12: Clasificación y funciones de las memorias en un microcontrolador 80
Tabla 2.13: Parámetros típicos de microcontroladores utilizados en módulos de control de motor de producción 89
Tabla 2.14: Comparativa de arquitecturas de hardware en proyectos de adquisición de datos OBD-II 90
Tabla 2.15: Comparativa de niveles de verificación xIL para sistemas embebidos automotrices 92
Tabla 2.16: Fragmento de la matriz de trazabilidad entre requisitos y pruebas para el sistema OBD-II 93
Tabla 2.17: Herramientas de integración y despliegue continuos para firmware embebido automotriz 94
Tabla 2.18: Comparativa de sensores empleados en sistemas avanzados de asistencia al conductor y sus características operativas 96
Tabla 2.19: Comparativa de buses de comunicación automotrices según sus especificaciones técnicas y aplicaciones 97
Tabla 2.20: Comparación de propiedades típicas de materiales termoplásticos para impresión FDM 99

Tabla 3.1: Comparativa técnica entre arquitecturas de adquisición CAN . 104

Tabla 3.2: Esquema de conexiones (Pinout) entre el puerto OBD-II, el transceptor y el microcontrolador . . . 105
Tabla 3.3: Comparativa de herramientas de análisis para tráfico CAN . . 107
Tabla 3.4: Comparativa de IDs candidatos para la determinación de la variable de RPM . . . 114
Tabla 3.5: Mapeo binario de la respuesta hexadecimal FE 3E B8 13 y descripción de PIDs . . . 118
Tabla 3.6: Componentes seleccionados y justificación técnica del sistema unificado . . . 121
Tabla 3.7: Distribución de la información en la pantalla OLED . . . 123
Tabla 3.8: Estructura real del registro CSV escrito por `sdLoggerTask()` (`sd_logger.cpp`) . . . 124
Tabla 3.9: Asignación de pines y código de colores del conector de expansión de 6 pines . . . 143
Tabla 3.10: Ancho de pista recomendado y resistencia estimada en función de la corriente . . . 151
Tabla 3.11: Distancias mínimas de aislamiento eléctrico (clearance) según el voltaje operativo y la altitud . . . 152
Tabla 3.12: Constante dieléctrica relativa de materiales comunes en sustratos de PCB . . . 152
Tabla 3.13: Separación mínima entre conductores externos sin recubrimiento (IPC-2221B, Tabla 6-1) . . . 154
Tabla 3.14: Parámetros de diseño del PCB conforme a IPC-2221B y capacidades de JLCPCB . . . 156
Tabla 3.15: Lista de materiales (BOM) del dispositivo . . . 158
Tabla 3.16: Capacidades de fabricación de JLCPCB (2 capas, 1 oz) frente al diseño . . . 160
Tabla 3.17: Árbol de decisión para la selección del método de estimación . 166
Tabla 3.18: Decodificación del byte B de la respuesta al PID 0x00 (ejemplo: B = 0x3E) . . . 167
Tabla 3.19: PIDs requeridos por cada método de estimación . . . 168
Tabla 3.20: Nomenclatura de las variables del modelo. . . . 173
Tabla 3.21: Señales OBD-II de realimentación empleadas por el estimador. 174
Tabla 3.22: Sintonización del coeficiente α del filtro EMA. . . . 177
Tabla 3.23: Valores ilustrativos de η_v por bin de régimen (medias por bin). Los valores definitivos se identifican para el vehículo de ensayo mediante el Algoritmo 3. . . . 180
Tabla 3.24: Variables de entrada, parámetros y salidas del modelo en lazo cerrado. . . . 182
Tabla 3.25: Mapa de pines definido en `include/config.h`. . . . 185
Tabla 3.26: Tareas FreeRTOS, definidas en `include/config.h` y creadas en `src/main.cpp`. . . . 186
Tabla 3.27: Códigos de `ErrorCode` (`include/types.h`). . . . 188
Tabla 3.28: PIDs Mode 01 consultados por `canObd2Task`. . . . 188
Tabla 3.29: `kVeTable`, `src/fuel_calc.cpp`. Interpolación lineal entre puntos.191

Tabla 3.30: Columnas escritas por `sdLoggerTask` (`src/sd_logger.cpp`). . 196
Tabla 3.31: Pines de *power-gating*, `include/config.h`. 197
Tabla 3.32: Contribuyentes al consumo promedio "sin contacto", cadena real (TPS54302 + AMS1117 + AP2112). 200
Tabla 3.33: Consumo estimado en modo activo, dos escenarios (misma cadena TPS54302+AMS1117+AP2112) 202
Tabla 3.34: Endpoints servidos por ESPAsyncWebServer (`src/web_server.cpp`). 204

Tabla 4.1: Componentes empleados en el prototipo sobre protoboard . . . 208
Tabla 4.2: Descripción de los componentes numerados en la Figura 4.1. . 210
Tabla 4.3: Asignación de pines GPIO del ESP32-S3 en el prototipo 211
Tabla 4.4: Pines relevantes del conector OBD-II (J1962) utilizados en el proyecto 215
Tabla 4.5: Vehículos utilizados en la fase de validación experimental . . . 217
Tabla 4.6: Lista de verificación previa a cada sesión de pruebas 219
Tabla 4.7: Formato de registro para la validación de la comunicación CAN (sesiones separadas) 221
Tabla 4.8: PIDs consultados y disponibilidad por vehículo. 223
Tabla 4.9: Comparación del MAF estimado por el modelo Speed-Density frente al MAF de referencia de la ECU (PID 0x10), Nissan Vanette 225
Tabla 4.10: Eficiencia volumétrica (VE) genérica frente a la VE real requerida, por punto de régimen 226
Tabla 4.11: Validación del sensor MAP en *key-on, engine-off* frente a la presión barométrica 229
Tabla 4.12: Integridad y tasa de registro de los archivos CSV generados en MicroSD, por sesión 231
Tabla 4.13: Latencia de actualización del panel web (intervalos entre mensajes WebSocket) 232
Tabla 4.14: Estabilidad de la conexión WebSocket en sesión prolongada . . 233
Tabla 4.15: Alcance del punto de acceso WiFi generado por el ESP32-S3 . 234
Tabla 4.16: Consumo de corriente en los distintos modos de operación, teórico y medido 234
Tabla 4.17: Métricas de validación dinámica del modelo Speed-Density (ruta real, Nissan Vanette) 236
Tabla 4.18: Validación del modelo Speed-Density en el Changan Honor . . 237
Tabla 4.19: Comparación de consumo de combustible entre el sistema propuesto, la referencia comercial y el consumo real, por trayecto (Nissan Vanette) 241
Tabla 4.20: Comparación de consumo de combustible entre el sistema propuesto, la referencia comercial y el consumo real, por trayecto (Changan Honor) 242
Tabla 4.21: Resumen de métricas de validación cuantitativa 248
Tabla 4.22: Formato de encuesta de usabilidad (escala de Likert, 1 = muy en desacuerdo, 5 = muy de acuerdo) 249

Tabla 5.1: Costo del módulo de procesamiento 252
Tabla 5.2: Costo de la etapa de alimentación de dos rieles 252
Tabla 5.3: Costo de la interfaz de bus CAN 253
Tabla 5.4: Costo de periféricos de almacenamiento, visualización y conexión 253
Tabla 5.5: Costo estimado de horas de ingeniería por etapa de desarrollo . 254
Tabla 5.6: Costo de fabricación de la placa preliminar en Robokit 255
Tabla 5.7: Componentes adicionales de la placa preliminar (etapa de alimentación y conectores) 256
Tabla 5.8: Costo de material de la carcasa del prototipo 256
Tabla 5.9: Costo de fabricación de PCB en JLCPCB 257
Tabla 5.10: Comparación de estrategias de ensamblaje SMD 258
Tabla 5.11: Costos asociados a la fase de pruebas y validación (Capítulo 4) 259
Tabla 5.12: Presupuesto total del proyecto, por categoría 260
Tabla 5.13: Precio de venta estimado, prototipo actual vs. unidad adicional 262
Tabla 5.14: Comparación funcional y de costo frente a alternativas comerciales 263
Tabla 5.15: Estimación del periodo de recuperación de la inversión 265

Tabla 1: Columnas adicionales de `trip_NNN.csv` no incluidas en la Tabla 3.30. 312
Tabla 2: `trip_003.csv` — muestra real, parámetros del motor (columnas 1–9). 313
Tabla 3: `trip_003.csv` — muestra real, consumo instantáneo y acumulado del viaje (columnas 10–14). 313
Tabla 4: `trip_003.csv` — muestra real, estado CAN, costo y presión barométrica (columnas 15–18). 313

GLOSARIO Y LISTA DE ABREVIATURAS

Se listan a continuación, en orden alfabético, las siglas y los términos técnicos empleados a lo largo del documento cuyo significado no siempre se repite en cada capítulo donde aparecen.

Siglas y acrónimos

Sigla	Significado
AFR	*Air/Fuel Ratio*, relación aire-combustible.
BOM	*Bill of Materials*, lista de materiales de la placa.
CAN	*Controller Area Network*, protocolo de bus serie usado por la ECU para comunicar sus módulos y, a través del puerto OBD-II, con equipos externos.
CSV	*Comma-Separated Values*, formato de archivo de texto con valores separados por comas, usado para el registro de datos en MicroSD.
DTC	*Diagnostic Trouble Code*, código de diagnóstico de fallas almacenado por la ECU (Modos 03/07 de OBD-II).
ECU	*Electronic Control Unit*, unidad de control electrónico del motor.
EMA	*Exponential Moving Average*, media móvil exponencial; filtro de suavizado usado en el factor de calibración adaptativo K y en la identificación de la curva $\eta_v(N)$.
ESD	*Electrostatic Discharge*, descarga electrostática; también se usa para nombrar a los diodos de protección contra ella.
GPIO	*General Purpose Input/Output*, pin de entrada/salida de propósito general del microcontrolador.
HMI	*Human-Machine Interface*, interfaz humano-máquina; en este proyecto, el panel de control web.
I2C	*Inter-Integrated Circuit*, bus serie de dos hilos usado para la comunicación con la pantalla OLED.
IAT	*Intake Air Temperature*, temperatura del aire de admisión (PID 0x0F).
IPC	*Institute for Printed Circuits*, organismo que define normas de manufactura de PCB (p. ej., IPC-2221B).
ISA	*International Standard Atmosphere*, Atmósfera Estándar Internacional; modelo de presión barométrica en función de la altitud.
ISO	*International Organization for Standardization*, organización responsable de normas como ISO 15765-4 (CAN sobre OBD-II).
JSON	*JavaScript Object Notation*, formato de intercambio de datos usado por el panel web.
KOEO	*Key-On, Engine-Off*, contacto del vehículo puesto con el motor apagado; estado usado para validar el sensor MAP contra la presión atmosférica real.

Sigla	Significado
LDO	*Low Drop-Out*, regulador de tensión de baja caída (p. ej., AMS1117, AP2112K).
LTFT	*Long Term Fuel Trim*, ajuste de combustible a largo plazo (PID 0x07).
MAF	*Mass Air Flow*, flujo másico de aire admitido por el motor (PID 0x10).
MAP	*Manifold Absolute Pressure*, presión absoluta del colector de admisión (PID 0x0B).
MAPE	*Mean Absolute Percentage Error*, error porcentual absoluto medio; métrica usada para cuantificar la exactitud del sistema propio frente a las referencias comercial y real.
MIL	*Malfunction Indicator Lamp*, luz indicadora de falla del vehículo ("check engine").
OBD-II	*On-Board Diagnostics*, segunda generación del estándar de diagnóstico a bordo del vehículo.
OLED	*Organic Light-Emitting Diode*, tecnología de la pantalla usada para el indicador local del sistema.
PCB	*Printed Circuit Board*, placa de circuito impreso.
PID	*Parameter ID*, identificador de parámetro consultable por el Modo 01 de OBD-II. En este documento **no** se refiere a un controlador Proporcional-Integral-Derivativo.
PWM	*Pulse Width Modulation*, modulación por ancho de pulso.
RPM	*Revolutions Per Minute*, régimen de giro del motor (PID 0x0C).
SAE	*Society of Automotive Engineers*, entidad que publica el estándar J1979 de PIDs OBD-II.
SLCAN	*Serial-Line CAN*, protocolo que transporta tramas CAN sobre un puerto serie, usado por SavvyCAN para capturar el bus.
SPI	*Serial Peripheral Interface*, bus serie usado para la comunicación con la tarjeta MicroSD.
STFT	*Short Term Fuel Trim*, ajuste de combustible a corto plazo (PID 0x06).
TWAI	*Two-Wire Automotive Interface*, nombre del controlador CAN integrado en el ESP32-S3 (equivalente funcional a un controlador CAN clásico).
UMSA	Universidad Mayor de San Andrés.
VE	Eficiencia volumétrica (η_v), cociente entre la masa de aire realmente admitida y la masa teórica que ocuparía la cilindrada a la densidad de admisión.

Términos técnicos

Término	Definición
Deep sleep	Modo de bajo consumo del ESP32-S3 en el que se desactivan la mayoría de sus periféricos, reduciendo el consumo de corriente a nivel de microamperios; el sistema despierta por temporizador para volver a comprobar si hay contacto en el vehículo.
ELM327	Circuito integrado comercial que traduce comandos AT en texto plano hacia tramas OBD-II/CAN; usado en este proyecto como adaptador de referencia comercial frente al sistema propio.
Lambda (λ)	Razón entre el AFR real y el AFR estequiométrico; $\lambda = 1$ indica una mezcla exactamente estequiométrica, $\lambda > 1$ una mezcla pobre y $\lambda < 1$ una mezcla rica.
Lazo cerrado (*closed-loop*)	Modo de operación de la ECU en el que corrige la mezcla aire-combustible en tiempo real a partir de la retroalimentación de los sensores de oxígeno, reflejada en los PID de STFT/LTFT.
SavvyCAN	Software de escritorio de código abierto para el análisis y envío de tramas CAN, usado en este proyecto para la ingeniería inversa de los PID soportados por cada vehículo.
Speed-Density	Método de estimación del flujo másico de aire admitido a partir de RPM, presión de colector (MAP) y temperatura de admisión (IAT), sin requerir un sensor de flujo de aire (MAF) directo.

RESUMEN

Este proyecto de grado desarrolla e implementa un sistema embebido de estimación y monitoreo del consumo de combustible mediante el protocolo OBD-II en vehículos a gasolina con inyección electrónica que carecen de esta funcionalidad de forma nativa, con aplicación, por ejemplo, en el transporte público urbano de la ciudad de La Paz.

El sistema se basa en un módulo ESP32-S3 con controlador CAN nativo (TWAI) y un transceptor SN65HVD230, que adquieren en tiempo real los parámetros del motor disponibles por el puerto OBD-II del vehículo. El consumo se estima mediante el modelo Speed-Density, corregido con los ajustes de combustible que reporta la propia ECU, y se registra por viaje en una tarjeta microSD. El dispositivo expone además un panel web propio mediante un punto de acceso WiFi local, sin depender de internet ni de aplicaciones de terceros.

El sistema se validó experimentalmente en un Nissan Vanette (con sensor MAF) y un Changan Honor (sin sensor MAF, evaluado con Speed-Density puro). Sobre nueve tramos de ruta real, concuerda con un adaptador ELM327 comercial dentro de un MAPE de 2,75 %, y con el aforo real de tanque lleno dentro de un MAPE de 4,84 %; a partir de estas cifras, estima además una autonomía por tanque lleno de 617 km para el Vanette y 489 km para el Changan Honor, cercana a la real de 575 km y 504 km respectivamente.

El análisis de costos estima un costo directo de 772,22 Bs por unidad adicional y un precio de venta de referencia de 1 047,13 Bs (utilidad del 20 % e IVA del 13 %), muy por debajo de las plataformas de telemetría de flota comerciales disponibles en el mercado.

Palabras clave: consumo de combustible, OBD-II, ESP32-S3, Speed-Density, autonomía vehicular, transporte público, La Paz.

Capítulo 1

GENERALIDADES

1.1. Introducción

En la actualidad, la incorporación de sistemas electrónicos en los vehículos ha permitido mejorar significativamente el control, diagnóstico y eficiencia en su funcionamiento. A través de diferentes sensores y unidades de control electrónico, es posible obtener información relevante sobre el desempeño del motor y otros subsistemas en tiempo real. En este contexto, el estándar OBD-II (On-Board Diagnostics) se ha consolidado como una herramienta fundamental para el acceso a datos vehiculares.

El presente proyecto desarrolla un sistema de monitoreo del consumo de combustible para vehículos a gasolina con inyección electrónica que no disponen de esta funcionalidad de forma nativa. Si bien algunos vehículos modernos ya incorporan sistemas que permiten visualizar este tipo de información directamente en el panel de instrumentos, existe una gran cantidad de automóviles que carecen de esta característica, como ocurre, por ejemplo, con buena parte del parque automotor de la ciudad de La Paz, lo que limita el control del gasto energético de combustible por parte del usuario.

Para dar solución a esta necesidad, se implementa un sistema basado en un microcontrolador ESP32, que en conjunto con un módulo de comunicación CAN, permite la adquisición de datos desde el puerto OBD-II del vehículo. A partir de estos datos, se realiza el procesamiento necesario para estimar el consumo de combustible en tiempo real.

Adicionalmente, el sistema cuenta con una interfaz de monitoreo accesible mediante una página web generada por el propio ESP32, disponible a través de una dirección IP local. Esto permite al usuario visualizar de manera sencilla e inmediata los parámetros obtenidos, sin necesidad de dispositivos adicionales o software especializado.

En conjunto, este proyecto ofrece una solución accesible y de bajo costo para el monitoreo del consumo de combustible, integrando conocimientos de electrónica, sistemas embebidos y comunicación de datos, con una aplicación directa en el ámbito automotriz.

1.2. Antecedentes

El desarrollo de sistemas de diagnóstico y monitoreo vehicular ha evolucionado significativamente en las últimas décadas, impulsado por la necesidad de mejorar el rendimiento, la seguridad y la eficiencia en el consumo de combustible. Uno de los avances más importantes en este ámbito es la implementación del estándar OBD-II (On-Board Diagnostics), el cual permite acceder a información generada por la unidad de control electrónico (ECU) del vehículo a través de un conjunto de parámetros definidos (McCord, 2011).

El protocolo OBD-II, ampliamente adoptado en vehículos modernos, utiliza diferentes estándares de comunicación, siendo uno de los más relevantes el basado en redes CAN (Controller Area Network), el cual permite una transmisión de datos eficiente y confiable entre los distintos módulos electrónicos del automóvil (Kaschel and Pinto, 2005). Gracias a este sistema, es posible obtener variables como la velocidad del vehículo, las revoluciones del motor (RPM), el flujo de aire y otros parámetros necesarios para el análisis del desempeño vehicular (Uvarov and Ponomarev, 2021) (McCord, 2011).

En los últimos años, se han desarrollado diversas soluciones comerciales orientadas al monitoreo vehicular, muchas de las cuales utilizan dispositivos basados en interfaces OBD-II junto con aplicaciones móviles para la visualización de datos en tiempo real (Machado et al., 2021). Asimismo, algunos vehículos de gama media y alta incorporan sistemas propios que permiten al conductor visualizar información relacionada con el consumo de combustible y el rendimiento del motor directamente en el panel de instrumentos .

Por otro lado, el avance de los sistemas embebidos y microcontroladores, como el ESP32, ha permitido el desarrollo de soluciones de bajo costo con capacidades de procesamiento y conectividad inalámbrica (Putra et al., 2025). Esto ha facilitado la implementación de sistemas personalizados capaces de adquirir, procesar y transmitir información en tiempo real mediante redes locales o interfaces web.

En este contexto, el presente proyecto se fundamenta en la integración de tecnologías existentes, tales como el estándar OBD-II, la comunicación CAN y el uso de microcontroladores con conectividad, y desarrolla una solución accesible que permite el monitoreo del consumo de combustible en vehículos que no cuentan con esta funcionalidad incorporada.

1.3. Planteamiento del problema

En la industria automotriz, una de las variables más importantes que se busca monitorear es el consumo de combustible, debido a su influencia directa en la eficiencia energética y el desempeño de los vehículos. Su análisis resulta fundamental tanto a nivel económico como operativo dentro del sector del transporte.

En Bolivia, el incremento en el uso del transporte ha generado un aumento significativo en la demanda de combustibles. De acuerdo con datos del Instituto Nacional

de Estadística (INE) (Instituto Nacional de Estadística (INE) de Bolivia, 2026), el uso del transporte presentó un incremento aproximado del 11,92 %, lo que refleja una mayor utilización de vehículos y, en consecuencia, un mayor consumo de carburantes.

A este contexto se suma la variabilidad reciente en la disponibilidad y calidad de la gasolina en el mercado nacional, lo cual ha generado incertidumbre en los conductores al momento de abastecer sus vehículos. Como se puede observar en la Figura 1.1, el precio de la gasolina en Bolivia ha experimentado un incremento sostenido durante los últimos meses, lo que no solo impacta en los costos operativos de los usuarios, sino que también puede afectar el proceso de combustión, provocando variaciones en el rendimiento del motor y en el consumo de combustible.

Figura 1.1

Incremento del precio de la gasolina en Bolivia

PRECIOS MERCADO INTERNO DE LOS PRODUCTOS REGULADOS AL CONSUMIDOR FINAL					
	GASOLINA PREMIUN	GASOLINA ESPECIAL	GASOLINA ESPECIAL +	GASOLINA PREMIUM +	GASOLINA ULTRA PREMIUM 100
	Bs/l	Bs/l	Bs/l	Bs/l	Bs/l
31/12/2010	7,51	6,47			
31/12/2011	4,79	3,74			
31/12/2012	4,79	3,74			
31/12/2013	4,79	3,74			
31/12/2014	4,79	3,74			
31/12/2015	4,79	3,74			
31/12/2016	4,79	3,74			
31/12/2017	4,79	3,74			
31/12/2018	4,79	3,74			
31/12/2019	4,79	3,74	3,74		
31/12/2020	4,79	3,74	3,74		
31/12/2021	4,79	3,74	3,74		
31/12/2022	4,79	3,74	3,74		
31/12/2023	4,79	3,74	3,74	4,79	
31/12/2024	4,79	3,74	3,74	6,18	6,82
06/10/25	4,79	3,74	3,74	7,22	7,98
17/12/2025	11,00	6,96	3,74	7,22	7,98
18/12/2025	11,00	6,96	6,96	7,22	7,98

Nota. El gráfico muestra la tendencia histórica del costo del combustible. Tomado de (Agencia Nacional de Hidrocarburos, 2024).

Asimismo, una combustión ineficiente asociada a combustibles de calidad variable puede influir en las lecturas de los sensores del vehículo, afectando la precisión de los sistemas de diagnóstico basados en OBD-II. Esto dificulta la interpretación adecuada de los parámetros del motor y limita la capacidad del usuario para evaluar el estado real de su vehículo.

En este escenario, surge una creciente preocupación sobre la eficiencia en el uso de la gasolina como fuente energética en el transporte, especialmente en vehículos que no cuentan con sistemas avanzados de monitoreo. La falta de herramientas accesibles que

permitan analizar el consumo de combustible en tiempo real incrementa la incertidumbre del usuario respecto al rendimiento de su vehículo.

Por lo tanto, se desarrolla un sistema que permite estimar y monitorear el consumo de combustible de manera precisa y en tiempo real, con el objetivo de brindar al usuario información confiable para la toma de decisiones (Lamby et al., 2026).

1.4. Justificación

Los vehículos a gasolina con inyección electrónica operan, en general, sin herramientas que permitan conocer de manera precisa y continua el consumo real de gasolina durante su funcionamiento; esta carencia se observa, por ejemplo, en el transporte urbano de la ciudad de La Paz. En la práctica, la evaluación del consumo se realiza de forma empírica, sin información técnica que permita identificar ineficiencias en el uso del combustible.

La ausencia de sistemas de monitoreo del consumo dificulta el análisis del comportamiento del motor, tales tramos recorridos y modos de conducción. Esta situación limita la posibilidad de optimizar el desempeño del vehículo y detectar posibles fallas asociadas a un consumo anómalo de combustible. El proyecto ayuda a tener un patrón o un control que indica si el vehículo consume más gasolina de la que debe, sin importar las condiciones particulares de uso.

El sistema desarrollado genera información confiable sobre el comportamiento del motor. Esta información contribuye a la mejora de la eficiencia operativa, al análisis técnico del consumo de combustible y al apoyo en procesos de mantenimiento y toma de decisiones técnicas.

- **Justificación técnica:** El proyecto propone el desarrollo de un sistema de monitoreo del consumo de gasolina basado en la adquisición de parámetros del motor a través del puerto OBD-II y su procesamiento mediante un modelo físico-matemático del proceso de inyección de combustible. A diferencia de los métodos convencionales basados únicamente en mantenimiento correctivo o preventivo (Reif, 2015), la propuesta integra una plataforma telemétrica capaz de adquirir datos del vehículo, permitiendo el análisis casi en tiempo real del comportamiento del motor.
- **Justificación económica:** El acceso a información precisa y continua sobre el consumo de combustible permite a conductores y operadores de transporte optimizar el uso del vehículo, reduciendo gastos innecesarios asociados al consumo excesivo de gasolina. Asimismo, el sistema contribuye a la toma de decisiones relacionadas con el mantenimiento del motorizado y la planificación de rutas, generando un impacto económico positivo tanto para transportistas como para usuarios.
- **Justificación social:** El sistema propuesto aporta a la mejora de la eficiencia de los vehículos a gasolina con inyección electrónica, tanto de uso particular

como de transporte urbano, al promover un uso más responsable del combustible. Esto contribuye indirectamente a una mayor transparencia en el desempeño de los vehículos, a la concienciación sobre el consumo energético y, en el caso del transporte público, a la mejora de la calidad del servicio, como ocurriría en la ciudad de La Paz.

1.5. Objetivos del proyecto

1.5.1. Objetivo General

- Implementar un sistema embebido capaz de adquirir datos en tiempo real desde el puerto de diagnóstico vehicular mediante el estándar OBD-II, con el fin de estimar el consumo instantáneo de combustible en vehículos a gasolina con inyección electrónica, generar un registro histórico de consumo y validar los resultados respecto a mediciones de referencia.

1.5.2. Objetivos específicos

- Determinar los parámetros del sistema OBD-II disponibles, particularmente el flujo de aire del motor (MAF) y/o presión de aire del motor (MAP), las RPM y la velocidad del vehículo, con el fin de estimar el consumo de combustible mediante un modelo matemático basado en la relación aire-combustible.
- Desarrollar un modelo matemático para la estimación del consumo de combustible en vehículos a gasolina con inyección electrónica, utilizando parámetros obtenidos mediante el sistema OBD-II, basado en la relación aire–combustible del proceso de combustión.
- Recopilar datos del funcionamiento del motor mediante el sistema OBD-II, particularmente el flujo másico de aire (MAF) y/o presión de aire del motor (MAP), las revoluciones por minuto (RPM) y la velocidad del vehículo, para su posterior análisis en el modelo de estimación del consumo de combustible. Como se menciona en (Shaw et al., 2019), es posible estimar el consumo instantáneo de combustible utilizando datos obtenidos del vehículo mediante interfaces OBD-II.
- Diseñar un sistema embebido de adquisición de datos para la captura, el guardado y posterior procesamiento de estos, facilitando la validación del modelo matemático de consumo de combustible en vehículos a gasolina con inyección electrónica.
- Integrar una plataforma telemétrica basada en arquitectura IoT para la adquisición de parámetros del motor a través de la interfaz OBD-II utilizando un módulo ESP32 conectado a un adaptador, permitiendo la transmisión de los datos mediante un protocolo de comunicación hacia un servidor local donde serán almacenados en una base de datos estructurada para su posterior análisis del consumo de combustible.
- Evaluar el sistema integral en vehículos a gasolina con inyección electrónica

bajo condiciones reales, validando su precisión en la estimación del consumo de combustible frente a mediciones de referencia.

1.6. Alcances y limites

1.6.1. Alcances

El presente proyecto abarca el desarrollo de un sistema de monitoreo del consumo de combustible basado en la adquisición de datos a través del puerto OBD-II del vehículo, utilizando un microcontrolador ESP32-S3 y un módulo de comunicación CAN. El sistema comprende los siguientes alcances:

- Lectura de parámetros vehiculares en tiempo real, tales como revoluciones del motor (RPM), velocidad del vehículo y otras variables disponibles mediante el protocolo OBD-II, las cuales se utilizan para la estimación del consumo de combustible. Este alcance se logra mediante el controlador CAN TWAI nativo del ESP32-S3 y el transceptor SN65HVD230, que permiten sondear el bus OBD-II del vehículo y obtener, entre otros, RPM, MAP, IAT y los ajustes de combustible (STFT/LTFT) reportados por la propia ECU.
- Procesamiento de los datos obtenidos con el fin de generar información comprensible para el usuario, permitiendo evaluar el rendimiento del vehículo en función de su consumo energético. Este alcance se logra mediante el modelo Speed-Density, corregido con los ajustes de combustible de la ECU, implementado y calibrado sobre los dos vehículos de prueba (Nissan Vanette y Changan Honor).
- Desarrollo de una interfaz de monitoreo accesible mediante una página web generada por el propio ESP32, disponible a través de una dirección IP local, que permite la visualización de los datos en tiempo real sin necesidad de software adicional. Este alcance se logra mediante un panel web propio, servido por el ESP32-S3 a través de un punto de acceso WiFi local (SoftAP), sin depender de conectividad a internet ni de aplicaciones de terceros.
- Diseño, implementación y validación del sistema en condiciones reales de funcionamiento, permitiendo verificar su desempeño en la estimación del consumo de combustible. Este alcance se logra mediante la validación experimental sobre nueve tramos de ruta real, en la cual el sistema propio concuerda con un adaptador ELM327 comercial dentro de un error porcentual absoluto medio (MAPE) de 2,75 %, y con el aforo real de tanque lleno dentro de un MAPE de 4,84 %.

1.6.2. Limites

La solución propuesta presenta ciertas limitaciones técnicas y de alcance que deben ser consideradas para una correcta interpretación de los resultados.

- Una de las principales limitaciones está relacionada con el tipo de combustible del vehículo. El desarrollo se enfoca exclusivamente en vehículos a gasolina con

inyección electrónica; tampoco se enfoca en vehículos con diésel o con sistemas de combustible combinado, conocidos como vehículos *bi-fuel*, los cuales pueden operar tanto con gasolina como con gas natural o GLP. Debido a que el comportamiento físico y energético de estos combustibles es diferente, el sistema propuesto se enfoca únicamente en vehículos que operan exclusivamente con gasolina, no siendo posible un control preciso del consumo en vehículos bi-fuel (Alternative Fuels Data Center, 2026). Esta limitación se confirma en la práctica, ya que las pruebas experimentales se realizan exclusivamente sobre los dos vehículos a gasolina con inyección electrónica disponibles (Nissan Vanette y Changan Honor), sin evaluarse vehículos diésel ni bi-fuel.

- Asimismo, los vehículos de modelos antiguos que cuentan con sistemas de diagnóstico OBD-I presentan una limitación importante, ya que no permiten el acceso a información estandarizada del motor. Por esta razón, el sistema se encuentra limitado a vehículos que disponen de puerto OBD-II o sistemas de diagnóstico compatibles. Esta limitación se confirma, dado que ambos vehículos empleados en las pruebas cuentan con puerto OBD-II, requisito verificado antes de iniciar cada sesión de adquisición.
- En los vehículos a gasolina con inyección electrónica, la compatibilidad con sistemas de diagnóstico a bordo OBD-II se encuentra en unidades de fabricación relativamente reciente; esto es particularmente relevante, por ejemplo, en el parque automotor del transporte público de la ciudad de La Paz. En el presente trabajo se consideran únicamente modelos que disponen de esta tecnología, sin especificar marcas o versiones comerciales particulares. No obstante, el análisis experimental se limita exclusivamente a unidades seleccionadas que cumplen con el requisito de compatibilidad OBD-II, limitación que se confirma en la práctica al restringirse las pruebas a los dos vehículos disponibles que cumplen dicho requisito.
- Una de las limitaciones del presente estudio es la disponibilidad de vehículos para la realización de las pruebas experimentales, al no disponerse de acceso directo y permanente a unidades específicas de un contexto de aplicación particular, como los minibuses del transporte público de la ciudad de La Paz, uno de los ejemplos de vehículos a gasolina con inyección electrónica a los que apunta este trabajo. Por esta razón, las pruebas experimentales se realizan también en otros vehículos con características mecánicas similares que forman parte del parque automotor urbano y que cuentan con sistemas de inyección electrónica compatibles con la interfaz OBD-II. Esta limitación se confirma durante el desarrollo del proyecto, en el que, en efecto, no se dispuso de acceso a minibuses del transporte público, por lo que las pruebas se realizan sobre los dos vehículos mecánicamente similares ya indicados (Nissan Vanette y Changan Honor).
- Adicionalmente, la precisión del sistema depende del estado mecánico del vehículo. En vehículos con alto kilometraje o desgaste significativo, los valores obtenidos pueden presentar variaciones respecto a los valores ideales. Esta limitación se confirma de forma indirecta en los propios resultados de validación: la exactitud del sistema se evalúa por separado en cada vehículo (Nissan Vanette y Changan

Honor), precisamente porque el estado mecánico y la arquitectura del motor de cada unidad afectan de forma particular la calibración y el error de estimación obtenidos.

Capítulo 2

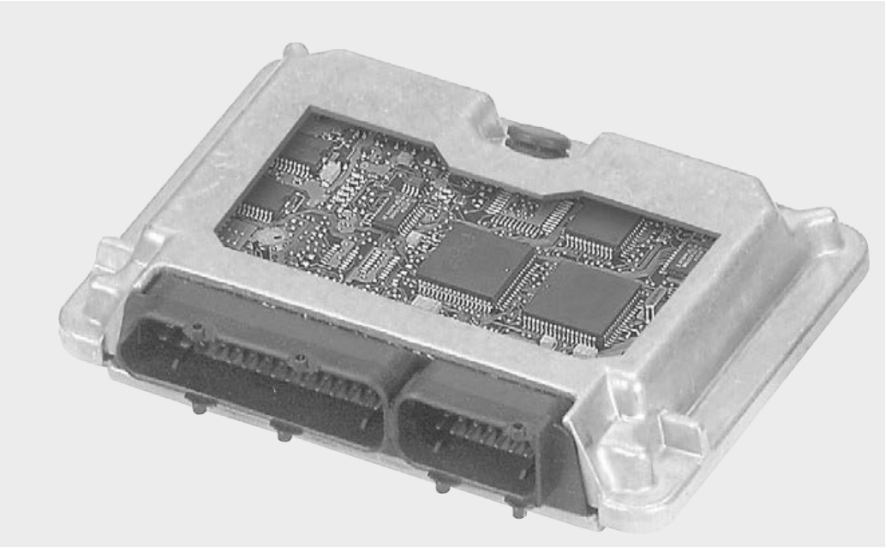
MARCO TEÓRICO

2.1. Unidad de control electrónica ECU

La *Engine Control Unit* (ECU), o Unidad de Control del Motor, es un módulo electrónico encargado de gestionar el funcionamiento del motor de combustión interna, así como de otros sistemas del vehículo (Robert Bosch GmbH, 2018a; Heywood, 2018).

En términos simples, la ECU funciona como la "computadora central" del motor y se ve fisicamente como en la Figura 2.1 recibe señales de sensores, las procesa y emite órdenes a actuadores como inyectores, válvulas de mariposa, sistemas de encendido y ventiladores para mantener el motor funcionando de manera eficiente, segura y dentro de los límites de emisiones permitidos (AcademiaLab, 2024; Club AutoDoc, 2022).

Figura 2.1
Estructura de la ECU



Nota. Cubierta de la carcasa abierta de una ECU. Fuente: (Kaiser, 2014).

La ECU integra principios de electrónica, control automático y termodinámica del motor, permitiendo regular la mezcla de aire y combustible, el encendido, las emisiones y el consumo energético (Heywood, 2018; Smith and Johnson, 2015). Su introducción en los vehículos modernos ha permitido optimizar el rendimiento del motor y cumplir con normas ambientales cada vez más estrictas (Smith and Johnson, 2015).

Entre las funciones más relevantes de una ECU automotriz se pueden mencionar (Robert Bosch GmbH, 2018a; Club AutoDoc, 2022; Autoavance, 2020):

- **Control de la inyección de combustible**: ajuste de la cantidad, el momento y la duración del chorro de combustible en cada cilindro según la carga del motor, la temperatura del refrigerante, la presión de admisión y otros parámetros.
- **Control del sistema de encendido**: determinación del avance del encendido (ángulo de bujía) para optimizar la combustión de la mezcla aire-combustible.
- **Control de la admisión de aire**: regulación de la válvula de mariposa, el acelerador electrónico y dispositivos de recirculación de gases (EGR, turbocompresión, etc.) para el caudal de aire al motor.
- **Diagnóstico y monitorización del motor**: detección de anomalías, activación de luces de advertencia en el panel (MIL) y almacenamiento de códigos de falla mediante el sistema OBD-II.
- **Gestión de otros sistemas**: en vehículos avanzados, la ECU o sus módulos asociados (ECM, PCM, VCM) pueden intervenir en el control de la transmisión automática, el sistema de carga y el aire acondicionado.

2.1.1. Tipos de modulo de control

- **ECM** (engine control module), modulo de control del motor.Controla y almacena únicamente los códigos de diagnóstico de fallas (DTCs) de los componentes del motor.
- **TCM** (transmission control module), modulo de control de la transmisión. Controla y almacena datos relacionados con el cambio de marchas, la presión hidráulica y el acoplamiento del convertidor de par en vehículos con transmisión automática.
- **PCM** (train power control module), modulo de control del tren de potencia. Controla y almacena datos del motor y la transmisión.
- **VCM** (vehicle control module), modulo de control del vehículo. Controla y almacena datos del motor, la transmisión y otros sistemas del vehículo como sistemas de frenos ABS.(Poudel and Munir, 2018)

2.1.2. Estructura interna de la ECU

La estructura interna de una ECU se basa en un sistema de hardware y software integrado, donde el núcleo principal es un microcontrolador (CPU) encargado de procesar

en tiempo real las señales provenientes de sensores y emitir órdenes a los actuadores del motor (Robert Bosch GmbH, 2018a; Heywood, 2018). Alrededor de este microcontrolador se disponen memorias de tipo flash y ROM, que almacenan el firmware y los mapas de encendido e inyección, así como memoria RAM utilizada para cálculos y variables de estado durante el funcionamiento (Club AutoDoc, 2022).

Además, la ECU integra módulos de acondicionamiento de señales de entrada (filtros, amplificadores y convertidores analógico-digital) y etapas de potencia de salida (drivers de transistores o MOSFET) para activar correctamente actuadores como inyectores de combustible, válvulas solenoides y ventiladores de refrigeración (Autoavance, 2020). Y se divide en los siguientes sistemas.

- **Periferia** Consta de elementos pasivos resistencias, capacitores.
- **Circuitos de filtrado** Filtros pasa bajos para contrarrestar ruido y mejorar la señal.
- **Fuente** Elementos pasivos y semiconductores resistencias, capacitores, diodos y reguladores de tensión.
- **Procesamiento de datos** Usa 2 sistemas clave, la memoria la que almacena todos los datos, y el procesador que lee todos los datos.
- **Driver** Maneja todos los actuadores: transistores, drivers para motores de paso.
- **Multiplexor** Comparte datos con otras ECUS.

Todos los sistemas mencionados se muestran en conjunto en la Figura 2.2

Procesamiento de señales

Para que el microprocesador pueda interpretar las señales de los sensores (señales de entrada), estas deben ser convertidas en el módulo de control electrónico de analógicas a digitales. Estas señales luego son acompañadas con los parámetros lógicos grabados en la memoria y luego será elaborada la orden o señales de salida a los actuadores.

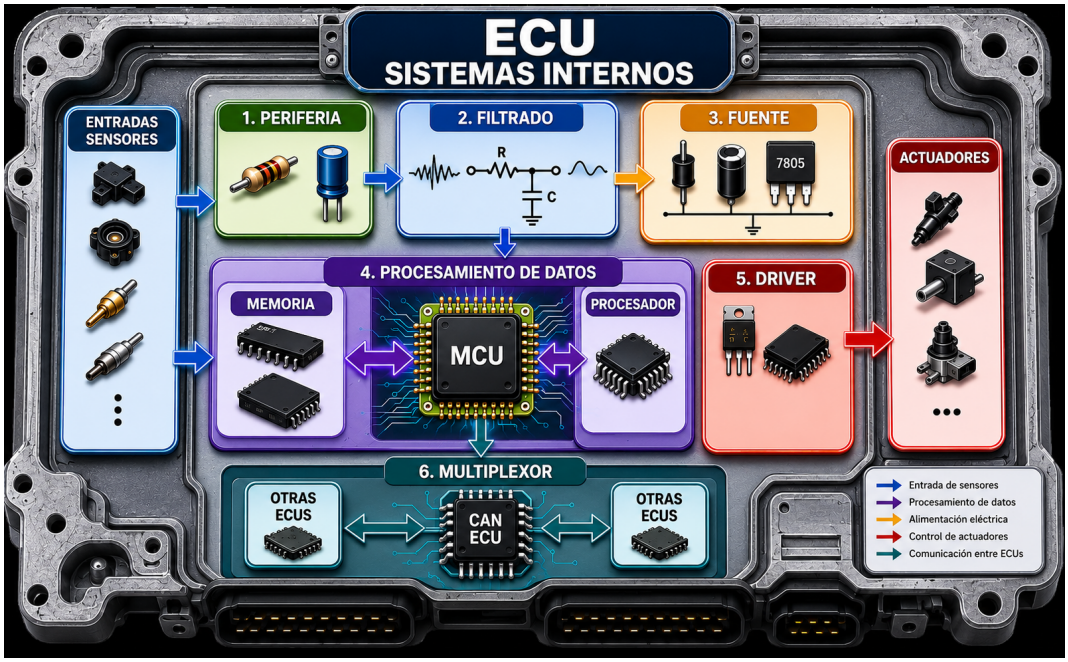
Se muestra un diagrama general del procesamiento de la ECU en la Figura 2.3.

Parámetros de control del motor

Es necesario diferenciar bien cada una de las fases que esta debe realizar para enviar señales a los diferentes actuadores del vehículo.
Para ello se divide el estudio en cuatro bloques de funcionamiento.

- **Bloque de entrada** Se denomina bloque de entrada a todos los circuitos que se encuentran como receptores de las diferentes señales que van a ingresar a la ECU y antes de que lleguen al microprocesador. Encontramos en este sentido, filtros, amplificadores, conformadores de impulsos, conversores análogos a digital.
    - Regulador de tensión.- El regulador de tensión (5V) es utilizado como señal de entrada para casitodos los sensores de inyección y para la operación interna de

Figura 2.2
Arquitectura interna de la ECU



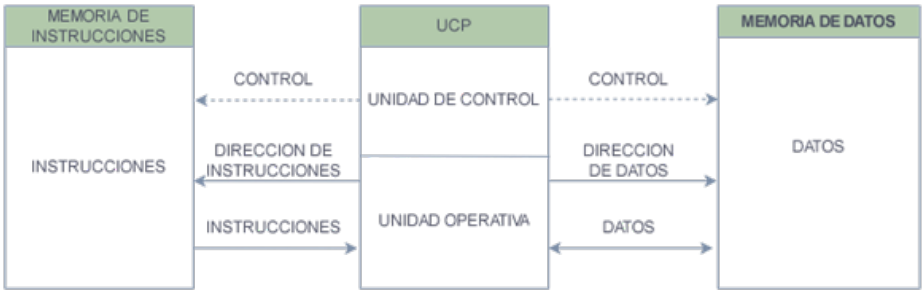
Nota. Arquitectura interna y conjunto de sistemas dentro de la unidad de control. Elaboración propia basada en (Poudel and Munir, 2018) y generada con apoyo de inteligencia artificial (Gemini).

memorias y el microprocesador del módulo de control electrónico. La tensión regulada es mantenida a pesar de las variaciones de tensión de la batería en 5V ± 15

- Filtrado de señales.- Los filtros digitales tienen como entrada una señal analógica o digital y en su salida tienen otra señal analógica o digital, pudiendo haber cambiado en amplitud, frecuencia o fase dependiendo de las características del filtro digital.
- Conformadores de impulso.- Actúa para recibir los impulsos de tensión de los órganos de información del encendido. Estos impulsos son modificados en magnitud y en forma, para dejarlos en condiciones que puedan ser procesados por el micro-ordenador.
- Convertidor Análogo – Digital.- La señal que recibe el módulo de control electrónico desde los sensores, no puede ser interpretada por el microprocesador, por lo tanto las señales de los sensores que son análogas, deben ser convertidas a digitales.
  Para esta conversión se utilizan transistores de saturación o corte, que se comportan exactamente igual a un relé activado-desactivado.

- **Bloque de procesamiento** Son todos los circuitos que desarrolla las funciones programadas y que están constituidos circuitalmente por el procesador, memorias

Figura 2.3
Diagrama de procesamiento de la ECU



Nota. Secuencia lógica de las etapas de entrada, procesamiento y salida de datos en la unidad de control. Elaboración propia.

y todo circuito que se vea involucrado en la ejecución del software.

- **Memorias.-** Es esta se realizan los cálculos matemáticos de acuerdo a las señales de entrada; la memoria se borra cuando se apaga el motor.
  - **Memoria RAM (Random Access Memory):** El microprocesador utiliza la memoria RAM para saber el estado general y las condiciones del motor y en función a esto hacer funcionar los actuadores.
  - **Memoria KAM (Keep Alive Memory):** La memoria KAM vive con la memoria RAM y su función es de guardar los datos que no se pueden perder al cerrar el contacto, como por ejemplo códigos de fallas aleatorias de sensores.

    A diferencia de la memoria RAM, la KAM no se borra al cerrar el contacto, pero si se borra al desconectar la batería.

    Cuando en la memoria KAM se almacena un defecto, este permanecerá instalado aun después de ser corregido, debiendo desconectar la batería para que se borre o en algunos casos, se debe proceder al borrado con un scanner.
  - **Memoria EPROM (Erasable Programmable Read Only Memory):** En la memoria EPROM (programable) o PROM (programada) es donde el microprocesador consulta todas las calibraciones del vehículo, tales como el peso del mismo y la compresión del motor. Al cerrar el contacto o desconectar la batería, esta memoria no se borra.
  - **Memoria ROM (Read Only Memory):** Esta memoria mantiene grabados los programas con todos los datos, curvas características y valores teóricos con los que ha de funcionar el sistema.

    Esta memoria es utilizada por el microprocesador para comparar los parámetro de cada sensor en cada situación y en los casos en que el

microprocesador detecte alguna anomalía en alguna señal, la reemplazará por un valor de la memoria ROM.

La memoria ROM es una memoria de consulta y al igual que la memoria EPROM no se borra ni al cerrar el contacto, ni al desconectar la batería.

- **Microprocesador:** Es el componente físico principal dentro de la unidad de control, encargado de ejecutar las instrucciones de software para optimizar el funcionamiento del motor. Este elemento realiza los cálculos matemáticos, la toma de decisiones y ejecuta las estrategias de emergencia. Contiene en su interior tres dispositivos fundamentales:
  - **Unidad lógica de calculo (ALU):**Realiza las operaciones aritméticas y las operaciones lógicas.
  - **Acumulador:** Es una memoria intermedia que permite al CPU guardar datos mientras trabaja con otros que están directamente relacionados con el proceso en curso.
  - **Unidad de control:** Es el elemento activo que solicita los datos, controla las entradas, las salidas y el desarrollo de las operaciones.

- **Bloque de salida** Así como las señales son tratadas al ingresar, existen circuitos que se encuentran entre las salidas del microprocesador y los diferentes elementos que van a ser actuados como por ejemplo: bobinas de encendido, inyectores, relés, etc.
  Tenemos los siguientes componentes importantes en este bloque:
  - Convertidor digital - análogo.- Así como las señales de los sensores deben ser transformadas en digitales, para que puedan ser interpretadas por el microprocesador y de esta forma "saber lo que está pasando La señal digital debe convertirse en analógica para controlar los actuadores(señales analógicas).

    De esta forma es que todas las señales micro procesadas son convertidas de digitales en análogas para controlar a los actuadores. El convertidor digital recibe un número de 8 BITS y la trasforma en una tensión continua de un valor proporcional al número digital.
  - Drivers (Controlador de dispositivo).- El drivers están básicamente diseñado para controlar los actuadores, como por ejemplo los inyectores, bobinas, relés, entre otros, estos circuitos deben cumplir con requisitos de manejo de potencia puesto que la corriente que se maneja en muchos de ellos alcanza los 5A y los voltajes operados pueden llegar a picos de hasta 400V.
    - Transistores
    - Circuitos integrados de control
  - Circuitos de potencia con transistores.- Se encargan de dar los pulsos de activación a las bobinas de encendido, inyectores, relés, etc. para de esta manera poner en funcionamiento a cada uno de los actuadores.

- **Bloque de soporte** Se denomina así al conjunto de componentes que tienen como función alimentar a los circuitos internos mencionados anteriormente. Lo constituye la fuente de alimentación de la ECU.
  - Regulador de voltaje.-
  - Fuente de alimentación de la ECU.- Los ECM disponen de una fuente de poder interna que proporciona distintos tipos de voltajes para energizar componentes como sensores y Actuadores. Estos voltajes pueden tener una variación.
    Conformados por:
    - Diodos rectificadores y zener.
    - Condensadores.
    - Reguladores de Tensión.
    - Resistencias.

  Datos importante de la parte de bloque de soporte:

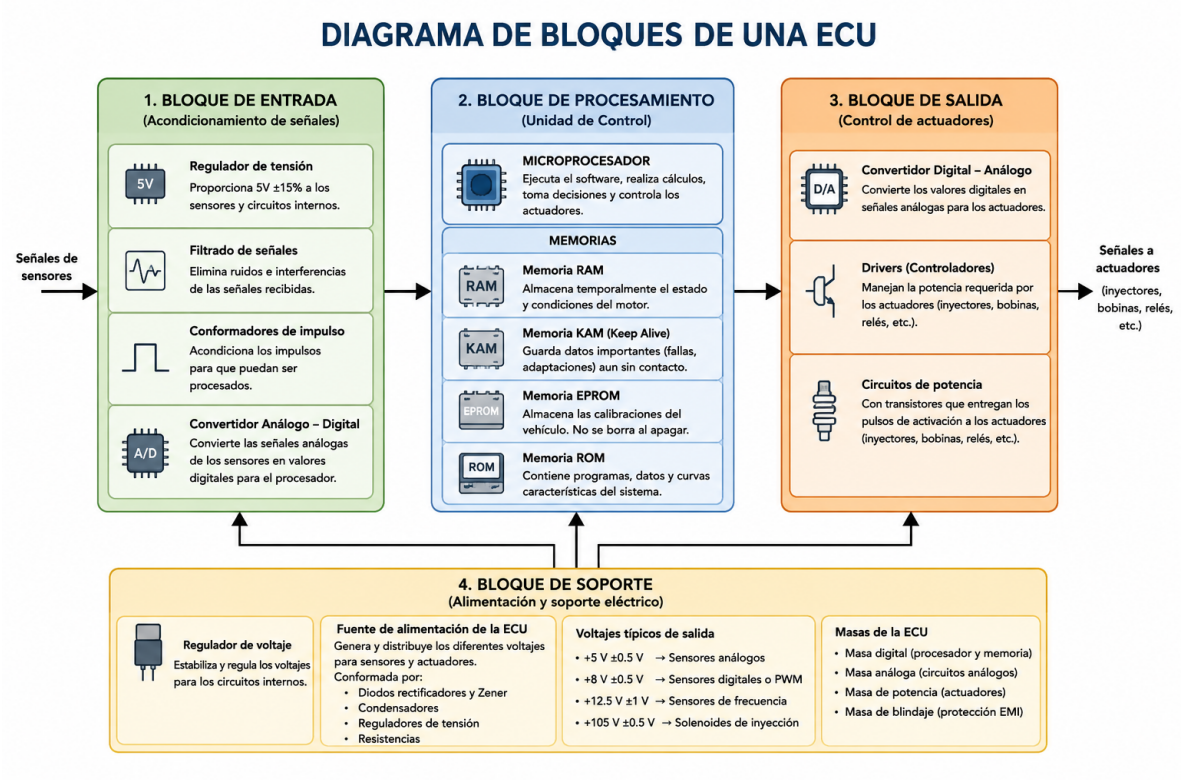
  - + 5 volt. + - 0,5 V voltaje de suministro para sensores análogos.
  - + 8 volt. + - 0,5 V voltaje de suministro para sensores digitales o PWM.
  - + 12,5 volt. + - 1V voltaje de suministro para sensores de frecuencia electrónicos.
  - + 105 volt. + - 0,5 voltaje de suministro para solenoides para inyección de combustible.

  Para el negativo, existen tres posibilidades de masas en una ECU, estas son:

  - **Masa digital:** Usada por la ECU para el sistema de procesamiento de datos, Procesador y Memoria.
  - **Masa análoga:** Usada por la ECU para circuitos análogos, por ejemplo conversores análogos a digital.
  - **Masa de potencia:** Usada por la ECU para circuitos de fuente y control de actuadores principalmente ej. , regulador de tensión y principalmente transistores.
  - **Masa de blindaje:** La Ecu posee un circuito de blindaje, el cual es capaz de transportar las señales dentro de la ECU sin la presencia de ruido electromagnético.

  Se presenta un resumen en la Figura 2.4

Figura 2.4
Esquema de la unidad de control electrónica ECU



Nota. Diagrama esquemático integral de los bloques funcionales de la unidad de control. Elaboración propia, generada con apoyo de inteligencia artificial (Gemini).

2.1.3. Importancia de la ECU en el consumo de combustible

La ECU desempeña un papel fundamental en la optimización del consumo de combustible, actúando como el elemento central que regula la mezcla aire-combustible, el encendido, la admisión de aire y la gestión general del motor (Robert Bosch GmbH, 2018a; Heywood, 2018). Mediante el uso continuo de sensores de presión de admisión, temperatura, posición del acelerador, velocidad y carga, la ECU ajusta en tiempo real la cantidad de combustible inyectado, el avance del encendido y el flujo de aire, favoreciendo siempre una combustión lo más completa y eficiente posible (Smith and Johnson, 2015; Club AutoDoc, 2022).

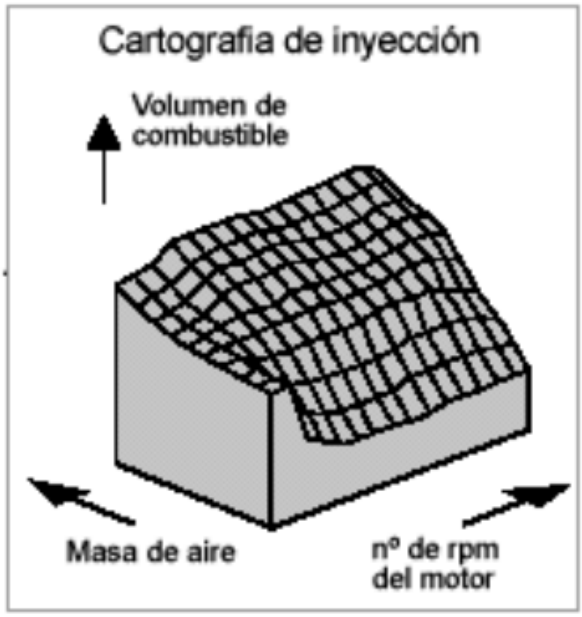
Esta regulación permanente permite que el motor funcione cerca de su punto óptimo de eficiencia en una amplia gama de condiciones de conducción, lo que se traduce en una reducción del consumo específico de combustible y en menores emisiones contaminantes (AcademiaLab, 2024; Centralita BSI, 2025).

Además, en vehículos con transmisión automática, la ECU colabora con otros módulos de control (por ejemplo, ECU de transmisión) para seleccionar el momento más

adecuado para cambiar de marcha, lo que contribuye a mantener el motor en rangos de revoluciones donde el consumo de combustible es más bajo (Smith and Johnson, 2015; Pedal Commander, 2025).

Por lo tanto la cantidad de combustible a inyectar es determinado teniendo en cuenta también la cartografía de inyección que esta memorizada en la unidad de control y que intenta en todo momento evitar la emisión de contaminantes (humo negro) ver Figura 2.5. Si el volumen de aire aspirado es demasiado bajo la cantidad de combustible inyectado es limitado a un valor que no provoque humos negros.

Figura 2.5
Cartografía de inyección



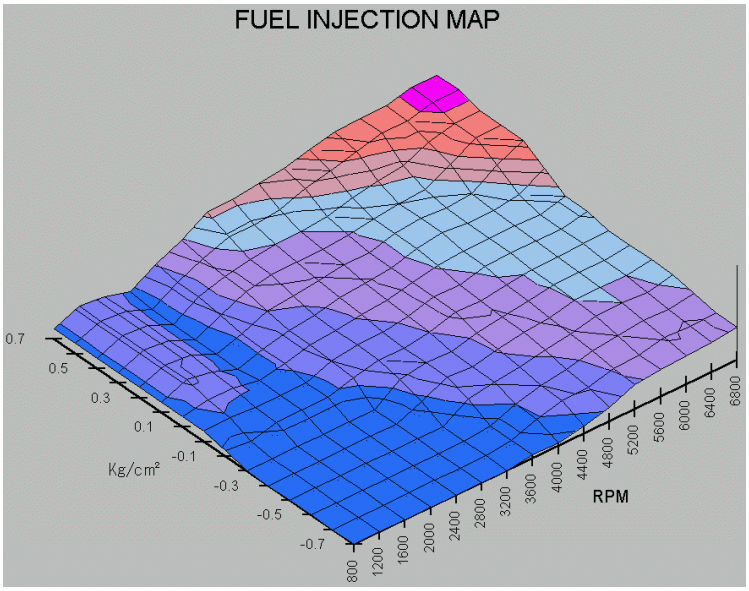
Nota. Mapa tridimensional de calibración que relaciona la carga del motor y las revoluciones por minuto para determinar el tiempo de inyección. Tomado de (Poudel and Munir, 2018).

La unidad de control tiene memorizado un mapa de comienzo de la inyección.Este mapa toma como referencia principal el nº de rpm del motor y la cantidad de combustible inyectado ver Figura 2.6 Como parámetro corrector se utiliza la temperatura del motor que actúa también sobre el comienzo de la inyección.

En resumen, la ECU no solo mejora el rendimiento y la respuesta del motor, sino que es un componente clave para lograr una **operación económica y sostenible** del vehículo, incrementando la eficiencia del combustible sin comprometer significativamente la potencia entregada (Robert Bosch GmbH, 2018a; SERNAUTO, 2024).

Figura 2.6

Relación de inyección entre el sensor MAP y las RPM



Nota. Comportamiento de la dosificación de combustible en un motor de cuatro tiempos operando a nivel del mar en función de la presión absoluta del colector y el régimen de giro. Tomado de (ClubJapo Forum, 2023).

2.2. Protocolos de comunicación en vehículos

En los vehículos modernos, la comunicación entre unidades de control electrónico (ECU) se realiza a través de diversos buses de datos en serie, que permiten integrar sistemas de motor, transmisión, chasis, seguridad y entretenimiento en una red coherente (Navixy, 2024; MSG Equipment, 2025). Entre los protocolos más utilizados se encuentran CAN, LIN, FlexRay y MOST, cada uno con características distintas de velocidad, ancho de banda, costo y aplicación específica (Scribd, 2025).

2.2.1. CAN

El protocolo *Controller Area Network* (CAN) es el estándar más extendido en la comunicación interna de vehículos, utilizado para conectar unidades de control del motor (ECM), de transmisión (TCM) y sistemas de frenado (ABS), entre otros (Navixy, 2024). CAN opera sobre un bus de dos hilos (CAN high y CAN low) y permite la transmisión de mensajes priorizados, lo que garantiza que los datos de seguridad y dinámica del vehículo tengan mayor prioridad que los de confort o información (Scribd, 2025). Su velocidad típica varía desde 125 kbit/s hasta 1 Mbit/s, convirtiéndolo en una solución robusta, económica y tolerante a fallos (MSG Equipment, 2025).

2.2.2. LIN

El bus *Local Interconnect Network* (LIN) es un protocolo de baja velocidad diseñado principalmente para aplicaciones de confort y comodidades, como ventanas eléctricas, espejos, luces interiores y sensores de climatización (Scribd, 2025). Comparado con CAN, LIN es más simple y barato, ya que trabaja con un solo maestro y varios esclavos en topología maestro-esclavo (MSG Equipment, 2025). Aunque su velocidad máxima suele estar alrededor de 20 kbit/s, LIN es ideal para tareas poco críticas donde la prioridad es reducir costos de cableado y electrónica (Scribd, 2025).

2.2.3. FlexRay

El protocolo FlexRay es un bus de alta velocidad y baja latencia orientado a aplicaciones críticas, como sistemas de conducción avanzada, control de chasis y vehículos autónomos (Navixy, 2024; Autoavance, 2025). FlexRay ofrece tiempos de respuesta deterministas y soporta tasas de transferencia de hasta 10 Mbit/s, lo que permite manejar grandes volúmenes de datos en tiempo real (MSG Equipment, 2025). Además, su topología dual-canal permite redundancia y alta disponibilidad, características clave para funciones de seguridad que no admiten interrupciones (Autoavance, 2025).

2.2.4. MOST

El bus MOST (*Media Oriented Systems Transport*) está dedicado a la transmisión de datos multimedia, como audio, video y señales de info-entretenimiento, dentro del vehículo (Navixy, 2024). MOST utiliza un anillo óptico o eléctrico y soporta velocidades de 25, 50 o 150 Mbit/s, permitiendo integrar pantallas, sistemas de navegación, reproductores de audio y dispositivos de comunicación sin afectar las redes de control principal (Scribd, 2025). Su diseño está orientado a mantener alta calidad de señal y bajo retardo en el transporte de flujos multimedios, separando esta red de la red de control motor-tracción construida sobre CAN o FlexRay (Scribd, 2025).

El resumen de lo abordado se presenta en la Tabla 2.1

2.3. Sistema OBD-II

2.3.1. Evolución del diagnóstico a bordo

El diagnóstico a bordo, conocido como OBD (*On-Board Diagnostics*), evolucionó desde sistemas muy básicos de monitoreo hasta convertirse en una herramienta estandarizada y esencial para la detección de fallas en vehículos modernos. Sus primeras aplicaciones aparecieron en la década de 1960 con sistemas de diagnóstico analógicos, y posteriormente, en las décadas de 1970 y 1980, los fabricantes comenzaron a incorporar funciones electrónicas capaces de almacenar información sobre el funcionamiento del motor y registrar anomalías (Geotab, 2020). La estandarización del sistema avanzó a finales de los años ochenta, cuando se impulsó el uso de conectores y protocolos comunes,

Tabla 2.1
Comparación de los protocolos de comunicación en vehículos

Ítem	CAN	LIN	FlexRay	MOST
Velocidad típica	125 kbit/s a 1 Mbit/s	Hasta 20 kbit/s	Hasta 10 Mbit/s	25, 50 o 150 Mbit/s
Aplicación	Control de motor, transmisión, ABS	Comodidades (ventanas, espejos, luces)	Sistemas de seguridad y conducción avanzada	Multimedia (audio, video, infoentretenimiento)
Topología	Bus multidifusión robusto	Maestro-esclavo, sencillo	Dual-canal redundante	Anillo óptico o eléctrico
Prioridad	Alta, con priorización de datos críticos	Baja, tareas no críticas	Alta, determinista	Media, orientado a multimedia
Costo	Moderado	Bajo	Alto	Alto
Latencia	Media	Alta	Muy baja	Media

Nota. Elaboración propia.

lo que permitió que los escáneres pudieran leer códigos de falla de manera uniforme en distintos vehículos.

Con la llegada del OBD-II en la década de 1990, el diagnóstico se volvió más completo, permitiendo supervisar sistemas relacionados con emisiones, rendimiento del motor y sensores críticos, facilitando así el mantenimiento preventivo y la reducción de contaminantes (Geotab, 2020).

Para llevar a cabo dichos diagnósticos, se emplean convencionalmente escáneres automotrices capaces de recopilar la información mencionada a través del puerto OBD-II; este instrumento se muestra en la Figura 2.7.

2.3.2. Estructura del sistema OBD-II

El sistema OBD-II está constituido por la unidad de control electrónico o ECU, el arnés de cableado, el conector de diagnóstico DLC y el protocolo de comunicación utilizado por el vehículo, que puede ser CAN, K-Line, PWM, VPW, entre otros (Autoavance, 2026; Autosoporte, 2021; FlexiHub, 2024). La ECU recibe señales de sensores distribuidos en el motor y, a partir del procesamiento interno, genera datos de funcionamiento, estados de actuadores y códigos de falla que pueden ser consultados a través del puerto OBD-II mediante una herramienta de diagnóstico (Autoavance, 2026; Universidad Mayor de San Andrés, sf).

La información no sale de la ECU de forma directa hacia el escáner, sino que primero es organizada por la unidad de control en mensajes estructurados según el

Figura 2.7
Escáner automotriz

Nota. Herramienta de diagnóstico electrónico utilizada para la lectura de códigos de falla y monitoreo de parámetros a través de la interfaz del puerto OBD-II. Tomado de (OBDTester, 2015).

protocolo de comunicación del vehículo. En sistemas basados en CAN, por ejemplo, la ECU transmite y recibe tramas de datos por las líneas CAN High y CAN Low, que llegan al conector OBD-II a través del arnés eléctrico del vehículo (FlexiHub, 2024; Equipo Automotriz JAVAZ, 2010). En otros sistemas, la comunicación puede realizarse por una línea de datos única, como la K-Line, o por otros protocolos definidos por el fabricante (FlexiHub, 2024; Autosoporte, 2021).

Entre la ECU y el puerto de diagnóstico intervienen principalmente el microcontrolador interno de la ECU, los circuitos de acondicionamiento de señal y los transceptores de comunicación, que adaptan los niveles eléctricos del microprocesador al bus del vehículo (Autoavance, 2026; Universidad Mayor de San Andrés, sf). Estos transceptores permiten que la información generada dentro de la ECU sea enviada correctamente hacia el DLC sin pérdida de integridad, mientras que el escáner OBD-II actúa como elemento externo que solicita los datos y los interpreta para mostrar parámetros en tiempo real, códigos de avería y estados de sensores o actuadores (Autoavance, 2026; Autosoporte, 2021) ver Figura 2.8. De esta manera, el sistema OBD-II funciona como una interfaz estandarizada entre la electrónica interna del vehículo y el equipo de diagnóstico.

2.3.3. Conector OBD-II y asignación de pines

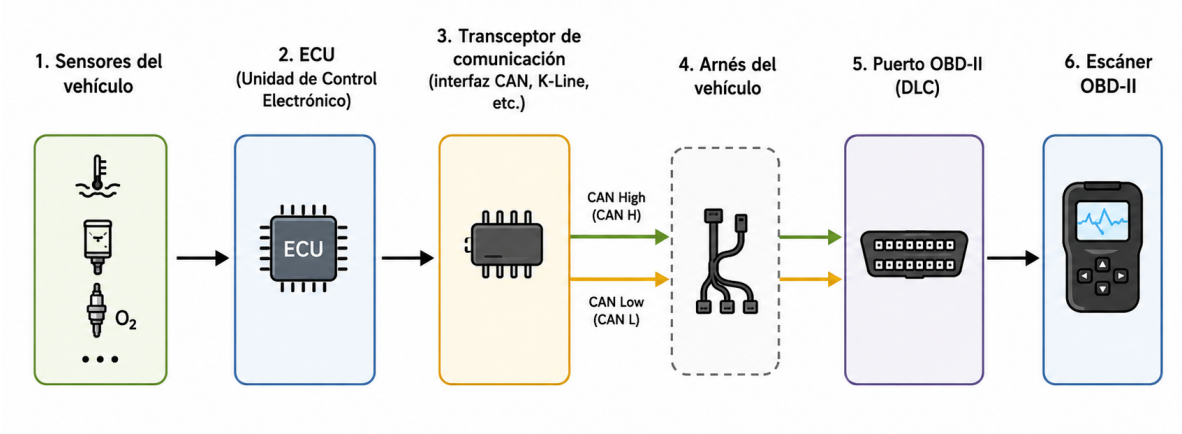
El puerto OBD-II es una interfaz de comunicación de 16 pines. Este puerto permite conectar dispositivos de diagnóstico, escáneres automotrices o sistemas de adquisición de datos para obtener información de las Unidades de Control Electrónico (ECU) del vehículo. En la Figura 2.9 se presenta los pines de conexión de este puerto.

El conector OBD-II posee 16 pines, cada uno con funciones específicas dependiendo del protocolo de comunicación utilizado el detalle de cada uno de estos pines:

1. LS-CAN High función específica del fabricante.
2. Bus SAE J1850.

Figura 2.8

Esquema general del flujo de información en el sistema OBD-II

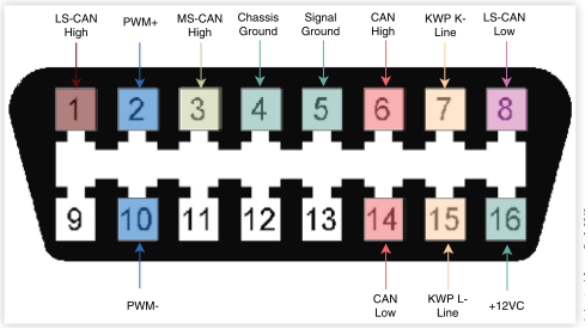


Nota. Diagrama de bloques que describe la transmisión de datos desde la ECU y sus sensores periféricos hacia la interfaz de diagnóstico a través del bus de datos. Elaboración propia.

3. MS-CAN High/función específica del fabricante.
4. Tierra (chasis).
5. Tierra de señal.
6. CAN High (CAN alto).
7. Línea K (ISO 9141 / ISO 14230).
8. LS-CAN Low/función específica del fabricante.
9. Función específica del fabricante.
10. Bus SAE J1850/PWM.
11. Función específica del fabricante.
12. Función específica del fabricante.
13. Función específica del fabricante.
14. CAN Low.
15. Línea L.
16. Alimentación +12 V batería.

En la mayoría de los vehículos modernos, los pines 6 (CAN High) y 14 (CAN Low) son utilizados por el protocolo Controller Area Network (CAN) para la transmisión de datos entre las ECU. El puerto OBD-II puede utilizar diferentes protocolos de comunicación dependiendo del fabricante y del año del vehículo. Entre los más comunes se encuentran:

Figura 2.9
Distribución de pines del conector OBD-II



Nota. Configuración y asignación de los 16 pines del conector estandarizado J1962 para el diagnóstico a bordo. Tomado de (OBDTester, 2015).

- SAE J1850 PWM.
- SAE J1850 VPW.
- ISO 9141-2.
- ISO 14230 (KWP2000).
- ISO 15765-4 (CAN).

Este puerto permite solicitar información especifica mediante parámetros de identificación (Parameter ID - PIDs), los mas comunes son:

- Velocidad del vehículo.
- Revoluciones del motor.
- Temperatura del refrigerante.
- Posición del acelerador.
- Códigos de falla (DTC).

Algunas marcas permiten la visualizar una mayor cantidad de parámetros que otras, dependiendo de la cantidad de módulos con los que cuente el vehículo.

2.3.4. Protocolos de comunicación

En el sistema OBD-II existen varios protocolos de comunicación estandarizados que permiten el intercambio de información entre la ECU y el equipo de diagnóstico. En total, se consideran cinco protocolos principales de comunicación OBD-II, aunque en la práctica algunos de ellos ya han sido sustituidos por tecnologías más modernas (Autoavance, 2026; Autosoporte, 2021; FlexiHub, 2024).

- **ISO 15765-4 CAN:** es el protocolo más utilizado en vehículos modernos. Se basa en la red CAN y utiliza las líneas CAN High y CAN Low del conector OBD-II. Se caracteriza por su alta velocidad, confiabilidad y capacidad para transmitir múltiples mensajes entre diferentes unidades de control (FlexiHub, 2024; CISE Electronics, sf).
- **ISO 9141-2:** se emplea principalmente en vehículos más antiguos, especialmente de fabricantes europeos y asiáticos. Utiliza una única línea de comunicación llamada K-Line, normalmente conectada al pin 7 del conector OBD-II. Su velocidad es menor que la de CAN, pero fue muy importante en las primeras implementaciones de diagnóstico a bordo (FlexiHub, 2024).
- **SAE J1850 PWM:** fue utilizado en algunos vehículos de origen estadounidense. Emplea una comunicación por modulación de ancho de pulso y presenta una velocidad intermedia. Aunque fue relevante en su momento, actualmente ha sido reemplazado en gran parte por CAN (Autosoporte, 2021; Equipo Automotriz JAVAZ, 2010).
- **SAE J1850 VPW:** también fue común en vehículos de fabricantes norteamericanos. Su principal diferencia respecto a PWM es la forma de codificación de la señal, utilizando modulación por ancho de pulso variable. Tiene una arquitectura simple, pero menor capacidad que CAN (Autosoporte, 2021; Equipo Automotriz JAVAZ, 2010).
- **ISO 14230-4 KWP2000:** conocido como *Keyword Protocol 2000*, fue utilizado en varios vehículos antes de la adopción masiva de CAN. Emplea comunicación en serie sobre K-Line y permite realizar diagnósticos, lectura de datos en tiempo real y borrado de códigos de falla (Autoavance, 2026; Autosoporte, 2021).

En resumen como se muestra en la Tabla 2.2, los protocolos OBD-II evolucionaron desde soluciones más simples como K-Line y J1850 hasta el uso predominante del protocolo ISO 15765-4 CAN, que ofrece mayor velocidad, estabilidad y compatibilidad con los sistemas electrónicos actuales del vehículo (Autoavance, 2026; FlexiHub, 2024).

2.3.5. Parámetros de diagnóstico (PIDs)

Los parámetros de diagnóstico, conocidos como PIDs (*Parameter IDs*), son códigos utilizados por el sistema OBD-II para solicitar y recibir información específica del funcionamiento del vehículo. Estos parámetros permiten acceder a datos en vivo provenientes de la ECU, tales como revoluciones del motor, velocidad del vehículo, temperatura del refrigerante, posición del acelerador, voltaje de sensores y otros valores relevantes para el diagnóstico (Wikibooks, 2021a; Geotab, 2020).

Dentro del OBD-II, los PIDs forman parte del llamado *Modo 01*, también denominado flujo de datos o datos en tiempo real. En este modo, el escáner de diagnóstico solicita a la ECU un parámetro concreto y la unidad de control responde con el valor correspondiente, que luego puede ser interpretado por el técnico para evaluar el estado de funcionamiento del motor y de los sistemas relacionados con emisiones (Wikibooks,

Tabla 2.2
Comparación de protocolos de comunicación OBD-II

Protocolo	Medio	Velocidad	Aplicación	Características
ISO 15765-4 CAN	CAN High / CAN Low	Alta	Vehículos modernos	Alta velocidad, confiabilidad y comunicación entre múltiples ECUs.
ISO 9141-2	K-Line	Baja	Vehículos europeos y asiáticos antiguos	Comunicación serial simple utilizada en primeras implementaciones OBD-II.
SAE J1850 PWM	PWM	Media	Vehículos estadounidenses	Comunicación mediante modulación por ancho de pulso.
SAE J1850 VPW	VPW	Media	Vehículos norteamericanos	Modulación por ancho de pulso variable y arquitectura simple.
ISO 14230-4 KWP2000	K-Line	Media	Vehículos previos a CAN	Permite diagnóstico, lectura de datos y borrado de códigos de falla.

Nota. Elaboración propia con base en (FlexiHub, 2024), (Autosoporte, 2021) y (Autoavance, 2026).

2021a,b). Por ejemplo, mediante un PID es posible conocer la temperatura del motor, la carga calculada, la presión absoluta del múltiple de admisión o el estado de los sensores de oxígeno (Wikibooks, 2021a,b).

Los PIDs no solo facilitan la lectura de variables operativas, sino que también permiten verificar si un vehículo soporta o no determinados parámetros. Esto se debe a que cada ECU responde únicamente a los PIDs que tiene implementados, lo que hace posible identificar qué información puede ser consultada en cada modelo de vehículo (Wikibooks, 2021a). En consecuencia, los PIDs constituyen una herramienta fundamental para el diagnóstico moderno, ya que transforman los datos internos de la ECU en información útil para el mantenimiento, la detección de fallas y el análisis de desempeño del motor (Geotab, 2020; Wikibooks, 2021b).

En la Tabla 2.3 se muestra los PIDs mas utilizados y mas accesibles de ver pero de forma mas concreta se muestran en la Tabla A1 del Apéndice A.

Tabla 2.3
PIDs OBD-II más utilizados en monitoreo vehicular

PID	Parámetro	Unidad	Aplicación principal
0C	Velocidad del motor (RPM)	rpm	Monitoreo del rendimiento térmico y dinámico.
0D	Velocidad del vehículo	km/h	Cálculo de desplazamiento y consumo de combustible.
05	Temperatura del refrigerante	°C	Supervisión térmica de las condiciones de operación.
0F	Temperatura del aire de admisión	°C	Correcios en la densidad de la mezcla aire-combustible.
10	Flujo de aire másico (MAF)	g/s	Estimación absoluta de la masa de aire ingresada.
11	Posición del acelerador	%	Detección de la carga solicitada por el conductor.
04	Carga calculada del motor	%	Análisis de la demanda volumétrica del motor.
2F	Nivel de combustible	%	Monitoreo volumétrico de las reservas del tanque.
42	Voltaje del módulo de control	V	Diagnóstico del estado del sistema eléctrico general.
03	Códigos de falla almacenados (DTC)	—	Diagnóstico y detección activa de averías.

Nota. Elaboración propia con base en la documentación del estándar SAE J1979 y los PIDs del Modo 01.

2.3.6. Importancia del OBD-II en el monitoreo del consumo de combustible

El sistema OBD-II desempeña un papel importante en el monitoreo del consumo de combustible, ya que permite supervisar en tiempo real los parámetros que influyen directamente en la eficiencia de la combustión, como la mezcla aire-combustible, el tiempo de inyección, la señal de los sensores de oxígeno, la carga del motor y el estado del sistema de encendido (MotoresUno, 2023; Webfleet, 2000). A través de los PIDs y de los monitores internos de la ECU, el diagnóstico a bordo identifica desviaciones que pueden aumentar el consumo, como fallos de combustión, problemas en el sistema de combustible o una mala regulación de las emisiones (MotoresUno, 2023).

Cuando el sistema detecta una condición anómala, la ECU almacena códigos de falla y activa indicadores de advertencia, lo que facilita la detección temprana de problemas que afectan el rendimiento del motor y elevan el gasto de combustible (Rodi, 2025). De esta manera, el OBD-II no solo cumple una función de diagnóstico, sino que también contribuye a optimizar la operación del vehículo, reduciendo el consumo

innecesario de combustible y ayudando a mantener los valores de emisiones dentro de los límites establecidos (Webfleet, 2000; Autoavance, 2018).

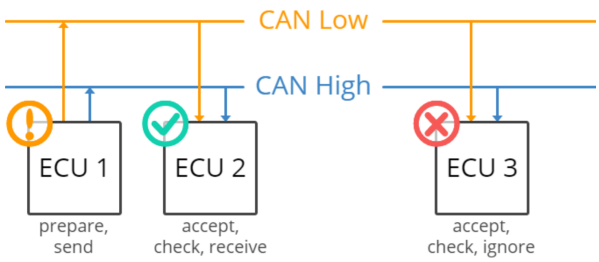
2.4. Red CAN

El Controller Area Network (CAN) es un protocolo de comunicación serial diseñado para permitir la transmisión eficiente y confiable de datos entre múltiples dispositivos (módulos de sistemas específicos) electrónicos dentro de un sistema distribuido. Fue desarrollado en la década de 1980 por la empresa Robert Bosch GmbH con el objetivo de reducir la complejidad del cableado en los sistemas electrónicos de los vehículos y mejorar la comunicación entre las diferentes unidades de control electrónico.

Este protocolo permite que diferentes nodos de una red intercambien información a través de un único canal de comunicación, denominado bus, sin necesidad de una computadora central que gestione el tráfico de datos. Gracias a su arquitectura distribuida, robustez frente a interferencias electromagnéticas y capacidad de detección de errores, el bus CAN se ha convertido en uno de los estándares más utilizados en la industria automotriz y en sistemas embebidos. En la Figura 2.10 se presenta la conexión de diferentes módulos mediante este protocolo con dos lineas de transmisión CAN-HIGH y CAN-LOW.

Figura 2.10

Conexión de múltiples módulos mediante el protocolo CAN



Nota. Topología en bus que ilustra la interconexión en paralelo de las unidades de control electrónico compartiendo las líneas diferenciales CAN High y CAN Low. Tomado de (Tope Parts, sf).

2.4.1. Arquitectura de la red CAN

La implantación de este modelo de comunicaciones supuso un avance en cuanto a la cantidad de conexiones entre dispositivos que eran necesarias para mantener todos los elementos comunicados, ya que el protocolo CAN emplea un único bus de comunicaciones, compartido por todos los dispositivos y evita la necesidad de establecer una conexión punto a punto con cada uno. Algunas de las características que permiten a este protocolo ser adecuado para aplicaciones automotrices e industriales son:

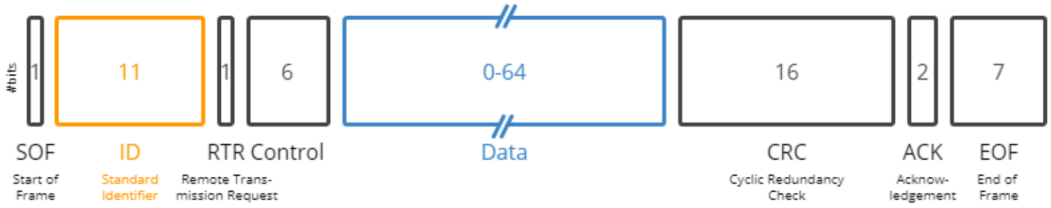
- Comunicación multi-maestro: En una red CAN no existe un nodo maestro que controle toda la comunicación. Todos los nodos conectados al bus tienen la capaci-

dad de transmitir información cuando el canal está disponible. Esto permite una mayor flexibilidad y confiabilidad en el sistema.

- Comunicación basada en mensajes: A diferencia de otros protocolos de red que utilizan direcciones específicas para cada dispositivo, el bus CAN utiliza un sistema basado en identificadores de mensajes (tramas). Cada mensaje transmitido en la red contiene un identificador que indica el tipo de información que se está enviando y su nivel de prioridad. En la Figura 2.11 se presenta la estructura de datos CAN, esta cuenta de las siguientes partes: a) Star of frame (inicio de trasmicion), b) Identifier: identificador, c) Control, d) Data (datos a transmitir), e) Cyclic Redundancy Check (detección de errores), f) Acknowledgement (confirmación) y e) End of frame (fin de trasmisión).

Figura 2.11

Estructura de la trama de datos del protocolo CAN



Nota. Distribución de los campos de bits dentro de una trama estándar CAN, incluyendo el identificador de arbitraje, campo de control, campo de datos y el código de verificación CRC. Tomado de (CarLightVision, 2025).

- Alta inmunidad al ruido: El bus CAN utiliza una señal diferencial transmitida a través de dos líneas trenzadas denominadas CAN High y CAN Low. Esto permite reducir los efectos de interferencias electromagnéticas, lo cual es fundamental en el entorno automotriz donde existen múltiples fuentes de ruido eléctrico.
- Detección y manejo de errores: incluye múltiples mecanismos de detección de errores que garantizan la integridad de los datos transmitidos como: i) Verificación de redundancia cíclica (CRC); ii) Confirmación de recepción (ACK-acknowledge) y iii) Monitoreo de bits. Cuando se detecta un error, el mensaje es retransmitido automáticamente

2.4.2. Arquitectura física del bus CAN

Una red CAN está compuesta por varios dispositivos denominados nodos (módulos), los cuales se conectan a un medio físico común denominado bus como se presenta en la figura 2.10. Cada nodo esta compuesto principalmente por tres nodos que son:

1. Microcontrolador: procesa la información del sistema y genera los datos que seran transmitidos.
2. Controlador CAN: implementa el protocolo de comunicación CAN y gestiona la

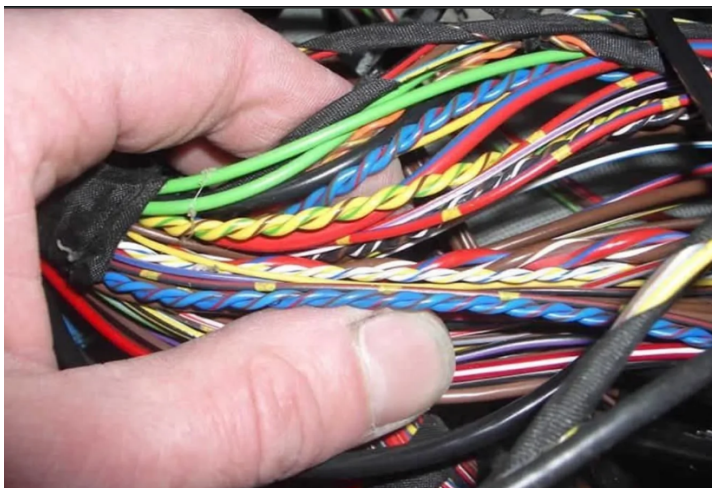
transmisión y recepción de mensajes puede estar incorporado en el microcontrolador o como un circuito externo.

3. Transceptor CAN: convierte las señales digitales del controlador CAN en señales diferenciales que pueden transmitirse a través del bus físico.

2.5. Arquitectura física del bus CAN

A nivel físico, el bus CAN está compuesto por una línea de comunicación diferencial formada por dos conductores. Estos dos cables transportan señales complementarias que permiten mejorar la inmunidad al ruido electromagnético. Los cables que transportan señales en protocolo CAN se identifican por ser pares trenzados como se presenta en la Figura 2.12.

Figura 2.12
Configuración física del cableado del protocolo CAN

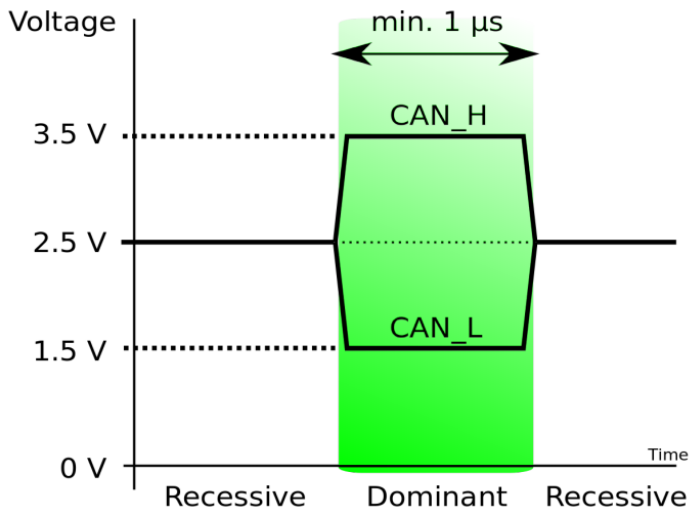


Nota. Disposición física de las líneas diferenciales de comunicación mediante par trenzado, diseñada para mitigar el ruido electromagnético en el entorno automotriz. Tomado de (CarLightVision, 2025).

El bus puede encontrarse en modo recesivo, con ambos cables con el mismo nivel de tensión o en modo dominante. En estado recesivo (inactivo), ambas líneas están cerca de 2,5V. En estado dominante (activo), CAN-H sube a 3,5V y CAN-L baja a 1,5V,Este modo de comunicación tiene como objetivo proporcionar una mayor protección frente a interferencias electromagnéticas. Esta protección viene dada, debido a que la lectura de los bits se basa en la diferencia de voltaje entre los dos cables trenzados, por lo que en caso de verse sometidas a la misma influencia electromagnética, a pesar de la variación señal en los cables, la diferencia de voltaje entre ellos se mantiene constante.

En la Figura 2.13 se presenta los niveles de voltaje de este protocolo. En el bus, los bits dominantes equivalen al nivel lógico "0", y los bits recesivos al valor lógico "1".

Figura 2.13
Niveles de voltaje diferencial en el bus CAN



Nota. Comportamiento eléctrico de las señales CAN High y CAN Low durante los estados lógicos dominante y recesivo para la transmisión física de datos. Tomado de (Jidoshaseibishi, sf).

2.5.1. Topología del bus CAN

Posee una topología en forma de bus, en la que únicamente son necesarios dos cables trenzados y todos los nodos se conectan a un mismo cable principal. En los extremos del bus se colocan resistencias de terminación, generalmente de 120 ohmios, con el objetivo de evitar reflexiones de señal y garantizar la estabilidad de la comunicación. (Lifeder, 2024; Industrial Monitor Direct, 2026)

La longitud máxima del bus depende de la velocidad de transmisión utilizada. Tabla 2.4 se presenta la velocidad, longitud y tiempo de transmisión mediante este protocolo de comunicación a diferentes distancias.

Tabla 2.4
Relación entre longitud del bus, velocidad y tiempo máximo de transmisión

Longitud del bus	Velocidad (bits/s)	Tiempo máximo de transmisión
Hasta 25 m	1 Mbit/s	129 μs
Hasta 100 m	500 Kbit/s	258 μs
Hasta 500 m	125 Kbit/s	1032 μs
Hasta 1000 m	50 Kbit/s	2580 μs

Nota. Elaboración propia.

2.5.2. Tramas del protocolo CAN

El protocolo CAN contempla diferentes formatos de tramas, cada una orientada a un propósito específico dentro del funcionamiento del protocolo en la figura 2.11 se presentan las partes mas importantes que son:

- Tramas de datos: utilizadas para el paso de información entre nodos, que puede ser enviada y recibida por uno o varios de ellos.
- Tramas de error: utilizadas cuando algún nodo de la red detecta un error en alguno de los mensajes transmitidos por otros nodos de la red conectada.
- Tramas de petición remota: utilizadas para solicitar el envío de información a otro nodo. Se envía una trama con el identificador (id) del nodo del que se requiere el envío de información, éste recibe la trama y devuelve la información con los datos solicitados.
- Tramas de sobrecarga: utilizadas para detectar errores de sobre carga. Es enviada por un nodo cuando éste está sobrecargado, provoca un retardo en la comunicación.

Cuando el bus esta en retardo no existe flujo de tramas y el bus se mantiene en estado recesivo. Cuando el bus requiere enviar mas de una trama estas deben estar separadas por una secuencia predeterminada (intertrama) que esta compuesta por tres bits recesivos.

Los datos enviados por este protocolo son de hasta 8 bytes. En la figigura 2.14 se presenta el formato de esta trama requiere 1 bit para el inicio de la secuencia se cuenta empleado para la sincronización con el resto de la secuencia. Posteriormente se encuentra el identificador de 11 bits. Conociendo los identificadores de todas las tramas que intentan ser transmitidas, se puede establecer de manera determinista el orden en el que son transmitidas. Se puede establecer un orden para priorizar algunas tramas sobre otras por ejemplo: una trama CAN con identificador más bajo (mayor número de bits dominantes en las primeras posiciones) tiene más prioridad que una trama con identificador más alto.

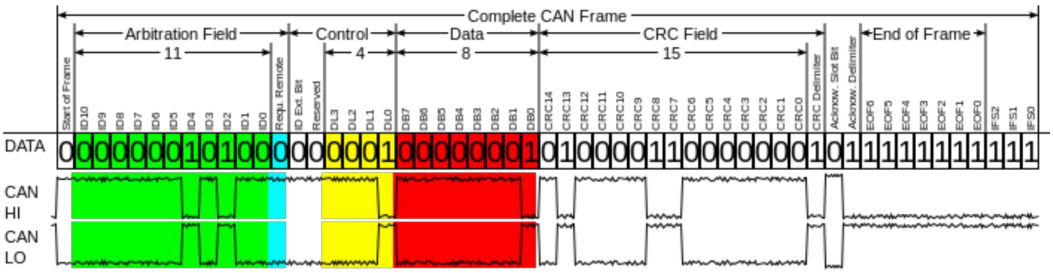
Para el campo de control los dos bits iniciales están reservados para un uso futuro, y posteriormente se encuentran cuatro bits, que definen el tamaño del campo de datos de la trama que se encuentra a continuación. La parte de la trama coloreada en rojo en la Figura 2.14 es la de datos que puede abarcar hasta 8 bytes. El campo CRC (Código de redundancia cíclica), se encarga de asegurar la integridad de los datos enviados esta cuenta con 15 bit y un bit adicional un bit recesivo como delimitador. La trama continua con 2 bits de confirmación o reconocimiento ACK (acknowledge) y finalmente se envían los bits de fin de trama EOF(end of frame) que contiene 7 bits. Adicionalmente la trama debe contener tres bits de espaciado al final de trama.

2.5.3. Errores en trama de datos

El protocolo CAN posee una norma de relleno de bits en las tramas, que consiste en que cada secuencia de cinco bits con el mismo valor, se introduce un bit de valor

Figura 2.14

Campos constitutivos del formato de trama de datos CAN

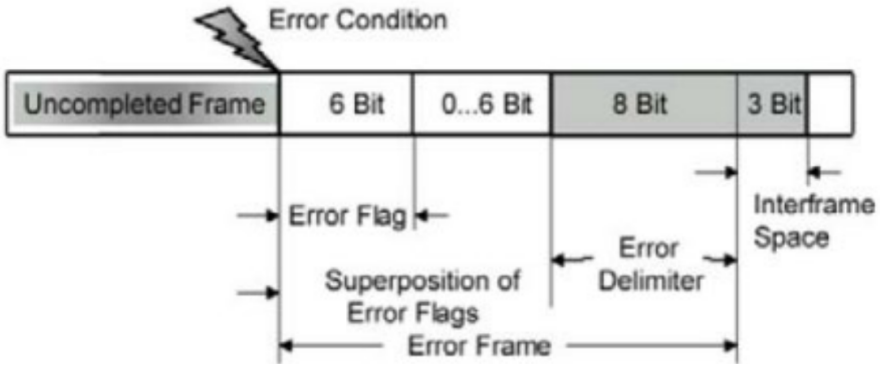


Nota. Segmentación detallada del formato de la trama estándar, especificando la longitud y función de los bits de inicio, arbitraje, control, datos, CRC, ACK y fin de trama (*EOF*). Tomado de (Jeong, sf).

inverso. Esto provoca que el formato de las tramas de error incumplan esta norma y puedan ser detectadas sobre las tramas de datos. Estas tramas de error pueden ser generadas por cualquier nodo que detecte un error en la comunicación. Se componen de dos campos. Se compone de dos campos primero se encuentra el indicador de error, o error flag, cuyo formato depende el tipo de nodo que haya detectado el error. En la Figura 2.15, se describen las partes de esta trama.

Figura 2.15

Estructura de una trama de error en el protocolo CAN



Nota. Configuración de los campos de la trama de error, detallando los bits de delimitación y la secuencia de banderas de error activas o pasivas utilizadas para la gestión de fallas en el bus. Tomado de (National Cheng Kung University, Department of Computer Science and Information Engineering, sf).

Para evitar que un nodo con errores condicione el funcionamiento de la red o pueda provocar fallos o saturaciones, el protocolo dispone de medidas para desconectar de la red este tipo de nodos. Los nodos pueden estar en tres tipos de estado diferente que son:

- Error activo: El nodo puede enviar mensajes y tramas de error normalmente. Es el estado normal.

- Error pasivo: El nodo cambia el formato de sus tramas de error, enviando 12 bits recesivos. De este modo, el resto de nodos no son capaces de detectarlas y no ralentiza las comunicaciones.
- Bus off o Anulado: El nodo se auto-desconecta del bus, se deshabilita su transceptor y no participa en la comunicación.

Cuando un nodo se desconecta de la red, se resetea, configura y se conecta de nuevo a la red, pero no podrá comenzar a transmitir de nuevo hasta no haber recibido 128 secuencias de 11 bits recesivos sin errores en el bus.

2.5.4. Red CAN en vehículos

La red CAN (*Controller Area Network*) es uno de los protocolos de comunicación más utilizados en vehículos modernos debido a su capacidad para permitir la comunicación entre múltiples unidades de control electrónico (ECU) mediante un bus compartido. A través de esta red, distintos módulos del vehículo intercambian información relacionada con el motor, transmisión, frenos, sistemas de seguridad y otros subsistemas electrónicos (FlexiHub, 2024; CISE Electronics, sf).

En los sistemas OBD-II modernos, especialmente aquellos basados en el protocolo ISO 15765-4, la comunicación de diagnóstico se realiza sobre la red CAN utilizando identificadores específicos para solicitudes y respuestas. Generalmente, las herramientas de diagnóstico envían solicitudes mediante el identificador funcional 0x7DF, mientras que las ECU responden utilizando identificadores comprendidos entre 0x7E8 y 0x7EF (FlexiHub, 2024). Mediante este mecanismo es posible consultar parámetros estandarizados conocidos como PIDs (*Parameter IDs*), los cuales permiten obtener datos como velocidad del vehículo, revoluciones del motor, temperatura del refrigerante o flujo de aire del motor.

Sin embargo, además de los PIDs normalizados definidos por el estándar OBD-II, los fabricantes implementan protocolos CAN propietarios para la comunicación interna entre las ECU del vehículo. Estos mensajes no se encuentran documentados públicamente y varían dependiendo de la marca, modelo y año del vehículo. Debido a ello, gran parte de la información transmitida en la red CAN no puede ser interpretada directamente sin realizar procesos de ingeniería inversa (*reverse engineering*) (FlexiHub, 2024; Verma et al., 2020).

La ingeniería inversa sobre redes CAN consiste en capturar y analizar las tramas transmitidas por las ECU con el objetivo de identificar qué mensajes corresponden a determinados parámetros del vehículo. Para ello, normalmente se registra el tráfico CAN mientras se modifica una variable específica del vehículo, por ejemplo acelerar, activar luces o mover el volante. Posteriormente se analizan los identificadores CAN y los bytes de datos que presentan variaciones relacionadas con dicha acción (Reddit - CarHacking, 2023; Reddit - Embedded, 2022).

Un aspecto importante en el análisis de tramas CAN es la interpretación de los datos binarios transmitidos por las ECU. Cada mensaje CAN contiene una secuencia de

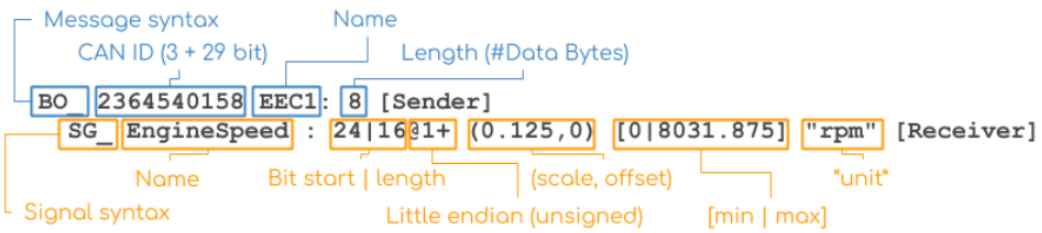
bytes que representan información específica del vehículo, la cual debe convertirse desde formato hexadecimal o binario a valores numéricos comprensibles conocidos como DBC (FlexiHub, 2024).

2.5.5. Archivo DBC CAN

Un archivo CAN DBC (CAN Database) es un archivo de texto que contiene la información necesaria para decodificar los datos sin procesar transmitidos por la red CAN y convertirlos en valores físicos interpretables. Este tipo de archivo define la estructura de los mensajes y de las señales asociadas, permitiendo establecer cómo debe leerse cada dato dentro de una trama CAN.

La Figura 2.16 presenta la estructura básica de un mensaje y de una señal dentro de un archivo DBC. Este tipo de archivo se utiliza para describir la forma en que los datos transmitidos a través de la red CAN deben ser interpretados, permitiendo asociar los valores binarios con variables físicas de interés.

Figura 2.16
Sintaxis básica de un mensaje y una señal en un archivo DBC



Nota. Estructura de codificación en bases de datos CAN (*.dbc*), donde se definen los identificadores de los mensajes (BO_), los nombres de las señales (SG_), el orden de los bits (*endianness*), el factor de escala y el offset de conversión. Elaboración propia.

En la parte superior de la figura se muestra la sintaxis correspondiente a un mensaje, identificada con el prefijo `BO_`. En esta definición se incluye el identificador CAN, el nombre del mensaje, la longitud expresada en número de bytes de datos y el nodo transmisor. Estos elementos permiten describir de manera ordenada cada trama que circula por el bus CAN.

En la parte inferior se presenta la sintaxis de una señal, indicada mediante el prefijo `SG_`. En este caso, se especifica el nombre de la señal, la posición inicial del bit, la longitud de la señal, el orden de bytes, el tipo de dato, el factor de escala, el desplazamiento, el rango físico, la unidad de medida y el nodo receptor. La combinación de estos parámetros define cómo debe interpretarse cada señal contenida dentro de un mensaje.

Por ejemplo, en la señal mostrada en la figura, `EngineSpeed` corresponde al nombre de la señal; `24|16` indica que la señal inicia en el bit 24 y tiene una longitud

de 16 bits; `@1+` señala una codificación little-endian y un dato sin signo; `(0.125,0)` representa el factor de escala y el desplazamiento; `[0|8031.875]` define el rango físico de la señal, y `rpm` establece la unidad de medida.

En este sentido, el archivo DBC funciona como una guía de decodificación que permite transformar los datos crudos del bus CAN en magnitudes físicas comprensibles, tales como velocidad, régimen del motor, temperatura o presión. Por ello, constituye una herramienta fundamental en aplicaciones de diagnóstico automotriz, monitoreo de variables vehiculares, telemetría e ingeniería inversa de redes CAN.

Un ejemplo simplificado de sintaxis DBC para la velocidad del vehículo es el siguiente:

```
BO_ 2024 OBD2: 8 Vector__XXX
SG_ VehicleSpeed : 0|8@1+ (1,0) [0|255] "km/h" Vector__XXX
```

En este ejemplo, `VehicleSpeed` es el nombre de la señal, `0|8` indica que la señal comienza en el bit 0 y tiene una longitud de 8 bits, `@1+` señala que usa little-endian y que es no firmada, `(1,0)` corresponde al factor de escala y al desplazamiento, `[0|255]` define el rango físico permitido, `"km/h"` representa la unidad y `Vector__XXX` actúa como nodo genérico.

Por ejemplo, una trama CAN correspondiente a una respuesta OBD-II puede representarse de la siguiente manera:

```
7E8 03 41 0D 37 FF FF FF FF
```

En esta trama, `7E8` corresponde al identificador de respuesta de la ECU, `41` indica una respuesta positiva al modo de diagnóstico 01, `0D` representa el PID de velocidad del vehículo y `37` contiene el valor de la señal. Al convertir `37` de hexadecimal a decimal se obtiene 55, por lo que la velocidad interpretada es de `55 km/h`.

De esta manera, los archivos DBC permiten transformar datos CAN sin procesar en variables físicas útiles para aplicaciones de monitoreo vehicular, diagnóstico automotriz, telemetría e ingeniería inversa de redes CAN.

2.6. Sistema de inyección electrónica

2.6.1. Evolución de los sistemas de alimentación de combustible

La evolución de los sistemas de alimentación de combustible en los vehículos ha pasado desde el carburador mecánico hasta sistemas electrónicos de inyección monopunto, multipunto e inyección directa, cada uno asociado a cambios normativos sobre emisiones y avances en electrónica del motor(Revista Autocrash, 2016; Tecnología del Automóvil, 2025; Wikipedia contributors, 2004).

- **Finales del siglo XIX – 1900**: Los primeros motores de gasolina utilizan mezcladores mecánicos rudimentarios; el carburador se consolida como dispositivo

estándar para crear la mezcla aire–combustible(Tecnología del Automóvil, 2025; Wikipedia contributors, 2004).

- **1912 – 1930s**: Primeros ensayos de bombas de inyección de gasolina en motores de aviación; en 1937 se aplica en serie la inyección de combustible en motores de avión, sentando las bases para los sistemas de inyección modernos(Wikipedia contributors, 2004).
- **1945 – 1950s**: Primera aplicación en serie de inyección de gasolina en vehículos de carretera; se mantienen sistemas principalmente mecánicos o electromecánicos, usados en modelos deportivos o de alta gama(Wikipedia contributors, 2004).
- **1967**: Bosch introduce el sistema D-Jetronic, el primer sistema de inyección electrónica de gasolina de producción masiva, sustituyendo parcialmente al carburador en Europa(Revista Autocrash, 2016).
- **1970s**: La crisis del petróleo y las normas de emisiones obligan a abandonar parcialmente el carburador; se extienden sistemas de inyección electrónica continuos (por ejemplo KE-Jetronic) y se empiezan a usar en modelos de serie de fabricantes como Mercedes-Benz y otros(Revista Autocrash, 2016; Wikipedia contributors, 2004).
- **1980s**: Aparecen sistemas digitales de gestión del motor (por ejemplo Motronic de Bosch), integrando inyección electrónica multipunto y encendido electrónico; marcas como Volkswagen, BMW y Mercedes adoptan estos sistemas en múltiples modelos(Escuela Politécnica Nacional, 2025).
- **1990**: Se difunden sistemas de inyección electrónica monopunto y multipunto indirecta en vehículos comerciales de bajo costo (por ejemplo Suzuki Samurai 1990 y Suzuki Jimny 2000)(Tecnología del Automóvil, 2025).
- **1990s – 2000s**: Implantación masiva del carburador en motores de gasolina reemplazado por inyección electrónica multipunto (por ejemplo General Motors, Ford, Honda, etc.); se adopta el catalizador de tres vías, lo que exige un control muy preciso de la mezcla aire–combustible(Revista Autocrash, 2016; Escuela Politécnica Nacional, 2025; Tecnología del Automóvil, 2025).
- **2000s – 2010s**: Desarrollo de sistemas de inyección directa de gasolina (GDI) en modelos de alta eficiencia y potencia (por ejemplo Toyota, Mazda, BMW, VW, GM); se combinan turbocompresores y control electrónico avanzado para reducir consumo y emisiones(Revista Autocrash, 2016; Escuela Politécnica Nacional, 2025).
- **2010 – actualidad**: La inyección directa se vuelve estándar en muchos motores de gasolina y se introducen sistemas flexibles de inyección múltiple por ciclo, combinados con sistemas híbridos y electrificación parcial de la alimentación (bomba eléctrica de alta presión, inyección piezoeléctrica en diesel)(Revista Autocrash, 2016; Escuela Politécnica Nacional, 2025).

Figura 2.17

Línea de tiempo de la evolución en los sistemas de alimentación de combustible

Nota. Cronología del desarrollo tecnológico en la dosificación de combustible, destacando la transición desde sistemas mecánicos por carburador hasta esquemas de inyección electrónica directa controlados por computadora. Elaboración propia con base en (Revista Autocrash, 2016), (Tecnología del Automóvil, 2025) y (Escuela Politécnica Nacional, 2025).

2.6.2. Tipos de sistemas de inyección electrónica

Los sistemas de inyección electrónica pueden clasificarse según la forma en que suministran el combustible hacia el motor. Entre los más utilizados se encuentran la inyección monopunto, multipunto y directa, cada una con diferentes características de funcionamiento, eficiencia y complejidad tecnológica (Robert Bosch GmbH, 2018b; Mecánica Fácil, 2024).

Inyección monopunto

La inyección monopunto, también conocida como *Single Point Injection* (SPI) o *Throttle Body Injection* (TBI), utiliza un único inyector ubicado en el cuerpo de aceleración para suministrar combustible a todos los cilindros del motor. Su funcionamiento es similar al carburador, pero con control electrónico de la dosificación de combustible (Robert Bosch GmbH, 2018b).

Este sistema fue ampliamente utilizado durante la transición entre carburadores y sistemas de inyección electrónica modernos debido a su bajo costo y simplicidad. Sin embargo, presenta menor precisión en la distribución del combustible y menor eficiencia energética en comparación con sistemas más avanzados (Autosoporte Capacitación, 2024).

Entre sus principales ventajas se encuentran:

- Menor costo de implementación.
- Diseño simple y mantenimiento reducido.
- Mejor control de emisiones respecto al carburador.

No obstante, sus desventajas incluyen:

- Distribución desigual del combustible.
- Menor rendimiento y eficiencia.
- Mayor consumo comparado con sistemas modernos.

Inyección multipunto

La inyección multipunto o *Multi Point Injection* (MPI) emplea un inyector independiente para cada cilindro del motor. Los inyectores se encuentran ubicados en el múltiple de admisión, permitiendo una dosificación más precisa de combustible hacia cada cilindro (Robert Bosch GmbH, 2018b; Mecánica Fácil, 2024).

Este sistema mejora significativamente el rendimiento del motor, el consumo de combustible y las emisiones contaminantes. Dependiendo de la estrategia de control de la ECU, la inyección puede realizarse de forma simultánea, secuencial o semisecuencial (Autosoporte Capacitación, 2024).

Entre sus ventajas destacan:

- Mejor atomización y distribución del combustible.
- Reducción del consumo y emisiones.
- Mayor eficiencia y respuesta del motor.

Sin embargo:

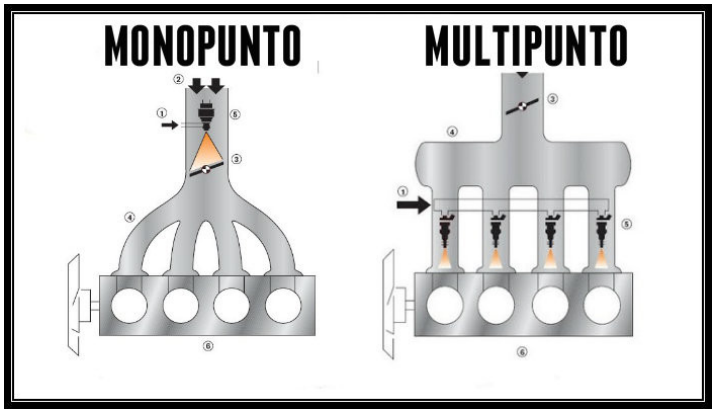
- Requiere mayor cantidad de sensores y componentes electrónicos.
- Presenta mayor complejidad de diagnóstico y mantenimiento.

Diferencia entre monopunto y multipunto

De acuerdo con Motorpasión, la inyección directa puede considerarse una evolución de la inyección multipunto, debido a que ambos sistemas utilizan un inyector por cilindro, pero difieren principalmente en la ubicación donde se introduce el combustible. En la inyección multipunto, el combustible es suministrado en el colector de admisión antes de ingresar a la cámara de combustión ver Figura 2.18, mientras que en la inyección directa el combustible es pulverizado directamente dentro del cilindro, permitiendo una combustión más eficiente y un mejor aprovechamiento energético (Buendía, 2017).

Figura 2.18

Diferencias estructurales entre los sistemas de inyección monopunto y multipunto



Nota. Comparación esquemática del posicionamiento y cantidad de inyectores en la galería de admisión, contrastando el suministro centralizado (monopunto) frente al flujo independiente por cilindro (multipunto). Tomado de (Buendía, 2017).

Inyección directa

La inyección directa de gasolina o *Gasoline Direct Injection* (GDI) introduce el combustible directamente dentro de la cámara de combustión a alta presión, eliminando el paso previo por el múltiple de admisión (Robert Bosch GmbH, 2014b; Motorpasión, 2024).

Este sistema permite un control mucho más preciso de la mezcla aire-combustible, incrementando la eficiencia térmica del motor y reduciendo el consumo de combustible. Además, puede trabajar junto con turbocompresores y estrategias avanzadas de control electrónico (Robert Bosch GmbH, 2014b).

Entre sus principales ventajas se encuentran:

- Mayor eficiencia energética.
- Mejor desempeño y potencia.
- Reducción de emisiones contaminantes.

Como desventajas:

- Mayor complejidad electrónica y mecánica.
- Costos elevados de mantenimiento.
- Sensibilidad a la calidad del combustible.

A continuacion en la Tabla 2.5 se compara los distintos tipos de sistemas de inyección electrónica.

Actualmente, la inyección directa es ampliamente utilizada en vehículos modernos debido a sus ventajas en eficiencia, desempeño y cumplimiento de normativas ambientales

Tabla 2.5
Comparación entre los tipos de sistemas de inyección electrónica

Característica	Inyección monopunto	Inyección multipunto	Inyección directa
Cantidad de inyectores	Un solo inyector para todos los cilindros	Un inyector por cilindro	Un inyector por cilindro
Ubicación de inyección	Cuerpo de aceleración	Colector de admisión	Dentro de la cámara de combustión
Formación de mezcla	Antes del múltiple de admisión	Antes de ingresar al cilindro	Dentro del cilindro
Precisión de dosificación	Baja	Media-Alta	Alta
Consumo de combustible	Mayor	Moderado	Menor
Eficiencia energética	Baja	Buena	Muy alta
Complejidad electrónica	Baja	Media	Alta

Nota. Elaboración propia con base en (Buendía, 2017), (Robert Bosch GmbH, 2018b) y (Robert Bosch GmbH, 2014b).

(Robert Bosch GmbH, 2014b; Motorpasión, 2024).

2.6.3. Componentes principales del sistema de inyección

El sistema de inyección electrónica está conformado por diversos componentes encargados de suministrar, regular y controlar la cantidad de combustible necesaria para el correcto funcionamiento del motor. Estos elementos trabajan de manera conjunta bajo la supervisión de la unidad de control electrónico (ECU), permitiendo optimizar el rendimiento, el consumo de combustible y las emisiones contaminantes (Robert Bosch GmbH, 2018b, 2014b).

Inyectores

Los inyectores son dispositivos electromecánicos encargados de pulverizar el combustible hacia el múltiple de admisión o directamente dentro de la cámara de combustión, dependiendo del tipo de sistema de inyección utilizado. Su funcionamiento es controlado electrónicamente por la ECU mediante pulsos eléctricos que determinan el tiempo de apertura y la cantidad de combustible inyectada (Robert Bosch GmbH, 2014b).

La correcta atomización del combustible es fundamental para lograr una combustión eficiente, reducir emisiones contaminantes y mejorar el desempeño del motor. En sistemas modernos de inyección directa, los inyectores trabajan a altas presiones y permiten múltiples inyecciones por ciclo de combustión.

Entre las principales funciones de los inyectores se encuentran:

- Dosificar la cantidad de combustible requerida.
- Pulverizar el combustible en partículas finas.
- Mejorar la mezcla aire-combustible.
- Reducir consumo y emisiones.

Bomba de combustible

La bomba de combustible tiene la función de suministrar el combustible desde el tanque hacia los inyectores manteniendo una presión adecuada dentro del sistema. En vehículos modernos generalmente se utiliza una bomba eléctrica ubicada dentro del tanque de combustible (Autodoc, 2024).

La presión generada por la bomba depende del tipo de sistema de inyección. Los sistemas multipunto trabajan con presiones moderadas, mientras que los sistemas de inyección directa requieren bombas de alta presión capaces de alimentar los inyectores directamente en la cámara de combustión.

Entre sus funciones principales destacan:

- Transportar combustible desde el tanque.
- Mantener presión constante en el sistema.
- Garantizar el suministro adecuado al motor.

Regulador de presión

El regulador de presión es el componente encargado de mantener estable la presión del combustible dentro del riel de inyectores ver Figura 2.19, evitando variaciones que puedan afectar la mezcla aire-combustible (Robert Bosch GmbH, 2018b).

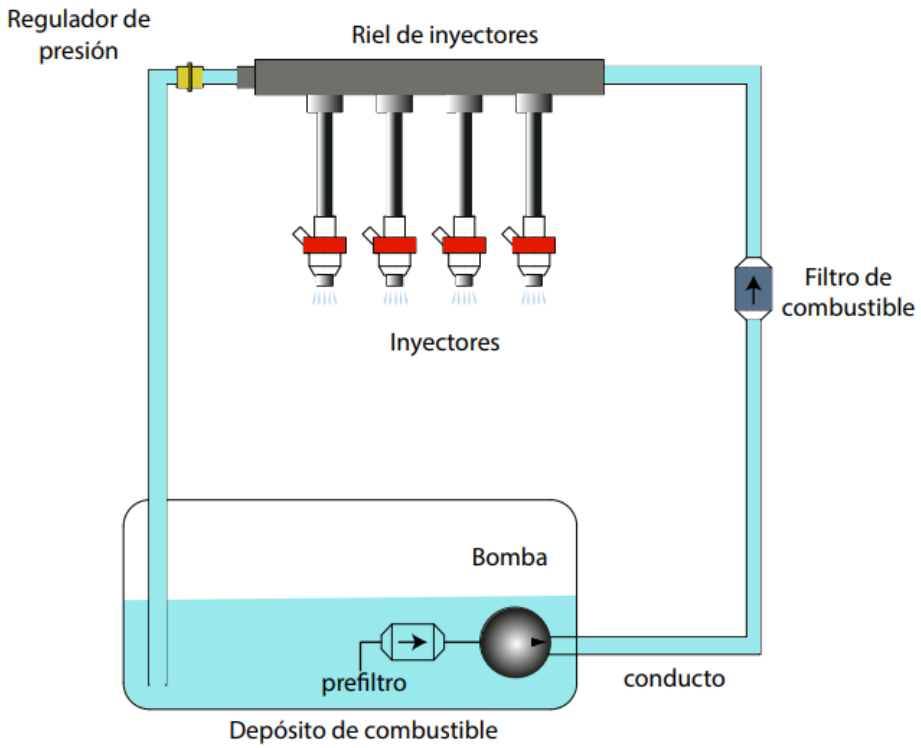
Cuando la presión excede el valor establecido, el regulador permite el retorno del exceso de combustible hacia el tanque. En sistemas modernos, esta regulación puede realizarse electrónicamente mediante sensores y módulos de control integrados.

Sus principales funciones son:

- Estabilizar la presión de combustible.
- Proteger inyectores y conductos.
- Garantizar una dosificación precisa.

Figura 2.19

Componentes principales y distribución del sistema de inyección electrónica



Nota. Disposición física y conectividad de los elementos constitutivos del sistema, incluyendo unidades de control, sensores de monitoreo de parámetros y actuadores para la dosificación de combustible. Elaboración propia con base en (Mamani Ticona et al., 2024).

Sensores

Los sensores son dispositivos encargados de medir diferentes variables de funcionamiento del motor y enviar esta información a la ECU para que pueda calcular la cantidad exacta de combustible necesaria en cada condición de operación (Robert Bosch GmbH, 2018b; HELLA Tech World, 2024b).

Entre los sensores más importantes del sistema de inyección electrónica se encuentran:

- Sensor MAF (*Mass Air Flow*): mide el flujo de aire que ingresa al motor.
- Sensor MAP (*Manifold Absolute Pressure*): mide la presión del múltiple de admisión.
- Sensor TPS (*Throttle Position Sensor*): detecta la posición de la mariposa de aceleración.
- Sensor ECT (*Engine Coolant Temperature*): mide la temperatura del refrigerante.

- Sensor de oxígeno o lambda: controla la cantidad de oxígeno presente en los gases de escape.

La información proporcionada por estos sensores permite que la ECU ajuste continuamente la mezcla aire-combustible, optimizando el rendimiento del motor y reduciendo emisiones contaminantes.

2.6.4. Control de la mezcla aire-combustible

El control de la mezcla aire-combustible constituye uno de los procesos fundamentales en los sistemas modernos de inyección electrónica, debido a su influencia directa sobre la eficiencia térmica del motor, el consumo de combustible y la formación de emisiones contaminantes durante la combustión (Mamani Ticona et al., 2024; Robert Bosch GmbH, 2014b).

La mezcla aire-combustible se define como la proporción entre la masa de aire y la masa de combustible que ingresa al cilindro para llevar a cabo el proceso de combustión. En motores a gasolina, la relación estequiométrica ideal se aproxima a 14,7:1, es decir, 14,7 partes de aire por cada parte de combustible (Robert Bosch GmbH, 2018b). Esta condición permite una combustión más eficiente y favorece el funcionamiento adecuado del convertidor catalítico.

Para evaluar la calidad de la mezcla, se utiliza el factor lambda (λ), definido como:

$$\lambda = \frac{(A/F)_{\text{real}}}{(A/F)_{\text{estequiometrico}}} \tag{2.1}$$

Donde $(A/F)_{\text{real}}$ representa la relación aire-combustible presente en el motor y $(A/F)_{\text{estequiometrico}}$ corresponde a la relación ideal para la combustión completa.

Las variaciones del factor lambda afectan de manera significativa tanto las emisiones contaminantes como el desempeño del motor. Las mezclas ricas incrementan la emisión de monóxido de carbono (CO) e hidrocarburos no quemados (HC), mientras que las mezclas pobres pueden favorecer la formación de óxidos de nitrógeno (NO_x) y provocar una disminución de la potencia entregada por el motor (Wang et al., 2020).

El control de esta mezcla es realizado por la unidad de control electrónico (ECU), la cual recibe información de distintos sensores del motor para ajustar de forma continua el tiempo de inyección y mantener condiciones de combustión adecuadas. Entre los sensores más relevantes se encuentran el sensor MAF, el sensor MAP, el sensor TPS, el sensor ECT y, de manera especial, el sensor de oxígeno o sensor lambda, encargado de medir la concentración de oxígeno presente en los gases de escape (Robert Bosch GmbH, 2018b; Mamani Ticona et al., 2024).

A partir de la señal proporcionada por el sensor lambda, la ECU puede implementar un control en lazo cerrado (*closed loop*), mediante el cual corrige de forma dinámica la cantidad de combustible inyectado para mantener el valor de λ cercano a

1. Este mecanismo mejora la eficiencia del motor, reduce el consumo de combustible y contribuye a la disminución de las emisiones contaminantes (Robert Bosch GmbH, 2014b).

Finalmente, diversos estudios han demostrado que un control preciso de la mezcla aire-combustible tiene un impacto significativo en la reducción de emisiones vehiculares y en la optimización del desempeño energético de los motores de combustión interna. En este sentido, el desarrollo de estrategias avanzadas de control electrónico ha permitido mejorar el funcionamiento del motor bajo diferentes condiciones de carga, velocidad y operación (Wang et al., 2020).

2.6.5. Relación del sistema de inyección con el consumo de combustible

El sistema de inyección electrónica tiene una relación directa con el consumo de combustible, debido a que la cantidad de combustible suministrada al motor depende del tiempo de apertura del inyector. En este contexto, la duración del pulso de inyección constituye una variable fundamental para estimar el consumo instantáneo, ya que representa el tiempo durante el cual el inyector permanece abierto y permite el paso del combustible hacia la cámara de combustión (art, 2021).

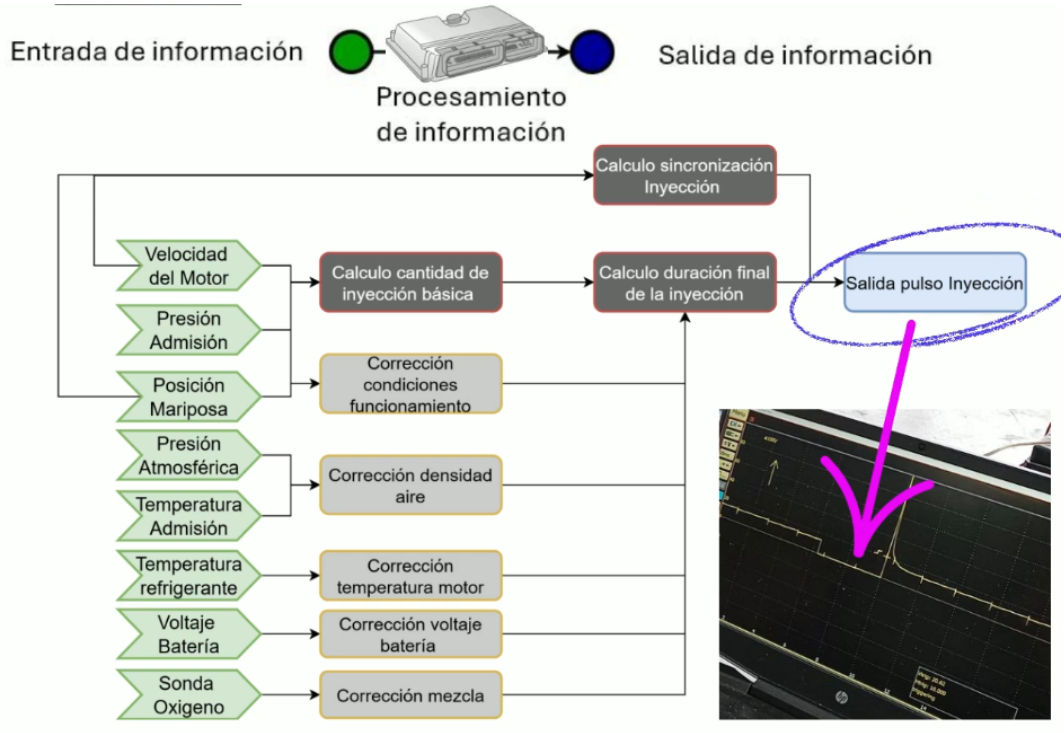
De acuerdo con el artículo de referencia, el consumo instantáneo puede determinarse a partir de la medición de la duración del pulso de inyección, estableciéndose una correlación muy alta entre ambas variables, con un coeficiente de 0,99 art (2021). Esto indica que, al incrementarse el ancho del pulso, aumenta también la cantidad de combustible inyectado; por el contrario, una reducción del tiempo de apertura disminuye el caudal de combustible y, en consecuencia, el consumo del motor (art, 2021).

La Figura 2.20 ilustra el proceso de adquisición y procesamiento de señales dentro del sistema de gestión del motor, donde las variables de entrada son procesadas por la unidad de control hasta generar la salida hacia el inyector. En particular, la señal de salida del pulso de inyección es el parámetro que permite regular la mezcla aire-combustible y, por tanto, influye de manera directa en el gasto de combustible durante la operación del motor (art, 2021).

Por ello, el análisis del pulso de inyección resulta útil para estudiar el comportamiento real del motor bajo diferentes condiciones de operación, así como para evaluar estrategias orientadas a mejorar la eficiencia energética y reducir el consumo específico de combustible (art, 2021).

Figura 2.20

Relación entre el sistema de inyección y el consumo de combustible



Nota. Diagrama de bloques que ilustra cómo los parámetros medidos por la ECU (tiempo de inyección y flujo de masa de aire) impactan en el cálculo matemático del consumo instantáneo de combustible. Tomado de (YouTube, 2024).

2.7. Modelos matemáticos para el consumo del combustible

2.7.1. Concepto de consumo de combustible

El consumo de combustible se define como la cantidad de combustible que utiliza un motor o vehículo para desarrollar un recorrido o mantener su operación durante un periodo determinado. Este parámetro constituye una de las variables más importantes en el análisis del desempeño automotriz, debido a que refleja de manera directa la eficiencia energética del sistema de propulsión y su impacto económico y ambiental (Pos, 2016; Est, 2019).

Desde el punto de vista técnico, el consumo puede expresarse en términos de volumen o masa de combustible por unidad de distancia recorrida, como litros por cada 100 km (L/100 km), o por unidad de tiempo, como litros por hora (L/h), dependiendo del tipo de evaluación que se requiera (Pos, 2016). En motores de combustión interna, este indicador no solo depende de la demanda de potencia, sino también de las condiciones de operación del vehículo, tales como la velocidad, la carga, la pendiente del terreno, la resistencia aerodinámica y el estado del sistema de inyección (Con, 2002; Pos, 2016).

En términos generales, un menor consumo de combustible se asocia con una mejor eficiencia del motor, siempre que se mantengan condiciones adecuadas de potencia y funcionamiento. Sin embargo, este comportamiento puede variar en función de la estrategia de control del motor, la calidad de la combustión y las características del trayecto o ciclo de conducción (Est, 2019; Con, 2002).

Por esta razón, el estudio del consumo de combustible constituye una base fundamental para el desarrollo de modelos matemáticos que permitan estimar, comparar y optimizar el desempeño de los motores en diferentes condiciones de operación (Pos, 2016).

2.7.2. Métodos de estimación del consumo

La estimación del consumo de combustible ha sido abordada mediante distintos enfoques matemáticos y experimentales, cuyo objetivo principal es determinar con precisión la cantidad de combustible utilizada por un vehículo durante su operación. En términos generales, estos métodos pueden clasificarse en modelos basados en variables de conducción, modelos fundamentados en parámetros del motor y modelos estadísticos o híbridos que combinan distintas señales del sistema vehicular (Est, 2004, 2019).

Los modelos basados en variables de conducción utilizan información como la velocidad, la distancia recorrida, la aceleración y las condiciones de operación del vehículo para aproximar el consumo. Este tipo de enfoque resulta útil cuando no se dispone de acceso directo a variables internas del motor, aunque su precisión puede verse limitada por las variaciones propias del ciclo de conducción y por la topografía de la ruta (Con, 2002; Est, 2004).

Por otro lado, los modelos basados en parámetros del motor emplean señales obtenidas desde la unidad de control electrónico o mediante el puerto OBD-II, tales como el flujo másico de aire (MAF), la posición del acelerador (TPS), las revoluciones por minuto (RPM) y la relación aire-combustible. Estos modelos presentan una mayor capacidad de representación del comportamiento real del motor, ya que incorporan variables directamente relacionadas con la formación de la mezcla y la entrega de combustible (MAF, 2024; Res, 2018).

Adicionalmente, existen métodos estadísticos y de aprendizaje automático que permiten construir relaciones empíricas entre variables de operación y consumo de combustible. Aunque estos enfoques pueden ofrecer buenos niveles de predicción, requieren bases de datos representativas y procesos de calibración adecuados para asegurar su validez en diferentes condiciones de uso (Est, 2018; Res, 2018).

En este contexto en la Tabla 2.6 se muestra, la selección del método de estimación depende de la disponibilidad de datos, el nivel de precisión requerido y la complejidad aceptable del modelo . Para aplicaciones en tiempo real, los modelos basados en sensores como el MAF representan una alternativa adecuada, debido a que permiten relacionar directamente la masa de aire admitida por el motor con la cantidad de combustible necesaria para mantener una combustión eficiente (MAF, 2024; Robert Bosch GmbH,

2014b).

Tabla 2.6
Comparación de métodos para la estimación del consumo de combustible

Método	Principio	Ventajas	Limitaciones
Estadístico	Relaciona consumo con variables observables como velocidad o distancia.	Simple y económico.	Menor precisión en condiciones variables.
Mecanicista	Usa fuerzas resistivas y demanda de potencia del vehículo.	Más físico y explicativo.	Requiere mayor cantidad de parámetros y calibración.
Basado en sensores	Utiliza señales directas de la ECU como MAF, MAP, TPS o sonda lambda.	Ideal para la estimación y monitoreo en tiempo real.	Depende de la tasa de refresco y calidad de los sensores.

Nota. Elaboración propia.

2.7.3. Relación aire-combustible (AFR)

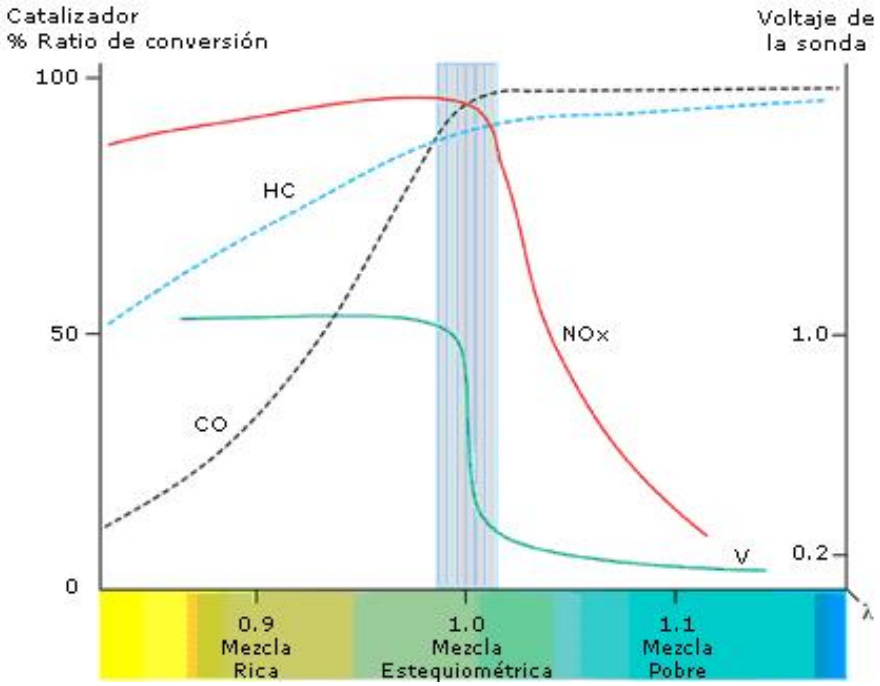
La relación aire-combustible, conocida como *Air-Fuel Ratio* (AFR), expresa la proporción existente entre la masa de aire y la masa de combustible que ingresan al cilindro durante el proceso de combustión. Este parámetro es fundamental en el análisis del funcionamiento de los motores de combustión interna, ya que determina la calidad de la mezcla y condiciona directamente la eficiencia térmica, el consumo de combustible y la generación de emisiones contaminantes (AFR, 2024, sf).

En motores a gasolina, la relación estequiométrica de referencia es aproximadamente 14,7:1, lo que significa que se requieren 14,7 partes de aire por cada parte de combustible para lograr una combustión ideal (AFR, 2024; Robert Bosch GmbH, 2018b). Bajo esta condición, el combustible puede oxidarse de manera más completa y se favorece el funcionamiento adecuado de los sistemas de postratamiento, como el convertidor catalítico.

Cuando la relación aire-combustible es inferior a la estequiométrica, la mezcla se considera rica, debido al exceso de combustible presente en la cámara de combustión. En este caso, aumenta la probabilidad de generar monóxido de carbono (CO) e hidrocarburos no quemados (HC), debido a una combustión incompleta (AFR, sf). Por el contrario, cuando la relación aire-combustible es superior a la estequiométrica, la mezcla es pobre, lo que implica un exceso de aire. Esta condición puede disminuir el consumo de combustible, pero también puede afectar la potencia del motor y favorecer la formación de óxidos de nitrógeno (NO_x) (AFR, 2024; Wang et al., 2020).

Figura 2.21

Curva estequiométrica del factor lambda y respuesta del sensor de oxígeno



Nota. Comportamiento de la señal de voltaje generada por el sensor de oxígeno en función del factor lambda, detallando las regiones de transición entre mezcla rica y mezcla pobre. Adaptado de (Mamani Ticona et al., 2024).

La AFR está estrechamente relacionada con el factor lambda (λ), el cual permite expresar qué tan lejos se encuentra la mezcla real respecto de la condición estequiométrica ver Tabla 2.7 En términos generales, el control de la AFR constituye una estrategia esencial para mantener una combustión eficiente y adaptar la entrega de combustible a las condiciones de carga, velocidad y demanda del motor (Robert Bosch GmbH, 2014b; Mamani Ticona et al., 2024).

Para el cálculo final del consumo, es fundamental definir la relación aire-combustible (AFR), la cual se expresa como la proporción entre la masa de aire y la masa de combustible:

$$AFR = \frac{m_{aire}}{m_{combustible}} \tag{2.2}$$

En los sistemas de gestión modernos, esta relación se normaliza mediante el factor λ (lambda), que relaciona el AFR real medido por la sonda de oxígeno con el valor estequiométrico ideal:

$$\lambda = \frac{AFR_{real}}{AFR_{estequiometrico}} \tag{2.3}$$

Si $\lambda = 1$, la mezcla es estequiométrica; si $\lambda < 1$, la mezcla es rica (exceso de combustible), y si $\lambda > 1$, la mezcla es pobre (exceso de aire) ver Figura 2.21.

Por esta razón, el análisis de la relación aire-combustible es indispensable dentro de los modelos matemáticos de estimación del consumo de combustible, ya que permite vincular la cantidad de aire admitida con la masa de combustible requerida para sostener una operación eficiente del motor.

Tabla 2.7
Interpretación de la relación aire-combustible y el factor lambda

Condición	Relación AFR	Factor λ	Efecto principal y aplicación
Mezcla rica	AFR < 14,7:1	$\lambda < 1$	Mayor consumo de combustible y aumento de emisiones de CO y HC. Típica en aceleraciones a plena carga.
Mezcla estequiométrica	AFR = 14,7:1	$\lambda = 1$	Combustión completa y eficiente, operación óptima del catalizador. Condición ideal de crucero.
Mezcla pobre	AFR > 14,7:1	$\lambda > 1$	Menor consumo de combustible, incremento de temperatura en la cámara y emisiones de NOx.

Nota. Comportamiento físico-químico de la dosificación de la mezcla aire-combustible en motores de gasolina y su impacto en la eficiencia y emisiones. Elaboración propia.

2.7.4. Modelo basado en el sensor MAF

El sensor MAF (*Mass Air Flow*) es uno de los sensores más importantes dentro de los sistemas modernos de inyección electrónica, debido a que permite medir la cantidad de aire que ingresa al motor. Esta información es utilizada por la unidad de control electrónico (ECU) para calcular la cantidad adecuada de combustible necesaria para mantener una combustión eficiente (Robert Bosch GmbH, 2018b; Mamani Ticona et al., 2024).

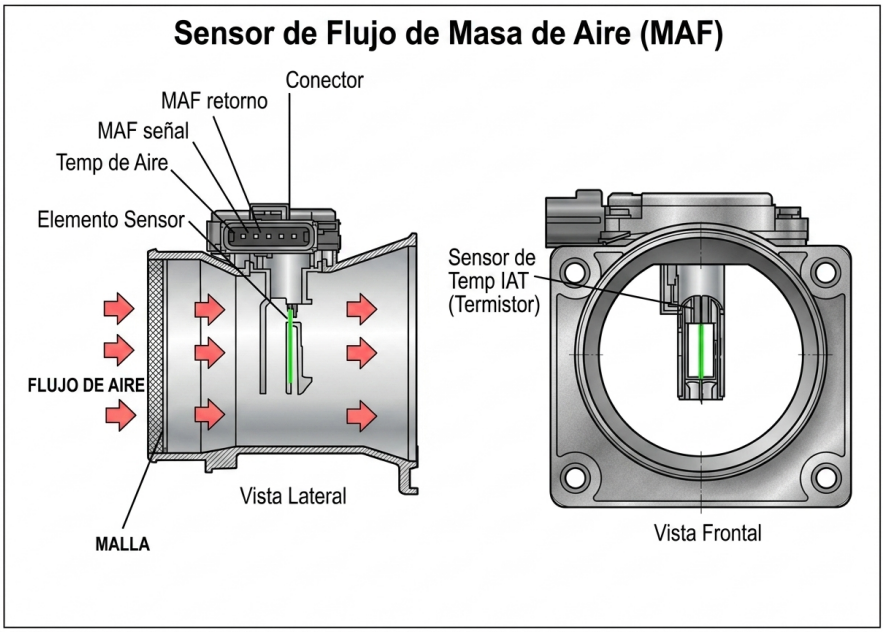
El funcionamiento del sensor MAF se basa generalmente en el principio de hilo caliente (*Hot Wire*). En este sistema, un filamento metálico es calentado eléctricamente a una temperatura constante. Cuando el aire atraviesa el conducto de admisión, enfría el filamento y provoca una variación en la corriente eléctrica necesaria para mantener la temperatura estable. La ECU interpreta esta variación como una medida del flujo de aire que entra al motor (HELLA Tech World, 2024a; Robert Bosch GmbH, 2014b).

La información proporcionada por el sensor MAF permite mejorar el control de la mezcla aire–combustible, optimizar el consumo de combustible y reducir emisiones contaminantes. Además, este sensor es ampliamente utilizado en estrategias de monitoreo y diagnóstico automotriz debido a que el flujo de aire está directamente relacionado con la cantidad de combustible consumida por el motor (Mamani Ticona et al., 2024).

La Figura 2.22 muestra la ubicación típica y el principio de funcionamiento de un sensor MAF dentro del sistema de admisión del motor.

Figura 2.22

Principio de funcionamiento eléctrico y térmico del sensor MAF



Nota. Diagrama del comportamiento del flujo de masa de aire sobre el elemento térmico (hilo caliente), ilustrando cómo la transferencia de calor altera la resistencia eléctrica y la señal de salida hacia la ECU. Adaptado de (HELLA Tech World, 2024a).

Uno de los métodos más utilizados para estimar el consumo de combustible en tiempo real consiste en emplear directamente la señal del sensor MAF. Este enfoque se basa en la relación estequiométrica aire–combustible, considerando que para motores a gasolina la mezcla ideal es aproximadamente 14,7:1 (Robert Bosch GmbH, 2018b).

El consumo de combustible puede calcularse mediante la siguiente expresión:

$$FuelConsumption_{(L/h)} = \frac{MAF_{(g/s)} \times 3600}{14{,}7 \times 745} \tag{2.4}$$

En esta expresión, los valores constantes representan:

- **3600**: Factor de conversión de segundos a horas.
- **14,7**: Relación aire-combustible estequiométrica ideal para la gasolina.

- **745**: Densidad aproximada de la gasolina en g/L (este valor puede oscilar entre 720 y 775 g/L según la temperatura y el octanaje).

donde:

Para el cálculo operativo en motores a gasolina, se asumen valores estándar que permiten simplificar la monitorización en tiempo real. En este modelo, se consideran las siguientes constantes:

$$AFR_{estequiometrico} = 14{,}7 \tag{2.5}$$

$$\rho_{fuel} \approx 750\,\mathrm{g/L} \tag{2.6}$$

Es importante destacar que la densidad de la gasolina es una variable sensible a la composición química, el porcentaje de aditivos y las condiciones ambientales. En el contexto de **Bolivia**, la gasolina comercial incorpora mezclas de bioetanol (como la Gasolina Especial Plus o la Súper Etanol), lo que puede generar variaciones en la densidad y en el poder calorífico respecto a las gasolinas convencionales puras (Agencia Nacional de Hidrocarburos, 2019).

Sustituyendo estos valores en la fórmula general, la ecuación de consumo instantáneo se simplifica mediante un factor de conversión aproximado:

$$FuelConsumption_{(L/h)} \approx MAF_{(g/s)} \times 0{,}33 \tag{2.7}$$

2.7.5. Ejemplo de Cálculo Instantáneo

Para ilustrar la sensibilidad del modelo, se propone el siguiente escenario: si el sistema (vía sensores MAP+IAT o MAF) registra un flujo de masa de aire de $10\,g/s$, el consumo instantáneo se calcula como:

$$FuelConsumption = \frac{10 \times 3600}{14{,}7 \times 745} \tag{2.8}$$

$$FuelConsumption \approx 3{,}29\,\mathrm{L/h} \tag{2.9}$$

Este resultado permite al sistema de gestión o al usuario final visualizar el gasto volumétrico de combustible en función de la carga del motor en un momento determinado.

Este modelo permite obtener estimaciones relativamente precisas del consumo instantáneo de combustible utilizando únicamente información proveniente del sistema OBD-II. Debido a su simplicidad y facilidad de implementación, es ampliamente utilizado en aplicaciones de telemetría, monitoreo vehicular y análisis de eficiencia energética (CSS Electronics, 2025; HELLA Tech World, 2024a).

En el presente proyecto, este modelo constituye una de las principales estrategias para estimar el consumo de combustible en tiempo real mediante la adquisición de datos desde la ECU del vehículo a través del protocolo OBD-II.

2.7.6. El Método Speed-Density: Origen y Evolución

El modelo basado en los sensores MAP e IAT es técnicamente denominado el método **Speed-Density** (Velocidad-Densidad). Recibe este nombre porque el flujo de masa de aire no se mide directamente, sino que se infiere a partir de la *velocidad* del motor (RPM) y la *densidad* del aire en el colector de admisión.

Aunque el concepto de densidad de gases se remonta a la termodinámica clásica, su aplicación en el control electrónico de motores (ECU) fue impulsada por la necesidad de cumplir con las normativas de emisiones contaminantes en la década de 1970.

Uno de los pioneros en la implementación masiva de sistemas electrónicos basados en presión de vacío fue la compañía Bendix Corporation con su sistema Electrojector, y posteriormente Bosch con el sistema D-Jetronic(donde la "D"proviene del alemán Druck, que significa presión).

Este sistema fue revolucionario porque reemplazó los carburadores mecánicos por una unidad de control analógica que leía la presión del colector (MAP) para determinar el pulso de inyección. A diferencia del sistema *L-Jetronic* (que usa flujo de aire directo o MAF), el sistema *D-Jetronic* sentó las bases del algoritmo Speed-Density moderno.

El nombre deriva directamente de las dos variables fundamentales que la computadora del motor utiliza para consultar sus mapas de combustible:

1. **Speed (Velocidad):** Se refiere a la velocidad angular del motor (N en RPM). Esta variable determina la frecuencia con la que los cilindros "bombean.ªire hacia adentro.
2. **Density (Densidad):** Calculada mediante la presión (MAP) y la temperatura (IAT). La densidad (ρ) nos dice cuánta masa de aire hay en un volumen determinado de espacio en el colector.

La relación fundamental del método establece que la masa de aire por cilindro (m_{cyl}) es proporcional a la densidad del aire en el colector multiplicada por el volumen del cilindro y corregida por la eficiencia volumétrica:

$$m_{cyl} = \eta_v(N, P) \cdot \rho_{intake} \cdot V_{cyl} \tag{2.10}$$

El uso de Speed-Density sobre otros métodos (como MAF) tiene implicaciones técnicas importantes:

- **Robustez Mecánica:** Los sensores MAP son extremadamente duraderos y no obstruyen el flujo de aire, a diferencia de los hilos calientes de los sensores MAF que pueden ensuciarse.

- **Respuesta Transitoria:** En motores con turbocompresores de alto rendimiento, el método Speed-Density suele ser más rápido para reaccionar a cambios súbitos de presión.
- **Dependencia de la Calibración:** La mayor desventaja es que el sistema "no sabe"si el motor ha sido modificado (por ejemplo, levas más agresivas). Cualquier cambio físico requiere recalibrar la tabla de eficiencia volumétrica (η_v) en la ECU, ya que el cálculo es puramente matemático y no una medición física directa del aire.

2.7.7. Densidad del combustible

La densidad del combustible es una propiedad física que relaciona la masa de un fluido con el volumen que ocupa. En el caso de los combustibles líquidos, esta magnitud es relevante porque influye en la energía contenida por unidad de volumen, en el caudal inyectado y en la estimación del consumo de combustible. Por esta razón, en los ensayos de calidad de carburantes se suele reportar la densidad en condiciones normalizadas de temperatura, generalmente a 15,6°C o 20°C, con el fin de garantizar la comparabilidad de los resultados (Agencia Nacional de Hidrocarburos, 2014; Fue, 2023).

A nivel del mar y en condiciones normalizadas de laboratorio, la densidad de la gasolina suele ubicarse aproximadamente entre 0,71 y 0,78 g/cm^3, mientras que en el diésel se encuentra en un rango aproximado de 0,82 a 0,85 g/cm^3, aunque estos valores pueden variar según la formulación, los aditivos y la temperatura de medición (Fue, 2023). En consecuencia, una misma masa de combustible puede ocupar distinto volumen dependiendo de su densidad, lo cual afecta la forma en que se expresa y compara el consumo en términos volumétricos.

En la práctica, la densidad del combustible disminuye cuando aumenta la temperatura, debido a la expansión térmica del líquido. Por ello, los organismos técnicos y las normas de control de calidad establecen valores de referencia y métodos estandarizados de medición, como D-1298 y D-4052, para asegurar que los resultados no dependan de variaciones ambientales durante el ensayo (Agencia Nacional de Hidrocarburos, 2014).

En la ciudad de La Paz, Bolivia, la evaluación de combustibles debe considerarse dentro de un contexto de altitud elevada, cercana a 3600 m. s. n. m., donde la presión atmosférica es menor que a nivel del mar (Bar, sf). Aunque la densidad del combustible como propiedad intrínseca no depende directamente de la altitud, las condiciones ambientales de operación del motor sí influyen en el comportamiento de la combustión y en la estimación real del consumo, por lo que resulta necesario trabajar con valores corregidos o medidos bajo condiciones controladas (Agencia Nacional de Hidrocarburos, 2014).

En la Tabla 2.8 se presenta valores aproximados de algunos carburantes utilizados en sistemas de combustión interna.

En este sentido, la densidad del combustible constituye una variable de apoyo dentro de los modelos matemáticos de consumo, especialmente cuando se desea convertir

Tabla 2.8
Densidad típica de combustibles líquidos a temperatura de referencia

Combustible	Densidad aproximada	Observación
Gasolina	0,715 – 0,780 g/cm^3	Varía según la formulación estacional y la temperatura.
Diésel	0,820 – 0,845 g/cm^3	Presenta mayor densidad y contenido energético que la gasolina.
Jet fuel	0,775 – 0,840 g/cm^3	Sujeto a especificaciones y normativas internacionales aeronáuticas.
GLP	Variable	Altamente dependiente de la proporción de la mezcla propano-butano.

Nota. Valores de referencia utilizados para la conversión volumétrica en el cálculo de tasas de consumo masivo en sistemas de combustión interna. Elaboración propia.

el caudal volumétrico a caudal másico o cuando se analizan diferencias de desempeño entre distintas zonas geográficas.

2.7.8. Efecto de la altura en el consumo de gasolina

La altura sobre el nivel del mar influye de manera significativa en el desempeño de los motores de combustión interna, debido a que al aumentar la altitud disminuyen la presión atmosférica y la densidad del aire. Esta reducción implica una menor cantidad de oxígeno disponible por unidad de volumen, lo que afecta directamente el proceso de combustión y la potencia desarrollada por el motor (López Ruano, 2018).

En motores de aspiración natural, la disminución de la presión barométrica reduce la masa de aire que ingresa al cilindro durante la carrera de admisión. Como consecuencia, la ECU debe modificar la cantidad de combustible inyectado para conservar una relación aire-combustible adecuada. Si esta compensación no es suficiente, la combustión puede volverse menos eficiente y el consumo específico de combustible puede incrementarse, especialmente en condiciones de carga elevada o conducción exigente (López Ruano, 2018).

Diversos estudios indican que, como referencia práctica, la potencia de un motor atmosférico puede disminuir aproximadamente un 10%por cada 1 000 m de incremento en la altitud, debido a la menor densidad del aire y a la reducción del oxígeno disponible para la combustión (López Ruano, 2018). Sin embargo, el efecto sobre el consumo de gasolina no siempre es lineal, ya que también intervienen la estrategia de inyección, la aerodinámica del vehículo, la topografía del recorrido y el estilo de conducción.

En la ciudad de La Paz, ubicada a gran altitud, esta condición tiene especial relevancia en el comportamiento de los vehículos con motor de gasolina. A menor presión

atmosférica, el motor recibe una masa de aire menor en cada ciclo, por lo que el sistema de inyección debe ajustar la entrega de combustible para evitar mezclas excesivamente ricas o pobres. En consecuencia, el análisis del consumo en esta ciudad debe considerar no solo el volumen de combustible utilizado, sino también el contexto altitudinal y las condiciones de operación del vehículo (López Ruano, 2018; Agencia Nacional de Hidrocarburos, 2014) ver Tabla 2.9.

Por ello, la altitud debe entenderse como una variable ambiental que modifica el rendimiento del motor y condiciona la estimación del consumo de gasolina, especialmente en vehículos no sobrealimentados y en trayectos urbanos donde las variaciones de carga y velocidad son frecuentes .

Tabla 2.9
Efecto de la altitud sobre los parámetros del motor y el consumo de gasolina

Condición	Efecto físico	Consecuencia en el consumo
Nivel del mar	Mayor presión barométrica y alta densidad del aire.	Combustión más estable y un óptimo llenado volumétrico del cilindro.
Altitud media	Menor densidad del aire y reducción moderada en la concentración de oxígeno.	Ajustes dinámicos en los tiempos de inyección y variación del consumo nominal.
Alta altitud	Reducción significativa de la presión barométrica y densidad atmosférica.	Pérdida de potencia efectiva (pérdida por bombeo) y potencial incremento del consumo específico.

Nota. Relación termodinámica general entre la altitud geográfica, la densidad del aire de admisión y el comportamiento del consumo de combustible en motores de combustión interna atmosféricos. Elaboración propia.

2.8. Vehículos bi-fuel

2.8.1. Principio de funcionamiento

Los vehículos bi-fuel o bicombustibles son aquellos cuyo sistema de propulsión está diseñado para operar con dos combustibles distintos, normalmente gasolina y un combustible gaseoso como GLP o GNV. Este tipo de configuración permite que el motor funcione con uno u otro combustible según las condiciones de operación, la disponibilidad energética y la estrategia de control implementada en la unidad electrónica del vehículo (Bif, 2022, 2021; Gas, sf).

En términos generales, el motor bi-fuel conserva un único sistema de combustión, pero incorpora un conjunto adicional de componentes para la gestión del segundo combustible, tales como tanque, válvulas, regulador, inyectores y elementos de conmutación

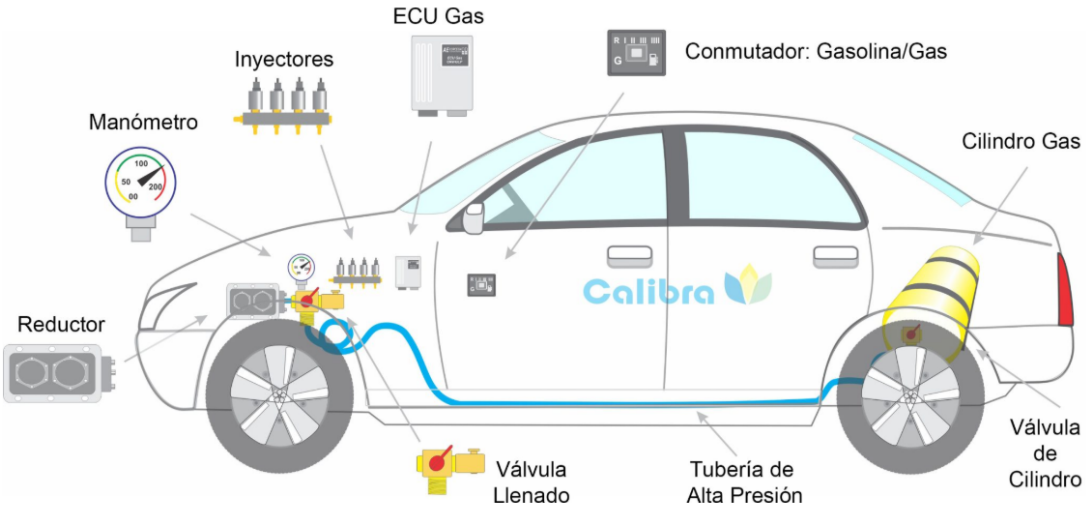
ver Figura 2.23. El arranque suele realizarse con gasolina y, una vez alcanzadas ciertas condiciones de temperatura o funcionamiento, el sistema puede cambiar automáticamente al combustible alternativo (DRB, 2026; Bif, 2021).

2.8.2. Tipos de combustibles utilizados

Los combustibles más comunes en los vehículos bi-fuel son la gasolina y el gas licuado de petróleo (GLP) o el gas natural vehicular (GNV). También existen aplicaciones con biocombustibles y mezclas de combustibles fósiles y alternativos, aunque en el contexto vehicular el término bi-fuel se asocia principalmente al uso de gasolina combinada con GLP o GNV (Tip, 2026; Bif, 2022).

Figura 2.23

Esquema de componentes y distribución en una adaptación de sistema Bi-Fuel (Gasolina/GLP)



Nota. Arquitectura de instalación y flujo de señales en un motor adaptado para doble combustible, detallando la integración de la ECU secundaria de gas, el regulador de presión, la rampa de inyectores de GLP y el tanque de almacenamiento con su respectiva multivalvula. Adaptado de (Gas, sf).

La elección del combustible alternativo depende de factores como la infraestructura disponible, el costo por unidad de energía, la autonomía requerida y la compatibilidad del sistema de inyección y encendido con las características del combustible (Gas, sf; Bif, 2022).

2.8.3. Ventajas y desventajas

Entre las principales ventajas de los vehículos bi-fuel se encuentra la posibilidad de extender la autonomía del vehículo al disponer de dos sistemas de alimentación, así como la reducción del costo operativo cuando se utiliza el combustible alternativo. Además, el uso de GLP o GNV suele asociarse con menores emisiones de CO_2 y NO_x, lo

cual representa una ventaja ambiental frente al uso exclusivo de gasolina (Gas, sf; Fed, 2024).

No obstante, este tipo de tecnología también presenta desventajas. En algunos casos, el motor puede experimentar una ligera reducción de potencia cuando opera con el combustible alternativo, especialmente en configuraciones a GLP. Asimismo, la instalación del sistema bi-fuel implica una mayor complejidad mecánica, costos iniciales más elevados y la necesidad de mantenimiento especializado (Bif, 2021).

2.8.4. Impacto en el consumo de combustible

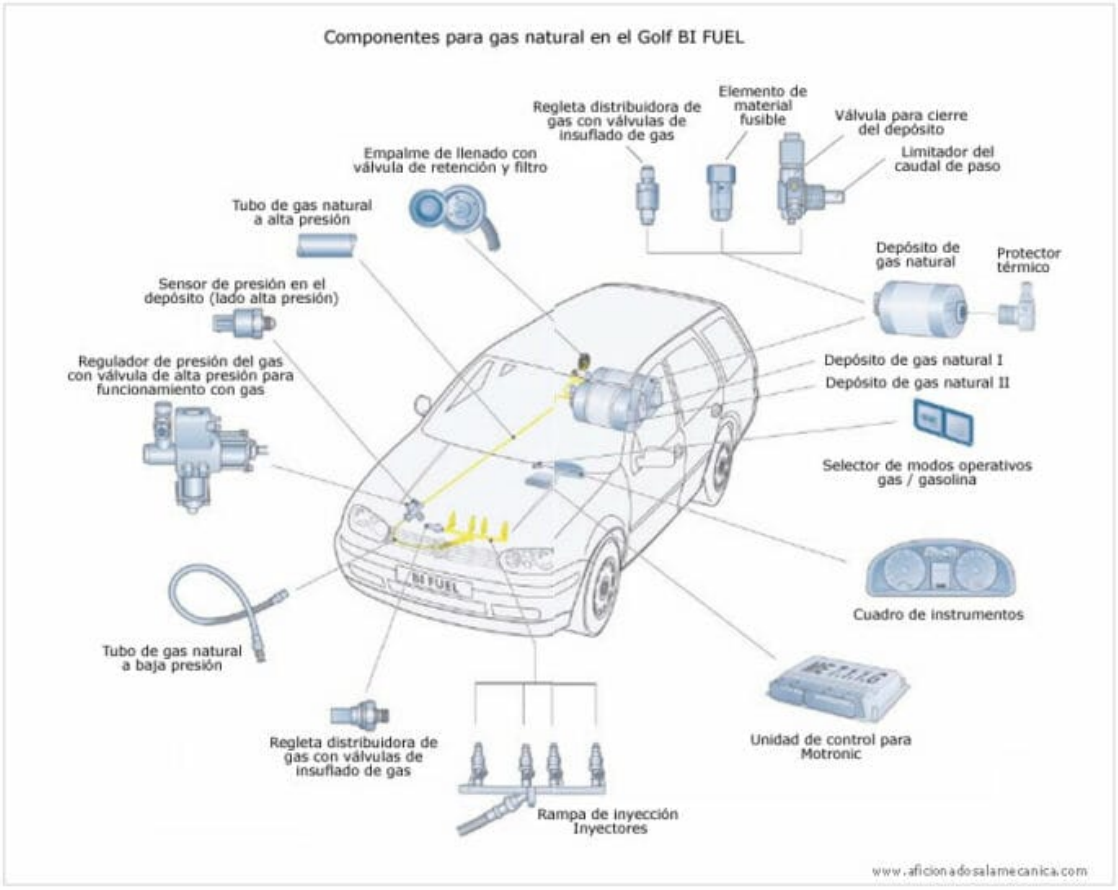
El impacto de los vehículos bi-fuel sobre el consumo de combustible debe analizarse considerando que cada combustible posee una densidad energética distinta. En general, el consumo volumétrico con GLP o GNV puede ser mayor que con gasolina, aunque el costo por kilómetro recorrido puede disminuir debido al menor precio del combustible alternativo (DRB, 2026).

En la práctica, la comparación del consumo debe realizarse en términos equivalentes de energía o costo operativo, ya que un mayor volumen consumido no necesariamente implica un mayor gasto económico. Por esta razón, los vehículos bi-fuel constituyen una alternativa interesante para aplicaciones donde se busca reducir el costo de operación y las emisiones, manteniendo la posibilidad de usar gasolina como respaldo energético (Gas, sf; Bif, 2021).

La Figura 2.24 puede utilizarse para mostrar la arquitectura general de un sistema bi-fuel, incluyendo los componentes principales del circuito de gasolina y del circuito de gas. Y seguidamente en la Tabla 2.10 su comparación con diferentes combustibles.

Figura 2.24

Esquema de componentes que conforman un vehículo bi-fuel, mostrando la coexistencia de sistemas de inyección paralelos y su gestión electrónica integrada



Nota. Diagrama de bloques que ilustra la arquitectura de un motor adaptado para doble combustible (Gasolina/Gas). Se detalla la integración de la ECU secundaria, el regulador de presión, la rampa de inyectores de gas y el tanque de almacenamiento con su multivalvula, operando en paralelo al sistema de inyección original. Elaboración propia con base en (Bif, 2021).

Tabla 2.10
Comparación general entre combustibles en vehículos bi-fuel

Combustible	Ventaja principal	Limitación principal	Uso típico
Gasolina	Mayor disponibilidad y arranque confiable en frío.	Mayor costo operativo y mayores emisiones de CO.	Operación principal o combustible de respaldo.
GLP	Menor costo por kilómetro y reducción de emisiones.	Ligera pérdida de potencia y mayor consumo volumétrico.	Uso alternativo en vehículos convencionales adaptados.
GNV	Alto rendimiento energético y bajas emisiones contaminantes.	Infraestructura de carga limitada y peso del tanque de almacenamiento.	Transporte urbano, flotas comerciales y transporte pesado.

Nota. Comparación de parámetros operativos, económicos y logísticos de los combustibles más utilizados en vehículos dotados con sistemas de conversión de doble combustible. Elaboración propia.

2.9. Microcontroladores y sistemas embebidos

2.9.1. Microprocesador vs microcontrolador

Un *microprocesador* es un circuito integrado que contiene únicamente la unidad central de procesamiento (CPU), encargada de ejecutar instrucciones y procesar datos. Su función principal consiste en interpretar operaciones almacenadas en memoria externa, realizar cálculos aritméticos y lógicos y coordinar la ejecución de un programa mediante señales de control y buses de datos, direcciones e instrucciones .

A diferencia de otros dispositivos, el microprocesador por sí solo no forma un sistema funcional completo, ya que requiere memoria externa (RAM, ROM/Flash), periféricos de entrada/salida y circuitos auxiliares para operar en una aplicación real. Por esta razón, suele emplearse en sistemas de propósito general o en plataformas que demandan mayor capacidad de cómputo y flexibilidad, como computadoras de escritorio o sistemas de infoentretenimiento de alto nivel (MikroElektronika, 2018).

En contraste, un *microcontrolador* integra en un solo chip la CPU, memoria (programa y datos) y periféricos de entrada y salida necesarios para ejecutar tareas específicas. Esta integración permite construir sistemas compactos, de bajo consumo y bajo costo, orientados a aplicación de control en tiempo real, como la gestión de sensores, actuadores y buses de comunicación. En el ámbito automotriz, los microcontroladores son ampliamente utilizados en unidades de control electrónico (ECU) para funciones de powertrain, chasis, seguridad y confort (Facultad de Ingeniería, Instituto de Computación, sf).

2.9.2. Sistema embebido

Un *sistema embebido* es un sistema de cómputo diseñado para realizar una función específica dentro de un dispositivo o equipo mayor, en lugar de servir como plataforma de propósito general. Está compuesto por hardware (microcontrolador o microprocesador, sensores, actuadores, conversiones de señal, etc.), software (firmware o RTOS) y, en algunos casos, elementos mecánicos, con restricciones propias de tiempo, consumo energético y recursos disponibles (Facultad de Ingeniería, Instituto de Computación, sf; ITESO, 2024).

Estos sistemas interactúan con el entorno físico mediante sensores y actuadores, y suelen operar en tiempo real o con tiempos de respuesta acotados. Debido a su naturaleza específica, se optimizan para resolver una tarea concreta (por ejemplo, medir el flujo de combustible, controlar el encendido o gestionar una bomba eléctrica) con alta confiabilidad, bajo costo y consumo reducido de energía. En la industria automotriz, prácticamente todas las ECUs presentes en el vehículo constituyen sistemas embebidos (Facultad de Ingeniería, Instituto de Computación, sf; Aut, 2008).

2.9.3. Arquitectura de microcontroladores

Como se ha señalado, un *microcontrolador* combina núcleo, memoria RAM y periféricos. Con el fin de profundizar en su funcionamiento, en esta sección se describirán teóricamente estos elementos y se analizarán las arquitecturas que definen su estructura.

Núcleo/CPU

El núcleo o CPU de un microcontrolador es el bloque encargado de ejecutar las instrucciones del programa, gestionar registros internos y controlar el flujo de datos entre los distintos componentes del sistema. En la actualidad, los microcontroladores más utilizados en aplicaciones automotrices se basan en arquitecturas de tipo RISC, como los núcleos ARM Cortex-M, AVR o PIC , que ofrecen un buen equilibrio entre rendimiento, consumo energético y facilidad de programación (MikroElektronika, 2018).

Estos últimos se detallan en la Tabla 2.11

Dentro de la CPU se encuentran la unidad de control, que coordina el ciclo de instrucción, y la unidad aritmético-lógica (ALU), que realiza operaciones aritméticas y lógicas sobre los datos. Los registros internos almacenan temporalmente direcciones, datos de trabajo e información de estado (flag de condición), lo que permite una ejecución rápida sin necesidad de acceder continuamente a la memoria principal.

Memoria on-chip

Los microcontroladores integran en el mismo chip varios tipos de memoria, cada uno orientado a una función específica. La memoria de programa, típicamente Flash o ROM, almacena el código ejecutable del firmware, permitiendo leer e, idealmente, reprogramar el dispositivo en campo. La memoria de datos (SRAM) guarda variables

Tabla 2.11

Comparación de arquitecturas de CPU en sistemas embebidos y computación

Arquitectur	Principio clave	Ventaja principal	Desventaja principal	Ejemplos
Harvard	Buses físicos separados para datos e instrucciones. Permite accesos simultáneos.	Mayor velocidad de procesamiento y paralelismo básico.	Hardware más complejo de diseñar y cablear.	PIC, AVR (Arduino), ARM Cortex-M (ESP32, Pi Pico).
Von Neumann	Un solo bus compartido para datos e instrucciones. Acceso de tipo secuencial.	Estructura más simple y mapa de memoria continuo.	Cuello de botella en el bus (ralentiza el sistema).	MSP430, Intel 8051 clásico, arquitectura de PC.
RISC	Juego de instrucciones simples y de tamaño fijo. Ejecución en un ciclo de reloj.	Ejecución rápida, eficiente y con bajo consumo energético.	Requiere mayor cantidad de líneas de código compilado.	AVR, ARM Cortex-M, PIC modernos, RISC-V.
CISC	Instrucciones complejas y de longitud variable para tareas múltiples.	Código binario más compacto (optimiza el uso de memoria).	Mayor consumo de potencia y tiempos de ejecución variables.	Procesadores de PC comerciales (x86), familias Renesas.

Nota. Comparación estructural de los criterios de diseño de hardware para el procesamiento de datos en microcontroladores y procesadores modernos. Elaboración propia.

temporales, pilas de ejecución y estructuras de datos mientras el sistema está activo, aunque su contenido se pierde al desconectar la alimentación, mas detalle acerca de estas memorias en la Tabla 2.12.

Tabla 2.12
Clasificación y funciones de las memorias en un microcontrolador

Tipo de memoria	Volatilidad	Función principal	Velocidad
Flash	No volátil	Almacenar el código del programa fuente (firmware) y constantes del sistema.	Alta / Media.
RAM (SRAM)	Volátil	Almacenar variables temporales, pilas de ejecución y datos de sensores en tiempo real.	Muy Alta.
EEPROM	No volátil	Almacenar parámetros de calibración y datos de configuración del usuario que deben persistir.	Baja.
Registros	Volátil	Espacio de trabajo directo del núcleo de la CPU para operaciones aritmético-lógicas.	Máxima (Inmediata).
ROM / Bootloader	No volátil	Almacenar el código de arranque y rutinas de fábrica inalterables para la programación del chip.	Alta.

Nota. Organización jerárquica y segmentación funcional del mapa de memoria interno característico en arquitecturas de sistemas embebidos modernos. Elaboración propia.

En muchos microcontroladores automotrices se incluye también memoria EEPROM o una sección de Flash configurada como memoria no volátil, utilizada para almacenar parámetros de calibración, configuraciones de usuario o datos de diagnóstico que deben preservarse entre encendidos. La coexistencia de Flash, SRAM y, en algunos casos, EEPROM permite una arquitectura de memoria eficiente y adecuada para aplicaciones de control críticas.

Periféricos integrados

Los periféricos integrados son módulos especializados que permiten al microcontrolador interactuar con el mundo externo de forma directa. Entre los más comunes se encuentran los convertidores analógico-digital (ADC), que permiten leer señales de sensores como presión, temperatura o voltaje; los convertidores digital-analógico (DAC), que generan señales continuas para controlar actuadores lineales; y los timers o temporizadores, que permiten la medición de tiempos, la generación de señales PWM o la

implementación de rutinas periódicas (Facultad de Ingeniería, Instituto de Computación, sf; Aut, 2008).

Además, los microcontroladores incluyen módulos de comunicación serie como UART, SPI e I2C, que permiten la comunicación con dispositivos externos (sensores, displays, módulos de radio, etc.) con bajo costo y alta integración. Otros periféricos típicos incluyen GPIO (pines de entrada/salida digitales configurables), comparadores analógicos, watchdog timers y módulos de captura/comparación, todos diseñados para simplificar el control de señales en tiempo real.

Controladores de comunicación

En el ámbito automotriz, los microcontroladores suelen integrar controladores de comunicación especializados para buses como CAN, LIN y, en algunos casos, FlexRay o Automotive Ethernet. El controlador CAN, por ejemplo, gestiona la trama CAN conforme a la norma ISO 11898, permitiendo la transmisión y recepción de mensajes sin que el núcleo tenga que gestionar cada bit, lo que alivia la carga de procesamiento y mejora la determinismo en la comunicación de datos entre ECUs (International Organization for Standardization, 2015; Aut, 2008).

Los módulos LIN permiten la creación de redes de bajo costo para subsistemas de confort y body electronics, mientras que módulos de Ethernet o FlexRay se utilizan en aplicaciones de mayor ancho de banda o requisitos de tiempo real estrictos, como sistemas de infoentretenimiento o ADAS. La presencia de estos controladores de comunicación integrados convierte al microcontrolador en un nodo de red listo para integrarse en la arquitectura de comunicación vehicular sin necesidad de circuitos externos complejos.

Sistema en chip (SoC): el caso del ESP32-S3

Un *sistema en chip* (*System on Chip*, SoC) constituye una evolución del microcontrolador clásico: además del núcleo de procesamiento, la memoria y los periféricos digitales típicos, integra en el mismo encapsulado subsistemas de radiofrecuencia (RF) completos, como transceptores de Wi-Fi y Bluetooth, junto con la circuitería analógica de acondicionamiento que estos requieren. Esta integración elimina la necesidad de módulos de comunicación inalámbrica externos, reduce el número de componentes discretos de la placa y el consumo asociado a las interfaces entre chips, a cambio de una mayor complejidad interna del silicio (MikroElektronika, 2018).

Un ejemplo representativo de esta arquitectura, y el empleado en el presente proyecto, es el ESP32-S3 de Espressif Systems, en su variante **ESP32-S3-WROOM-1-N16R8**. Este SoC integra un procesador de doble núcleo Xtensa® LX7 de hasta 240 MHz, un controlador de bus CAN nativo (TWAI, *Two-Wire Automotive Interface*) compatible con ISO 11898-1, y radios integrados de Wi-Fi de 2,4 GHz y Bluetooth Low Energy (BLE) 5.0, además de 16 MB de memoria Flash y 8 MB de PSRAM integrados en el propio módulo, de donde proviene la nomenclatura “N16R8” (16 MB de Flash NOR, 8 MB de PSRAM) (Espressif Systems, 2023a). La disponibilidad de un controlador CAN nativo resulta particularmente relevante para este proyecto, ya que permite prescindir

de un controlador CAN externo por SPI, como el MCP2515, reduciendo el conteo de componentes de la placa y eliminando una fuente adicional de fallas en la interfaz entre ambos circuitos integrados.

Subsistemas de energía y gestión

Los microcontroladores incluyen subsistemas de energía y gestión que permiten operar en entornos de alto consumo dinámico y garantizar la confiabilidad del sistema. Entre estos se encuentran módulos de regulación de voltaje interna, circuitos de brown-out detection (que detectan caídas de tensión peligrosas), osciladores de reloj internos y externos, y módulos de bajo consumo que permiten activar distintos modos de ahorro de energía (sleep, standby, power-down) según el estado de la aplicación (MikroElektronika, 2018).

El watchdog timer (WDT) es un módulo fundamental en sistemas críticos, ya que reinicia el microcontrolador si el software deja de ejecutar periódicamente una rutina de "alimentación" del temporizador, corrigiendo así bloqueos o estados indecisos del firmware. Estos mecanismos combinados (modos de bajo consumo, brown-out detection, WDT y supervisión de reloj) son esenciales en aplicaciones automotrices, donde la falencia de un sistema de control puede tener consecuencias de seguridad y operativas significativas.

En la Figura 2.25 se presenta un esquema genérico de la arquitectura de un *microcontrolador*

2.9.4. Programación y desarrollo de firmware

Lenguajes y paradigmas de programación

El desarrollo de firmware para microcontroladores se realiza principalmente en lenguajes de bajo nivel pero de alto rendimiento, destacando el uso de *C* y, en menor medida, *C++*. Estos lenguajes permiten un control fino sobre el hardware, manejo directo de registros y periféricos, y optimización de memoria y tiempo de ejecución, aspectos críticos en sistemas embebidos de recursos limitados (Facultad de Ingeniería, Instituto de Computación, sf).

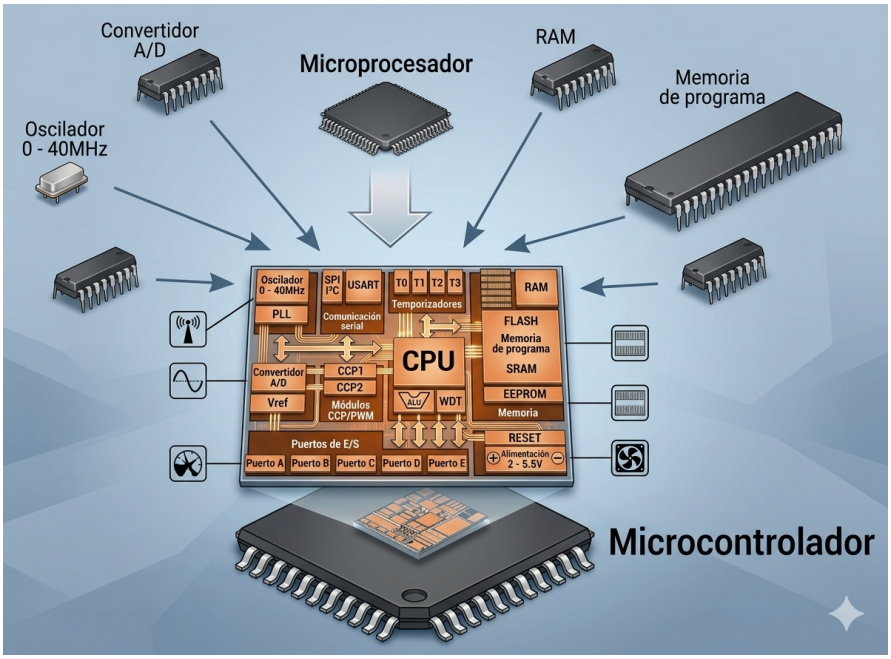
En secciones críticas, como manejo de interrupciones o rutinas de tiempo real extremadamente acotado, se emplea también *ensamblador* para reducir la latencia y aprovechar al máximo las características del núcleo. Además, se utilizan paradigmas de programación orientados a eventos y a tareas, mediante el uso de rutinas de servicio de interrupción (ISR), bucles principales con polling y, cuando se emplea un RTOS, ejecución de tareas concurrentes con prioridades bien definidas .

Software bare-metal y RTOS

El *bare-metal programming* se refiere al desarrollo de firmware sin sistema operativo, donde el programador gestiona directamente el hardware, los registros y el flujo

Figura 2.25

Diagrama de bloques general de la arquitectura interna de un microcontrolador estándar



Nota. Interconectividad de los componentes modulares del dispositivo, ilustrando el núcleo de procesamiento (CPU), las unidades de memoria volátil y no volátil, las líneas de los buses internos y la distribución de los periféricos de entrada y salida (I/O). Adaptado de (Aprendiendo Arduino, 2016).

de ejecución. Este enfoque minimiza el uso de memoria y la latencia de respuesta, resultando adecuado para sistemas relativamente simples o de máxima crítica en tiempo real, como ciertas ECUs de sensado y control básico (Facultad de Ingeniería, Instituto de Computación, sf).

Cuando la complejidad del sistema crece (múltiples tareas, comunicación asincrónica, gestión de recursos compartidos), se recurre a *sistemas operativos en tiempo real* (RTOS) como FreeRTOS, que permiten ejecutar tareas concurrentes, gestionar colas de mensajes, semáforos y prioridades de forma determinista. Los RTOS son ampliamente utilizados en ECUs automotrices donde se requiere coordinar diversas funcionalidades (lectura de sensores, control de actuadores, comunicación CAN) con tiempos de respuesta garantizados (Aut, 2008).

Herramientas de desarrollo y depuración

El desarrollo de firmware se apoya en una cadena de herramientas conocida como *toolchain*, compuesta por compiladores (por ejemplo, GCC para ARM o herramientas del fabricante), ensambladores, enlazadores y librerías de bajo nivel. Estas herramientas transforman el código fuente en un binario que se graba en la memoria del microcontrolador y se ejecuta en el sistema objetivo .

La depuración se realiza mediante interfaces como JTAG o SWD, que permiten

detener la ejecución, inspeccionar variables, modificar el flujo de control y monitorear señales de control en tiempo real. Entornos de desarrollo integrados (IDE) y simuladores ayudan a realizar pruebas tempranas y a validar algoritmos sin necesidad de hardware físico, aumentando la productividad y reduciendo errores en el diseño de firmware (Facultad de Ingeniería, Instituto de Computación, sf).

Pruebas y validación de firmware

La validación de firmware incluye pruebas unitarias, pruebas de integración y, en sistemas críticos, pruebas en configuración hardware in the loop (HIL), donde la ECU se conecta a un simulador de vehículo que replica el comportamiento de sensores y actuadores. Estas pruebas permiten verificar el cumplimiento de requisitos de funcionalidad, seguridad y tiempo real antes de la puesta en marcha en el vehículo real (Aut, 2008).

Además, se emplean técnicas de análisis estático de código, inspección de cubrimiento de pruebas y auditorías de seguridad funcional para detectar errores tempranos y asegurar que el firmware cumpla con normas de calidad y estándares de la industria automotriz.

2.9.5. Comunicación y redes embebidas en automoción

CAN / CAN-FD

El bus *CAN* (Controller Area Network), definido por la norma ISO 11898, es el estándar de comunicación más extendido en vehículos modernos. Permite interconectar múltiples unidades de control electrónico (ECU) mediante un par trenzado y protocolo de comunicación de mensajes orientado a datos, con alta inmunidad al ruido y mecanismos de arbitraje basados en la prioridad de la identificación de mensaje (Aut, 2008).

CAN-FD (CAN with Flexible Data Rate) es una evolución que mantiene compatibilidad con CAN clásico, pero permite velocidades de línea y tamaños de payload mayores, lo que mejora el ancho de banda disponible para datos de diagnóstico, sensores de alto volumen y sistemas ADAS. En sistemas embebidos automotrices, el uso de CAN/CAN-FD es fundamental para la coordinación de ECUs de motor, transmisión, chasis y seguridad de forma robusta y determinista.

LIN, FlexRay y Automotive Ethernet

El bus *LIN* (Local Interconnect Network) se utiliza para redes de bajo costo y baja velocidad, orientadas a subsistemas de confort y body electronics, como ventanas, cerraduras, sensores de puerta y luces interiores. LIN funciona sobre un solo cable y emplea una arquitectura maestro-esclavo, lo que lo hace adecuado para aplicaciones no críticas en tiempo real pero sensibles al costo (Aut, 2008).

FlexRay ofrece un bus de alta velocidad y alta fiabilidad, orientado a sistemas de tiempo real estricto como control de estabilidad, suspensión activa y algunas arquitecturas

de seguridad. Proporciona latencias predecibles y tolerancia a fallos, pero con un costo de hardware y desarrollo más elevado respecto a CAN. En paralelo, *Automotive Ethernet* aparece como solución para sistemas de alto ancho de banda, como infoentretenimiento, telemetría avanzada y comunicaciones entre dominios mediante redes IP dentro del vehículo.

Protocolos de diagnóstico (OBD-II, UDS)

La normativa internacional de diagnósticos vehiculares se basa principalmente en el *OBD-II* (On-Board Diagnostics II), que define formatos de conectores, mensajes básicos y códigos de fallo genéricos accesibles mediante lectores comerciales. Este protocolo permite la lectura remota de códigos DTC, el monitoreo de parámetros de funcionamiento y el borrado de fallos, facilitando el mantenimiento y la verificación emisiones (Aut, 2008).

Por encima de la capa física y de transporte se encuentra el protocolo *UDS* (Unified Diagnostic Services, ISO 14229), que define servicios estandarizados como lectura y escritura de datos de diagnóstico, arranque de rutinas, acceso a memoria y reprogramación de firmware vía CAN (ISO 15765).

2.9.6. Requisitos eléctricos y ambientales

Rangos de tensión y transientes

Los microcontroladores empleados en entornos automotrices deben operar de forma confiable dentro de un sistema eléctrico cuya tensión nominal es de 12 V en vehículos de combustión interna convencionales, 24 V en vehículos comerciales y camiones pesados, y 48 V en arquitecturas de red de a bordo híbridas de tipo *mild-hybrid* (Zimmermann and Schmidgall, 2014).

La norma **ISO 16750-2** (ISO, 2023) establece los perfiles de tensión que todo componente electrónico automotriz debe tolerar, clasificándolos en condiciones de operación normal, condiciones límite y eventos transitorios. En condiciones normales, la tensión de batería de un sistema de 12 V oscila entre 9 V y 16 V durante la operación del alternador.

Los principales fenómenos transitorios reconocidos por **ISO 7637-2** (ISO, 2011) e **ISO 16750-2** se describen a continuación:

- **Pulso 1 — Load Dump:** Ocurre cuando la batería se desconecta abruptamente mientras el alternador está en plena carga. Genera pulsos de tensión que pueden superar los +100 V con una duración de hasta 400 ms. Es el transitorio de mayor energía y requiere supresores de tensión tipo TVS (*Transient Voltage Suppressor*) o varistores de óxido metálico (MOV) en la etapa de entrada de alimentación (Tarter, 1991).
- **Pulso 2a/2b — Interrupción inductiva:** Producido por la apertura de cargas inductivas (relés, solenoides). Genera pulsos negativos de hasta −100 V y pulsos

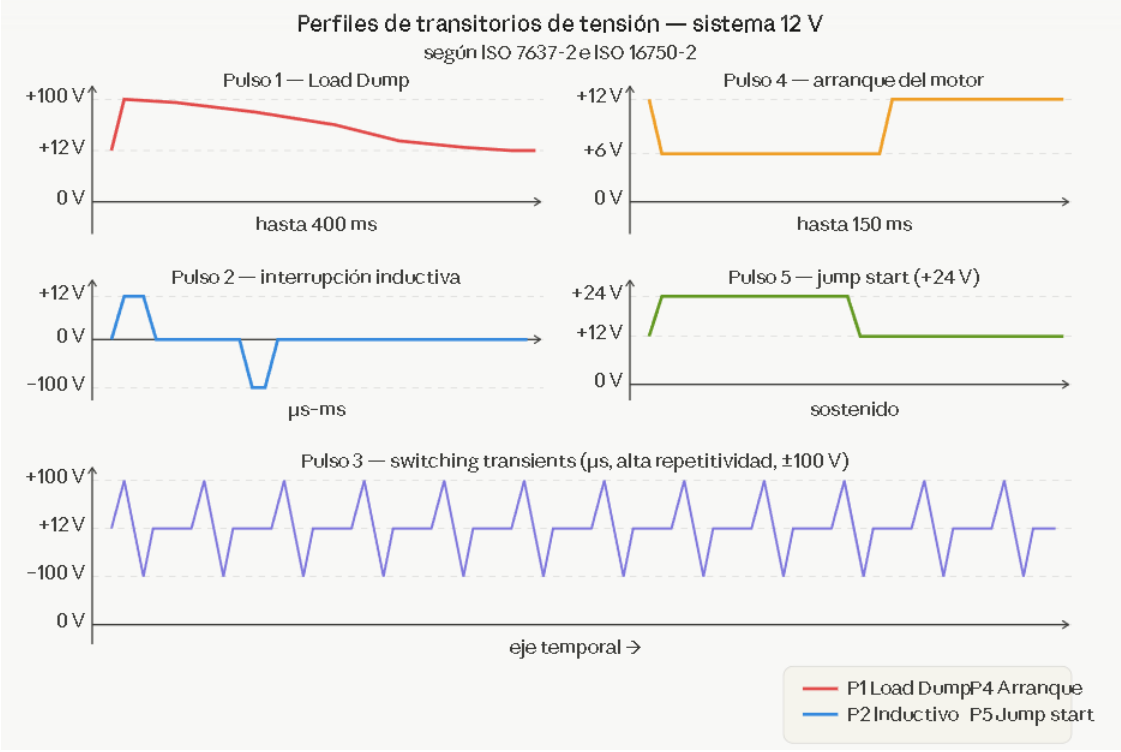
positivos de corta duración en el bus.

- **Pulso 3a/3b — Switching transients:** Originados por la conmutación de cargas en el mismo bus. Presentan amplitudes de ±100 V con duraciones del orden de los microsegundos y alta repetitividad.
- **Pulso 4 — Arranque del motor:** Durante el encendido, el motor de arranque produce una caída de tensión que puede llevar el bus de 12 V hasta valores de 4–6 V por intervalos de hasta 150 ms. Los microcontroladores deben mantener su estado operativo o ejecutar un *reset* controlado en este escenario (Robert Bosch GmbH, 2014a).
- **Pulso 5a/5b — Jump start / sobretensión de carga:** Un arranque con cables de puente puede elevar la tensión hasta 24 V en un sistema de 12 V. Los reguladores de tensión de la ECU deben absorber esta condición sin degradarse.

La Figura 2.26 ilustra de forma esquemática la morfología temporal de los pulsos más representativos sobre un bus de 12 V.

Figura 2.26

Perfiles morfológicos de los principales transitorios de tensión en un sistema eléctrico automotriz de 12 V



Nota. Comportamiento temporal y formas de onda de los pulsos de voltaje críticos conforme a las normativas ISO 7637-2 e ISO 16750-2. Los pulsos 1 y 5 representan las condiciones de mayor descarga de energía (como el fenómeno de *Load Dump*), mientras que el pulso 3 detalla las ráfagas de alta frecuencia provocadas por conmutaciones de relés y acoplamientos capacitivos. Elaboración propia.

Para garantizar la integridad de las ECU frente a estos eventos, los diseñadores recurren a una cadena de protección en la etapa de entrada de alimentación que combina varios dispositivos (Robert Bosch GmbH, 2014a):

1. **Fusible o PTC:** Primera barrera contra sobrecorriente sostenida.
2. **MOV / TVS bidireccional:** Absorbe la energía del Load Dump y los pulsos positivos de alta tensión. Los diodos TVS (p. ej., serie SMBJ) ofrecen tiempos de respuesta del orden de los picosegundos.
3. **Inductor de modo común y condensador de desacoplo:** Filtra los transitorios de alta frecuencia (pulso 3) y desacopla el bus de la electrónica interna.
4. **Regulador de tensión (LDO o SMPS):** Provee la tensión estabilizada de 5 V o 3,3 V al MCU, con rechazo de rizado (PSRR) y protección de subvoltaje (*undervoltage lockout*, UVLO) para gestionar correctamente la caída producida durante el arranque (pulso 4).
5. **Supervisor de tensión:** Circuito dedicado (p. ej., TPS3840) que monitoriza continuamente la alimentación y genera un *reset* controlado ante condiciones fuera de rango, garantizando la integridad del firmware almacenado en memoria no volátil.

Los microcontroladores automotrices modernos, como el Renesas RH850 o el NXP S32K3, incorporan internamente circuitos de UVLO y detectores de *brown-out* conformes con los requisitos de ISO 16750-2, reduciendo la cantidad de componentes externos necesarios en el diseño de la ECU (Ren, 2021; NXP, 2022).

2.9.7. Aplicaciones en la industria automotriz

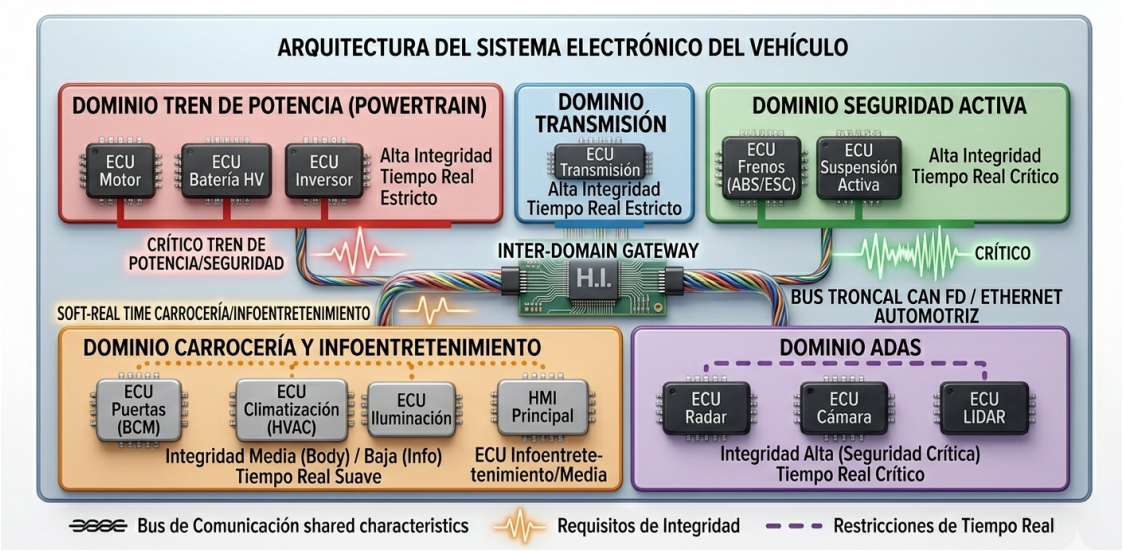
Los microcontroladores (MCU) constituyen el núcleo computacional de las unidades de control electrónico (*Electronic Control Unit*, ECU) que gobiernan la práctica totalidad de los subsistemas de un vehículo moderno. Según estimaciones de la industria, un automóvil de gama media-alta incorpora entre 70 y 100 ECUs independientes, cifra que en vehículos premium o eléctricos puede superar las 150 unidades (Robert Bosch GmbH, 2014a). Estas ECUs se agrupan funcionalmente en dominios que comparten buses de comunicación, requisitos de integridad y restricciones de tiempo real, tal como se ilustra en la Figura 2.27.

2.9.8. ECUs de powertrain

El dominio de *powertrain* agrupa las ECUs responsables de la gestión energética del tren de potencia, siendo la más relevante la unidad de control del motor (*Engine Management System*, EMS o ECM). Su función principal consiste en optimizar los procesos de dosificación de combustible, encendido y control de emisiones, garantizando simultáneamente el rendimiento dinámico exigido por el conductor y el cumplimiento de las normativas de emisiones EURO 6d y EPA Tier 3 (Reif, 2015).

Figura 2.27

Arquitectura y distribución de dominios de ECUs en un vehículo moderno



Nota. Organización funcional del sistema electrónico del vehículo dividida en cinco dominios principales: *powertrain*, transmisión, seguridad activa, *body*/infoentretenimiento y sistemas avanzados de asistencia al conductor (ADAS). Se ilustra la interconectividad y el flujo de datos masivo a través de un bus troncal central basado en las tecnologías CAN FD y Ethernet automotriz. Elaboración propia.

Funciones principales de la ECM

La ECM adquiere señales de un conjunto de sensores —cigüeñal, árbol de levas, masa de aire (MAF), presión absoluta en colector (MAP), sonda lambda de banda ancha, temperatura de refrigerante y presión de combustible— y ejecuta, en tiempo real, los algoritmos de:

- **Cálculo de tiempo de inyección (*injection pulse width*, IPW):** basado en modelos de carga volumétrica y correcciones adaptativas por desvíos lambda (Robert Bosch GmbH, 2014a).
- **Avance de encendido:** optimizado ciclo a ciclo mediante el detector de detonación (*knock sensor*); en motores modernos con encendido por compresión (HCCI), el control se realiza por temperatura en cámara (Heywood, 2018).
- **Gestión del par:** coordinación con la TCU y con el sistema ESP para garantizar transferencia de par coherente en maniobras de cambio o corrección de estabilidad.
- **Control de emisiones:** regeneración del filtro de partículas (DPF), dosificación de AdBlue para reducción catalítica selectiva (SCR) y monitorización del sistema OBD-II conforme a ISO 15031 (SAE International, 2014).

Requerimientos computacionales

La ECM opera con ciclos de control de hasta 1 ms para las tareas de alta prioridad (inyección, encendido) y ciclos de 10–100 ms para las tareas de diagnóstico y adaptación. Esto impone el uso de MCUs de 32 bits con núcleos duales en configuración *lockstep* —como el Infineon AURIX TC3xx o el NXP S32K3— capaces de ejecutar entre 200 y 800 DMIPS con memorias Flash de hasta 16 MB y RAM de hasta 6 MB (Inf, 2022; NXP, 2022) ver Tabla 2.13.

Tabla 2.13
Parámetros típicos de microcontroladores utilizados en módulos de control de motor de producción

Familia	Núcleos	Frec. (MHz)	Flash (MB)	RAM (KB)	ASIL
AURIX TC39x	6 × TriCore	300	16	6 272	D
S32K396	4 × CM7	320	16	4 096	D
RH850/U2A	8 × G4MH	400	16	7 680	D
SPC58xEx	4 × e200z7	200	8	1 536	D

Nota. Criterios de rendimiento, almacenamiento y niveles de integridad de seguridad de la arquitectura automotriz (ASIL) en semiconductores integrados para la gestión del *powertrain*. Elaboración propia.

La arquitectura *lockstep* duplica la ejecución en dos núcleos idénticos comparando los resultados en cada ciclo de reloj; cualquier discrepancia activa un manejador de fallo seguro, lo que constituye un requisito indispensable para alcanzar el nivel ASIL-D según ISO 26262 (ISO, 2023).

2.9.9. Registrador de datos vehiculares vía CAN/OBD-II con ESP32

International Council on Clean Transportation (2013) documentaron el diseño de un registrador de datos portátil basado en el estándar OBD-II sobre CAN, capaz de solicitar hasta seis PIDs por trama CAN en vehículos modernos, lo que permite un ciclo de adquisición multiplex eficiente. Este principio se aplica directamente en el presente proyecto: el ESP32 envía tramas de solicitud (ID 0x7DF) al ECU y procesa las respuestas (ID 0x7E8) interpretando los bytes de datos según las fórmulas de conversión de SAE J1979 (SAE International, 2014).

A diferencia de arquitecturas que emplean un controlador CAN externo por SPI —como el MCP2515—, el ESP32 integra un controlador TWAI (*Two-Wire Automotive Interface*) nativo conforme a ISO 11898-1. La capa física del bus se implementa mediante el transceptor SN65HVD230 de Texas Instruments (Texas Instruments, 2015), un circuito de 3,3 V que proporciona la señalización diferencial en los hilos CAN-H y CAN-L sin

requerir adaptación de niveles de tensión, dado que la lógica de entrada/salida del ESP32 opera a la misma tensión de alimentación. La conexión es directa: los pines GPIO configurados como TWAI_TX y TWAI_RX del ESP32 se conectan, respectivamente, a los terminales TXD y RXD del SN65HVD230, cuyos pads diferenciales se llevan al conector OBD-II (pin 6 CAN-H, pin 14 CAN-L). Esta topología elimina el *overhead* del protocolo SPI, simplifica el árbol de componentes y reduce la latencia de inserción de tramas al bus.

La arquitectura hardware típica de estos sistemas embebidos se muestra en la Tabla 2.14:

Tabla 2.14
Comparativa de arquitecturas de hardware en proyectos de adquisición de datos OBD-II

Referencia	MCU / SoC	Interfaz CAN	Características principales
Rimpas et al. (2020)	Arduino Uno (ATmega328P)	ELM327	Extracción de variables MAF, lambda y velocidad; cálculo de consumo en L/100 km.
Hackaday.io (2017)	STM32F103	ELM327 clone	Despliegue en pantalla gráfica ILI9341; compatibilidad con el protocolo heredado KW1281 (Audi).
Al-Nuaimi et al. (2020)	Raspberry Pi + OBD logger	USB–CAN	Registro de RPM y TPS; procesamiento adaptativo mediante modelo SVM en modo *offline*.
Presente proyecto	ESP32 (TWAI nativo)	SN65HVD230 (TI)	Controlador CAN integrado sin *overhead* SPI; transceptor 3,3 V con acople directo al bus diferencial; adquisición de PIDs OBD-II y captura de tramas CAN raw; algoritmo de corrección por altitud barométrica; conectividad Wi-Fi con interfaz web vía WebSocket.

Nota. Estado del arte comparativo enfocado en los componentes de procesamiento, métodos de acoplamiento al transceptor físico del bus de datos y características funcionales del firmware embebido desarrollado. Elaboración propia.

2.9.10. Validación, verificación y certificación

La ingeniería de sistemas embebidos automotrices exige metodologías formales de verificación que garanticen el correcto funcionamiento del software antes de su despliegue en hardware final. El estándar ISO 26262 (International Organization for Standardization, 2018) define la *Automotive Safety Integrity Level* (ASIL) como marco regulatorio de seguridad funcional, mientras que la normativa IEC 61508 orienta el diseño de sistemas de seguridad en aplicaciones críticas (International Electrotechnical Commission, 2010).

Pruebas MIL, SIL y HIL

Las pruebas del ciclo de vida de desarrollo de software embebido automotriz siguen una jerarquía de cuatro niveles denominada *xIL* (Model/Software/Processor /Hardware-in-the-Loop). VVDN Technologies (2024) describen estas etapas:

1. **MIL (*Model-in-the-Loop*):** Valida la lógica del controlador en un entorno de simulación de alto nivel (p. ej., MATLAB/Simulink) antes de cualquier generación de código. El modelo de la planta (vehículo) y el controlador corren en el mismo simulador.
2. **SIL (*Software-in-the-Loop*):** El código fuente o el código generado automáticamente (model-based) sustituye al modelo del controlador, pero el entorno de simulación permanece en PC. Verifica que la traducción modelo–código no introduce errores semánticos.
3. **PIL (*Processor-in-the-Loop*):** El código compilado se ejecuta en el microcontrolador objetivo conectado al simulador de planta en PC mediante UART u otro canal. Detecta problemas de compilación cruzada y tiempos de ejecución reales.
4. **HIL (*Hardware-in-the-Loop*):** El ECU o prototipo real interactúa con un simulador en tiempo real (dSPACE, National Instruments) que emula los sensores y actuadores del vehículo. VVDN Technologies (2024) señalan que esta etapa incluye pruebas de inyección de fallas, validación de I/O (CAN, LIN, FlexRay) y verificación de tiempo real.

Chawda et al. (2025) aplicaron la metodología MIL–SIL–HIL en el diseño de un controlador SMC-MPC para motores BLDC, obteniendo un tiempo de establecimiento de 0,02 s y un sobrepico del 0,066 %, lo que ilustra la efectividad de estas metodologías para validar controladores embebidos en entornos de alta fiabilidad.

Un resumen de estos sistemas en la Tabla 2.15.

Para el presente proyecto, la etapa PIL resulta especialmente relevante dado que el ESP32 opera a 240 MHz con el controlador TWAI configurado a 500 kbps; las pruebas PIL permiten verificar que el ciclo de adquisición OBD-II (solicitud + respuesta + cálculo de consumo) se completa dentro del intervalo de muestreo configurado de 100 ms, y que el buffer de recepción del controlador TWAI no incurre en desbordamiento bajo carga máxima del bus.

Trazabilidad de requisitos y pruebas

La trazabilidad de requisitos es el proceso mediante el cual cada requisito del sistema queda vinculado, de forma bidireccional, a los casos de prueba que lo verifican y a los módulos de código que lo implementan (International Organization for Standardization, 2018). Esta práctica es obligatoria en sistemas de seguridad funcional ASIL B o superior.

En sistemas de adquisición de datos OBD-II, los requisitos típicos cubren:

- **Requisitos de comunicación:** La trama CAN de solicitud PID debe generarse con ID 0x7DF y una latencia máxima de 50 ms desde el evento de disparo.
- **Requisitos de precisión:** El error en la estimación de consumo no debe superar el 7 %en ciclo urbano (Rimpas et al., 2020).

Tabla 2.15
Comparativa de niveles de verificación xIL para sistemas embebidos automotrices

Nivel	¿Qué se ejecuta en HW real?	Herramientas típicas	Defectos detectables
MIL (*Model*)	Nada (todo el entorno opera en simulación matemática).	MATLAB / Simulink, Modelica.	Errores en la lógica algorítmica y control del modelo.
SIL (*Software*)	Nada (el código C/C++ compilado se ejecuta en una PC).	GCC, IAR Embedded Workbench.	Errores de sintaxis, punteros y fallas en la generación de código.
PIL (*Processor*)	El código binario se ejecuta en el ESP32 objetivo.	ESP-IDF, Arduino-ESP32, monitor serie USB.	Tiempos de ejecución del ciclo TWAI y posibles desbordamientos de buffer en recepción CAN.
HIL (*Hardware*)	El sistema completo (ESP32 + SN65HVD230) interactúa con hardware de simulación física.	dSPACE, National Instruments (NI PXI).	Errores de integración I/O, latencia de red y fallas de comunicación en el bus CAN.

Nota. Clasificación de las etapas de validación en lazo (*In-the-Loop*) aplicadas al ciclo de desarrollo en V para sistemas electrónicos embebidos de grado automotriz. Adaptado de (VVDN Technologies, 2024).

- **Requisitos de interfaz:** La interfaz web debe actualizar los valores cada 500 ms sin causar bloqueos en el ciclo principal del firmware.
- **Requisitos de robustez:** El sistema debe recuperarse de un error de bus CAN (mensaje ausente) dentro de 200 ms sin reiniciar el microcontrolador.

La Tabla 2.16 muestra un ejemplo de matriz de trazabilidad aplicable a este proyecto:

2.9.11. Buenas prácticas de firmware y mantenimiento

El desarrollo de firmware para sistemas embebidos vehiculares requiere disciplinas de ingeniería de software que faciliten el mantenimiento, la reproducibilidad y la trazabilidad a lo largo del ciclo de vida del producto.

Tabla 2.16

Fragmento de la matriz de trazabilidad entre requisitos y pruebas para el sistema OBD-II

ID Req.	Descripción del requisito	Caso de prueba asociado	Estado
REQ-COM-01	Solicitud de PID en intervalos ≤ 50 ms.	TC-COM-01: Medición de la latencia de respuesta mediante osciloscopio en los hilos físicos del bus CAN.	Verificado.
REQ-CAL-01	Margen de error en cálculo de consumo ≤ 7 %.	TC-CAL-01: Comparación analítica de la estimación del firmware frente a las lecturas de un flujómetro Coriolis patrón.	Verificado.
REQ-HMI-01	Tasa de refresco de la interfaz web cada 500 ms.	TC-HMI-01: Verificación de la temporización mediante captura de tramas WebSocket con Wireshark.	Verificado.
REQ-ROB-01	Recuperación del nodo ante fallos en el bus ≤ 200 ms.	TC-ROB-01: Inyección de fallas físicas en la red mediante conmutación de resistencia de carga espuria.	Pendiente.

Nota. Mapeo bidireccional para el control de calidad del firmware y hardware, asegurando la correlación directa entre los objetivos funcionales de diseño y los protocolos de ensayo experimentales. Elaboración propia.

Versionado y integración continua

El versionado semántico (*Semantic Versioning*, SemVer) es el esquema más extendido para firmware embebido, estructurado en tres componentes `MAYOR.MENOR.PARCHE` (Preston-Werner, 2013):

- **MAYOR:** se incrementa ante cambios incompatibles con versiones anteriores (p. ej., modificación del protocolo CAN o del formato de trama OBD-II).
- **MENOR:** nuevas funcionalidades retrocompatibles (p. ej., incorporación del PID `0x33` de presión barométrica).
- **PARCHE:** correcciones de errores sin cambio de interfaz (p. ej., corrección de cálculo CRC-8 en la trama SPI del MCP2515).

Swedish Embedded (2024) señalan que en proyectos de firmware embebido es crítico garantizar que, en 2025, sea posible reproducir una compilación de 2023 con exactamente el mismo binario: esto exige versionar no solo el código fuente, sino también el compilador (p. ej., AVR-GCC 7.3), las bibliotecas y el toolchain de construcción.

La *Integración Continua* (CI) en firmware embebido se implementa mediante sistemas como GitHub Actions, GitLab CI o Jenkins (Dojo Five, 2025). Un pipeline típico para un proyecto ATmega328P incluye:

1. **Compilación cruzada** con AVR-GCC en contenedor Docker para reproducibilidad.
2. **Pruebas unitarias** de las funciones de cálculo (conversión PID, corrección altitudinal) con `Unity` o `CMock` sobre PC.

3. **Análisis estático** con `cppcheck` o MISRA-C lint para detectar comportamientos indefinidos.
4. **Generación automática** de artefactos (`.hex`, `.elf`) con etiqueta de versión SemVer integrada en el binario mediante macro `FIRMWARE_VERSION`.

Infolitz (2025) destacan que los equipos de firmware automotriz utilizan plataformas HIL con racks de hardware y simulación del bus CAN para validar componentes críticos de seguridad en pruebas nocturnas automatizadas, lo que permite detectar regresiones de forma temprana.

En la Tabla 2.17 las funciones de algunas herramientas en el monitoreo OBD-II.

Tabla 2.17
Herramientas de integración y despliegue continuos para firmware embebido automotriz

Etapa de CI/CD	Herramienta seleccionada	Función en el proyecto de monitoreo OBD-II
Control de versiones	Git + GitHub / GitLab	Gestión del historial de cambios y etiquetado estructurado (*SemVer*) por cada lanzamiento (*release*).
Compilación cruzada	AVR-GCC (Contenedor Docker)	Compilación automatizada y aislada para la generación del archivo ejecutable `firmware.hex` de forma reproducible.
Pruebas unitarias	Unity / CppUTest	Verificación aislada de las funciones lógicas del *parser* de tramas OBD-II y ecuaciones de consumo másico.
Análisis estático	`cppcheck` / Reglas MISRA-C	Auditoría automatizada del código fuente para la detección temprana de accesos fuera de rango en los *buffers* del bus CAN.
Despliegue y flasheo	`avrdude` + *Runner* de CI	Programación automatizada del binario en el microcontrolador ATmega328P dispuesto en el banco de pruebas de hardware.

Nota. Configuración de la tubería (*pipeline*) de automatización para la optimización de la calidad del software embebido y la mitigación de errores en entornos físicos de operación. Adaptado de (Dojo Five, 2025; Infolitz, 2025).

2.9.12. Documentación y soporte a largo plazo

La documentación técnica de un sistema embebido automotriz abarca cuatro niveles complementarios (International Organization for Standardization, 2018):

1. **Documentación de requisitos:** especificación funcional de cada módulo (p. ej., módulo de comunicación CAN, módulo de cálculo de consumo, módulo de visualización LCD).
2. **Documentación de arquitectura:** diagramas de bloques hardware (esquemático EasyEDA, capas PCB) y diagramas de flujo del firmware.

3. **Documentación de API interna:** comentarios Doxygen en el código fuente que describen cada función, sus parámetros y valores de retorno.
4. **Notas de versión (*Release Notes*):** tabla de cambios por versión SemVer con la referencia al ticket o requisito modificado.

El *soporte a largo plazo* (LTS) es la práctica de mantener una rama de software estable, recibiendo únicamente correcciones de errores y parches de seguridad, durante un periodo extendido (típicamente 3 años) sin incorporar nuevas características (NVIDIA Networking, 2024). En el contexto del proyecto, una política LTS implica que la versión de firmware que se certifica para el vehículo permanece congelada, y cualquier nueva funcionalidad (p. ej., soporte de PIDs adicionales o integración con plataforma IoT) se desarrolla en una rama paralela.

ipXchange (2026) señalan que en el sector automotriz e industrial, los clientes evalúan los productos embebidos no solo por sus prestaciones iniciales sino también por la garantía de mejora durante el ciclo de vida del dispositivo, lo que convierte al LTS en un factor competitivo crítico.

2.9.13. Futuro y tendencias

En la actualidad, prácticamente todo vehículo moderno incorpora al menos una computadora embebida que gestiona su funcionamiento, independientemente de su tipo de propulsión: los vehículos de combustión interna la emplean para el control del motor y la inyección de combustible, los vehículos híbridos suman a esto la gestión de la transición entre el motor térmico y el eléctrico, y los vehículos eléctricos dependen por completo de ella para el control del motor eléctrico, la batería y la electrónica de potencia. La diferencia entre estas arquitecturas no radica, por tanto, en la presencia o ausencia de electrónica de control, sino en el número, la complejidad y el grado de centralización de dichas unidades de procesamiento, tendencia que se profundiza con la electrificación del tren motriz, tal como se describe a continuación.

Electrificación y arquitecturas de zona

La industria automotriz está experimentando una transformación profunda en su arquitectura eléctrica/electrónica (E/E), impulsada por la electrificación del tren motriz y la proliferación de funciones definidas por software (*Software-Defined Vehicles*, SDV).

Tech Briefs (2024) reportan que la electrificación y las presiones de sostenibilidad han motivado regulaciones gubernamentales que aceleran la adopción de vehículos eléctricos: en 2022, Noruega alcanzó el 80 % de cuota de mercado EV, mientras las ventas globales de automóviles eléctricos totalizaron 14 millones de unidades en 2023 según la Agencia Internacional de Energía (Global Market Insights, 2025).

Este avance impulsa la transición de las arquitecturas E/E distribuidas (un ECU por función) hacia **arquitecturas de zona** (*Zonal Architecture*), en las que unos pocos controladores de zona gestionan todos los actuadores y sensores de una región física del vehículo (Promwad, 2025). Los beneficios reportados son:

- Reducción del arnés de cableado (hasta 30 %de ahorro en peso y coste).
- Gestión centralizada de energía con menor caída de tensión.
- Actualizaciones OTA (*Over-the-Air*) más simples al consolidar el software en pocos nodos.

Bosch Mobility (2024) (Bosch Mobility) describen un kit modular de arquitectura E/E centralizada y orientada a zonas que es escalable según el tamaño del vehículo: los vehículos pequeños emplean menos ECUs de zona que los de mayor tamaño, reduciendo la complejidad de desarrollo gracias a la *arquitectura orientada a servicios* (SOA) diseñada para iteraciones rápidas de software y actualizaciones OTA.

La Tabla 2.18 resume la evolución histórica de las arquitecturas E/E:

Tabla 2.18
Comparativa de sensores empleados en sistemas avanzados de asistencia al conductor y sus características operativas

Sensor	Alcance	Res. ang.	Rango vel.	Condic. adv.	Costo rel.
Cámara mono/estéreo	0 – 250 m	Alta (píxel)	Media	Bajo (lluvia, niebla)	Bajo
Radar 77 GHz	0 – 300 m	Baja – media	Alto	Alto	Medio
LiDAR (ToF)	0 – 200 m	Muy alta	Bajo	Medio (lluvia)	Alto
Ultrasónico	0 – 5 m	Baja	Estático	Alto	Muy bajo
IMU (6 ejes)	—	—	—	Muy alto	Bajo
GNSS (RTK)	—	0,01 m	Medio	Bajo (túneles)	Medio

Nota. Análisis comparativo de los rangos de operación, inmunidad a perturbaciones climáticas y costos relativos de los componentes de hardware que integran la capa de percepción del entorno vehicular. Elaboración propia.

Implicación para el proyecto. Aunque el presente sistema emplea CAN 2.0 clásico a través del SN65HVD230, la tendencia hacia arquitecturas zonales sugiere que futuras versiones del sistema podrían migrar a CAN-FD o incluso a Ethernet automotriz, manteniendo compatibilidad con el estándar OBD-II a través de capas de adaptación de protocolo.

Evolución de buses y estándares

El bus CAN, introducido por Bosch en 1986 y estandarizado en ISO 11898, sigue siendo el protocolo dominante en redes de control automotriz. Sin embargo, la creciente demanda de ancho de banda de aplicaciones como ADAS, cámaras y comunicaciones V2X está acelerando la adopción de protocolos de mayor velocidad (Wu et al., 2024).

Wu et al. (2024) documentan que el **Ethernet automotriz** (*100BASE-T1*, *1000BASE-T1*) opera sobre un par de hilos no apantallados y ofrece latencia baja y

baja variación de retardo (*jitter*), lo que lo hace adecuado para transmisión de audio y video en tiempo real dentro del vehículo. El protocolo SOME/IP, estandarizado por AUTOSAR en 2016, opera sobre TCP/UDP sobre Ethernet automotriz y proporciona una arquitectura de middleware orientada a servicios (SOA) (Wu et al., 2024).

La Tabla 2.19 compara los principales buses automotrices en uso y proyectados:

Tabla 2.19

Comparativa de buses de comunicación automotrices según sus especificaciones técnicas y aplicaciones

Bus	**Velocidad máx.**	**Estándar**	**Aplicación típica en el vehículo**
CAN 2.0	1 Mbps	ISO 11898-1	Diagnóstico mediante el puerto OBD-II y redes de control secundarias.
CAN FD	8 Mbps	ISO 11898-2	Redes de tren motriz (*powertrain*) y sistemas de chasis de nueva generación.
LIN	20 kbps	ISO 17987	Control de actuadores de carrocería local (espejos, ventanas, asientos).
FlexRay	10 Mbps	ISO 17458	Sistemas de seguridad activa de alta criticidad determinista (*steer-by-wire*).
MOST	150 Mbps	MOST Spec.	Sistemas de infoentretenimiento, pantallas multimedia y transferencia de audio digital.
100BASE-T1	100 Mbps	IEEE 802.3bw	Transmisión de vídeo en cámaras ADAS y comunicación dentro de la arquitectura E/E zonal.
1000BASE-T1	1 Gbps	IEEE 802.3bp	Interconexión troncal (*backbone*) en arquitecturas E/E centralizadas y sistemas V2X.

Nota. Análisis comparativo de los principales protocolos de red y buses de datos que estructuran la comunicación interna de los vehículos modernos. Adaptado de (Wu et al., 2024; Embien Technologies, 2024; International Organization for Standardization, 2015).

En paralelo, la estandarización AUTOSAR (AUTomotive Open System ARchitecture) avanza hacia su plataforma *Adaptive*, que incorpora POSIX sobre procesadores multinúcleo y soporta SOA y actualizaciones OTA, en complemento con la plataforma clásica (*Classic Platform*) basada en microcontroladores como el ATmega328P o familias de 32 bits (Embien Technologies, 2024; Wu et al., 2024).

Para el contexto de proyectos de adquisición de datos basados en OBD-II, Global Market Insights (2025) señalan que el mercado global de arquitecturas E/E automotrices fue valorado en 79,4 miles de millones de USD en 2024, con una tasa de crecimiento compuesto anual (CAGR) del 6,6 %proyectada hasta 2034, impulsada principalmente por la demanda de EVs y los sistemas ADAS. Esto garantiza la vigencia y el interés continuo en proyectos de integración de sensores y diagnóstico vehicular como el desarrollado en el presente trabajo.

2.10. Manufactura aditiva: impresión 3D

2.10.1. Modelado por deposición fundida (FDM/FFF)

La manufactura aditiva, comúnmente conocida como impresión 3D, es un conjunto de procesos de fabricación que construyen piezas físicas de manera incremental, agregando material capa sobre capa a partir de un modelo digital, en contraposición a los procesos sustractivos tradicionales (mecanizado, fresado) que parten de un bloque de material y remueven el excedente. Entre las distintas tecnologías de manufactura aditiva, el *modelado por deposición fundida* (*Fused Deposition Modeling*, FDM, también referido como *Fused Filament Fabrication*, FFF por su nombre genérico no registrado) es la más extendida en aplicaciones de prototipado de bajo costo, tanto a nivel industrial como de escritorio.

El proceso FDM consiste en alimentar un filamento termoplástico sólido, típicamente de 1,75 mm de diámetro, hacia un cabezal de extrusión (*hotend*) que lo calienta hasta su temperatura de fusión y lo deposita de forma controlada, capa por capa, sobre una plataforma de impresión, siguiendo la trayectoria definida por el modelo tridimensional (archivo STL) y procesada por un software de laminado (*slicer*) que lo convierte en instrucciones de movimiento para la impresora (código G). Una vez depositado, el material se solidifica rápidamente por enfriamiento, y el proceso se repite capa tras capa hasta completar la geometría de la pieza (Wevolver, 2023).

2.10.2. Materiales termoplásticos comunes en FDM

La elección del material termoplástico determina directamente las propiedades mecánicas, térmicas y de durabilidad de la pieza impresa, así como la dificultad del propio proceso de impresión (temperatura de extrusión, necesidad de cama caliente, propensión al alabeo). Los tres materiales más utilizados en impresión FDM de escritorio son el ácido poliláctico (PLA), el acrilonitrilo butadieno estireno (ABS) y el tereftalato de polietileno modificado con glicol (PETG), cuyas propiedades características se resumen en la Tabla 2.20.

Tabla 2.20

Comparación de propiedades típicas de materiales termoplásticos para impresión FDM

Propiedad	PLA	ABS	PETG
Temperatura de transición vítrea (T_g)	55 – 60 °C	100 – 105 °C	75 – 80 °C
Resistencia a la tracción	50 – 70 MPa	35 – 50 MPa	45 – 53 MPa
Rigidez (módulo de Young)	Alta (2300 – 3000 MPa)	Media (1800 – 2300 MPa)	Media-baja (1000 – 2100 MPa)
Elongación a la rotura	Baja (≈6 %, frágil)	Media-alta (≈20 %)	Alta (tenaz, absorbe impactos)
Temperatura de impresión	190 – 220 °C	220 – 250 °C	220 – 250 °C
Cama caliente	No estrictamente necesaria	Necesaria (≈90 – 110 °C)	Recomendada (≈70 – 80 °C)
Alabeo (*warping*)	Bajo	Alto (requiere recinto cerrado)	Bajo-medio
Resistencia UV / intemperie	Baja (se degrada y fragiliza con el sol)	Media	Alta
Resistencia química / humedad	Baja (biodegradable por hidrólisis)	Media	Alta

Nota. Valores típicos reportados en la literatura técnica y por fabricantes de filamento; varían según la marca específica, los parámetros de impresión (temperatura, velocidad, orientación de capas) y el porcentaje de relleno de la pieza. Adaptado de (Wevolver, 2023; All3DP, 2023).

El **PLA** es un poliéster termoplástico derivado de recursos renovables (almidón de maíz o caña de azúcar), lo que lo hace parcialmente biodegradable bajo condiciones de compostaje industrial. Es el material más sencillo de imprimir en FDM: su baja temperatura de extrusión y su mínima contracción al enfriarse reducen drásticamente el alabeo, no requiere cama caliente ni recinto cerrado, y no emite vapores significativos durante la impresión. A cambio, es un material relativamente frágil (baja elongación a la rotura) y con baja resistencia a la temperatura y a la radiación ultravioleta: una pieza de PLA expuesta de forma prolongada al sol o a temperaturas sostenidas por encima de su T_g (como el interior de un vehículo estacionado al sol, que puede superar los 60 °C) tiende a ablandarse, deformarse y volverse quebradiza con el tiempo (Wevolver, 2023;

All3DP, 2023).

El **ABS** es un terpolímero amorfo ampliamente utilizado en piezas automotrices e industriales (paneles interiores, carcasas de electrodomésticos) por su mayor resistencia al impacto y a la temperatura frente al PLA. Su principal desventaja en el contexto de impresión FDM de escritorio es su tendencia al alabeo durante el enfriamiento, lo que exige una cama caliente a temperatura elevada y, idealmente, un recinto cerrado que mantenga una temperatura ambiente estable alrededor de la pieza; además, su impresión libera vapores de estireno que requieren ventilación adecuada (Wevolver, 2023).

El **PETG** combina características de ambos materiales anteriores: conserva una temperatura de impresión y una facilidad de manejo similares a las del ABS, pero con un alabeo considerablemente menor (cercano al del PLA), a la vez que ofrece una resistencia química, a la humedad y a la radiación ultravioleta notablemente superior a la del PLA. Esta combinación lo convierte en una opción frecuente para piezas funcionales destinadas a permanecer a la intemperie o en ambientes con variaciones térmicas, como carcasas automotrices (All3DP, 2023).

La selección concreta de material para la carcasa del prototipo desarrollado en este proyecto, y la justificación de por qué un material distinto sería preferible para una carcasa de uso permanente dentro del vehículo, se detalla en la Sección 3.3.6 del Capítulo 3.

Capítulo 3

INGENIERÍA DEL PROYECTO

3.1. Descripción general del sistema

El sistema propuesto tiene como uno de los objetivos principales optimizar el consumo energético, limitando el uso de componentes innecesarios y priorizando la eficiencia del sistema embebido. Debido a que la fuente de alimentación proviene directamente de la batería del vehículo, un consumo excesivo de energía podría ocasionar la descarga de la misma, afectando el arranque y funcionamiento del automóvil.

Con el propósito de mejorar la organización, modularidad y escalabilidad del diseño, el sistema se divide en los siguientes subsistemas funcionales:

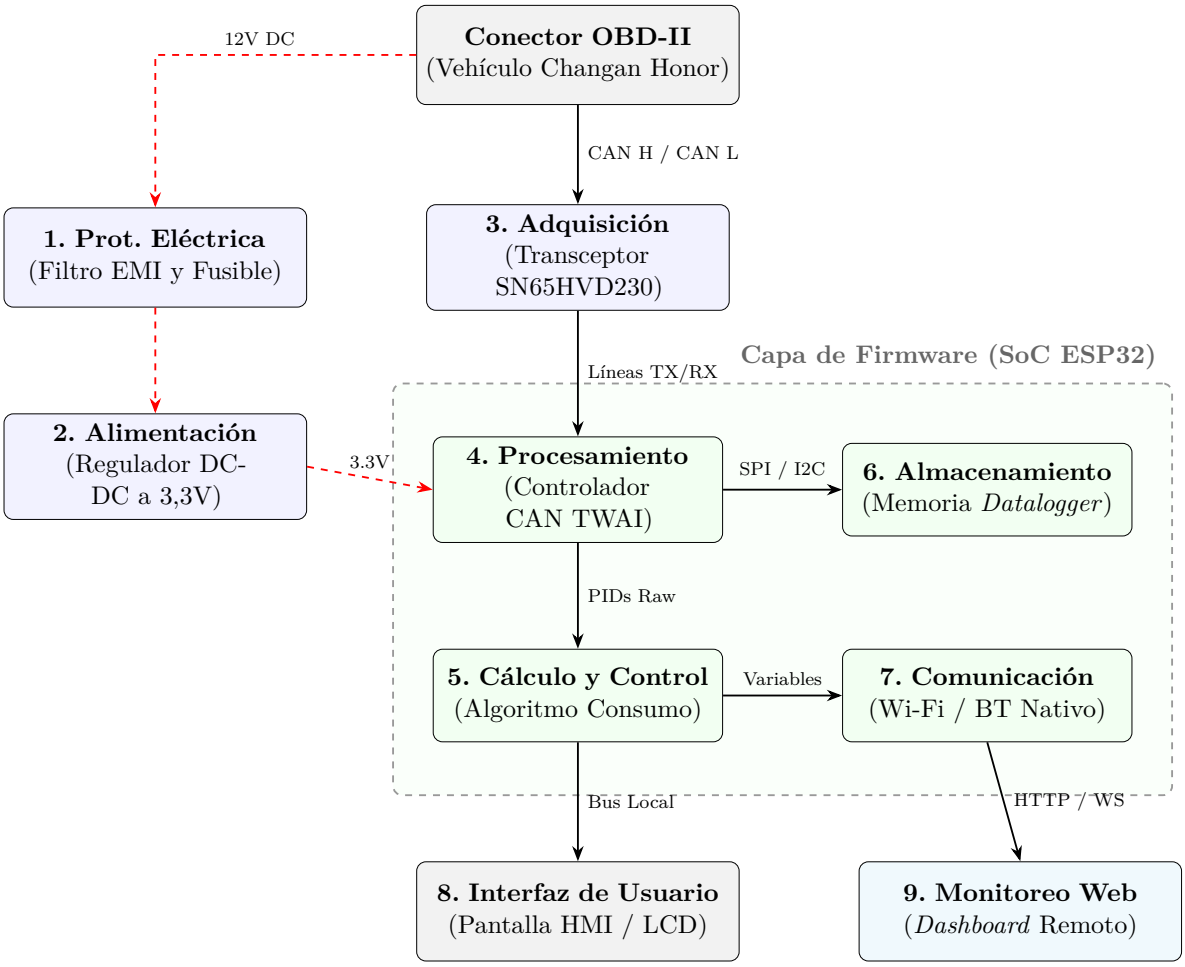
- **Sistema de protección eléctrica.** Etapa de aislamiento analógico encargada de salvaguardar los componentes lógicos frente a transitorios de tensión, picos de sobrecorriente, inversión de polaridad y ruido electromagnético (EMI) inherente a la red del vehículo.
- **Sistema de alimentación.** Módulo de acondicionamiento de potencia que convierte la tensión nominal de 12 V extraída del puerto OBD-II a niveles lógicos estables (ej. 3,3 V o 5 V) mediante regulación DC-DC.
- **Sistema de adquisición de datos.** Capa física (*PHY*) acoplada a las líneas diferenciales CAN High y CAN Low (pines 6 y 14 del conector OBD-II) para la intercepción, lectura y transmisión de las tramas de diagnóstico del automóvil.
- **Sistema de procesamiento y filtrado.** Etapa algorítmica donde el firmware central extrae la carga útil (*payload*) de los mensajes de red, descartando el ruido o tramas irrelevantes y decodificando los PIDs hexadecimales a valores físicos operables.
- **Sistema de cálculo y control.** Subsistema lógico que aplica modelos estequiométricos de dinámica de fluidos utilizando los parámetros extraídos (como flujo de aire y RPM) y la retroalimentación del sensor lambda para estimar la tasa real de consumo de combustible.

- **Sistema de almacenamiento de datos.** Unidad de gestión de memoria (*datalogger*) que resguarda los registros en buffers temporales o memoria no volátil, garantizando la preservación de los históricos de conducción ante cortes de energía.
- **Sistema de interfaz de usuario (HMI).** Módulo de visualización en cabina que otorga retroalimentación directa de las variables calculadas e instrumentación en tiempo real, permitiendo la interacción física del conductor.
- **Plataforma de monitoreo web.** Servidor de interfaz de red (*Dashboard*) diseñado para la recepción, graficación y análisis remoto de los datos telemétricos del vehículo.

Teniendo finalmente un diagrama ver Figura 3.1

Figura 3.1

Diagrama de bloques de la arquitectura funcional y flujo de señales del sistema



Nota. Arquitectura del sistema embebido. La columna izquierda procesa la etapa de potencia; la central gobierna el flujo del bus CAN y el núcleo del procesador; la derecha y la base gestionan el almacenamiento local y la telemática. Elaboración propia.

3.1.1. Configuración del entorno de captura

Para validar experimentalmente los fundamentos teóricos del protocolo CAN, se propone el desarrollo de un prototipo orientado a la captura y lectura de tramas de datos a través del puerto OBD-II. Este sistema requiere de una interfaz física capaz de realizar la transición entre el medio físico de transmisión y el nivel lógico digital. Dado que el protocolo CAN opera bajo un esquema de señalización diferencial a través de los hilos CAN-H y CAN-L, es indispensable integrar un transceptor que convierta dichas diferencias de potencial en niveles lógicos (TTL/CMOS) compatibles con un microcontrolador.

Este prototipo se fundamenta en los principios del hacking ético aplicado a sistemas automotrices (Ocsaly, 2022), cuyo enfoque permite la extracción, monitoreo y análisis de datos vehiculares mediante el uso de hardware embebido. El flujo operativo de esta arquitectura de adquisición se ilustra en la Figura 3.2.

Figura 3.2

Diagrama de flujo lógico para la adquisición y procesamiento de datos

Nota. Esquema del flujo de información. Elaboración propia.

Según Ocsaly (2022), para la implementación de este tipo de sistemas se pueden emplear principalmente dos arquitecturas de interfaz. La primera consiste en el uso del controlador CAN MCP2515 en conjunto con el transceptor físico TJA1050. Por otra parte, está la alternativa basada en el transceptor SN65HVD230, que opera a un nivel de voltaje de 3,3V y por eso resulta directamente compatible con los microcontroladores de la serie ESP32, sin necesitar conversores de nivel externos.

En la Tabla 3.1 se detallan las ventajas y desventajas técnicas de ambas configuraciones, facilitando la selección de la arquitectura más eficiente para este desarrollo.

Finalmente, se ha seleccionado la Opción B para la implementación del sistema de adquisición de datos del bus CAN, la cual integra el transceptor SN65HVD230 con el microcontrolador ESP32. Esta elección se fundamenta en las ventajas técnicas expuestas en la Tabla 3.1 y en las recomendaciones metodológicas propuestas por Ocsaly (2022).

3.1.2. Esquema de conexión ESP32 – OBD-II

Para establecer la comunicación entre el microcontrolador y la red CAN del vehículo, es necesario realizar una interconexión precisa entre el puerto OBD-II, el transceptor SN65HVD230 y el ESP32. El diseño del cableado debe garantizar la integridad de la señal diferencial y minimizar el ruido electromagnético.

Tabla 3.1
Comparativa técnica entre arquitecturas de adquisición CAN

Característica	Opción A: MCP2515 + TJA1050 + ATmega328p	Opción B: SN65HVD230 + ESP32
Controlador CAN	Externo (MCP2515 vía SPI).	Interno (TWAI nativo en ESP32).
Nivel de Tensión	5V (requiere conversores si el MCU es 3,3V).	3,3V (nativo, sin conversores).
Complejidad	Alta (más componentes, más cableado).	Baja (arquitectura minimalista).
Velocidad	Limitada por la interfaz SPI.	Alta (gestión directa por hardware).
Eficiencia Energética	Consumo mayor por múltiples integrados.	Muy eficiente (modo Standby avanzado).
Facilidad de Implementación	Requiere librerías externas de terceros.	Integración nativa con framework ESP-IDF.

Nota. Comparativa entre la arquitectura clásica de 8 bits y la arquitectura SoC de 32 bits de alto rendimiento. Elaboración propia.

3.1.3. Interfaz con el puerto OBD-II

La conexión al puerto de diagnóstico requiere la identificación exacta de los pines normalizados en el conector macho:

- **Pin 6 (CAN-H):** Línea de señal diferencial alta.
- **Pin 14 (CAN-L):** Línea de señal diferencial baja.
- **Pin 5 (Signal Ground):** Masa de referencia para la señal de datos. Se recomienda estrictamente el uso del Pin 5 en lugar del Pin 4 (masa de chasis), debido a que este último suele presentar un alto nivel de ruido eléctrico derivado de los componentes auxiliares del vehículo, como el alternador o los limpiaparabrisas.

3.1.4. Configuración del transceptor SN65HVD230

El transceptor actúa como el puente físico para interpretar los niveles de tensión del bus CAN. Los cables provenientes del puerto OBD-II deben conectarse a los terminales correspondientes del módulo (H, L y GND). Es imperativo que las líneas CAN-H y CAN-L mantengan una configuración de **par trenzado** a lo largo de todo el cableado, ya que esta técnica permite la cancelación de interferencias electromagnéticas externas.

3.1.5. Enlace lógico con el ESP32

La comunicación digital entre el transceptor y el microcontrolador se realiza mediante el protocolo TWAI (*Two-Wire Automotive Interface*) nativo del ESP32, siguiendo la configuración de pines descrita a continuación:

- **VCC (3,3V):** Alimentación del módulo transceptor.
- **GND:** Punto de masa común; es fundamental que el ESP32, el transceptor y la señal de referencia proveniente del vehículo compartan una misma referencia a tierra.
- **CTRX/CRX (Transmisión/Recepción):** Conexión a los pines de entrada/salida (IO4 e IO5), configurados mediante el *firmware* para la gestión de las tramas de datos.

3.1.6. Consideraciones de montaje y seguridad

Previo a la integración definitiva en el vehículo, se deben realizar las siguientes validaciones físicas:

- **Prueba de tracción:** Verificar la solidez de todas las soldaduras mediante una tracción mecánica suave para asegurar conexiones estables.
- **Alivio de tensión:** Aplicar fijaciones mecánicas (como abrazaderas o adhesivo termofusible) en los puntos de entrada de los cables hacia los módulos para prevenir daños por tensión mecánica accidental.

Y finalmente mostramos en la Tabla 3.2 las conexiones.

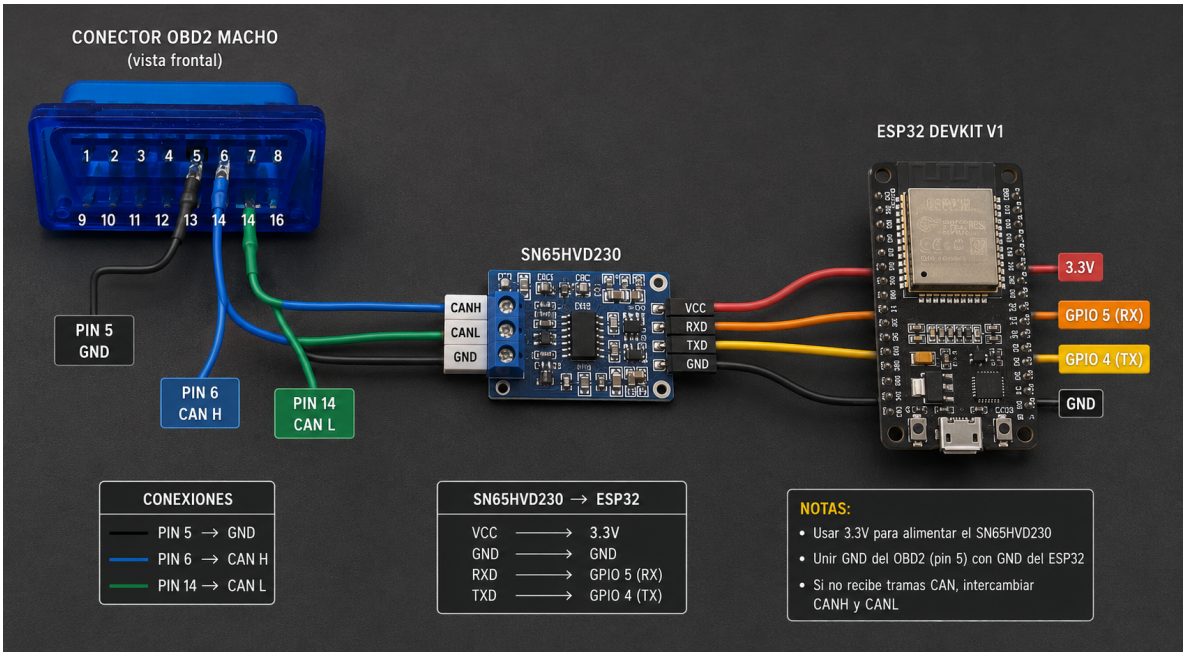
Tabla 3.2
Esquema de conexiones (Pinout) entre el puerto OBD-II, el transceptor y el microcontrolador

Origen	Destino	Función/Nota
OBD-II Pin 6	SN65HVD230 CAN-H	Señal diferencial alta.
OBD-II Pin 14	SN65HVD230 CAN-L	Señal diferencial baja.
OBD-II Pin 5	GND Común	Referencia a tierra (no usar Pin 4).
SN65HVD230 VCC	ESP32 3,3V	Alimentación lógica del módulo.
SN65HVD230 TX	ESP32 GPIO 4	Transmisión de datos (TX).
SN65HVD230 RX	ESP32 GPIO 5	Recepción de datos (RX).

*Nota.*Elaboración propia.

En la Figura 3.3 se muestra el esquema de conexión:

Figura 3.3
Esquema de conexión de los pines CAN-H y CAN-L al microcontrolador



Nota. Esquema de conexión entre los componentes de la Tabla 3.2. Elaboración propia asistida con IA (Gemini).

3.1.7. Configuración SLCAN y SavvyCAN para el análisis de datos

Una vez completada la implementación física del *hardware* de adquisición basado en el ESP32, el siguiente paso crítico es la interfaz lógica con un entorno de análisis. Debido a que las tramas CAN en vehículos como el Changan Honor 2023 operan a velocidades de entre 250 y 500 kbps, el uso de un monitor serial convencional resulta insuficiente, ya que la alta tasa de datos satura la visualización y dificulta la interpretación en tiempo real.

Para solventar esta limitación, se implementa el protocolo **SLCAN (Serial CAN)**, basado en el estándar *LAWICEL*. SLCAN permite encapsular tramas CAN dentro de un flujo de datos serie (UART), facilitando que el microcontrolador actúe como un adaptador CAN-USB transparente. Esta arquitectura es fundamental para el uso de **SavvyCAN**, una herramienta de código abierto especializada en ingeniería inversa, *fuzzing* y análisis de tráfico vehicular, la cual es compatible de forma nativa con el protocolo SLCAN (EVTV, 2026).

La Tabla 3.3 presenta una comparativa de las herramientas evaluadas para este estudio:

Tabla 3.3
Comparativa de herramientas de análisis para tráfico CAN

Herramienta	Licencia	Características principales
SavvyCAN	Open Source	Ingeniería inversa, *fuzzing*, soporte SLCAN.
Busmaster	Open Source	Simulación de nodos y redes.
webCAN	Propietario	Visualización web y gráficos rápidos.
CANtact App	Open Source	Sencillez y diagnóstico básico.

*Nota.*Elaboración propia.

Para la ejecución en entornos Linux (Ubuntu), es necesario configurar el puerto serie mediante *udev* o asignar permisos de lectura/escritura al usuario actual sobre el dispositivo (ej. `/dev/ttyUSB0`). SavvyCAN, al ser ejecutado en Ubuntu, permite la conexión mediante el *Serial/SLCAN Connection Manager*. La configuración recomendada es la siguiente:

1. **Selección de puerto:** Identificar el puerto serie donde está conectado el ESP32.
2. **Baudrate:** Configurar el *baudrate* del puerto serie a 115200 o superior (dependiendo del *firmware* del ESP32).
3. **Configuración SLCAN:** Habilitar el formato *LAWICEL* y establecer la velocidad del bus CAN (500 kbps para este caso específico).

La elección de SavvyCAN se justifica por su capacidad de manejar grandes volúmenes de datos con una sobrecarga mínima, permitiendo capturar y filtrar tramas de forma eficiente bajo el protocolo SLCAN (EVTV, 2026).

Para que SavvyCAN pueda interpretar estos datos en tiempo real, es necesario utilizar el protocolo slcan a través del puerto serial. Este protocolo emplea una estructura de trama específica con el formato txxxxxxxxxx, la cual permite al software decodificar y mostrar la actividad del bus CAN de manera continua.

Con este objetivo, se configuró el ESP32 mediante un código diseñado para leer los datos recibidos por el transceptor SN65HVD230 (aprovechando los pines dedicados para hardware CAN del microcontrolador) y retransmitirlos hacia SavvyCAN a través del puerto serial.

Para la integración del hardware se desarrolló un firmware en el entorno de desarrollo de Arduino utilizando el controlador nativo de dos cables para redes automotrices (TWAI). En la Figura 1 se detalla la lógica de programación orientada a la bidireccionalidad de datos entre el bus del vehículo y el software de análisis de diagnóstico.

Figura 3.4

Firmware bidireccional entre el driver TWAI del ESP32 y SavvyCAN.

```
#include <Arduino.h>
#include "driver/twai.h"

static const gpio_num_t CAN_TX_PIN = GPIO_NUM_4;
static const gpio_num_t CAN_RX_PIN = GPIO_NUM_5;

void handleSavvyCANTransmit(String cmd) {

    if (cmd.length() < 5 || cmd[0] != 't') return;
    twai_message_t tx_msg;
    memset(&tx_msg, 0, sizeof(tx_msg));
    tx_msg.identifier = strtol(cmd.substring(1, 4).c_str(), NULL, 16);
    tx_msg.data_length_code = cmd.substring(4, 5).toInt();
    if (tx_msg.data_length_code > 8) tx_msg.data_length_code = 8;
    for (int i = 0; i < tx_msg.data_length_code; i++) {
        tx_msg.data[i] = strtol(cmd.substring(5 + (i * 2), 7 + (i * 2)).c_str(), NULL,
    ↪ 16);
    }
    esp_err_t res = twai_transmit(&tx_msg, pdMS_TO_TICKS(5));
    if (res != ESP_OK) {
    }
}

void setup() {

    Serial.begin(115200);
    twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(CAN_TX_PIN, CAN_RX_PIN,
    ↪  TWAI_MODE_NORMAL);
    twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
    twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

    if (twai_driver_install(&g_config, &t_config, &f_config) == ESP_OK) {
        twai_start();
    }
}

void loop() {

    twai_message_t rx_msg;
    while (twai_receive(&rx_msg, 0) == ESP_OK) {
        if (!(rx_msg.flags & TWAI_MSG_FLAG_RTR)) {
            Serial.printf("t03Xd", rx_msg.identifier, rx_msg.data_length_code);
            for (int i = 0; i < rx_msg.data_length_code; i++) {
                Serial.printf("02X", rx_msg.data[i]);
            }
            Serial.print('\r');
        }
    }

    while (Serial.available()) {
        char c = Serial.read();
        if (c == '\r') {
            handleSavvyCANTransmit(inputBuffer);
            inputBuffer = "";
        } else if (c != '\n') {
            inputBuffer += c;
        }
    }
}

```

Nota. Algoritmo estructurado para la conversión asíncrona de tramas del bus de datos bajo el estándar *slcan*. Utiliza el retorno de carro (`\r`) con referencia de (Ocsaly, 2022).

Después de cargar el firmware al esp32 y montar el circuito planteado en la Figura 3.3 se procede a obtener tramas CAN, los siguientes son usados en el sistema operativo Ubuntu 22.04 para tener mas control de los comunicadores.

- Primero se descarga el archivo ejecutable para Ubuntu de SAvvyCAN de la pagina oficial de la misma (EVTV, 2026).
- Posteriormente se conceden permisos al archivo con el siguiente comando por la terminal ver Figura 3.5

Figura 3.5
Comando para los permisos de administrador a SavvyCAN.

```
$ sudo chmod +x SavvyCAN-x86_64.AppImage
$ sudo ./SavvyCAN-x86_64.AppImage
```

Nota. Secuencia de comandos ejecutada en terminal Bash sobre Ubuntu 22.04 para la instalación y apertura del software SAvvyCAN.

- Después se observa que el software se abre y queda listo para recibir las tramas.
- Mientras cargamos el código de la Figura 3.4 igual atravesar de la terminal al ESP32 con el comando de la Figura 3.6.

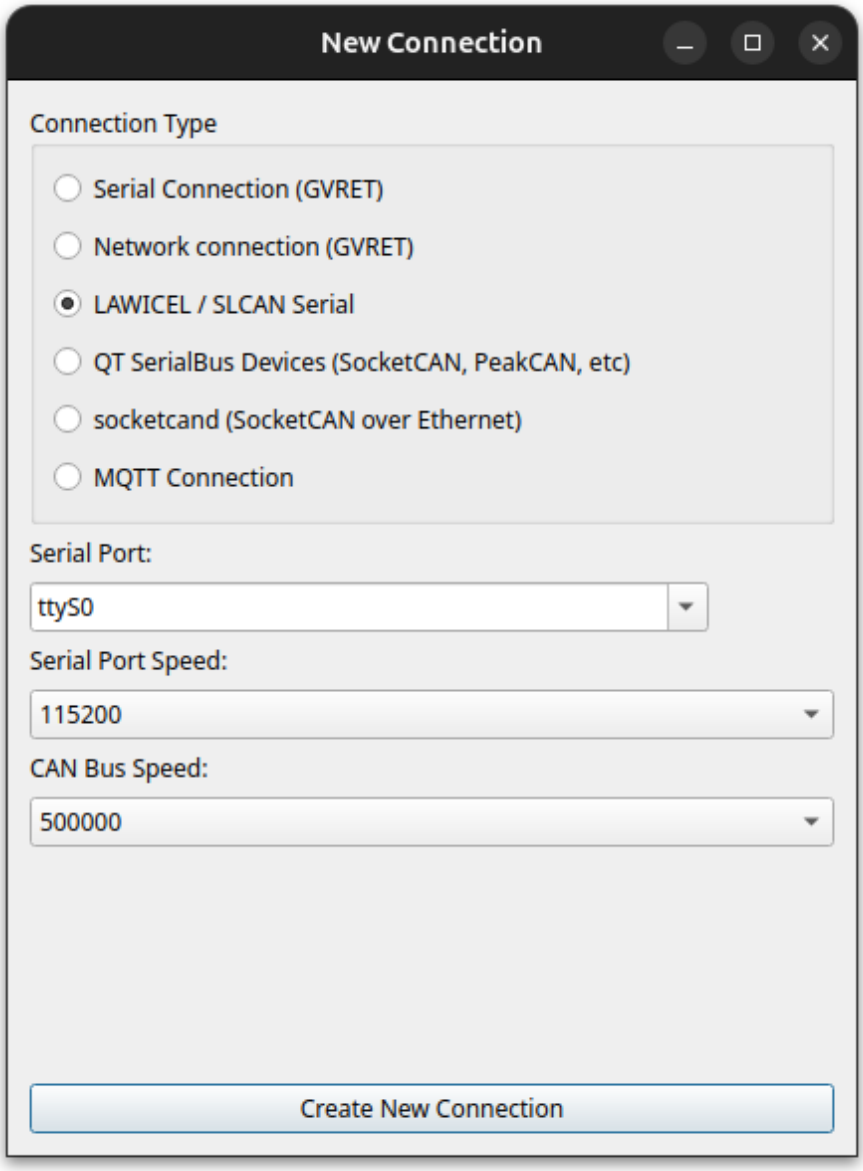
Figura 3.6
Comando para la carga del firmware al microcontrolador ESP32 mediante PlatformIO.

```
$ pio run --target upload --upload-port /dev/ttyUSB0
```

Nota. Comando ejecutado en terminal Bash sobre Ubuntu 22.04. El puerto `/dev/ttyUSB0` corresponde a la conexión serial del ESP32; puede variar según el sistema. Se requiere tener PlatformIO CLI instalado y el archivo `platformio.ini` configurado en el directorio del proyecto.

- A continuación se conecta al SavvyCAN con la siguiente configuración como se ve en la Figura 3.7.

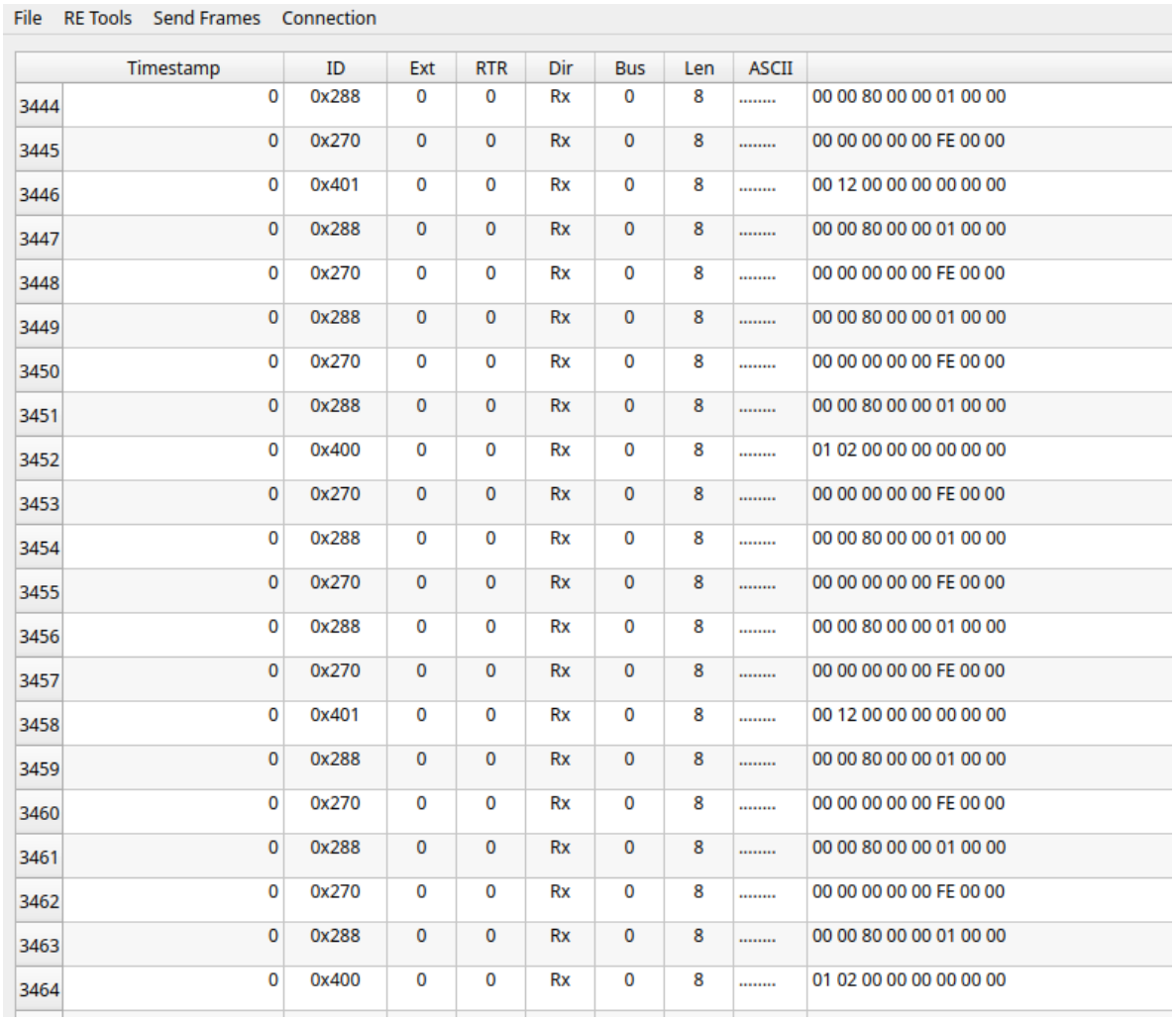
Figura 3.7
Configuración en el software SavvyCAN



Nota. Configuración de la conexión en SavvyCAN mediante el protocolo SLCAN, utilizando un ESP32 como puente serial hacia el bus CAN.

- Después de crear la conexión podremos notar como las tramas can se muestran en la pantalla como se ve en la Figura 3.8.

Figura 3.8
Conexion exitosa con el software SavvyCAN



	Timestamp	ID	Ext	RTR	Dir	Bus	Len	ASCII	
3444	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3445	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3446	0	0x401	0	0	Rx	0	8	.....	00 12 00 00 00 00 00 00
3447	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3448	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3449	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3450	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3451	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3452	0	0x400	0	0	Rx	0	8	.....	01 02 00 00 00 00 00 00
3453	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3454	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3455	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3456	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3457	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3458	0	0x401	0	0	Rx	0	8	.....	00 12 00 00 00 00 00 00
3459	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3460	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3461	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3462	0	0x270	0	0	Rx	0	8	.....	00 00 00 00 00 FE 00 00
3463	0	0x288	0	0	Rx	0	8	.....	00 00 80 00 00 01 00 00
3464	0	0x400	0	0	Rx	0	8	.....	01 02 00 00 00 00 00 00

Nota. Se detalla como los valores en hexadecimal están en cambio porque el vehículo al estar en contacto recibe muchas tramas CAN.

- Finalmente se demuestra exitosamente que si existen estas tramas CAN y en la siguiente parte veremos sus análisis y compresión del mismo.

3.1.8. Análisis de tramas capturadas

Una vez establecida la comunicación con el vehículo (cuya arquitectura puede compararse con la de un sistema robótico compuesto por múltiples subsistemas interconectados), se procedió a la captura de tramas del bus de datos.

Como se observa en la Figura 3.8, los valores obtenidos en bruto no resultan interpretables de forma directa, por lo que es necesario aplicar ingeniería inversa para

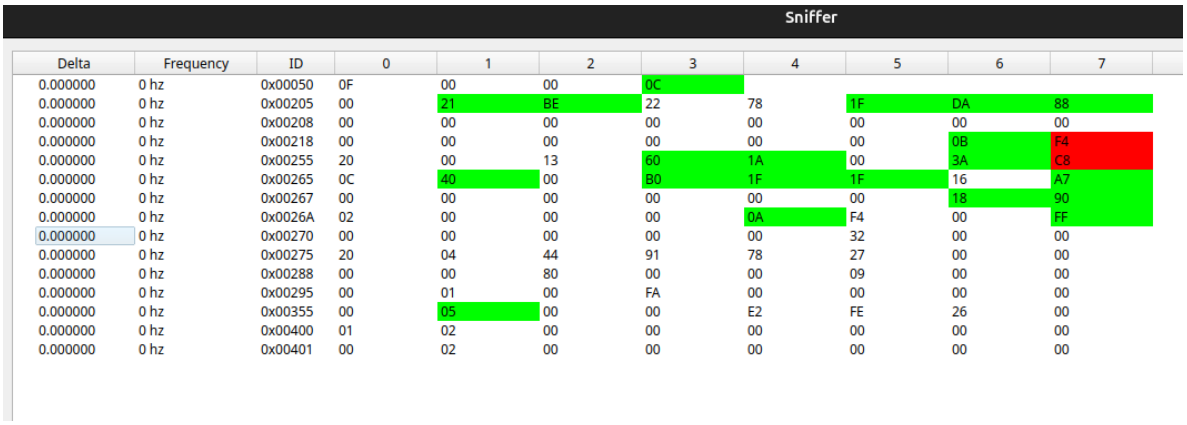
descifrar el significado de cada parámetro. Con base en los PIDs normalizados presentados en la Tabla 2.3, se identificaron en primera instancia aquellos parámetros conocidos como punto de partida para el análisis.

Identificación de IDs

Para este análisis, se utilizó la herramienta Sniffer integrada en SavvyCAN, la cual permite monitorear e identificar en tiempo real las variaciones en los datos del bus. Un ejemplo práctico de su aplicación consiste en la conmutación de las luces altas del vehículo. Como se observa en la Figura 3.9, la interfaz resalta dinámicamente los cambios en los bytes: los valores decrecientes se tiñen de color rojo, mientras que aquellos que se incrementan se muestran en verde, facilitando así la detección visual de las transiciones de datos.

Figura 3.9

Interfaz de la herramienta Sniffer en SavvyCAN para el monitoreo del bus



Nota. Monitoreo pasivo en tiempo real de los identificadores de CAN (CAN IDs). La interfaz destaca dinámicamente las variaciones de los bytes empleando un código de colores: verde para incrementos en los valores y rojo para decrementos.

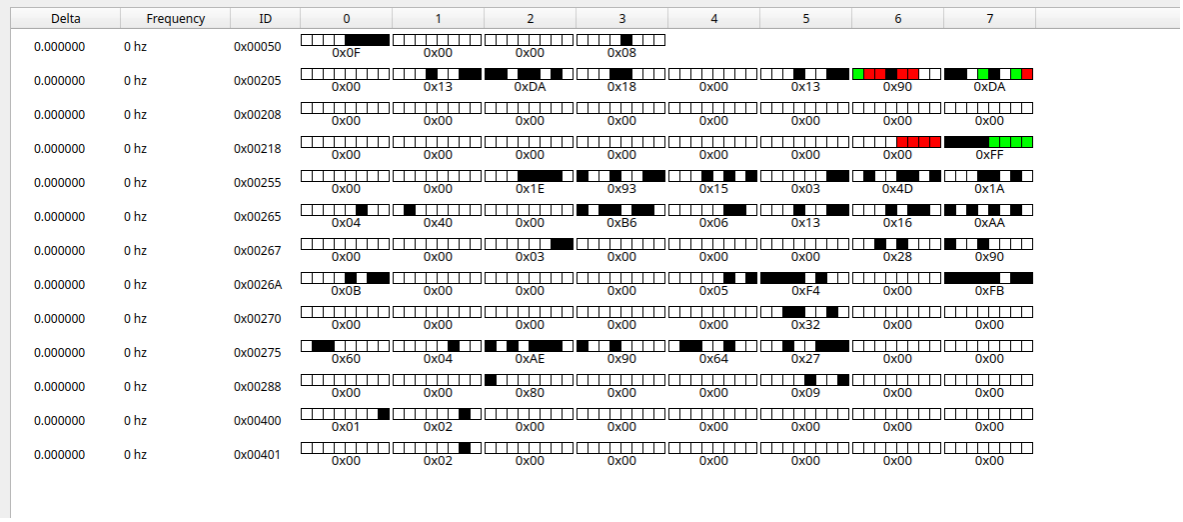
Este procedimiento es fundamental para el proceso de ingeniería inversa; por ejemplo, al activar las luces de parqueo, se examinan las modificaciones en el Sniffer con el objetivo de aislar e identificar el identificador de CAN (CAN ID) y la trama hexadecimal específica que comanda dicha función.

Posteriormente, se analizó el comportamiento de las revoluciones por minuto (RPM) del motor en SavvyCAN. Aunque el estándar OBD-II define conceptualmente el PID 0x0C para solicitar de forma activa las RPM (como se listó en la Tabla A1 del Apéndice A), mediante el escaneo pasivo (Sniffer) se identificó que el vehículo transmite de forma autónoma estos datos bajo el CAN ID como muestra su estructura en la Figura 2.11. Para validar este hallazgo, se capturaron las tramas en tiempo real bajo diferentes regímenes de operación (ver Figura 3.10).

Utilizando los datos crudos extraídos en un instante específico de la captura, se aplicaron las ecuaciones de ingeniería inversa correspondientes para decodificar los

Figura 3.10

Monitoreo del bus CAN a un régimen constante de 2000 RPM



Nota. Captura de datos en el software SavvyCAN con el motor retenido a 2000 RPM. Las variaciones cromáticas en los bytes del ID `0x00255` permiten aislar las señales dinámicas vinculadas a la velocidad angular del cigüeñal del motor frente al estado de ralentí.

bytes y comparar el resultado matemático con el valor reflejado simultáneamente en el tacómetro del tablero de instrumentos del vehículo.

La ecuación matemática estándar que rige esta conversión, considerando la resolución universal de 0,25 RPM/bit (factor de división por 4), se define en la Ecuación 3.1:

$$\text{RPM} = \frac{(\text{MSB} \times 256) + \text{LSB}}{4} \tag{3.1}$$

Donde MSB representa el Byte Más Significativo (*Byte Alto*) y LSB representa el Byte Menos Significativo (*Byte Bajo*).

Candidato 1: ID `0x00255` (Bytes 2 y 3)

A partir de la captura en vivo con el motor retenido, los bytes en estado activo para este ID presentaron los valores hexadecimales $\text{Byte}_2 = 1\text{E}_{16}$ y $\text{Byte}_3 = 93_{16}$. Convirtiendo dichos valores a base decimal se obtiene:

- $\text{MSB} = 1\text{E}_{16} = (1 \times 16^1) + (14 \times 16^0) = 30$
- $\text{LSB} = 93_{16} = (9 \times 16^1) + (3 \times 16^0) = 147$

Sustituyendo estos valores en la ecuación de ingeniería inversa:

$$\text{RPM}_{0255} = \frac{(30 \times 256) + 147}{4} = \frac{7680 + 147}{4} = \frac{7827}{4} = \mathbf{1956{,}75}\,\text{RPM} \tag{3.2}$$

Candidato 2: ID `0x00218` (Bytes 6 y 7)

Este ID mostró alta volatilidad durante la aceleración. Evaluando sus bytes finales, cuyos valores registrados fueron $\text{Byte}_6 = 00_{16}$ y $\text{Byte}_7 = \text{FF}_{16}$, se procede con la conversión decimal:

- $\text{MSB} = 00_{16} = 0$
- $\text{LSB} = \text{FF}_{16} = 255$

Sustituyendo en la ecuación fundamental:

$$\text{RPM}_{0218} = \frac{(0 \times 256) + 255}{4} = \frac{255}{4} = \mathbf{63{,}75}\,\text{RPM} \tag{3.3}$$

Candidato 3: ID `0x00275` (Bytes 1 y 2)

Este identificador, vinculado tentativamente a la posición del acelerador, marcó variaciones lineales leves. Tomando sus primeros bytes modificados, $\text{Byte}_1 = 00_{16}$ y $\text{Byte}_2 = \text{AE}_{16}$, se calcula su equivalente decimal:

- $\text{MSB} = 00_{16} = 0$
- $\text{LSB} = \text{AE}_{16} = (10 \times 16^1) + (14 \times 16^0) = 174$

Sustituyendo en la ecuación:

$$\text{RPM}_{0275} = \frac{(0 \times 256) + 174}{4} = \frac{174}{4} = \mathbf{43{,}50}\,\text{RPM} \tag{3.4}$$

Comparación de Resultados y Validación

En la Tabla 3.4 se organizan los datos calculados frente al valor real de referencia observado en el tacómetro analógico del vehículo (2000 RPM).

Tabla 3.4
Comparativa de IDs candidatos para la determinación de la variable de RPM

CAN ID	Datos Hexadecimales	Valor Calculado	Desviación Absoluta	¿Es el ID de RPM?
`0x00255`	`0x1E 0x93`	**1956,75 RPM**	**43,25 RPM (2,1%)**	**SÍ (Confirmado)**
`0x00218`	`0x00 0xFF`	63,75 RPM	1936,25 RPM	NO
`0x00275`	`0x00 0xAE`	43,50 RPM	1956,50 RPM	NO

Nota. Comparativa de los identificadores del bus CAN analizados bajo un régimen constante de operación frente al valor nominal del tacómetro. Elaboración propia.

A partir del análisis comparativo, se determina que el identificador del bus CAN perteneciente a las revoluciones por minuto es contundentemente el **ID `0x00255`**.

La resolución matemática arroja un valor de 1956,75 RPM, lo cual representa una precisión del 97,9% respecto al valor objetivo visualizado en el tablero de instrumentos.

El margen de desviación residual ($\approx 2{,}1\%$) no manifiesta un error en el modelado de la ecuación, sino que responde al comportamiento dinámico real del motor de combustión interna, cuyas micro-oscilaciones son captadas de forma pura por el bus de datos en milisegundos, mientras que la aguja física del cuadro de instrumentos actúa como un filtro mecánico amortiguado que suaviza la lectura visual para el conductor.

Los resultados demuestran que, mediante el análisis por ingeniería inversa de los identificadores (IDs) provistos por la herramienta SavvyCAN, es posible aislar e interpretar la estructura de las tramas CAN del vehículo.

3.1.9. Selección final de PIDs para monitoreo de consumo

A continuación, se abordará la consulta de Identificadores de Parámetro (PIDs). A diferencia de los identificadores de trama (IDs) del bus CAN, que definen pasivamente la procedencia o prioridad de un mensaje, los PIDs constituyen peticiones activas enviadas hacia la Unidad de Control Electrónico (ECU), la cual procesa la solicitud y devuelve la información bajo un esquema de pregunta-respuesta. Este mecanismo interactivo representa el principio operativo fundamental de la mayoría de los escáneres automotrices comerciales. Con el propósito de determinar qué variables de telemetría están disponibles en el vehículo de prueba, se desarrolló inicialmente un algoritmo general diseñado para explorar y decodificar el mapa de bits de los PIDs soportados.

El código fuente en C++ presentado en la Figura 3.11 implementa la inicialización, inyección y lectura de datos sobre el bus CAN del vehículo. Este algoritmo configura el periférico nativo TWAI (Espressif Systems, 2023a) a un régimen de 500 kbps para garantizar la sincronización temporal con la red automotriz a través del transceptor físico (Texas Instruments, 2015).

El procesamiento de datos se sintetiza en tres etapas operativas esenciales:

- **Inicialización:** En la rutina `setup()`, se asignan los pines de control (GPIO 4 y 5) y se establece un filtro de aceptación total para habilitar la escucha completa del tráfico en el bus.
- **Transmisión Funcional:** La función `solicitarPIDsSoportados()` inyecta una trama de diagnóstico única (*Single Frame*) bajo el identificador estándar `0x7DF` conforme a la norma ISO 15765-4 (ISO, 2021). La solicitud invoca al Servicio 01 y al PID `0x00`, rellenando los bytes restantes con ceros (*padding*) para mantener la simetría obligatoria de 8 bytes.
- **Filtrado Lógico:** En el bucle principal `loop()`, el sistema evalúa las tramas entrantes en el búfer. El algoritmo descarta el tráfico secundario y aísla exclusivamente las respuestas de la ECU de motor (`0x7E8`) que validen el servicio (`0x41 0x00`), extrayendo finalmente los 4 bytes del mapa de bits.

Figura 3.11

Algoritmo en C++ para la detección activa y decodificación del mapa de bits de PIDs soportados en el bus CAN.

```
#include "driver/twai.h"
#define CAN_TX_PIN GPIO_NUM_5
#define CAN_RX_PIN GPIO_NUM_4

void setup() {
    Serial.begin(115200);
    while(!Serial);
    Serial.println("Iniciando Escaner de PIDs OBD-II...");
    twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(CAN_TX_PIN, CAN_RX_PIN,
    ↪  TWAI_MODE_NORMAL);
    twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
    twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();
    if (twai_driver_install(&g_config, &t_config, &f_config) == ESP_OK) {
        Serial.println("Driver TWAI instalado con exito.");
    } else {
        while (1);
    }
    if (twai_start() == ESP_OK) {
        Serial.println("Driver TWAI iniciado.");
    } else {
        while (1);
    }
    delay(2000);
    solicitarPIDsSoportados();
}
void loop() {
    twai_message_t rx_msg;
    if (twai_receive(&rx_msg, pdMS_TO_TICKS(1000)) == ESP_OK) {
        // Filtrar respuestas OBD2 estandar (0x7E8) y Servicio 01
        if (rx_msg.identifier == 0x7E8 && rx_msg.data[1] == 0x41 && rx_msg.data[2] == 0
    ↪ x00) {
            Serial.print("Respuesta Recibida de la ECU ID: 0x");
            Serial.println(rx_msg.identifier, HEX);
            Serial.print("Mapa de bits de PIDs (Hex): ");
            for(int i = 3; i < 7; i++) {
                if(rx_msg.data[i] < 0x10) Serial.print("0");
                Serial.print(rx_msg.data[i], HEX);
                Serial.print(" ");
            }
            Serial.println("\n----------------------------------------");}}
}
void solicitarPIDsSoportados() {
    twai_message_t tx_msg;
    tx_msg.identifier = 0x7DF;
    tx_msg.extd = 0;
    tx_msg.data_length_code = 8;
    tx_msg.data[0] = 0x02;
    tx_msg.data[1] = 0x01;
    tx_msg.data[2] = 0x00;
    for(int i=3; i<8; i++) tx_msg.data[i] = 0x00;
    if (twai_transmit(&tx_msg, pdMS_TO_TICKS(1000)) == ESP_OK) {
        Serial.println("Solicitud PID 0x00 enviada correctamente.");
    }}

```

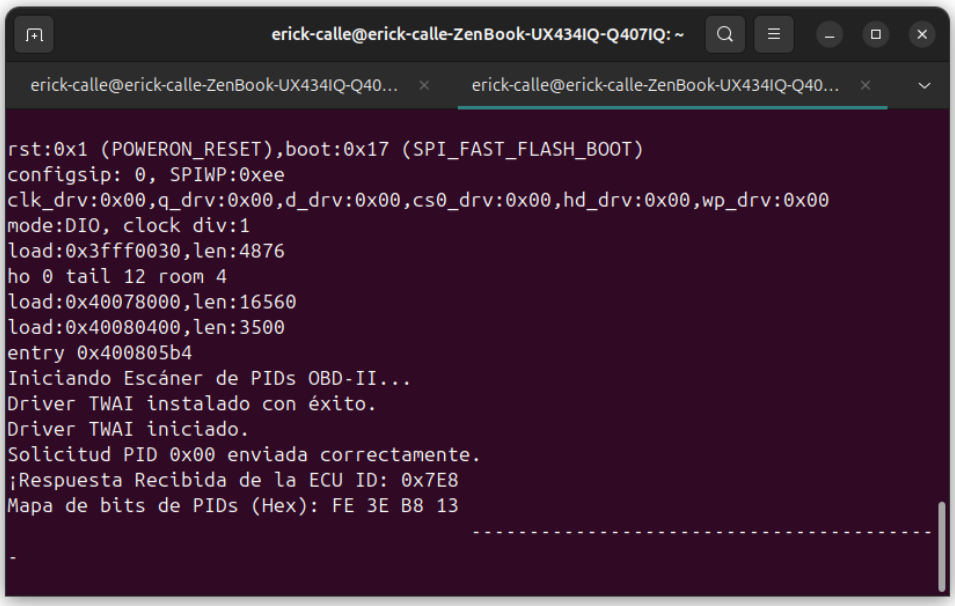
Nota. Algoritmo estructurado para la inicialización y el control del periférico de comunicación automotriz mediante la inyección activa de tramas de diagnóstico funcional (ID `0x7DF`). Desarrollado a partir de las especificaciones de bajo nivel para el manejo del buffer y sincronización de bits con referencias de (Espressif Systems, 2023b) y (SAE International, 2014).

Resultados Obtenidos

Tras compilar el código y conectar el sistema al puerto OBD-II del Changan Honor con el contacto en posición *ON*, el monitor serie arrojó la siguiente salida a 115200 baudios ver Figura 3.12:

Figura 3.12

Captura de pantalla del monitor serie durante la ejecución del escáner de PIDs OBD-II (Changan Honor).



Nota. Registro de depuración en tiempo real, tomado sobre el Changan Honor, que muestra la inicialización correcta del controlador TWAI, el envío de la solicitud funcional (PID 0x00) y la respuesta hexadecimal (`FE 3E B8 13`) emitida por la unidad de control del motor (ECU).

Interpretación de Resultados

La respuesta del bus CAN indica que la comunicación bidireccional fue exitosa. El identificador `0x7E8` confirma, bajo la arquitectura de segmentación de IDs de diagnóstico físico ISO (2021), que la trama proviene directamente del módulo de control del motor (ECM).

Para determinar la disponibilidad de los comandos, el mapa de bits devuelto en formato hexadecimal, `FE 3E B8 13`, debe ser descompuesto en su representación binaria de 32 bits. Siguiendo el estándar SAE J1979 (SAE International, 2014), cada bit se evalúa de izquierda a derecha (desde el bit más significativo hasta el menos significativo), donde un estado lógico de 1 indica que el PID está soportado por la ECU, y un estado de 0 indica su ausencia.

A continuación, se detalla la conversión de los cuatro bytes y el mapeo completo de los bits correspondientes al primer bloque de diagnóstico (PIDs del 0x01 al 0x20):

- **Byte 1 (`FE`):** `1111 1110` → Soportados PIDs del 0x01 al 0x07; el PID 0x08 está ausente.
- **Byte 2 (`3E`):** `0011 1110` → Ausentes PIDs 0x09 y 0x0A; soportados del 0x0B al 0x0F; el PID 0x10 está ausente.
- **Byte 3 (`B8`):** `1011 1000` → Soportados PIDs 0x11, 0x13, 0x14 y 0x15; los demás están ausentes.
- **Byte 4 (`13`):** `0001 0011` → Soportados PIDs 0x1C, 0x1F y 0x20; los demás están ausentes.

La Tabla 3.5 describe la función específica y el impacto analítico de las variables críticas recuperadas en este mapa de bits para la caracterización dinámica del tren motriz.

Tabla 3.5
Mapeo binario de la respuesta hexadecimal FE 3E B8 13 y descripción de PIDs

Byte	Hex	Binario	PID Asociado	Descripción Técnica Resumida
Byte 1	`FE`	`11111110`	PID 0x01 - 0x08	Estado de monitores y códigos de falla (DTC).
			PID 0x0B (1)	**MAP:** Presión del múltiple de admisión; clave para la densidad del aire.
			PID 0x0C (1)	**RPM:** Régimen de giro del motor y ciclos de llenado.
Byte 2	`3E`	`00111110`	**PID 0x0D (1)**	**VSS:** Velocidad del vehículo; convierte el consumo a L/km.
			PID 0x0F (1)	**IAT:** Temperatura de admisión; corrige la densidad por factor térmico.
			`PID 0x10 (0)`	**MAF:** Sensor de flujo de masa de aire (Ausente).
Byte 3	`B8`	`10111000`	PID 0x11 - 0x18	Posición del acelerador y sensores de oxígeno activos.
Byte 4	`13`	`00010011`	PID 0x19 - 0x20	Tiempo de operación y habilitación del bloque 0x21-0x40.

Nota. El bit final del Byte 4 (`PID 0x20 = 1`) confirma que la ECU admite la consulta del siguiente rango de parámetros (PIDs 0x21 a 0x40) para expandir la adquisición de datos.

Conclusión de la prueba

La ausencia absoluta del sensor MAF (`PID 0x10 = 0`), combinada con la disponibilidad simultánea y mandatoria de los sensores MAP, RPM e IAT, revela un

aspecto fundamental de la arquitectura de la ECU: el sistema de inyección electrónica del vehículo no mide el flujo de aire de forma masiva directa, sino que calcula el caudal utilizando el método indirecto de Densidad de Velocidad (*Speed Density*).

Este hallazgo define y valida el desarrollo matemático del sistema embebido. Al no contar con el PID 0x10, el microcontrolador deberá ser programado para resolver la ecuación de estado de los gases ideales combinada con la eficiencia volumétrica del motor en tiempo real. Al disponer de los datos de presión (MAP), temperatura (IAT) y velocidad angular (RPM), se confirma que el hardware desarrollado tiene acceso a todas las variables de estado requeridas para modelar el flujo de aire y estimar el consumo instantáneo de combustible de manera matemática y precisa.

El mismo procedimiento de escaneo se repitió sobre el segundo vehículo de prueba (Nissan Vanette), confirmando además que este sí soporta el PID 0x10 (MAF), tal como se resume en la Tabla 4.8 del Capítulo 4. El listado completo y decodificado de los PIDs soportados por ambos vehículos se presenta en el Apéndice O.

3.2. Selección y justificación de componentes

En esta sección se describen los componentes seleccionados para el sistema, priorizando su disponibilidad local en la ciudad de La Paz y su alineación con los objetivos del proyecto. La elección se basa en criterios de compatibilidad eléctrica, facilidad de integración, costo y rendimiento, con énfasis en soluciones robustas y accesibles.

3.2.1. Interfaz OBD-II y controlador CAN

Para leer datos del puerto OBD-II del vehículo, se requiere una conexión estable al bus CAN (Controller Area Network). Los componentes clave son:

1. **Conector OBD-II**: Se utiliza un conector extraído de un escáner defectuoso para garantizar fiabilidad. Alternativamente, dos cables directos a las líneas CANH y CANL podrían emplearse, pero presentan riesgos de falsos contactos o conexiones inestables.Este posteriormente sera remplazado para una versión final del sistema.
2. **Transceptores CAN**: Para convertir las señales diferenciales CANH/CANL en líneas digitales RX/TX compatibles con el microcontrolador, se evaluaron dos opciones:
   - TJA1050 : Actúa como transceptor bidireccional, convirtiendo señales eléctricas del bus CAN en RX/TX como muestra en la Tabla B1 del Apéndice B. Es de bajo costo y disponible. Y su rango de voltajes de señal nos da el primer obstáculo como se ve en la Tabla B2 del Apéndice B al no ser de rango 3,3 V.
   - SN65HVD230 (ver Apéndice C): Similar al anterior, pero optimizado para 3,3 V; ideal, entonces, para microcontroladores de bajo voltaje.

Se seleccionó el SN65HVD230 por su stock local, popularidad en aplicaciones automotrices y compatibilidad con 3,3 V.

3. **Controlador CAN (Periférico TWAI)**: Se seleccionó el ESP32 como unidad central debido a que integra periféricamente un controlador CAN a nivel de silicio, una característica ausente en la mayoría de microcontroladores convencionales de desarrollo ver Apéndice D. Este módulo nativo asume de forma autónoma la gestión de la capa de enlace de datos: empaqueta el payload, ejecuta el filtrado de mensajes por hardware para no saturar la CPU y administra la corrección de errores.

A todo esto se descartó el uso del escáner ELM327 de código abierto por su mayor costo y dependencia de conectividad inalámbrica (Bluetooth/Wi-Fi), que requeriría un ESP32 o Raspberry Pi con Python/MicroPython, complicando la integración.

3.2.2. Microcontrolador del Sistema

A diferencia de los esquemas de procesamiento distribuido, el núcleo de este sistema centraliza la adquisición de datos, el modelado matemático y la conectividad inalámbrica en un único circuito integrado, optimizando el espacio físico y suprimiendo la latencia de comunicación inter-procesador:

- **ESP32**: Seleccionado como la unidad central de control debido a su arquitectura de alto rendimiento y su versatilidad periférica. El dispositivo asume de forma nativa dos funciones críticas para el proyecto: la gestión del bus CAN automotriz mediante su controlador interno TWAI (*Two-Wire Automotive Interface*) y la transmisión inalámbrica de la telemetría hacia el servidor local a través de su pila de protocolos Wi-Fi integrada.

3.2.3. Periféricos de visualización

Se evaluaron varias pantallas para mostrar datos en tiempo real, priorizando tamaño, facilidad de conexión (I^2C o paralelo) y legibilidad para conductores:

- **Pantalla LCD 20x4 o 16x2** : Ofrece el tamaño óptimo para datos vehiculares, mejorando la usabilidad sin sacrificar simplicidad.
- **Pantalla OLED**: Se selecciono esta por ser compacta, conexión I^2C simple y compatible con 3,3 V según el Apéndice E.

3.2.4. Lenguaje de programación

El lenguaje de programación seleccionado es C++, por ser mas optimo para su programación y flexibilidad en entornos embebidos.

Finalmente en la Tabla 3.6 se muestran los componentes seleccionados :

Tabla 3.6
Componentes seleccionados y justificación técnica del sistema unificado

Componente	Selección	Ventajas y justificación en el proyecto
Interfaz OBD-II	Conector OBD-II estándar	Permite una conexión mecánica y eléctrica estable directamente con el puerto de diagnóstico del vehículo.
Microcontrolador y Controlador CAN	ESP32	Centraliza el procesamiento, el modelado matemático de consumo y la conectividad Wi-Fi. Su periférico nativo TWAI elimina la necesidad de chips controladores externos como el MCP2515.
Transceptor CAN	SN65HVD230	Opera nativamente a 3,3 V, acoplándose de forma segura a los pines del ESP32. Traduce las señales lógicas a los niveles diferenciales del bus CAN automotriz.
Pantalla de visualización	OLED	Ofrece excelente legibilidad en entornos con alta iluminación solar y mejor adaptación a ser 3,3 V.
Lenguaje de programación	C++ (Entorno Arduino)	Cuenta con librerías maduras y de bajo nivel para el manejo del periférico TWAI, optimizando el tiempo de desarrollo y la estabilidad del sistema embebido.

Nota. Arquitectura de hardware optimizada y simplificada a partir de la centralización de procesos en un único núcleo de ejecución.

3.3. Diseño de hardware

3.3.1. Módulo de Interfaz CAN Bus (SN65HVD230 + ESP32)

El transceptor SN65HVD230 de Texas Instruments fue seleccionado como interfaz física entre el microcontrolador ESP32 y el bus CAN del vehículo. A diferencia de soluciones basadas en el MCP2515, este transceptor no requiere controlador CAN externo ni comunicación SPI, dado que el ESP32 integra nativamente el periférico TWAI® (*Two-Wire Automotive Interface*), compatible con el estándar ISO 11898-1 (CAN 2.0B). La conexión entre ambos dispositivos se reduce a dos líneas lógicas: TX y RX entre el ESP32 y el transceptor, y dos líneas diferenciales CANH y CANL hacia el bus del vehículo.

Configuración del Periférico TWAI® del ESP32

La velocidad del bus CAN automotriz opera típicamente a 500 kbps bajo el estándar ISO 11898-1. La configuración del periférico TWAI® se realiza directamente mediante el driver oficial de Espressif en el framework ESP-IDF, sin necesidad de calcular

registros de temporización manualmente. La inicialización del bus se estructura en tres parámetros fundamentales, tal como se muestra en la Figura 3.13:

Figura 3.13
Inicialización del periférico TWAI® del ESP32 a 500 kbps.

```
twai_general_config_t g_config =
TWAI_GENERAL_CONFIG_DEFAULT(GPIO_NUM_4, GPIO_NUM_5, TWAI_MODE_NORMAL);
twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

twai_driver_install(&g_config, &t_config, &f_config);
twai_start();

```

Nota. La macro `TWAI_TIMING_CONFIG_500KBITS()` configura automáticamente los parámetros de bit-timing (TQ, PropSeg, PhaseSeg1, PhaseSeg2 y SJW) para operar a 500 kbps sobre el reloj interno del ESP32 a 80 MHz, eliminando el cálculo manual de registros CNF.

Filtrado de Tramas CAN por Identificador

El periférico TWAI® incorpora un filtro de aceptación por hardware configurable mediante una máscara de 32 bits, equivalente funcional a los registros RXF y RXM del MCP2515. Para recibir únicamente la respuesta de la ECU principal en consultas OBD-II (ID `0x7E8`), se configura el filtro de la siguiente manera (ver Figura 3.14):

Figura 3.14
Configuración del filtro de aceptación para recepción exclusiva de respuestas OBD-II.

```
twai_filter_config_t f_config = {
    .acceptance_code = (0x7E8 << 21), // ID aceptado: respuesta ECU
    .acceptance_mask = ~(0x7FF << 21), // Mascara exacta de 11 bits
    .single_filter   = true
};

```

Nota. El desplazamiento de 21 bits se debe al formato interno del registro de filtro del TWAI®, que ubica el identificador de 11 bits en los bits más significativos de la palabra de 32 bits. La máscara `0x7FF` garantiza coincidencia exacta del identificador, descartando cualquier otra trama presente en el bus y reduciendo la carga de procesamiento del sistema.

3.3.2. Módulo de Visualización (OLED I2C)

Para la visualización local de parámetros se empleó una pantalla OLED de 128×64 píxeles controlada mediante el protocolo I2C, conectada directamente al ESP32. La dirección esclava del dispositivo es `0x3C`, configurable a `0x3D` mediante el puente de soldadura en la placa del módulo. La gestión del display se realizó mediante la biblioteca `Adafruit_SSD1306`, que abstrae el protocolo de comunicación y provee funciones de renderizado de texto y gráficos primitivos.

Distribución de la información en pantalla

A diferencia de un esquema de múltiples pantallas navegables, el firmware mantiene una única pantalla fija que se redibuja por completo cada `OLED_REFRESH_PERIOD_MS` (400 ms por defecto), priorizando en todo momento los datos que más se consultan al conducir: el consumo y el costo acumulado del viaje. La distribución de la información en esa pantalla se detalla en la Tabla 3.7.

Tabla 3.7
Distribución de la información en la pantalla OLED

Zona	Contenido mostrado
Línea superior	RPM del motor (PID `0x0C`) y velocidad del vehículo (PID `0x0D`), en texto pequeño.
Centro (texto grande)	Consumo instantáneo: L/100 km si el vehículo se mueve (velocidad > 2 km/h), o L/h si está detenido o en ralentí, para no dividir entre una velocidad cercana a cero.
Centro (texto grande)	Costo acumulado del viaje, en bolivianos, calculado a partir del combustible ya consumido (`trip_fuel_L` $\times$ `FUEL_PRICE_BS_PER_L`), no de la tasa instantánea.
Línea inferior 1	Distancia y combustible acumulados del viaje.
Línea inferior 2	Estado de la comunicación CAN (`CAN:OK`/`CAN:-`) y estado del panel web: dirección IP del punto de acceso si está activo, o `srv:-` en caso contrario.

Nota. No hay navegación entre pantallas ni pulsador físico asociado a la OLED: toda la información relevante para conducir se muestra simultáneamente en esta única vista, y el detalle adicional (MAP, IAT, eficiencia volumétrica, flujo de aire y combustible calculados, etc.) queda disponible en el panel web y en el registro de la tarjeta microSD (Tabla 3.8).

3.3.3. Módulo de Registro en Memoria (SSD)

Para el almacenamiento persistente de los datos adquiridos se utilizó una memoria de estado sólido conectada al ESP32 mediante el protocolo SPI. Los registros se almacenan en formato CSV con marca de tiempo, permitiendo su posterior análisis en herramientas de ofimática o procesamiento de datos. Cada sesión genera un archivo independiente identificado por número de sesión correlativo, preservando el historial completo de adquisiciones.

Estructura del Archivo de Registro

Cada fila del archivo CSV registra una muestra completa de los parámetros activos en ese instante, con la estructura definida en la Tabla 3.8.

Tabla 3.8
Estructura real del registro CSV escrito por `sdLoggerTask()` (`sd_logger.cpp`)

Campo	Tipo	Descripción
`uptime_ms`	uint32	Tiempo transcurrido desde el arranque del sistema, en milisegundos (`millis()`).
`rpm`	uint16	Revoluciones por minuto del motor (PID `0x0C`).
`speed_kmh`	float	Velocidad del vehículo, en km/h (PID `0x0D`).
`map_kpa`	float	Presión absoluta del colector de admisión, en kPa (PID `0x0B`).
`iat_c`	float	Temperatura del aire de admisión, en °C (PID `0x0F`).
`coolant_c`	float	Temperatura del líquido refrigerante del motor, en °C (PID `0x05`).
`ve_pct`	float	Eficiencia volumétrica interpolada de `kVeTable` para el régimen actual, en %.
`maf_gs`	float	Flujo másico de aire estimado por el modelo Speed-Density, en g/s.
`fuel_gs`	float	Flujo másico de combustible calculado, en g/s.
`instant_Lh`	float	Consumo instantáneo en L/h (válido en ralentí o vehículo detenido).
`instant_L100km`	float	Consumo instantáneo en L/100 km (válido con el vehículo en movimiento).
`trip_km`	float	Distancia acumulada del viaje, en km.
`trip_fuel_L`	float	Combustible acumulado del viaje, en L.
`trip_avg_L100km`	float	Consumo promedio acumulado del viaje, en L/100 km.
`can_status`	uint8	Estado de la comunicación CAN (`0`=INIT, `1`=OK, `2`=SIN_RESPUESTA, `3`=ERROR_BUS, `4`=BUS_OFF).
`trip_cost_bs`	float	Costo acumulado del viaje, en bolivianos (`trip_fuel_L` × `FUEL_PRICE_BS_PER_L`).
`baro_kpa`	float	Presión barométrica de referencia, en kPa (PID `0x33` real, o estimación del modelo ISA si no está soportado).
`baro_is_estimated`	bool	`1` si `baro_kpa` proviene de la estimación ISA; `0` si es una lectura real por PID `0x33`.

Nota. Fila exacta escrita cada `SD_LOG_INTERVAL_MS` (1000 ms por defecto) mientras hay una lectura CAN válida; el archivo se abre una sola vez por sesión y cada fila se escribe en modo *append* con `flush()` inmediato, de forma que una pérdida de energía no descarta filas ya escritas. Los ajustes de combustible STFT/LTFT (PIDs `0x06`/`0x07`) se usan internamente para corregir el cálculo (Capítulo 3, lazo cerrado) pero no se registran como columnas propias de este CSV.

3.3.4. Módulo de Comunicación Inalámbrica y Servidor Web

El mismo ESP32 que gestiona la adquisición CAN opera simultáneamente como punto de acceso WiFi (*Access Point*), exponiendo un servidor web embebido para la visualización remota de parámetros en tiempo real desde cualquier dispositivo conectado a la red local generada. No se requiere conexión a internet ni infraestructura de red externa.

Arquitectura del Servidor Web Embebido

A diferencia del modelo tradicional de petición-respuesta de HTTP, en el que el cliente debe solicitar cada actualización de forma individual, el protocolo WebSocket (Fette and Melnikov, 2011) establece un canal de comunicación bidireccional y persistente sobre una única conexión TCP. Una vez completado el saludo inicial (*handshake*), que sí se realiza sobre HTTP, el servidor puede enviar datos al cliente en cualquier momento sin que este los solicite explícitamente, con un sobrecosto de encabezado mucho menor que el de una petición HTTP completa repetida. Esta característica lo hace especialmente adecuado para la actualización en tiempo real del panel web del presente sistema, donde el ESP32-S3 necesita empujar el estado del vehículo al navegador de forma periódica sin que este tenga que preguntar repetidamente si hay datos nuevos.

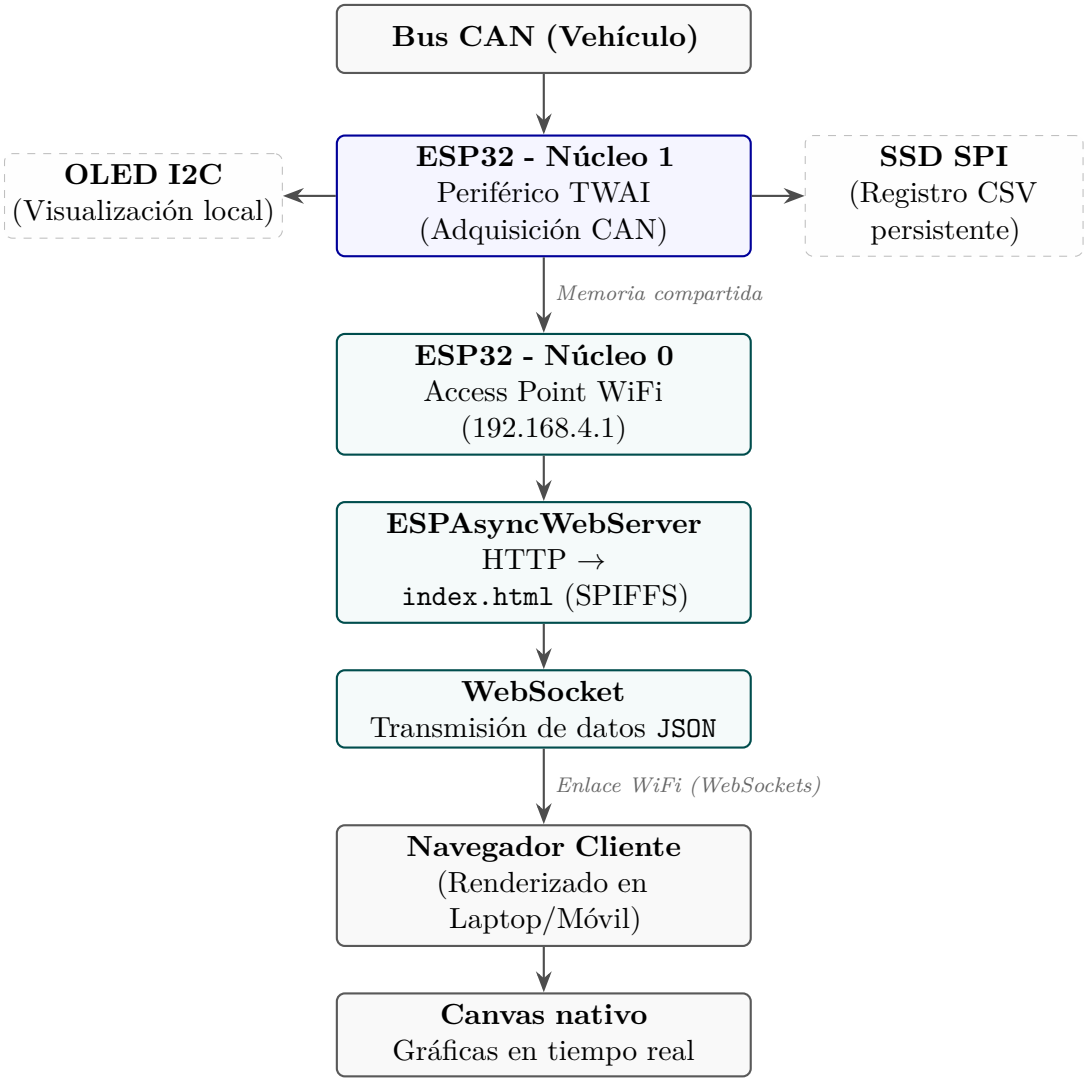
El servidor web se implementó con la biblioteca `ESPAsyncWebServer`, que permite atender peticiones HTTP y mantener conexiones WebSocket de forma asíncrona sin bloquear el bucle principal de adquisición CAN. La interfaz gráfica HTML/CSS/JS se almacena en la partición SPIFFS del ESP32 y se sirve estáticamente al cliente. La actualización de datos en tiempo real se realiza mediante WebSocket, evitando el *polling* HTTP y reduciendo la latencia de visualización. El flujo de datos completo se ilustra en la Figura 3.15.

Formato de Trama JSON por WebSocket

Cada 500 ms el ESP32 serializa los parámetros activos en un objeto JSON y lo transmite a todos los clientes conectados mediante difusión (*broadcast*) WebSocket con la estructura en código como se muestra en la Figura 3.16:

Figura 3.15

Flujo de datos desde el bus CAN hasta el cliente web



Nota. Todos los módulos operan sobre el mismo microcontrolador ESP32, aprovechando el núcleo dual del procesador: el núcleo 0 gestiona la comunicación WiFi y el servidor web, mientras que el núcleo 1 ejecuta la adquisición CAN y la escritura en SSD.

Figura 3.16
Estructura de la trama JSON transmitida por WebSocket al cliente web.

```
// Serializacion en el ESP32
String buildJSON(OBDData& d) {
    return "{\"rpm\":"      + String(d.rpm, 0)       +
        ",\"vel\":"      + String(d.velocidad)     +
        ",\"temp\":"     + String(d.temp_ref)      +
        ",\"carga\":"    + String(d.carga, 1)      +
        ",\"maf\":"      + String(d.maf, 2)        +
        ",\"stft\":"     + String(d.stft_b1, 1)    +
        ",\"ltft\":"     + String(d.ltft_b1, 1)    +
        ",\"ts\":"       + String(millis())        + "}";
}

// Envio broadcast a todos los clientes WebSocket
ws.textAll(buildJSON(currentData));

```

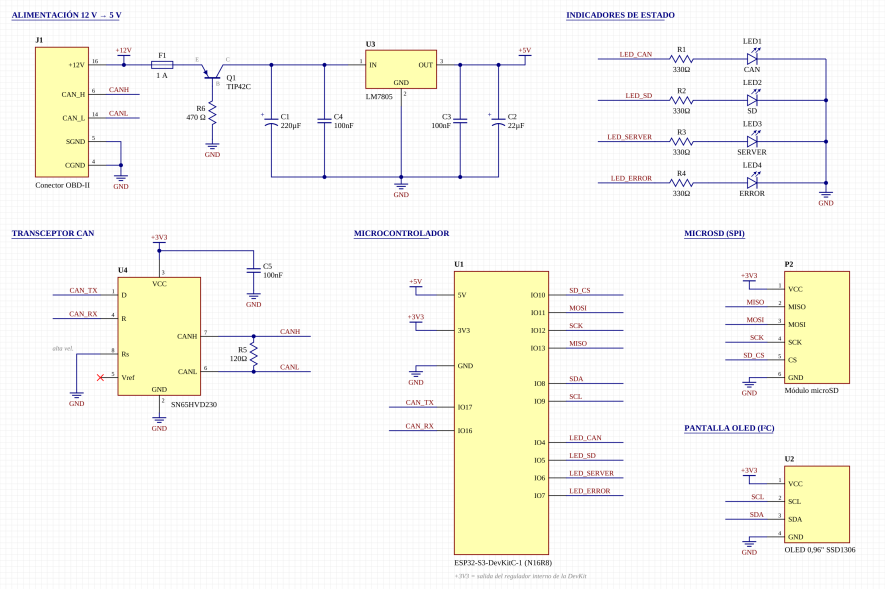
Nota. El cliente web recibe el JSON y actualiza las gráficas mediante Chart.js sin recargar la página, logrando una visualización fluida con latencia inferior a 100 ms en condiciones normales de red local.

3.3.5. Placa preliminar (Altium Designer)

Antes de proceder con el diseño de la placa final descrito en el resto de esta sección, se desarrolló en Altium Designer una placa preliminar que replica el mismo circuito ya validado sobre protoboard (Capítulo 4, Sección 4.1.3), con el fin de contar con una versión física más robusta que las conexiones sueltas de una protoboard para las pruebas de campo. A diferencia de la placa final, esta versión preliminar resuelve la alimentación con un regulador lineal simple (7805 más un transistor de paso TIP42C en configuración de refuerzo de corriente) en lugar de la conversión conmutada de dos rieles, y conecta el transceptor CAN, la pantalla OLED y el módulo microSD –ya validados en la etapa de protoboard– mediante conectores de pines en vez de integrarlos directamente en la placa. El esquemático completo se muestra en la Figura 3.17.

Figura 3.17

Esquemático completo de la placa preliminar.

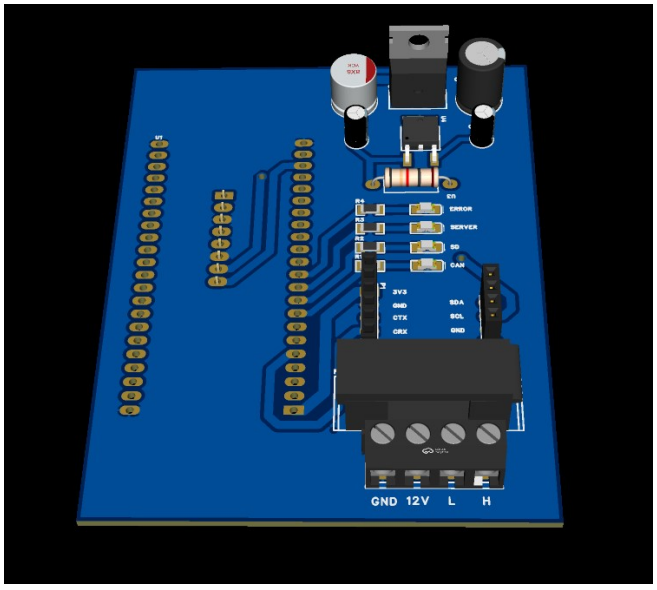


Nota. Desarrollado en Altium Designer. El costo detallado de cada componente se presenta en el Capítulo 5 (Tabla 5.7).

La Figura 3.18 muestra la vista 3D generada por Altium Designer a partir de ese esquemático y del enrutado de la placa, previo al envío a fabricar.

Figura 3.18

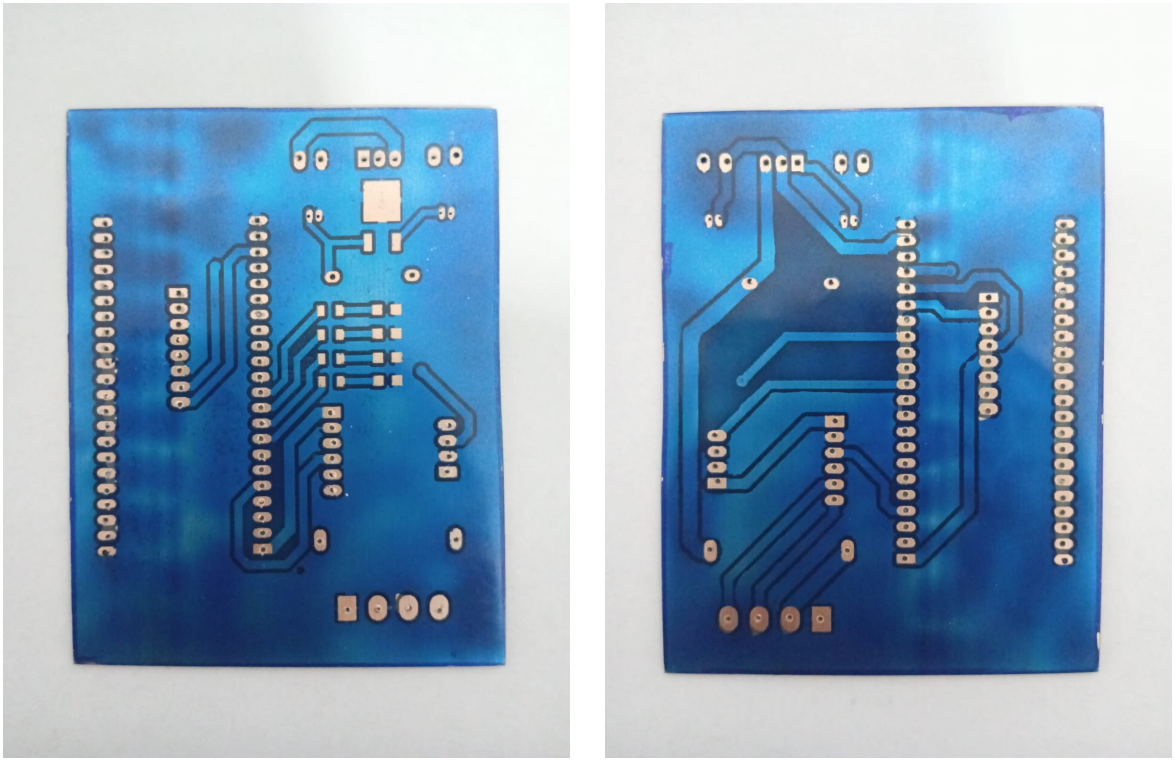
Simulación 3D de la placa preliminar, generada en Altium Designer antes de la fabricación.



La fabricación de la placa preliminar se encargó a Robokit (La Paz); la Figura 3.19 muestra las caras superior e inferior de la placa recibida, previo al ensamblaje de los componentes.

Figura 3.19

Placa preliminar fabricada por Robokit, sin ensamblar

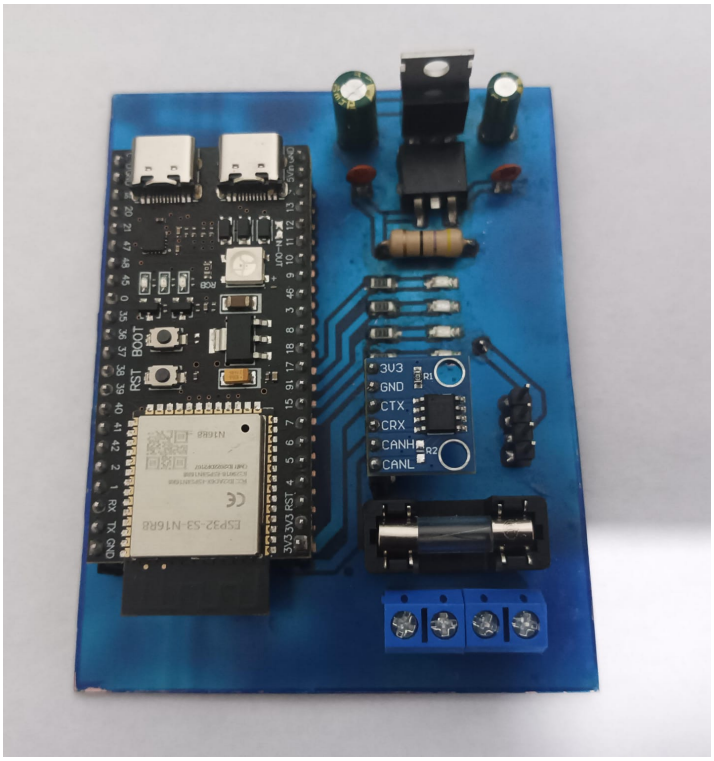


(a) *Cara superior (top)* **(b)** *Cara inferior (bottom)*

Nota. Placa de una sola cara fabricada por Robokit (La Paz), 150 Bs (Capítulo 5, Tabla 5.6); el ensamblaje de los componentes se realizó manualmente. Elaboración propia.

Una vez soldados los componentes de la Tabla 5.7 (Capítulo 5) sobre esta placa, y reutilizando el módulo ESP32-S3, el transceptor SN65HVD230 y la pantalla OLED ya validados en protoboard, se obtiene la placa ensamblada de la Figura 3.20.

Figura 3.20
Placa preliminar ensamblada



Nota. Placa preliminar tras el ensamblaje manual de los componentes. El **módulo ESP32-S3-N16R8** es el microcontrolador principal, que ejecuta el firmware completo: adquisición de datos por OBD-II, cálculo de consumo, registro en microSD y servidor web. El **regulador 7805** junto con el **transistor de paso TIP42C** forman la etapa de alimentación de esta placa preliminar: reducen los 12 V constantes del vehículo a los 5 V que requiere el resto del circuito, con el transistor aportando la capacidad de corriente adicional que el 7805 por sí solo no entrega; es una solución más simple que la conversión conmutada de dos rieles de la placa final (Sección 3.6.7), suficiente para esta etapa de pruebas de campo. El **módulo transceptor SN65HVD230** (conectado mediante el header de 6 pines) adapta la señal diferencial del bus CAN del vehículo a los niveles lógicos TTL que el ESP32-S3 puede leer. El **fusible reseteable F1** protege la placa ante una sobrecorriente o un cortocircuito accidental, cortando el paso de corriente y restableciéndose solo una vez retirada la falla. El **conector de entrada de 4 pines** (GND, 12 V, CAN_L, CAN_H) es el punto de conexión del cable hacia el puerto OBD-II del vehículo. Elaboración propia.

3.3.6. Diseño de la carcasa del prototipo (impresión 3D)

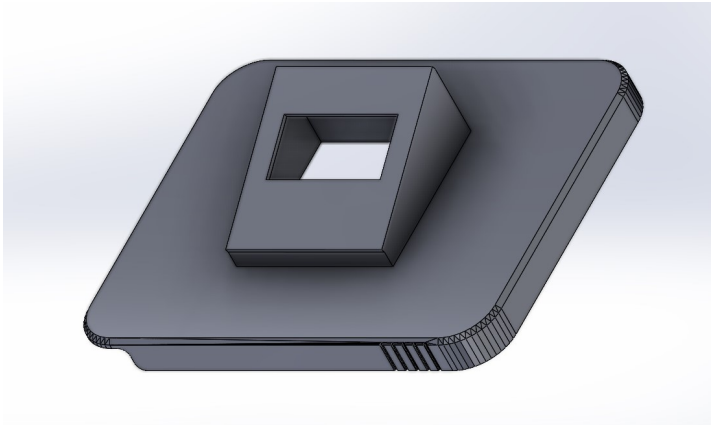
Para esta presentación se diseñó e imprimió una carcasa únicamente para la placa preliminar; la carcasa de la placa final queda pendiente y se elaborará si el tribunal la solicita. Como punto de partida se tomó un modelo STL de una caja de dos piezas (base y tapa) publicado en MakerWorld, el repositorio de modelos de Bambu Lab, mostrado en la Figura 3.21.

Figura 3.21
Modelo STL base, tomado de MakerWorld

Nota. Modelo de caja de dos piezas (base y tapa) publicado en MakerWorld (Bambu Lab), usado como punto de partida antes de la edición en SolidWorks.

La tapa del modelo original no contaba con una abertura para la pantalla OLED; se editó en SolidWorks para agregar el recorte rectangular necesario, manteniendo el resto de la geometría del modelo original (Figura 3.22).

Figura 3.22
Tapa modificada en SolidWorks, con abertura para la pantalla OLED



La impresión se realizó en PLA, con una impresora propia Creality Ender V3 KE (Figura 3.23). Las propiedades generales de este y otros materiales termoplásticos de impresión FDM se detallan en la Sección 2.10 del Capítulo 2 (Tabla 2.20); aquí se retoman solo para justificar la elección concreta hecha en este prototipo.

Se eligió PLA principalmente por dos razones prácticas, ambas específicas del contexto de un prototipo de presentación y no de una pieza de uso permanente. Primero, es el material más sencillo de imprimir con el equipo disponible (una impresora de escritorio sin recinto cerrado ni cama caliente de alta temperatura): su bajo alabeo y su ausencia de requisitos térmicos especiales permitieron obtener piezas dimensionalmente

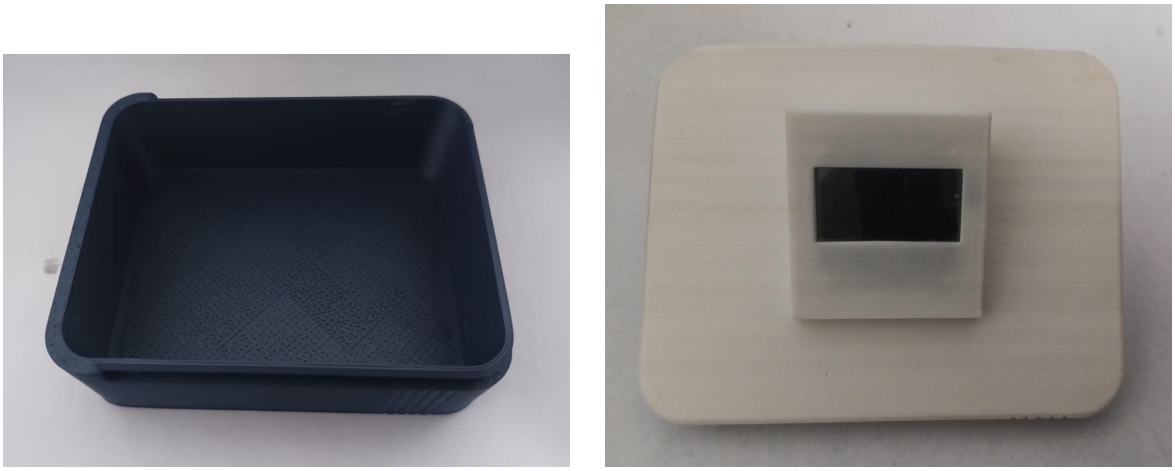
correctas al primer intento, sin necesidad de calibrar un perfil de impresión más exigente como el que requiere el ABS. Segundo, su baja resistencia a la temperatura y a la radiación ultravioleta (Tabla 2.20) no representa un riesgo relevante en esta etapa, porque la carcasa del prototipo permanece sobre un banco de pruebas o dentro del vehículo solo durante sesiones cortas de validación, nunca expuesta al sol de forma prolongada ni estacionada con el vehículo cerrado.

Esa misma debilidad del PLA sí sería determinante para una carcasa de producción, instalada de forma permanente bajo el tablero o en el habitáculo del vehículo: ahí la pieza queda expuesta a ciclos térmicos sostenidos (el interior de un vehículo estacionado al sol puede superar ampliamente la temperatura de transición vítrea del PLA, de 55 a 60 °C según la Tabla 2.20) y, en instalaciones cercanas al parabrisas o a una ventana, a radiación ultravioleta directa. Por ello, para una eventual carcasa de la placa final se recomienda **PETG** en lugar de PLA: ofrece una resistencia a la temperatura y a la intemperie notablemente mayor, manteniendo al mismo tiempo una facilidad de impresión cercana a la del PLA (bajo alabeo, sin necesidad de un recinto cerrado). El **ABS** queda como alternativa secundaria, principalmente si se buscara una resistencia al impacto todavía mayor, ya que su alabeo más pronunciado exige un recinto cerrado y una cama caliente de mayor temperatura, equipamiento con el que no se contaba para este prototipo.

Figura 3.23
Impresión 3D en la Creality Ender V3 KE

La base y la tapa, ya impresas (21 g y 16 g de PLA, respectivamente), se muestran en la Figura 3.24. El costo del material se presenta en el Capítulo 5 (Tabla 5.8).

Figura 3.24
Carcasa impresa del prototipo



(a) *Base (impresa, 21 g)* **(b)** *Tapa (impresa, 16 g)*

Nota. Impresión propia en PLA. La tapa incorpora el recorte rectangular para la pantalla OLED editado en SolidWorks (Figura 3.22). Elaboración propia.

3.3.7. Diagrama esquemático general

En esta sección se presentará el diseño esquemático del sistema, realizando el cálculo y selección de los valores óptimos para cada componente con base en las recomendaciones de sus respectivas hojas de datos. Para facilitar el análisis y diseño, el sistema se ha dividido en los siguientes subsistemas:

- **Sistema de alimentación:** En este subsistema se incorporarán las etapas de protección, regulación y filtrado de energía. Su principal objetivo es garantizar una alimentación estable y segura para todos los componentes del sistema, minimizando los efectos de ruido eléctrico, interferencias y transitorios de tensión.
- **Sistema transceptor CAN:** Se analizará el funcionamiento del transceptor CAN SN65HVD230, implementando una configuración optimizada que permita reducir el espacio ocupado en la PCB y mantener una comunicación confiable con la red CAN del vehículo.
- **Sistema de procesamiento:** Comprende el microcontrolador ESP32 y todos los componentes necesarios para su correcto funcionamiento, incluyendo circuitos auxiliares, interfaces de comunicación y conexiones para los distintos periféricos del sistema.
- **Sistema de visualización:** Se desarrollará la interfaz de conexión hacia una pantalla externa ubicada en el tablero del vehículo, para que el usuario disponga de la información relevante.
- **Sistema de almacenamiento de datos:** Se implementará una interfaz para memoria microSD destinada al almacenamiento de los datos obtenidos por el

sistema, de modo que puedan analizarse y procesarse más adelante.

- **Sistema de programación:** Incluye los componentes necesarios para la configuración, programación y actualización del firmware del ESP32; con esto se simplifica el desarrollo y mantenimiento del dispositivo.
- **Sistema de monitoreo y estado:** Se incorporarán indicadores luminosos (LEDs) que permitirán visualizar el estado operativo del sistema y facilitar tareas de diagnóstico durante su funcionamiento.

Todos estos subsistemas han sido concebidos para su implementación en una placa de circuito impreso (PCB). A continuación, se desarrollan los esquemáticos correspondientes a cada uno de ellos.

Sistema de alimentación

Se contempla la obtención del voltaje de alimentación principal directamente desde el pin 16 del conector OBD-II del vehículo, el cual proporciona una tensión nominal de 12 V proveniente de la batería (ver Figura 2.9). Debido a que los circuitos electrónicos internos operan a tensiones significativamente menores, es indispensable implementar una etapa previa de acondicionamiento, protección y reducción de voltaje mediante un convertidor reductor (*step-down*).

Con el propósito de mitigar perturbaciones eléctricas severas comunes en el entorno automotriz, tales como transitorios de conmutación o picos de tensión, se diseñó una etapa de protección robusta previa a la regulación síncrona a niveles de 5 V y 3,3 V. El flujo lógico y secuencial planificado para esta etapa de potencia se detalla en la Figura 3.25.

Figura 3.25
Diagrama de bloques del sistema de protección y regulación de energía

Nota. Topología de conversión directa de alta eficiencia basada en el regulador síncrono TPS54302. Esta configuración reduce directamente la tensión del bus automotriz de 12 V a 3,3 V nominales, optimizando la disipación térmica y garantizando un suministro eléctrico estable para el microcontrolador y sus periféricos críticos, previniendo caídas de tensión (*brownouts*) bajo las exigencias de consumo del transmisor Wi-Fi Espressif Systems (2023a).

Con base en el diagrama de bloques estructurado, se procedió a la selección rigurosa de los componentes electrónicos para la etapa de potencia. Esta etapa es crítica para adaptar los 12 V nominales provenientes del pin 16 del conector OBD-II a los niveles lógicos del sistema, garantizando fiabilidad frente a las exigentes condiciones eléctricas del entorno automotriz:

1. **Fusible rearmable (PTC):** Debido a que la red eléctrica del vehículo está expuesta a cortocircuitos y sobrecorrientes transitorias, se descartó el uso de fusibles tradicionales de un solo uso. Se optó por el fusible rearmable polimérico PTC modelo LUTE 1812L150/33GR, el cual soporta una tensión máxima de operación de 33 V, una corriente de mantenimiento (*hold current*) de 1,5 A y una corriente de disparo (*trip current*) de 3,0 A en encapsulado SMD 1812,datos que muestra la hoja de datos del componente en el Apendice F (Bourns, Inc., 2022). Su principal ventaja radica en su capacidad de restablecer automáticamente la conducción una vez que la falla desaparece y el componente se enfría, eliminando la necesidad de reemplazo físico en el campo.

2. **Diodo Schottky de protección de polaridad inversa (D1):** Para evitar daños irreversibles ante una conexión accidental con polaridad invertida en el puerto OBD-II, se integró el diodo Schottky SS34 en serie con la línea de alimentación positiva. Según su hoja de datos mostrada en el Apéndice G , este componente soporta una tensión inversa repetitiva máxima (V_{RRM}) de 40 V y una corriente media rectificada (I_O) de 3 A, con una caída de tensión directa típica de 550 mV a plena carga (onsemi, 2023). Su encapsulado DO-214AB (SMA) ofrece excelente disipación térmica en condiciones de alta corriente.

3. **Diodo TVS (*Transient Voltage Suppressor*):** Para suprimir el ruido electromagnético y proteger la circuitería contra picos de voltaje severos (tales como los transitorios de desconexión de carga, o *load dump*, propios del entorno automotriz, que pueden superar los 40 V) se integró el diodo TVS unidireccional SMAJ18A en paralelo con la línea de alimentación, inmediatamente después del fusible PTC y el diodo de polaridad. De acuerdo con su hoja de datos (ver Apéndice H), su tensión de trabajo (*stand-off voltage*) de 18 V permite la libre operación del voltaje nominal de la batería del vehículo, mientras que su tensión de sujeción máxima (*clamping voltage*) de 29,2 V a una corriente de pico de 13,7 A (Vishay General Semiconductor, 2023) garantiza que ningún transitorio destructivo alcance al regulador. La potencia de disipación de pulso pico es de 400 W con una forma de onda de 10/1000 µs, suficiente para absorber los transitorios definidos por la norma ISO 7637-2 en entornos automotrices.

4. **Regulador reductor de 5 V (TPS54302):** Inicialmente se evaluaron reguladores lineales clásicos como el AMS1117-3,3, pero fueron descartados debido a su ineficiente disipación térmica al intentar reducir directamente tensiones del bus automotriz de 12 V a 3,3 V, ya que esto representaría disipar más del 72% de la potencia en calor. En su lugar, se seleccionó el convertidor reductor síncrono (*buck converter*) TPS54302 de Texas Instruments.

   Este integrado opera en un rango de tensión de entrada de 4,5 V a 28 V, entrega una corriente de salida continua de hasta 3 A, y opera a una frecuencia de conmutación fija de 400 kHz (con espectro expandido para reducción de EMI). Incorpora dos MOSFETs de canal N integrados, de bajo $R_{DS(on)}$ (85 mΩ en el lado alto o *high-side*, y 40 mΩ en el lado bajo o *low-side*), y con ello configura un convertidor *buck* síncrono, además de una función de modo Eco (*Eco-mode*) que maximiza la

eficiencia en condiciones de carga ligera. El rango de temperatura de juntura de operación es de −40 °C a +125 °C (ver Apéndice I). Se encuentra disponible en encapsulado SOT-23-6, de fácil soldadura y montaje en superficie.

Cálculo de la tensión de salida: La tensión de salida se programa mediante un divisor resistivo conectado al pin FB del integrado. Según la hoja de datos del TPS54302DDCR, la tensión de referencia interna es $V_{ref} = 0{,}596$ V, y la expresión que gobierna la tensión de salida es:

$$V_{out} = V_{ref} \times \left(1 + \frac{R_{upper}}{R_{lower}}\right) \tag{3.5}$$

Para obtener una salida de 5 V se seleccionaron los valores de resistencia $R_{upper} =$ 100 kΩ y $R_{lower} = 13{,}3$ kΩ. Sustituyendo en la ecuación 3.5:

$$V_{out} = 0{,}596 \times \left(1 + \frac{100000}{13300}\right) = 0{,}596 \times 8{,}519 \approx 5{,}078 \text{ V} \tag{3.6}$$

Este valor representa un error del +1,56%respecto al valor nominal de 5 V, dentro de la tolerancia aceptable para alimentar la etapa posterior de reguladores lineales (AMS1117-3,3 y AP2112K-3,3), que a su vez entregan la tensión regulada de 3,3 V requerida por el microcontrolador ESP32-S3, cuyo rango de operación es de 3,0 V a 3,6 V (Espressif Systems, 2023a).

Habilitación (EN): A diferencia de otros reguladores de la familia TPS543xx que permiten configurar un *undervoltage lockout* (UVLO) ajustable mediante un divisor resistivo externo en el pin EN, el TPS54302DDCR habilita la conmutación simplemente dejando el pin EN sin conexión (*floating*), según lo indicado en su hoja de datos. En esta condición, el dispositivo inicia la conmutación de forma automática cuando la tensión de entrada supera el umbral interno de UVLO, típicamente 4,5 V, sin necesidad de componentes adicionales. Esta configuración fue la adoptada en el diseño, dado que la tensión de entrada protegida (`12V_PROT`) proveniente del riel del vehículo (12 V a 14,4 V nominales) se mantiene ampliamente por encima de este umbral durante la operación normal del sistema.

5. **Filtro de salida LC:** El filtro de salida está compuesto por un inductor de 10 µH y dos condensadores de 22 µF en paralelo, formando una inductancia total de 10 µH y una capacitancia de salida efectiva de 44 µF. Esta combinación proporciona la atenuación del rizado de corriente y tensión necesaria para garantizar una alimentación limpia y estable al microcontrolador ESP32-S3 y demás periféricos del sistema.

6. **Compensación del lazo de control:** A diferencia de otros reguladores de la familia TPS543xx que requieren una red de compensación externa tipo II en un pin COMP dedicado, el TPS54302DDCR emplea control en modo de corriente pico (*peak current-mode control*) con una red de compensación interna optimizada de fábrica. Esto elimina la necesidad de componentes externos de compensación,

reduce el conteo de piezas y simplifica el diseño del lazo de control, según lo indicado en su hoja de datos. Como complemento al filtro de salida, se incorporó un condensador adicional de 75 pF en paralelo, cuya función es mejorar la respuesta ante transitorios de carga.

7. **Regulación de 5 V a 3,3 V:** Para mejorar la eficiencia del convertidor DC-DC la regulación de 5 V a 3,3 V se realizara con 2 componentes muy usados los cuales se detallan:

   - **AMS1117-3,3**

     Se seleccionó el regulador AMS1117-3,3 para alimentar exclusivamente el módulo ESP32 debido a su capacidad de suministrar hasta 1 A (ver Apéndice J)de corriente, proporcionando un margen suficiente para soportar los picos de consumo generados por el microcontrolador durante la transmisión Wi-Fi, el funcionamiento del servidor web local y el procesamiento de datos provenientes del sistema OBD-II. Aunque el consumo promedio del ESP32 es inferior, durante las comunicaciones inalámbricas puede presentar picos de corriente superiores a 400 mA, por lo que disponer de un regulador con mayor capacidad mejora la estabilidad del sistema y reduce la posibilidad de caídas de tensión.

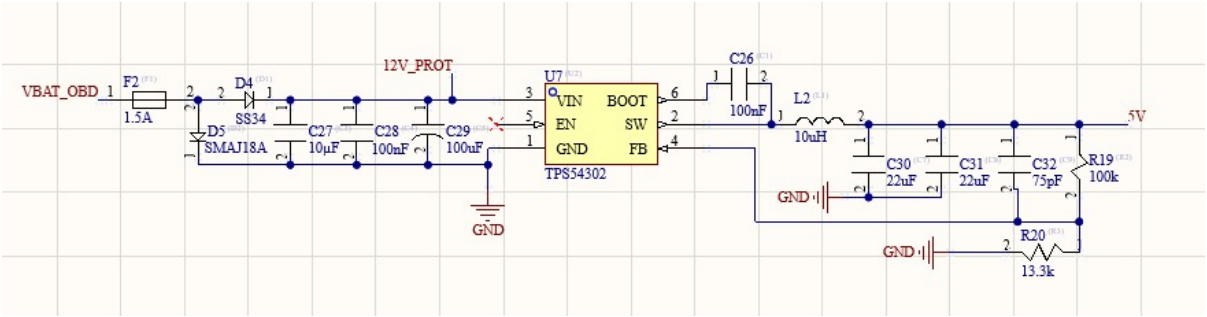
   - **AP2112K-3,3TRG1**

     El regulador AP2112K-3,3TRG1 fue destinado a la alimentación de los dispositivos periféricos, como la pantalla OLED, la memoria microSD y otros circuitos auxiliares. Este regulador puede suministrar hasta 600 mA (ver Apéndice K), corriente suficiente para estos dispositivos, además de ofrecer una baja caída de tensión (Low Dropout) y un bajo nivel de ruido, características que favorecen el correcto funcionamiento de los periféricos. Separar la alimentación del ESP32 y de los periféricos evita que los picos de corriente generados por el módulo Wi-Fi afecten el funcionamiento de los demás dispositivos, incrementando la estabilidad y confiabilidad del sistema.

Consolidada la selección y el dimensionamiento métrico de los componentes, se procedió a la transcripción topológica en el entorno de diseño electrónico profesional Altium Designer. El resultado es una arquitectura de alimentación robusta y desacoplada, cuyo esquema electrónico del sistema de alimentación se ilustra en la Figura 3.26, junto con el convertidor a 3,3 V que se ilustra en la Figura 3.27.

Figura 3.26

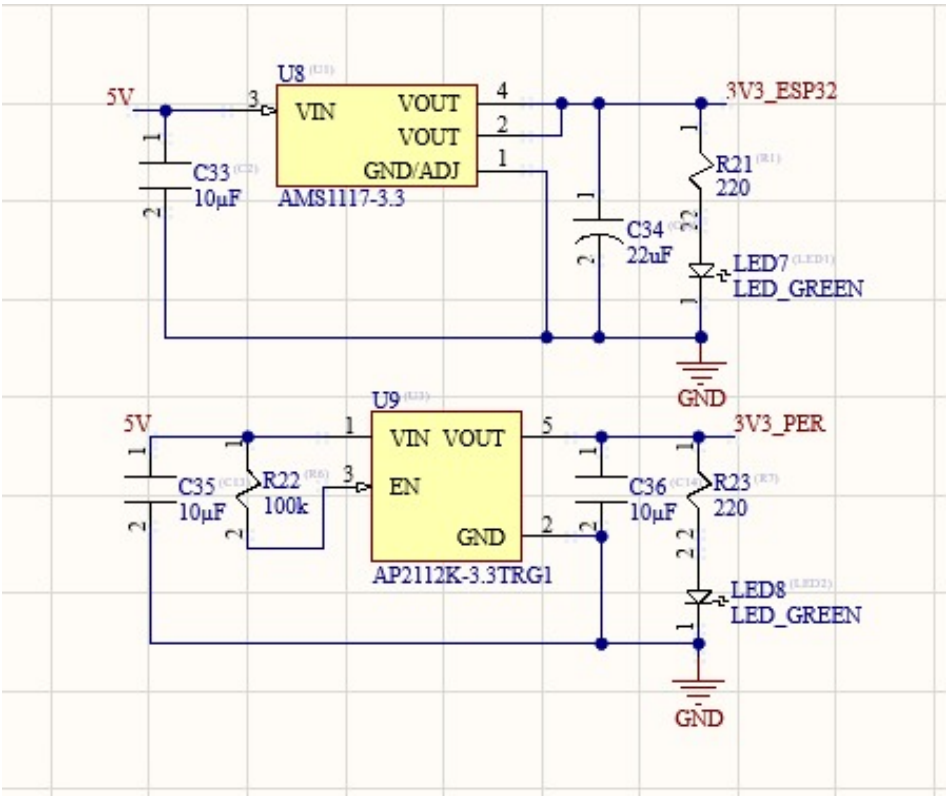
Esquemático del circuito convertidor DC/DC Buck para alimentación de 5 V.



Nota. Diagrama eléctrico de la etapa de potencia regulada tipo reductora (*Buck*) basada en el controlador TPS54302. Incluye las etapas de protección de entrada (fusible PTC rearmable, diodo Schottky de protección contra inversión de polaridad y diodo TVS), el filtro de potencia LC y el lazo de realimentación ajustado para una salida estable de 5 V/3 A.

Figura 3.27

Esquemático del circuito de regulación lineal para la alimentación de 3,3 V.

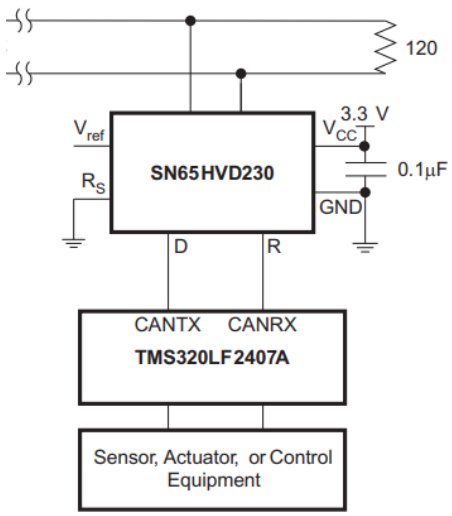


Nota. Diagrama eléctrico de la etapa de regulación lineal que reduce el voltaje de 5 V a 3,3 V. El diseño emplea los reguladores de baja caída de tensión (LDO, por sus siglas en inglés) AMS1117 y AP2112 para suministrar energía de forma estable al microcontrolador y periféricos de 3,3 V.

Sistema transceptor CAN

Para la comunicación en el bus CAN, se seleccionó el transceptor SN65HVD230 (ver Tabla 3.6). La configuración del circuito se realizó siguiendo las recomendaciones técnicas detalladas en su hoja de datos (Texas Instruments, 2015), la cual describe la implementación óptima para garantizar la integridad de la señal. La configuración del esquema de conexión se ilustra en la Figura 3.28

Figura 3.28. *Configuración de conexión óptima del transceptor CAN SN65HVD230*

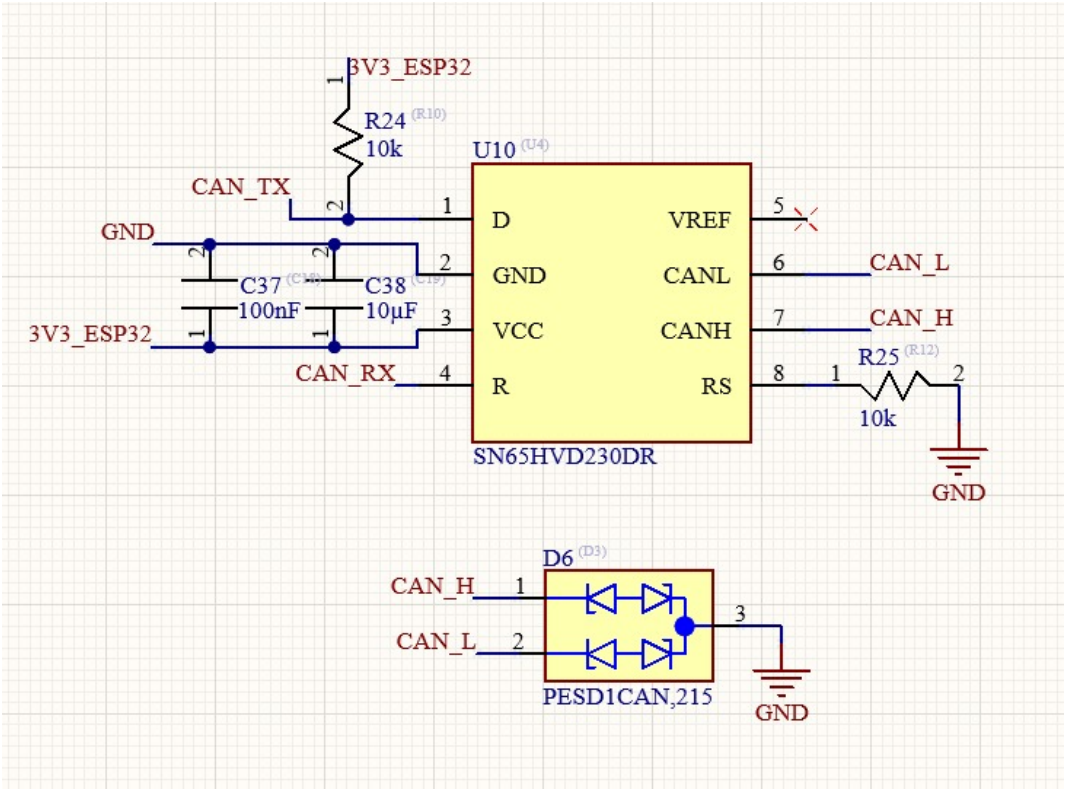


Nota. Diagrama de conexión típica para el transceptor SN65HVD230, incluyendo la resistencia de terminación de 120 Ω entre las líneas CANH y CANL para evitar reflexiones en el bus, y la configuración de los pines de control para el modo de operación de alta velocidad. Adaptado de Texas Instruments (2015).

Tras analizar el esquema de los módulos basados en el SN65HVD230, se identificó una resistencia adicional conectada al pin RS (Rate Select). Asimismo, se añadió un PediScan, cuya función es actuar como un escáner y herramienta de diagnóstico automotriz, permitiendo la comunicación con la ECU del vehículo a través del puerto OBD-II para la lectura de parámetros, códigos de diagnóstico (DTC) y datos en tiempo real. Su incorporación permitió tomar como referencia un dispositivo comercial ampliamente utilizado para validar el diseño de la interfaz CAN implementada. Al integrar esta configuración en el diseño, el sistema resultante se ilustra en la Figura 3.29.

Figura 3.29

Esquemático del circuito transceptor CAN basado en el SN65HVD230.



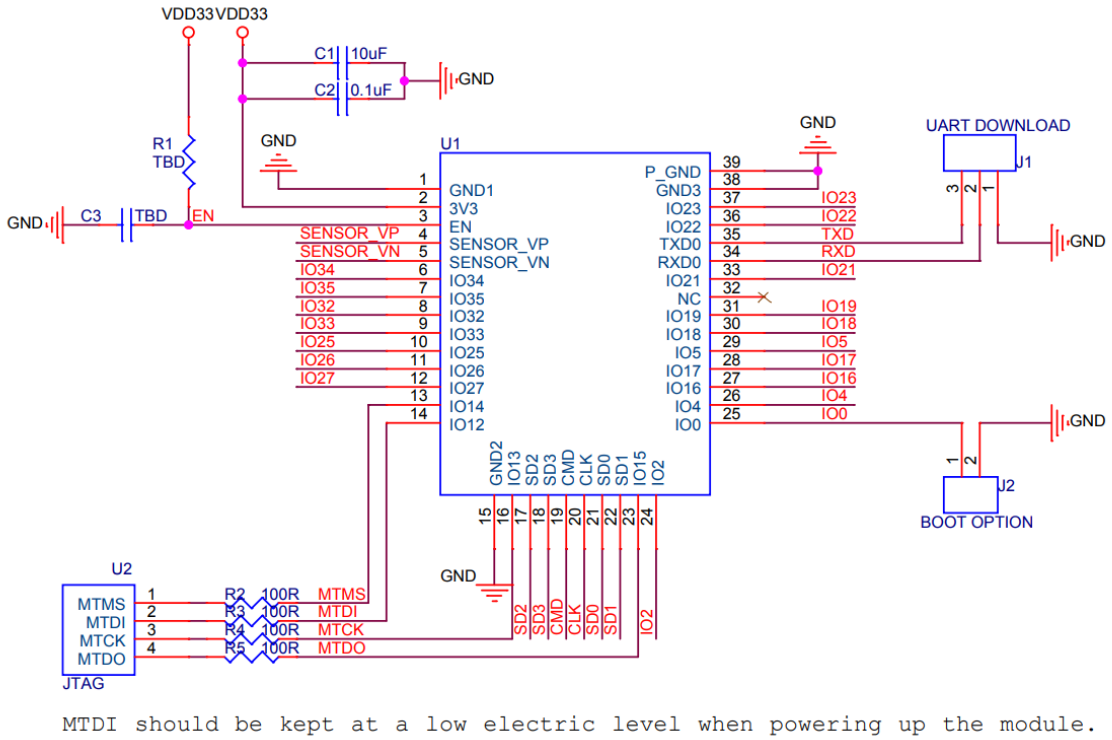
Nota. Diagrama eléctrico de la interfaz de comunicación CAN basada en el transceptor SN65HVD230. Incluye la configuración de los pines de control, la red de desacoplamiento de alimentación y la resistencia de terminación de 120 Ω necesaria para la integridad del bus, siguiendo las especificaciones de diseño recomendadas en la hoja de datos del fabricante.

Sistema de procesamiento

Para este sistema se optó por el módulo **ESP32-S3-WROOM-1-N16R8**, el cual integra el microcontrolador ESP32-S3 de Espressif Systems. Este dispositivo incorpora un procesador de doble núcleo basado en la arquitectura Xtensa® LX7 de hasta 240 MHz, conectividad Wi-Fi de 2,4 GHz y Bluetooth Low Energy (BLE) 5.0, además de una memoria Flash integrada de 16 MB y 8 MB de PSRAM. Estas características proporcionan la capacidad de ejecutar simultáneamente las tareas de adquisición de datos del vehículo, almacenamiento en memoria microSD, comunicación mediante el protocolo CAN y la implementación de un servidor web local para la visualización de información en tiempo real; el desempeño del sistema se mantiene estable incluso con estas tareas corriendo en paralelo (Espressif Systems, 2023a). Para asegurar el correcto funcionamiento del microcontrolador se debe implementar, además, la configuración mínima de hardware recomendada por el fabricante, tal como se ilustra en la Figura 3.30.

Figura 3.30

Circuito de conexión mínima y configuración óptima del módulo ESP32-S3-WROOM-1-N16R8.

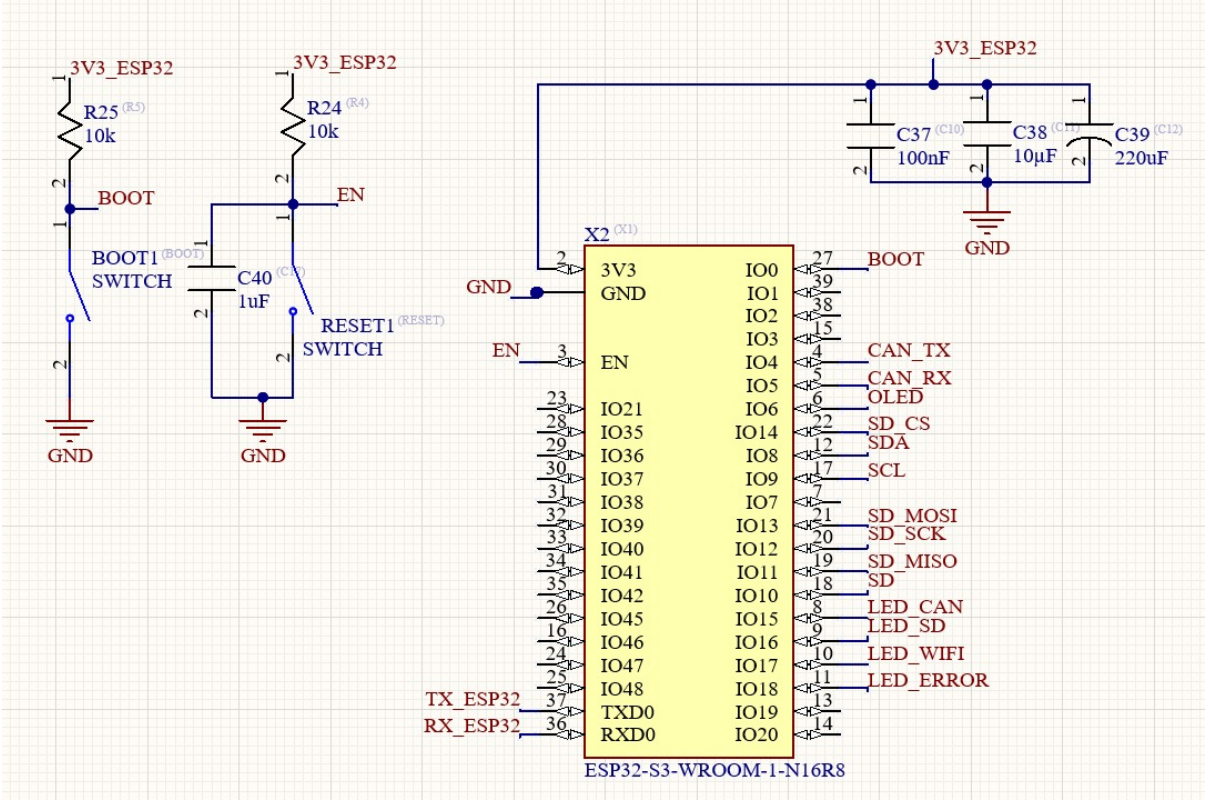


Nota. Esquemático de la configuración de hardware requerida para el correcto funcionamiento del módulo ESP32-S3-WROOM-1-N16R8. Se muestra el circuito de habilitación y reinicio mediante el pin EN, los condensadores de desacoplamiento de la alimentación de 3,3 V ubicados próximos al módulo, así como la configuración recomendada de los pines de arranque y programación para garantizar una inicialización estable del sistema. Adaptado de *ESP32-S3-WROOM-1 Datasheet*, por Espressif Systems, 2024.

La integración definitiva de la etapa de procesamiento se consolidó mediante el diseño del circuito impreso en la plataforma Altium Designer. Este esquema incorpora las recomendaciones establecidas por el fabricante para el módulo ESP32-S3-WROOM-1-N16R8 y la integración de los diferentes periféricos empleados en el sistema, optimizando la distribución de las señales de alimentación, comunicación y control para mejorar la estabilidad eléctrica y el rendimiento general del dispositivo. El resultado de esta integración y la topología eléctrica final del sistema basado en el microcontrolador se presentan detalladamente en la Figura 3.31.

Figura 3.31

Esquemático completo del circuito de control y periféricos basados en el ESP32-S3-WROOM-1-N16R8.



Nota. Diagrama eléctrico final desarrollado en Altium Designer para la etapa de procesamiento basada en el módulo ESP32-S3-WROOM-1-N16R8. El diseño integra las recomendaciones de la hoja de datos del fabricante, el acondicionamiento de la alimentación, el circuito de programación y reinicio automático, así como la conexión de los periféricos utilizados en el sistema para garantizar un funcionamiento confiable durante la adquisición, procesamiento y visualización de los datos obtenidos desde el puerto OBD-II.

Sistema de visualización

Una vez definida la distribución de señales del conector de expansión, se procedió al diseño del esquemático de la etapa de visualización utilizando el software Altium Designer. En este diseño se integran la pantalla OLED, el conector JST-GH, las resistencias *pull-up* del bus I^2C y el condensador de desacoplamiento, obteniéndose el circuito mostrado en la Figura 3.32.

Tabla 3.9

Asignación de pines y código de colores del conector de expansión de 6 pines

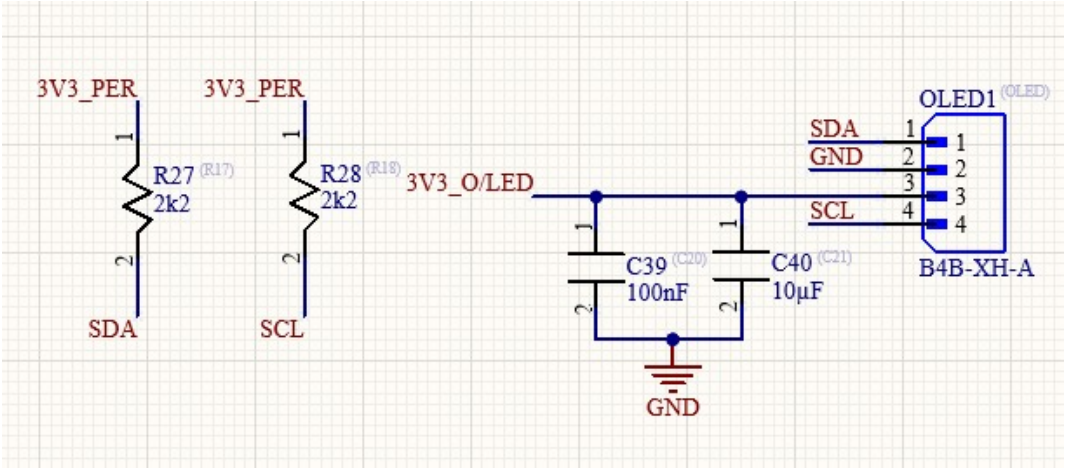
Pin / Función	Color	Descripción
`VCC`	Rojo	Alimentación principal del módulo externo regulada a 3,3 V.
`GND`	Negro	Referencia común de tierra del sistema.
`SCL`	Verde	Línea de reloj de la interfaz de comunicación serial I^2C.
`SDA`	Azul	Línea de datos bidireccional de la interfaz de comunicación serial I^2C.

Nota. La distribución de señales del conector de expansión garantiza la compatibilidad con sensores periféricos a través del bus de comunicación I^2C. Las líneas auxiliares permiten la integración posterior de interrupciones de hardware o señales de control adicionales sin necesidad de modificar el diseño base del circuito impreso.

Definida el uso de este conector se procede a dibujar el esquemático (ver Figura 3.32) donde su conexión sera al oled.

Figura 3.32

Esquemático del circuito de visualización física y conector de expansión periférico.



Nota. Nota. Diagrama esquemático desarrollado en Altium Designer para la etapa de visualización del sistema. El diseño incorpora la pantalla OLED, el conector JST-GH de seis pines para su conexión externa, las resistencias *pull-up* de las líneas SDA y SCL del bus I^2C, así como el condensador de desacoplamiento de 100 nF ubicado próximo al periférico para mejorar la estabilidad de la alimentación y la integridad de la comunicación en aplicaciones con cableado de mayor longitud..

Sistema de almacenamiento de datos

Con el propósito de almacenar de forma local los datos obtenidos desde el sistema OBD-II, se incorporó una memoria MicroSD al diseño electrónico. Su utilización permite registrar de manera permanente las variables adquiridas por el ESP32-S3-WROOM-1-N16R8, facilitando el almacenamiento de grandes volúmenes de información para su posterior análisis sin depender de una conexión inalámbrica o de un servidor externo. Esta característica resulta especialmente útil durante las pruebas de campo, donde es necesario conservar el historial completo de los parámetros del vehículo para realizar comparaciones, validar modelos matemáticos y analizar el comportamiento del sistema en diferentes condiciones de operación.

La implementación de esta etapa se basó en las especificaciones de la SD Card Association (Transcend Information Inc., 2006). Debido a que tanto el módulo ESP32-S3-WROOM-1-N16R8 como las tarjetas MicroSD operan con niveles lógicos de 3,3 V, fue posible realizar la conexión directa entre ambos dispositivos sin necesidad de incorporar convertidores de nivel lógico, simplificando el diseño y reduciendo el número de componentes.

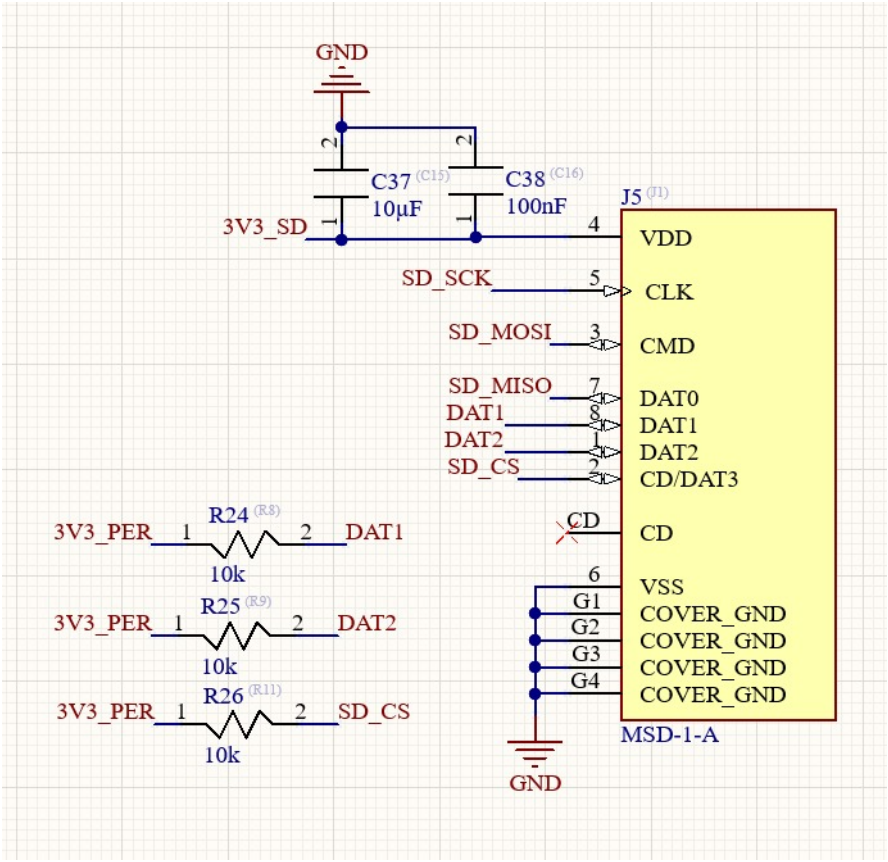
Para la comunicación entre el microcontrolador y la memoria se seleccionó el protocolo SPI (Serial Peripheral Interface), debido a su elevada velocidad de transferencia, simplicidad de implementación y reducido número de líneas de comunicación. En esta configuración se emplean las señales SCK, MOSI, MISO y CS para gestionar el acceso a la memoria durante las operaciones de lectura y escritura.

Con el objetivo de garantizar un funcionamiento confiable del sistema, se incorporaron condensadores de desacoplamiento de 10 μF y 100 nF próximos al conector MicroSD. El condensador de 10 μF proporciona una reserva de energía para compensar los picos de corriente que se presentan durante las operaciones de escritura, mientras que el condensador cerámico de 100 nF filtra las componentes de alta frecuencia presentes en la alimentación, reduciendo el ruido eléctrico y mejorando la estabilidad del sistema.

También se implementaron resistencias de 10 kΩ conectadas a las líneas DAT1, DAT2 y DAT3/CS de la tarjeta MicroSD. Estas resistencias mantienen dichas líneas en un nivel lógico alto cuando no están siendo utilizadas, evitando estados flotantes durante el encendido del sistema, la inserción o extracción de la tarjeta y mejorando la confiabilidad del proceso de inicialización, de acuerdo con las recomendaciones de diseño para dispositivos compatibles con tarjetas SD. El esquema eléctrico final de esta etapa se presenta en la Figura 3.33.

Figura 3.33

Esquemático de la etapa de almacenamiento de datos mediante tarjeta MicroSD.



Nota. Diagrama esquemático desarrollado en Altium Designer para la interfaz de almacenamiento mediante memoria MicroSD. El diseño incorpora los condensadores de desacoplamiento de 10 μF y 100 nF para estabilizar la alimentación durante las operaciones de lectura y escritura, así como resistencias de 10 kΩ conectadas a las líneas DAT1, DAT2 y DAT3/CS, las cuales mantienen dichas señales en un estado lógico definido durante el arranque e inicialización de la tarjeta, incrementando la confiabilidad de la comunicación SPI con el módulo ESP32-S3-WROOM-1-N16R8.

Sistema de programación de uso de periféricos

Control de alimentación de periféricos

Con el propósito de optimizar el consumo energético del sistema y permitir una administración independiente de los dispositivos auxiliares, se incorporó un interruptor electrónico de carga TPS22918. Este circuito integrado permite conectar o desconectar la alimentación de los periféricos mediante una señal de control proveniente del microcontrolador ESP32-S3-WROOM-1-N16R8.

En el diseño propuesto, dos pines GPIO del ESP32-S3 controlan el estado lógico de habilitación del TPS22918, permitiendo energizar únicamente los dispositivos cuando son requeridos por la aplicación. Los principales periféricos alimentados mediante este circuito

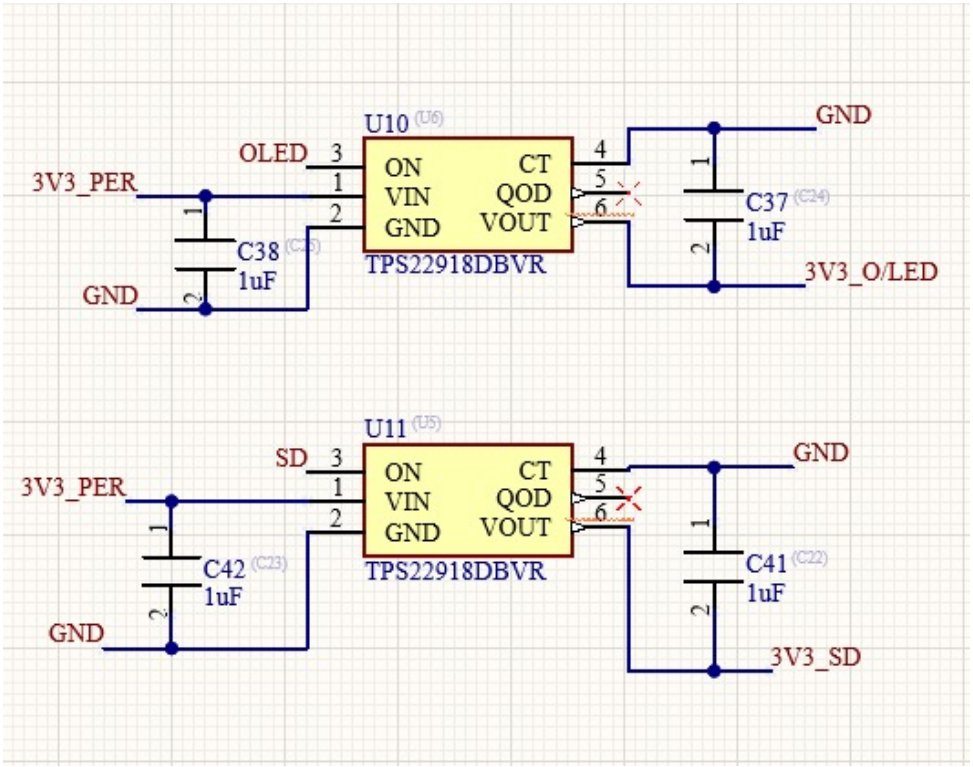
son la memoria MicroSD y la pantalla OLED, los cuales permanecen desenergizados durante el proceso de inicialización del sistema o cuando no se encuentran en uso; así se reduce el consumo de corriente y se evitan inicializaciones indeseadas.

La utilización del TPS22918 también proporciona un encendido controlado de los periféricos gracias a su baja resistencia de conducción (R_{ON}) y a su mecanismo interno de control de la corriente de irrupción (*Inrush Current*) (ver Apéndice L), evitando caídas bruscas de tensión sobre la línea de alimentación de 3,3 V durante la conexión de cargas capacitivas. Estas características contribuyen a mejorar la estabilidad del sistema y la confiabilidad del funcionamiento del microcontrolador durante el arranque y la operación normal.

El esquemático correspondiente a la etapa de control de alimentación se presenta en la Figura 3.34.

Figura 3.34

Esquemático de la etapa de control de alimentación mediante el interruptor electrónico TPS22918.



Nota. Diagrama esquemático desarrollado en Altium Designer para el control de alimentación de los periféricos del sistema. El circuito emplea un interruptor electrónico TPS22918, controlado mediante señales GPIO del módulo ESP32-S3-WROOM-1-N16R8, para habilitar o deshabilitar la alimentación de la memoria MicroSD y la pantalla OLED. Esta estrategia permite reducir el consumo energético, controlar la secuencia de encendido de los periféricos y mejorar la estabilidad de la línea de alimentación de 3,3 V al limitar la corriente de irrupción durante la conexión de las cargas.

Sistema de monitoreo y estado

Indicadores de estado

Con el propósito de facilitar el diagnóstico visual y el monitoreo del estado operativo del sistema, se incorporó un conjunto de indicadores luminosos basados en diodos emisores de luz (LED) de montaje superficial (SMD). Estos indicadores permiten conocer de forma inmediata el estado de los principales módulos del dispositivo durante su funcionamiento: las tareas de depuración, mantenimiento y validación del sistema pueden hacerse así sin herramientas externas de diagnóstico.

Cada LED es controlado mediante un pin de propósito general (GPIO) del módulo ESP32-S3-WROOM-1-N16R8, permitiendo que el firmware indique diferentes estados de operación de acuerdo con los eventos generados por el sistema. Para limitar la corriente que circula por cada diodo y proteger tanto los LED como las salidas digitales del microcontrolador, se incorporó una resistencia de 220 Ω conectada en serie con cada indicador.

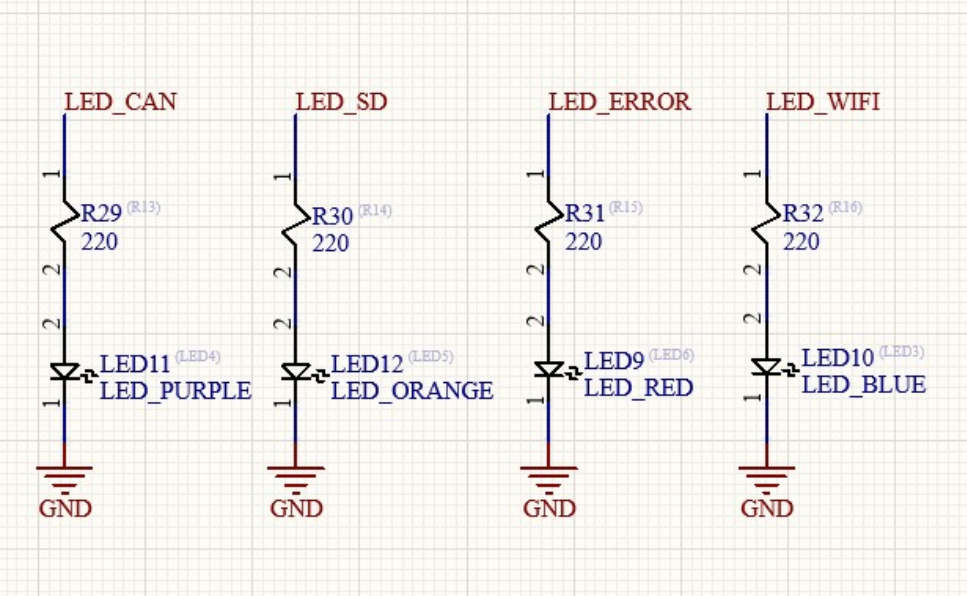
El sistema de señalización está conformado por los siguientes indicadores:

- **LED CAN (morado):** Indica la actividad de comunicación sobre el bus CAN, encendiéndose o parpadeando durante la transmisión y recepción de tramas provenientes de la ECU del vehículo.
- **LED MicroSD (naranja):** Señaliza las operaciones de acceso a la memoria MicroSD, incluyendo procesos de inicialización, lectura y escritura de archivos de registro.
- **LED ERROR (rojo):** Indica la presencia de fallas detectadas por el sistema, como errores de comunicación, problemas durante la inicialización de los periféricos o cualquier condición anómala registrada por el firmware.
- **LED Wi-Fi (azul):** Muestra el estado de la conectividad inalámbrica y del servidor web local implementado en el ESP32-S3, indicando la disponibilidad del sistema para la visualización de datos mediante un navegador web.

El esquemático correspondiente a esta etapa se desarrolló en Altium Designer y se presenta en la Figura 3.35.

Figura 3.35

Esquemático del circuito de indicadores LED para el monitoreo del estado del sistema.



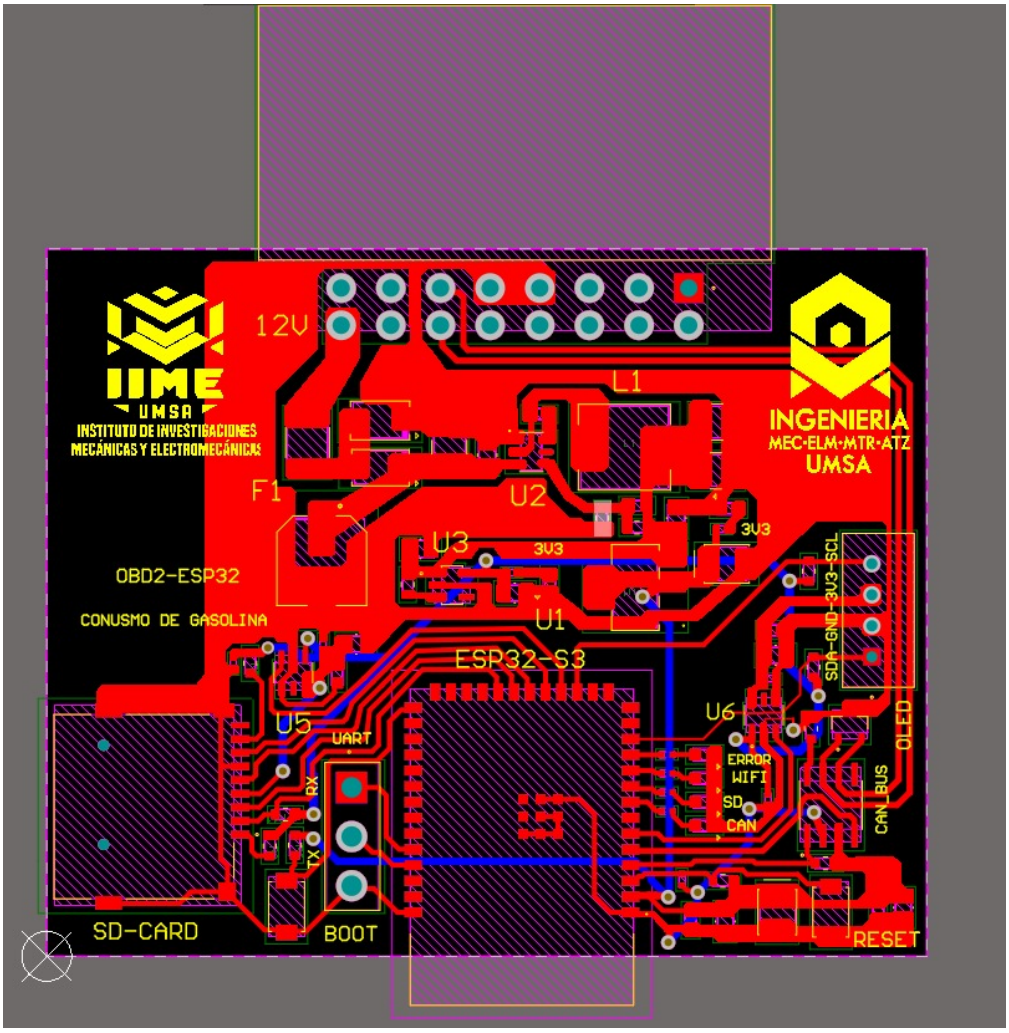
Nota. Diagrama esquemático desarrollado en Altium Designer para el sistema de indicadores visuales. Cada LED se encuentra conectado a un GPIO del módulo ESP32-S3-WROOM-1-N16R8 mediante una resistencia limitadora de 220 Ω, la cual restringe la corriente que circula por el diodo, protege las salidas digitales del microcontrolador y prolonga la vida útil de los indicadores. Los cuatro LED permiten visualizar el estado de la comunicación CAN, la actividad de la memoria MicroSD, la conectividad Wi-Fi y la presencia de condiciones de error detectadas durante el funcionamiento del sistema.

3.3.8. Diseño de la PCB (Altium Designer)

Una vez concluidos los diagramas esquemáticos de cada subsistema, se procedió a la fase de diseño y distribución de la placa de circuito impreso (PCB). Para ello, se utilizó la función 'Import to PCB' integrada en Altium Designer, la cual permite transferir la conectividad lógica y las huellas físicas (footprints) de los componentes. Posteriormente, se realizó la disposición estratégica de los elementos en las diferentes capas (layers) de la placa. En la Figura 3.36 se ilustra detalladamente el proceso de transición de los componentes desde el entorno esquemático hacia el lienzo de diseño definitivo de la PCB. Todos estos documentos tanto como el diagrama esquematico y el layout de la PCB se muestran en el Apendice M

Figura 3.36

Interfaz de transición y mapeo desde el diagrama esquemático hacia el entorno de diseño de la PCB.



Nota. Proceso de transferencia y sincronización de componentes mediante la netlist en Altium Designer. Se ilustra cómo los símbolos lógicos del esquema eléctrico se transforman en sus respectivas huellas físicas (footprints) sobre el lienzo bidimensional de la placa de circuito impreso, quedando dispuestos para la organización de capas y el ruteado definitivo de las pistas de cobre.

Previo a la fase de distribución (layout) y ruteo de las pistas de cobre, es fundamental establecer ciertos criterios técnicos de diseño. Estas consideraciones preliminares permiten optimizar la integridad de las señales, asegurar el manejo correcto de la potencia y garantizar la viabilidad de manufactura de la PCB.

Criterios de integridad de señal

El diseño de circuitos impresos está regulado por estándares internacionales que garantizan su fiabilidad, rendimiento y manufacturabilidad. Entre ellos, la norma IPC-2221 (Generic Standard on Printed Board Design) constituye el pilar fundamental que establece las directrices lógicas y físicas para el desarrollo de PCB. Dentro de las buenas prácticas estipuladas por esta normativa, destacan los siguientes criterios:

- **Sistemas de unidades:** Definir el sistema de medición es un paso preliminar crítico en el desarrollo de la PCB. En la práctica de la ingeniería electrónica, se prioriza el uso del sistema imperial (mils y pulgadas), debido a que el paso (pitch) y la distribución de los pines en la gran mayoría de los componentes y encapsulados comerciales están estandarizados bajo este sistema de medidas.
- **Configuración de rejillas (Grids):** La correcta gestión del grid es indispensable para garantizar la precisión geométrica, simetría y alineación del circuito. Un grid de 100 mils constituye el estándar industrial para la distribución de componentes de inserción (Through-Hole), ya que se acopla perfectamente con los centros de perforación. Para el trazo y ruteado de pistas, se adopta de forma general una rejilla de 50 mils, la cual puede reducirse a 25 mils o valores inferiores para diseños de alta densidad que requieran un nivel de detalle más fino.
- **Dimensionamiento de pistas:** El ancho de los conductores no está condicionado por un valor absoluto, sino que depende estrictamente de los requerimientos eléctricos del circuito (capacidad de corriente), el espacio disponible para el enrutamiento y las distancias de aislamiento (clearance). En la Tabla 3.10 se presentan los valores geométricos sugeridos; el cálculo matemático para determinar el ancho óptimo en función de la corriente y la elevación térmica es, de por sí, complejo, así que la industria se fundamenta en ecuaciones empíricas y curvas de comportamiento no lineal que han sido recopiladas y validadas experimentalmente por organismos normativos a lo largo de décadas de pruebas de laboratorio.

Tabla 3.10

Ancho de pista recomendado y resistencia estimada en función de la corriente

Corriente (Amp)	1 oz (mils)	2 oz (mils)	Resistencia (mΩ/plg)
1	10	5	52,0
2	30	15	17,2
3	50	25	10,3
4	80	40	6,4
5	110	55	4,7
6	150	75	3,4
7	180	90	2,9
8	220	110	2,3
9	260	130	2,0
10	300	150	1,7

Nota. Parámetros de diseño electrónico sugeridos para el ruteado de pistas de potencia en la PCB. Los anchos están calculados en milésimas de pulgada (mils) para los dos espesores de cobre más comunes (1 oz y 2 oz), considerando una elevación de temperatura admisible estándar ($\Delta T = 10°\text{C}$) bajo criterios normativos de diseño industrial.

- **Pads:** La geometría, dimensiones y morfología de los pads (islas de soldadura) no se determinan únicamente en función del componente seleccionado; existe un marco normativo y teórico riguroso que rige su diseño para diferentes aplicaciones electrónicas. Aunque cada fabricante de PCB (fab house) establece sus propias tolerancias y capacidades tecnológicas mínimas, una regla empírica ampliamente aceptada en la industria dicta que el diámetro del pad debe ser al menos 1,8 veces el diámetro de la perforación, o bien, presentar un excedente dimensional mínimo de 0,5 mm.
- **Vías:** Las vías constituyen las interconexiones eléctricas verticales que permiten comunicar pistas conductoras ubicadas en diferentes capas (layers) de la PCB a través de perforaciones metalizadas. En la industria del prototipado comercial (inclusive en servicios de bajo costo), esta continuidad se logra mediante un proceso de chapado electrolítico, técnicamente denominado PTH (Plated Through Hole). A diferencia de los métodos de fabricación puramente artesanales (donde se recurre a puentes de alambre o remaches mecánicos), este metalizado químico garantiza una unión uniforme, robusta y de baja resistencia eléctrica dentro de la perforación.
- **Distancias de aislamiento (Clearance):** El clearance representa un parámetro crítico y mandatorio en el diseño de cualquier PCB. Para un diseño estándar seguro, se recomienda adoptar un valor umbral de 15 mils; si bien es técnicamente viable reducir este margen a 8 o 10 mils, el diseño se encuadra en tecnologías de alta densidad (HDI), lo que exige procesos de fabricación más estrictos y costosos una referencia es la siguiente Tabla 3.11.

Tabla 3.11

Distancias mínimas de aislamiento eléctrico (clearance) según el voltaje operativo y la altitud

Voltaje (DC o pico AC)	Capas Interiores	Capa Externa (< 3050 m s.n.m.)	Capa Externa (> 3050 m s.n.m.)
0–15 V	0,05 mm	0,10 mm	0,10 mm
16–30 V	0,05 mm	0,10 mm	0,10 mm
31–50 V	0,10 mm	0,60 mm	0,60 mm
51–100 V	0,10 mm	0,60 mm	1,50 mm
101–150 V	0,20 mm	0,60 mm	3,20 mm
151–170 V	0,20 mm	1,25 mm	3,20 mm
171–250 V	0,20 mm	1,25 mm	6,40 mm
251–300 V	0,20 mm	1,25 mm	12,50 mm
301–500 V	0,25 mm	2,50 mm	12,50 mm

Nota. Directrices de espaciamiento mínimo (clearance) adaptadas de la norma IPC-2221B (IPC – Association Connecting Electronics Industries, 2012).

- **Constante dieléctrica:** Entre los parámetros más importantes en los PCB está también la constante dieléctrica, usada para cálculos de parámetros en líneas de alta velocidad como referencia se tiene la Tabla 3.12.

Tabla 3.12

Constante dieléctrica relativa de materiales comunes en sustratos de PCB

Material	Constante Dieléctrica (ε_r)
FR4	3,8 – 4,8
FR5	4,2
Vidrio	6,0
Resina	3,0

Nota. Valores típicos de la permitividad eléctrica relativa (ε_r) evaluados a frecuencias estándar de operación.

3.3.9. Parámetros de diseño del PCB según la norma IPC-2221B

Después de revisar estos conceptos y ser validados con la norma IPC-2221B (IPC – Association Connecting Electronics Industries, 2012), se definieron los criterios de diseño que permiten obtener un PCB conforme a dicho estándar. La IPC-2221B constituye el documento genérico de referencia para el diseño de placas de circuito impreso de una, dos o más capas, y establece, entre otros aspectos, los requisitos de ancho de pista, separación entre conductores (*clearance*), distancias de fuga (*creepage*) y dimensiones de vías (IPC – Association Connecting Electronics Industries, 2012).

Con estos criterios claros, los parámetros adoptados para la fabricación de la placa de dos capas (35 µm de cobre, 1 oz) se resumen en la Tabla 3.14. A continuación se justifica cada grupo de valores a partir del estándar, de las capacidades del fabricante (JLCPCB, 2025) y de las corrientes reales estimadas en el presupuesto de potencia del sistema.

Ancho de pistas (capacidad de corriente)

El ancho mínimo de una pista se determina por la corriente que debe conducir y por el incremento de temperatura admisible debido al efecto Joule. La IPC-2221B deriva el área transversal de cobre necesaria a partir de la Ecuación 3.7, donde la corriente, el sobrecalentamiento y un coeficiente ligado a la ubicación de la pista (externa o interna) fijan la sección de cobre requerida (IPC – Association Connecting Electronics Industries, 2012).

Para diseños de alta exactitud en capacidad de corriente, el estándar moderno IPC-2152 reemplaza a estas curvas históricas; sin embargo, la formulación de la IPC-2221B sigue siendo válida y conservadora para placas de baja potencia como la presente (IPC, 2009).

$$A = \left(\frac{I}{k \cdot \Delta T^{\,0,44}} \right)^{1/0,725} \tag{3.7}$$

donde:

A : área transversal de la pista [mils^2]
I : corriente que circula por la pista [A]
ΔT : incremento de temperatura admisible sobre el ambiente [°C]
k : constante ($k = 0{,}048$ para pistas externas; $k = 0{,}024$ internas)

A partir del área obtenida, el ancho de la pista se calcula con la Ecuación 3.8, considerando el espesor de cobre de 1 oz ($t = 35$ µm $= 1{,}378$ mils):

$$W = \frac{A}{t} \tag{3.8}$$

Para la línea de mayor consumo del sistema (el riel de 3,3 V, que alimenta de forma conjunta al ESP32-S3-WROOM-1-N16R8, el transceptor CAN, la tarjeta MicroSD y la pantalla OLED), el presupuesto de potencia arrojó una corriente pico de aproximadamente 0,76 A. Diseñando con un margen de seguridad hasta $I = 1$ A, un sobrecalentamiento conservador de $\Delta T = 10\,°\text{C}$ y $k = 0{,}048$ (pista externa), la Ecuación 3.7 entrega $A \approx 16{,}3$ mils^2, que con la Ecuación 3.8 corresponde a un ancho mínimo de $W \approx 11{,}8$ mils $\approx 0{,}30$ mm. En otras palabras, 0,30 mm bastarían para 1 A; no obstante, se adoptó un ancho de **1,0 mm** para las pistas de potencia (entrada de 12 V y riel de 3,3 V).

Esta decisión, holgada respecto al mínimo, responde a tres criterios de diseño: (i) reducir la caída de tensión IR y la resistencia serie del riel, evitando caídas que

afecten la regulación de 3,3 V; (ii) disminuir la elevación de temperatura para mejorar la fiabilidad térmica, dado que un ancho de 1,0 mm en cobre de 1 oz admite del orden de 2,4 A con el mismo $\Delta T = 10\,^{\circ}\mathrm{C}$ (más de tres veces la corriente pico real); y (iii) ofrecer una vía de retorno de baja impedancia que mitiga el ruido del regulador conmutado. Para las pistas de señal de baja corriente (GPIO, I^2C, SPI de la MicroSD) se adoptó 0,20 mm, ancho gobernado por la densidad de ruteo y el límite de fabricación (no por la corriente, ya que estas líneas conducen apenas algunos miliamperios), y se mantiene por encima del mínimo de 0,127 mm del fabricante (JLCPCB, 2025). Finalmente, el par diferencial del bus CAN (CANH/CANL) se ruteó con pistas de 0,30 mm, acopladas y de igual longitud; a 500 kbps y sobre trazos cortos la impedancia diferencial nominal de 120 Ω no es crítica, por lo que prima el apareamiento geométrico y de longitud del par sobre el control estricto de impedancia.

Separación entre conductores (*clearance*)

La separación mínima entre conductores se establece a partir de la tensión de trabajo pico entre ellos, según la Tabla 6-1 de la IPC-2221B (IPC – Association Connecting Electronics Industries, 2012). El riel de mayor tensión del diseño es la entrada de 12 V proveniente del conector OBD-II; aun considerando transitorios del sistema eléctrico automotriz, la tensión de trabajo se mantiene por debajo de 30 V. La Tabla 3.13 reproduce los valores relevantes del estándar para conductores externos sin recubrimiento.

Tabla 3.13

Separación mínima entre conductores externos sin recubrimiento (IPC-2221B, Tabla 6-1)

Tensión pico [V]	**Hasta 3050 m (B1)** [mm]	**Sobre 3050 m (B2)** [mm]
≤ 15	0,10	0,10
≤ 30	0,10	0,10
≤ 50	0,60	0,60
≤ 100	0,60	1,50
≤ 150	0,60	3,20

Nota. Valores derivados de la Tabla 6-1 de la norma IPC-2221B (IPC – Association Connecting Electronics Industries, 2012). La columna B1 corresponde a conductores externos sin recubrimiento desde el nivel del mar hasta 3050 m; la columna B2, a la misma condición por encima de 3050 m.

Un aspecto particular de este proyecto es la altitud de La Paz (≈ 3600 m s. n. m.), superior al umbral de 3050 m que separa las categorías B1 y B2 de la norma. Dado que la reducción de presión atmosférica disminuye la rigidez dieléctrica del aire (ley de Paschen), la IPC-2221B exige mayores separaciones por encima de 3050 m. Sin embargo, como se observa en la Tabla 3.13, para tensiones de trabajo ≤ 30 V (rango en el que opera la totalidad de este diseño) el valor exigido es idéntico en ambas categorías (0,10 mm): el penalti por altitud solo se vuelve significativo a partir de ~ 100 V (por

ejemplo, 0,60 mm frente a 1,50 mm). Por tanto, se concluye que la altitud de La Paz no impone requisitos adicionales de separación para esta placa de baja tensión, aunque sí es un factor determinante en el modelo de cálculo de consumo de combustible por su efecto sobre la densidad del aire.

Sobre esta base, y siempre por encima del mínimo del fabricante de 0,127 mm (JLCPCB, 2025), se adoptaron separaciones conservadoras: 0,20 mm como *clearance* general de señal y 0,50 mm en la sección de entrada de 12 V, esta última para absorber con holgura eventuales sobretensiones del bus de 12 V del vehículo y facilitar el ensamblaje. Todos los conductores se mantienen además a $\geq 0{,}30$ mm del borde de la placa, según la capacidad del fabricante (JLCPCB, 2025).

Una vez establecidos estos criterios y reglas de diseño, se procedió con su implementación en el software Altium Designer. Como resultado, se consolidó el esquemático completo del circuito y el ruteado de la placa de circuito impreso (PCB), garantizando la correcta disposición de vías y distancias de separación (clearance) bajo los lineamientos de la norma IPC. El esquemático eléctrico completo, la disposición de capas y el ruteado de la PCB, y el reporte de la pila de capas se presentan en el Apéndice M.

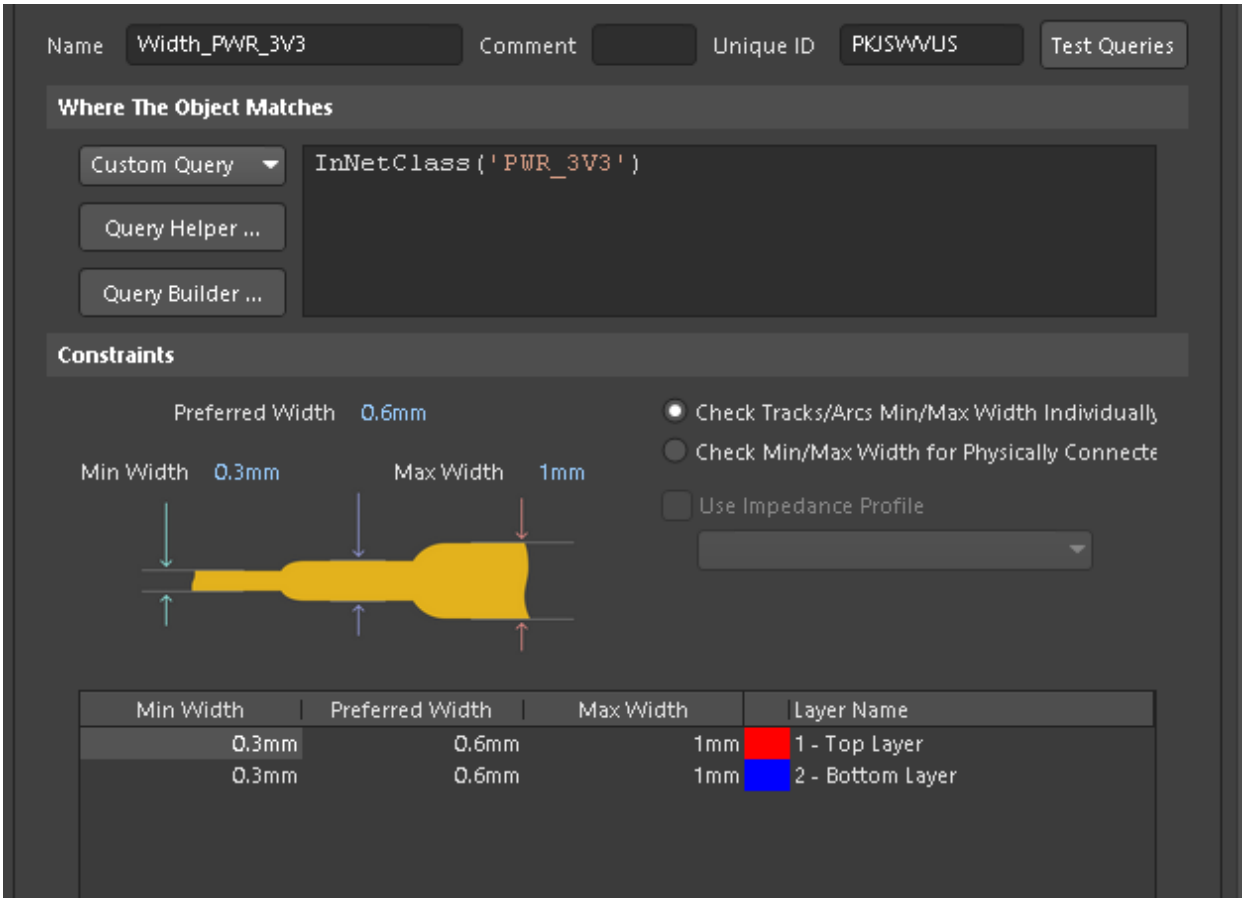
Tabla 3.14

Parámetros de diseño del PCB conforme a IPC-2221B y capacidades de JLCPCB

Parámetro	Valor adoptado	Criterio / Referencia
Ancho de pistas de potencia (12 V, 3,3 V)	1,00 mm	Margen ≈ 3× sobre 0,76 A pico; baja caída IR (IPC – Association Connecting Electronics Industries, 2012)
Ancho de pistas de señal (GPIO, I^2C, SPI)	0,20 mm	Densidad de ruteo; > mínimo de fábrica (JLCPCB, 2025)
Ancho del par diferencial CAN (CANH/CANL)	0,30 mm	Par acoplado y apareado en longitud
Separación general de señal	0,20 mm	> mínimo IPC y de fábrica (IPC – Association Connecting Electronics Industries, 2012; JLCPCB, 2025)
Separación en sección de 12 V	0,50 mm	Margen ante transitorios automotrices
Separación mínima absoluta (fábrica)	0,127 mm	Límite 2 capas, 1 oz (JLCPCB, 2025)
Diámetro de perforación de vía	0,30 mm	Vía estándar fiable (JLCPCB, 2025)
Diámetro de *pad* de vía	0,60 mm	Anillo anular 0,15 mm (JLCPCB, 2025)
Anillo anular mínimo	0,15 mm	Capacidad de fábrica (JLCPCB, 2025)
Distancia cobre–borde de placa	0,30 mm	Capacidad de fábrica (JLCPCB, 2025)
Espesor de cobre	1 oz (35 µm)	Estándar de uso general (JLCPCB, 2025)
Espesor de placa	1,60 mm	FR-4 estándar
Acabado superficial	HASL sin plomo	Estándar de bajo costo (JLCPCB, 2025)
Dimensiones del PCB	40 × 60 mm	Restricción mecánica del encapsulado OBD-II

Nota. Parámetros validados contra la norma IPC-2221B (IPC – Association Connecting Electronics Industries, 2012) y las capacidades de fabricación de JLCPCB (JLCPCB, 2025). Los anchos de pista se verificaron con la Ecuación 3.7 para la corriente pico estimada del sistema.

Figura 3.37
Anchos de pista en el ruteo del PCB



Nota. Elaboración propia a partir del diseño en Altium Designer.

3.3.10. Lista de materiales (BOM)

La lista de materiales (*Bill of Materials*, BOM) consolida todos los componentes del dispositivo, organizados por bloque funcional: etapa de alimentación de dos rieles basada en el convertidor reductor síncrono TPS54302DDCR (12 V → 5 V) seguido de los reguladores lineales AMS1117-3,3 y AP2112K-3,3TRG1 (5 V → 3,3 V), unidad de procesamiento ESP32-S3-WROOM-1-N16R8 (Espressif Systems, 2023a), interfaz de bus CAN con el transceptor SN65HVD230 (Texas Instruments, 2015), almacenamiento en MicroSD, interfaz de usuario OLED y conector OBD-II. Los valores de los componentes pasivos de la etapa de potencia se seleccionaron siguiendo el procedimiento de diseño y el esquemático típico de aplicación del fabricante del regulador, detallado en la subsección anterior. La Tabla 3.15 presenta un resumen; el detalle de fabricación (huellas, designadores agrupados y códigos de parte LCSC) se presenta en el Apéndice N.

Tabla 3.15
Lista de materiales (BOM) del dispositivo

Ref.	Componente	Valor / Parte	Encap.	Cant.
Etapa de alimentación, riel 1 de 2 (12 V → 5 V, síncrona)				
U1	Convertidor reductor (buck) síncrono	TPS54302DDCR	SOT-23-6	1
L1	Inductor de potencia	10 µH	SMD	1
D2	Diodo protección polaridad inv.	SS34 (3 A/40 V)	SMA	1
F1	Fusible reseteable PTC	$I_{hold} \approx 1{,}5$ A	1812	1
C1,C2	Condensador de entrada	4,7 µF / 50 V X7R	1206	2
C3	Condensador de entrada (HF)	0,01 µF	0603	1
C4,C5	Condensador de salida (44 µF efectivos)	22 µF / 16 V X5R	1206	2
C6	Condensador de mejora transitoria	75 pF	0603	1
R1	Resistor divisor realim. (sup.)	100 kΩ	0603	1
R2	Resistor divisor realim. (inf.)	13,3 kΩ	0603	1
Etapa de alimentación, riel 2 de 2 (5 V → 3,3 V, dos rieles LDO)				
U4	LDO riel 1 (ESP32-S3, hasta 1 A), siempre activo	AMS1117-3,3	SOT-223	1
U5	LDO riel 2 (periféricos, hasta 600 mA)	AP2112K-3,3TRG1	SOT-23-5	1
C14,C15	Desacople entrada/salida AMS1117	10 µF	1206	2
C16,C17	Desacople entrada/salida AP2112	1 µF	0603	2
Unidad de procesamiento				
U2	Microcontrolador (módulo)	ESP32-S3-WROOM-1-N16R8	Módulo	1
R3	*Pull-up* EN	10 kΩ	0603	1
C7	Condensador EN	0,1 µF	0603	1
C8–C11	Desacople de alimentación	0,1 µF	0603	4
C12	Reserva de carga (*bulk*) 3,3 V	10 µF	0805	1
SW1	Pulsador de *reset*	—	SMD	1
SW2	Pulsador BOOT (GPIO0)	—	SMD	1
Interfaz de bus CAN				
U3	Transceptor CAN	SN65HVD230	SOIC-8	1
R4	Resistor de control de pendiente	10 kΩ	0603	1
R5	Resistor de terminación (opcional)[a]	120 Ω	0805	1
C13	Desacople	0,1 µF	0603	1
Almacenamiento y pantalla				
J2	Zócalo MicroSD (push-push)	—	SMD	1
R6–R9	*Pull-up* líneas SPI	10 kΩ	0603	4
DISP1	Pantalla OLED I²C 0,96"	SSD1306	Módulo	1
R10,R11	*Pull-up* I²C (SDA/SCL)	4,7 kΩ	0603	2

Continúa en la siguiente página...

Tabla 3.15 (continuación)

Ref.	Componente	Valor / Parte	Encap.	Cant.
Conectores e indicadores				
J1	Conector OBD-II (12 V, GND, CANH, CANL)	4 pines	—	1
LED1	LED indicador de alimentación	Verde	0805	1
LED2	LED indicador de estado	Azul	0805	1
R12,R13	Resistor limitador de LED	1 kΩ	0805	2

Nota. Elaboración propia. Los valores de la etapa de alimentación siguen el procedimiento de diseño del fabricante para el TPS54302DDCR, detallado en la subsección de diseño de hardware de este capítulo. [a]El resistor de terminación de 120 Ω se deja como *footprint* opcional (no poblado por defecto), dado que el dispositivo se conecta como nodo en derivación a un bus CAN que ya posee sus terminaciones en el vehículo.

3.3.11. Capacidades de fabricación de JLCPCB

El PCB se fabricó mediante el servicio de prototipado de JLCPCB, integrado de forma nativa con la herramienta EasyEDA empleada en el diseño. Para garantizar una producción correcta y de alto rendimiento (*Right-First-Time*), las reglas de diseño (DRC) del proyecto se ajustaron a las capacidades del fabricante para placas de dos capas en cobre de 1 oz (JLCPCB, 2025). La Tabla 3.16 compara los límites de fabricación con los valores adoptados en este diseño, evidenciando que todos los parámetros se mantienen dentro de la ventana de proceso estándar (la más económica y de mayor rendimiento de producción).

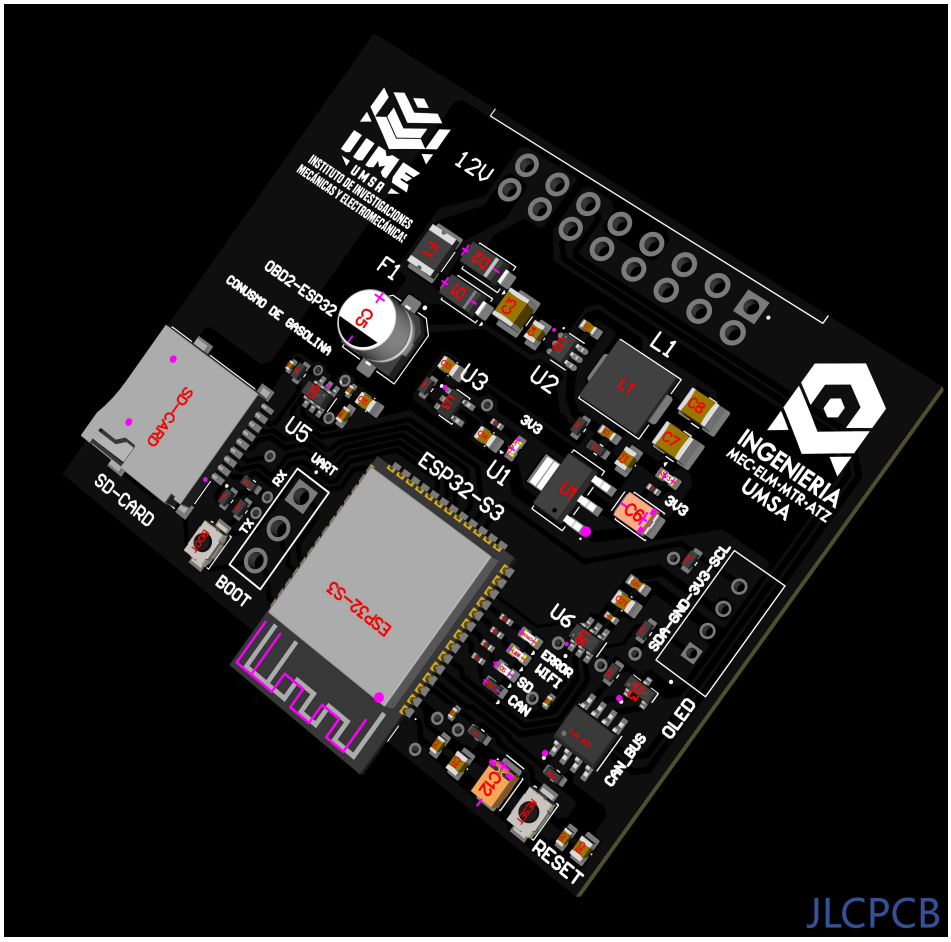
Tabla 3.16

Capacidades de fabricación de JLCPCB (2 capas, 1 oz) frente al diseño

Parámetro de fabricación	Mínimo JLCPCB	Diseño
Ancho mínimo de pista	0,127 mm (5 mil)	0,20 mm
Separación mínima de pista	0,127 mm (5 mil)	0,20 mm
Diámetro mínimo de perforación	0,20 mm	0,30 mm
Anillo anular mínimo	0,15 mm	0,15 mm
Distancia cobre–borde	0,30 mm	0,30 mm
Altura mínima de texto (serigrafía)	1,00 mm	≥ 1 mm
Ancho mínimo de línea (serigrafía)	0,15 mm	$\geq 0{,}15$ mm
Expansión de máscara antisoldante	0,05 mm	0,05 mm
Espesor de cobre	1 oz (35 µm)	1 oz
Tamaño de placa (rango)	5×5 a 400×500 mm	40×60 mm

Nota. Capacidades según la documentación de fabricación de JLCPCB (JLCPCB, 2025). Todos los valores del diseño cumplen o superan el mínimo del fabricante, situándose en la categoría de proceso estándar.

Figura 3.38
Verificación de fabricabilidad (DFM) del PCB en JLCPCB



Nota. Captura obtenida del servicio de fabricación JLCPCB (JLCPCB, 2025).

3.4. Modelo matemático de estimación de consumo de combustible

En la sección 2.7 se desarrolló el marco teórico de los modelos de estimación de consumo de combustible. En este apartado, en cambio, se deducen y formalizan las ecuaciones que se implementarán directamente en el software del dispositivo, partiendo de principios físicos (conservación de masa, ley de los gases ideales y relación aire-combustible) y aplicando los criterios de ingeniería que rigen la selección de variables, parámetros y fuentes de datos (PIDs OBD-II). El objetivo es obtener un conjunto de expresiones cerradas, computables en tiempo real sobre el ESP32, junto con conclusiones operativas sobre qué PIDs deben consultarse y con qué prioridad.

3.4.1. Método por sensor MAF

El método más directo parte de la medición del flujo másico de aire admitido por el motor, $\dot{m}_{air}$ [g/s], reportado por el sensor MAF (*Mass Air Flow*) a través del PID 0x10 (SAE International, 2014). La estimación de combustible se fundamenta en la relación aire-combustible (*Air-Fuel Ratio*, AFR), definida como el cociente entre la masa de aire y la masa de combustible de la mezcla (Heywood, 2018):

$$\mathrm{AFR} = \frac{\dot{m}_{air}}{\dot{m}_{fuel}} \tag{3.9}$$

Para la gasolina, la relación estequiométrica es $\mathrm{AFR}_s \approx 14{,}7$ (14,7 kg de aire por cada kg de combustible) (Heywood, 2018; Robert Bosch GmbH, 2014a). La mezcla real rara vez es exactamente estequiométrica; la ECU la regula en torno a $\lambda = 1$, donde λ es el factor lambda (relación de equivalencia inversa) disponible en el PID 0x44. La relación real se expresa entonces como $\mathrm{AFR} = \lambda \cdot \mathrm{AFR}_s$, y despejando $\dot{m}_{fuel}$ de la Ecuación 3.9 se obtiene el flujo másico de combustible:

$$\dot{m}_{fuel} = \frac{\dot{m}_{air}}{\lambda \cdot \mathrm{AFR}_s} \tag{3.10}$$

El flujo másico se convierte a flujo volumétrico dividiendo por la densidad del combustible, $\rho_{fuel} \approx 745$ kg/m^3 para la gasolina (Robert Bosch GmbH, 2014a). Considerando que 1 kg/m^3 = 1 g/L y que $\dot{m}_{air}$ se expresa en g/s, el caudal de combustible en litros por hora (FuelRate) resulta de incorporar el factor de conversión temporal 3600 s/h:

$$\mathrm{FuelRate\ [L/h]} = \frac{3600 \cdot \dot{m}_{air}}{\lambda \cdot \mathrm{AFR}_s \cdot \rho_{fuel}} \tag{3.11}$$

Finalmente, el consumo normalizado por distancia, métrica de referencia para el usuario, se obtiene relacionando el caudal con la velocidad del vehículo v [km/h] (PID 0x0D):

$$\mathrm{FC\ [L/100\,km]} = \frac{\mathrm{FuelRate\ [L/h]}}{v\ \mathrm{[km/h]}} \cdot 100 \tag{3.12}$$

donde:

- $\dot{m}_{air}$: flujo másico de aire [g/s] (PID 0x10)
- $\dot{m}_{fuel}$: flujo másico de combustible [g/s]
- AFR_s : relación aire-combustible estequiométrica ($\approx 14{,}7$)
- λ : factor lambda (PID 0x44; $\lambda = 1$ en estequiometría)
- ρ_{fuel} : densidad de la gasolina (≈ 745 kg/m^3)
- v : velocidad del vehículo [km/h] (PID 0x0D)

Criterio de ingeniería. El método MAF es el más exacto porque mide *directamente* la masa de aire, sin requerir el modelado de la eficiencia volumétrica ni el conocimiento de la cilindrada; su única dependencia paramétrica es ρ_{fuel} y el supuesto sobre λ. Su limitación es la disponibilidad: exige que el vehículo soporte el PID 0x10, lo cual no está garantizado (véase la Subsección 3.4.5).

3.4.2. Método Speed-Density

Cuando el vehículo no dispone de sensor MAF (caso del Changan Honor analizado), el flujo másico de aire no se mide sino que se *estima* a partir de la presión absoluta del múltiple (MAP, PID 0x0B), la temperatura del aire de admisión (IAT, PID 0x0F) y el régimen del motor (RPM, PID 0x0C), junto con dos parámetros del motor: la cilindrada V_d y la eficiencia volumétrica η_v. Este es el método *Speed-Density* (Heywood, 2018; Pulkrabek, 2014).

El punto de partida es la ecuación de los gases ideales en su forma de densidad, $P = \rho RT$, de la cual se despeja la densidad del aire de admisión (Çengel and Boles, 2019):

$$\rho_{air} = \frac{\text{MAP}}{R_{air} \cdot \text{IAT}} \tag{3.13}$$

con $R_{air} = 287$ J/(kg·K) (constante específica del aire), MAP en Pa e IAT en kelvin. A continuación se cuantifica el aire admitido por unidad de tiempo. En un motor de cuatro tiempos, cada cilindro completa una carrera de admisión una vez cada dos vueltas del cigüeñal; por lo tanto, el número de ciclos de admisión completos por segundo es:

$$n_{ciclos} = \frac{\text{RPM}}{2} \cdot \frac{1}{60} = \frac{\text{RPM}}{120} \tag{3.14}$$

El factor 120 surge del producto de dos conversiones: el factor 2 del ciclo de cuatro tiempos (una admisión cada dos revoluciones) y el factor 60 del paso de minutos a segundos. **Es crítico aplicar este factor una sola vez**: dividir adicionalmente entre dos (un error frecuente, que confunde el ciclo de cuatro tiempos con una segunda corrección) deja el resultado en la mitad de lo que debería ser.

En condiciones ideales, el volumen de aire admitido por ciclo es la cilindrada V_d; la masa real admitida se corrige con la eficiencia volumétrica η_v. Combinando la densidad (Ecuación 3.13) y la frecuencia de ciclos (Ecuación 3.14), el flujo másico de aire estimado es:

Ecuación más importante del modelo de estimación

$$\dot{m}_{air} = \eta_v \cdot \rho_{air} \cdot V_d \cdot \frac{\text{RPM}}{120} = \eta_v \cdot \frac{\text{MAP}}{R_{air} \cdot \text{IAT}} \cdot V_d \cdot \frac{\text{RPM}}{120} \tag{3.15}$$

donde:

ρ_{air} : densidad del aire de admisión [kg/m^3]
MAP : presión absoluta del múltiple [Pa] (PID 0x0B)
R_{air} : constante específica del aire (287 J/(kg·K))
IAT : temperatura del aire de admisión [K] (PID 0x0F)
V_d : cilindrada total del motor [m^3]
η_v : eficiencia volumétrica [adimensional]
RPM : régimen del motor [rev/min] (PID 0x0C)

Una vez estimado $\dot{m}_{air}$ con la Ecuación 3.15, el flujo de combustible y el consumo se calculan exactamente igual que en el método MAF, mediante las Ecuaciones 3.10 a 3.12. De este modo, ambos métodos comparten la misma cadena de cálculo aguas abajo y solo difieren en cómo obtienen $\dot{m}_{air}$.

Criterio de ingeniería. El método Speed-Density habilita la estimación en vehículos sin MAF, a costa de introducir dos dependencias paramétricas: V_d (conocida y constante) y η_v (variable con el punto de operación). Su exactitud queda condicionada por la calidad del modelo de η_v y por la correcta lectura de la presión, factor especialmente sensible en altura, como se analiza a continuación.

3.4.3. Eficiencia volumétrica y corrección por altitud

La eficiencia volumétrica η_v se define como el cociente entre la masa de aire realmente admitida y la masa teórica que ocuparía la cilindrada a la densidad del aire de admisión (Heywood, 2018):

$$\eta_v = \frac{m_{air,\,real}}{\rho_{air} \cdot V_d} \tag{3.16}$$

En motores atmosféricos η_v se sitúa típicamente entre 0,70 y 0,95, y varía con el régimen y la carga (Heywood, 2018; Pulkrabek, 2014). En la implementación se adopta un valor calibrado en función del punto de operación, ajustado de forma adaptativa mediante la realimentación de los *fuel trims* de la ECU (lazo cerrado descrito en la arquitectura de control del sistema).

Efecto de la altitud de La Paz

El parámetro físico determinante en altura es la presión atmosférica. La densidad del aire (Ecuación 3.13) es directamente proporcional a la presión; una sobreestimación de esta se traduce, punto por punto, en una sobreestimación del aire admitido y, por tanto, del combustible calculado. La Paz se encuentra a ≈ 3600 m s. n. m., muy por encima de la referencia a nivel del mar empleada por defecto en la mayoría de las implementaciones genéricas. La presión a dicha altitud puede predecirse con el modelo de la Atmósfera Estándar Internacional (ISA) (International Organization for Standardization, 1975):

$$P(h) = P_0 \left(1 - \frac{L\,h}{T_0}\right)^{\dfrac{g\,M}{R\,L}} \tag{3.17}$$

con $P_0 = 101{,}32$ kPa, $T_0 = 288{,}15$ K, $L = 0{,}0065$ K/m, $g = 9{,}80$ m/s^2, $M = 0{,}029$ kg/mol y $R = 8{,}32$ J/(mol·K). Evaluando la Ecuación 3.17 en $h = 3600$ m se obtiene $P(3600) \approx 64{,}9$ kPa, valor que coincide con la presión barométrica medida en tiempo real mediante el PID 0x33 (≈ 65 kPa). Esta concordancia valida tanto el modelo ISA como la lectura del sensor y confirma la magnitud de la desviación respecto al nivel del mar.

El error cometido al asumir la presión estándar a nivel del mar ($P_0 = 101{,}3$ kPa) en lugar de la presión real de La Paz ($P_{baro} = 65$ kPa) es:

$$\varepsilon = \frac{P_0 - P_{baro}}{P_{baro}} = \frac{101{,}3 - 65}{65} \approx 0{,}558 = 55{,}8\% \tag{3.18}$$

Es decir, un modelo que *asumiera* una presión de admisión constante de nivel del mar en vez de medirla sobrestimaría el consumo en aproximadamente un **55,8%** en La Paz, una desviación inadmisible para la finalidad del dispositivo.

Precisión importante. La Ecuación (3.15) de este proyecto *no* incurre en este error, porque no asume P_0: emplea en cada ciclo la lectura real y absoluta del colector (MAP, PID 0x0B) tomada directamente de la ECU del vehículo. A mariposa cerrada, el MAP no puede superar la presión atmosférica local, sea cual sea esta; por construcción, la densidad del aire calculada con la Ecuación 3.13 ya refleja la altitud de operación sin necesitar ningún factor adicional. Aplicar además un multiplicador $K_{alt} = P_{baro}/P_0$ sobre el resultado (como haría una implementación que combinara MAP real y un factor de altitud) penalizaría la altitud dos veces sobre el mismo dato, y así se incurriría en un error de signo contrario. Por esta razón, el firmware evaluado en el Capítulo 4 no aplica K_{alt} como corrector multiplicativo.

El valor de P_{baro} conserva, no obstante, una utilidad real: como **dato de validación** del propio sensor MAP, no como corrector del cálculo. Con el contacto puesto y el motor apagado (*key-on, engine-off*), no existe vacío de admisión, por lo que el MAP debería leer un valor muy cercano a la presión atmosférica real; contrastar esa lectura contra el PID 0x33 (o, en su defecto, contra el modelo ISA evaluado en la altitud del sitio) permite verificar que el sensor MAP del vehículo está correctamente calibrado para la altitud de La Paz, antes de confiar en él durante la conducción. Esta es la prueba que se documenta en el Capítulo 4.

Criterio de ingeniería. Se prioriza la medición en tiempo real del PID 0x33 sobre el modelo ISA estático como fuente de P_{baro} para esta validación, ya que la primera captura también las variaciones meteorológicas de presión; el modelo ISA se reserva como respaldo (*fallback*) cuando el vehículo no soporta el PID 0x33 (implementado así en `can_obd2.cpp`, función `isaPressureKpaAt()`). La corrección de altitud del *consumo*

es, en cambio, automática e implícita en el método Speed-Density con MAP real, y no requiere un paso adicional de software.

3.4.4. Comparativa de métodos y criterio de selección

La Tabla 2.6 sintetiza las diferencias entre ambos métodos. El criterio de selección es jerárquico y se fundamenta en un principio de ingeniería: preferir siempre la medición directa frente a la estimación basada en modelos. En consecuencia, si el vehículo soporta el sensor MAF (PID 0x10) se emplea el método MAF; en caso contrario, se recurre al método Speed-Density. La Tabla 3.17 resume este árbol de decisión, y la Subsección 3.4.5 detalla cómo se determina el soporte de PIDs en tiempo de ejecución.

Tabla 3.17
Árbol de decisión para la selección del método de estimación

Condición evaluada	Resultado	Método seleccionado
PID 0x10 (MAF) soportado	Verdadero	Método MAF (medición directa)
PID 0x10 (MAF) soportado	Falso	Método Speed-Density (estimación)
PID 0x0B, 0x0C, 0x0F soportados	Requerido	Habilita Speed-Density como respaldo

Nota. Elaboración propia. La evaluación se realiza una sola vez durante la inicialización, a partir de la respuesta al PID 0x00 (SAE International, 2014).

El pseudocódigo del criterio de conmutación, en su forma más simple, es:

Algorithm 1: Criterio de selección del método de estimación

Entrada : Mapa de PIDs soportados (respuesta a PID 0x00)
Salida : Método de estimación seleccionado

```
if PID_0x10_soportado then
    metodo ← MAF
else
    metodo ← SPEED-DENSITY
return metodo
```

3.4.5. Consulta de soporte de PIDs (PID 0x00)

La decisión anterior exige conocer, en tiempo de ejecución, qué PIDs implementa cada vehículo. El estándar SAE J1979 resuelve esto mediante el PID 0x00, cuya respuesta es un campo de bits (*bitmask*) de cuatro bytes (denominados A, B, C y D) que codifica el soporte de los PIDs 0x01 a 0x20 (SAE International, 2014). La convención de codificación es la siguiente: el byte A es el más significativo, y dentro de cada byte el bit 7 es el

más significativo y el bit 0 el menos significativo. El bit más significativo del byte A corresponde al PID 0x01, y así sucesivamente hasta el bit menos significativo del byte D, que corresponde al PID 0x20. Un bit en 1 indica que el PID asociado está soportado.

De esta convención se desprende un resultado de gran utilidad para este proyecto: el soporte del sensor MAF (PID 0x10) queda determinado por el bit menos significativo del byte B. La Tabla 3.18 muestra el mapeo de los bits del byte B (que abarca los PIDs 0x09 a 0x10, precisamente los más relevantes para la estimación de consumo) y un ejemplo de decodificación.

Tabla 3.18

Decodificación del byte B de la respuesta al PID 0x00 (ejemplo: B = 0x3E)

Bit (byte B)	PID	Parámetro	Ejemplo (0x3E)
7	0x09	Presión de combustible	0 (no)
6	0x0A	Presión de combustible (ECU)	0 (no)
5	0x0B	MAP — presión del múltiple	1 (sí)
4	0x0C	RPM — régimen del motor	1 (sí)
3	0x0D	Velocidad del vehículo	1 (sí)
2	0x0E	Avance de encendido	1 (sí)
1	0x0F	IAT — temperatura de admisión	1 (sí)
0	0x10	**MAF — flujo másico de aire**	**0 (no)**

Nota. Codificación según SAE J1979 (SAE International, 2014). En el ejemplo, $0x3E = 0011\,1110_2$: el bit 0 (MAF) está en 0, por lo que el vehículo *no* soporta el PID 0x10 y el sistema conmuta a Speed-Density; los PIDs MAP, RPM, velocidad e IAT (bits 5 a 1) sí están disponibles, habilitando dicho método. Este patrón ilustra el caso de un vehículo sin sensor MAF.

Cabe señalar que el PID 0x33 (presión barométrica), necesario para la corrección por altitud, pertenece al rango 0x21–0x40; su soporte se consulta con el PID 0x20 de forma análoga. A partir de esta lectura de soporte, el dispositivo determina si dispone de los PIDs que requiere cada método de estimación. La Tabla 3.19 reúne los PIDs necesarios para el método MAF y para el método Speed-Density, indicando en cada caso si la lectura es obligatoria u opcional.

Tabla 3.19
PIDs requeridos por cada método de estimación

PID	Parámetro	Método MAF	Speed-Density
0x00	Descubrimiento de PIDs soportados	Obligatorio	Obligatorio
0x0C	RPM — régimen del motor	—	Obligatorio
0x0B	MAP — presión del múltiple	—	Obligatorio
0x0F	IAT — temperatura de admisión	—	Obligatorio
0x10	MAF — flujo másico de aire	Obligatorio	—
0x0D	Velocidad del vehículo	Obligatorio	Obligatorio
0x33	Presión barométrica (altitud)	—	Obligatorio[a]
0x44	Lambda — relación de equivalencia	Opcional	Opcional

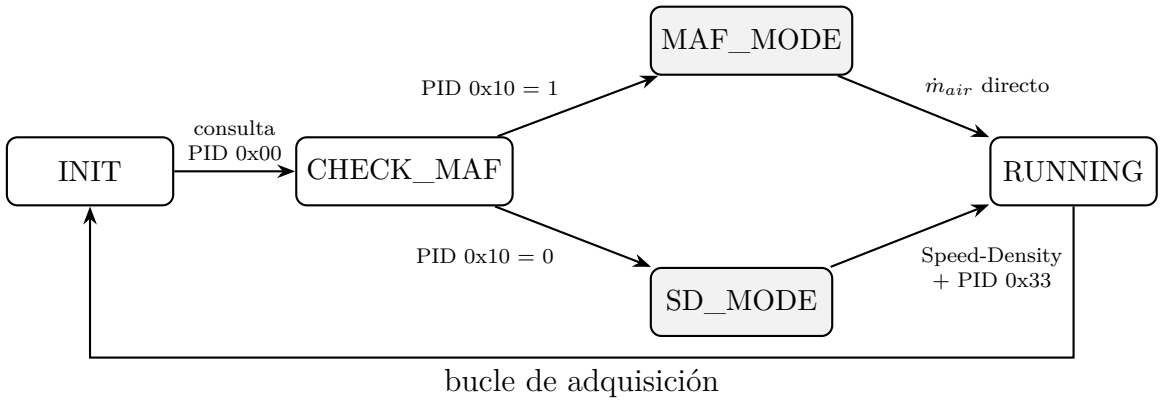
Nota. Elaboración propia con base en SAE J1979 (SAE International, 2014).

Conclusión sobre los PIDs. El conjunto mínimo de PIDs que el dispositivo debe consultar de forma obligatoria es: 0x00 (descubrimiento de soporte), 0x0C (RPM), 0x0D (velocidad), y según el método, 0x10 (MAF) o bien {0x0B, 0x0F} (MAP e IAT) más 0x33 (corrección por altitud). El PID 0x44 (lambda) se consulta para refinar la mezcla cuando está disponible; si no lo está, se asume $\lambda = 1$.

3.4.6. Algoritmo de conmutación automática

La lógica completa de operación se modela como una máquina de estados finitos que integra el descubrimiento de PIDs, la selección de método y el bucle de adquisición. El sistema transita por cuatro estados, ilustrados en la Figura 3.39: inicialización (INIT), verificación de MAF (CHECK_MAF), y los dos modos de operación (MAF_MODE o SD_MODE), que convergen al estado de ejecución continua (RUNNING).

Figura 3.39
Máquina de estados del algoritmo de conmutación automática



Nota. Elaboración propia. La transición desde CHECK_MAF se decide con el bit 0 del byte B de la respuesta al PID 0x00.

El Algoritmo 2 formaliza el comportamiento de la máquina de estados, incluyendo la corrección por altitud en el modo Speed-Density.

Algorithm 2: Conmutación automática y cálculo de consumo

Entrada : Tramas OBD-II del bus CAN; parámetros del motor V_d, η_v
Salida : FuelRate [L/h], FC [L/100 km]

Estado INIT
  Inicializar bus CAN (TWAI, 500 kbps); solicitar PID 0x00 y 0x20
  soporte ← decodificar_bitmask(respuesta)
Estado CHECK_MAF
if *soporte[0x10]* **then**
  modo ← MAF_MODE
else
  modo ← SD_MODE
Estado RUNNING **(bucle periódico)**
while *sistema activo* **do**
  if *modo = MAF_MODE* **then**
    $\dot{m}_{air}$ ← leer_PID(0x10)
  else
    MAP ← leer_PID(0x0B); IAT ← leer_PID(0x0F)
    RPM ← leer_PID(0x0C)
    if *soporte[0x33]* **then**
      P_{baro} ← leer_PID(0x33) `// corrección por altitud`
    $\rho_{air} \leftarrow \mathrm{MAP}/(R_{air} \cdot \mathrm{IAT})$
    $\dot{m}_{air} \leftarrow \eta_v \cdot \rho_{air} \cdot V_d \cdot \mathrm{RPM}/120$
  λ ← leer_PID(0x44) **si** disponible **si no** 1
  $\dot{m}_{fuel} \leftarrow \dot{m}_{air}/(\lambda \cdot 14{,}7)$
  FuelRate $\leftarrow 3600 \cdot \dot{m}_{fuel}/\rho_{fuel}$
  v ← leer_PID(0x0D)
  if $v > 0$ **then**
    FC ← 100· FuelRate / v
  Mostrar/registrar (FuelRate, FC); acumular litros y costo

Criterio de ingeniería. El descubrimiento de PIDs y la selección de método se ejecutan una sola vez en la inicialización (no en cada iteración), minimizando el tráfico en el bus CAN y la carga de cómputo del ESP32. El bucle RUNNING opera a una frecuencia de muestreo acorde con la dinámica del consumo (del orden de 1–5 Hz), suficiente para integrar litros y costo con exactitud sin saturar el canal de diagnóstico.

Alcance de la implementación evaluada. El Algoritmo 2 formaliza el *criterio de diseño* que justifica la elección del método de estimación: dado que el vehículo de prueba principal (Changan Honor) no soporta el PID 0x10 (Subsección 3.4.5), Speed-Density es la única vía disponible sin instalar un sensor adicional, y ese es el método que el

firmware descrito en la Sección 3.6 fija de forma permanente para ambos vehículos de validación del Capítulo 4 (incluido el Nissan Vanette, que sí soporta el PID 0x10, pero se evalúa igualmente en modo Speed-Density para mantener un único método comparable entre ambas unidades). La conmutación *automática y en tiempo de ejecución* entre MAF_MODE y SD_MODE que describe el estado CHECK_MAF de la máquina de estados no está implementada en esta versión del firmware; se documenta aquí como la extensión natural del sistema y se retoma como recomendación de trabajo futuro en el Capítulo 6.

3.5. Sistema de estimación con retroalimentación en lazo cerrado

Alcance de esta sección: estudio y diseño, no implementación. Todo lo desarrollado a continuación —el factor de calibración adaptativo K con filtro EMA (Subsección 3.5.6), la identificación en línea de la curva $\eta_v(N)$ por regresión (Subsección 3.5.9) y la conmutación automática MAF/Speed-Density (Subsección 3.4.5)— es un estudio y diseño formal de una arquitectura de estimador en lazo cerrado, no una implementación evaluada en este proyecto. Del contenido de esta sección, el firmware probado en el Capítulo 4 incorpora únicamente el corrector directo por ajustes de combustible del ECU (STFT + LTFT aplicados sobre el AFR efectivo, ya descrito en la Sección 3.6.5 del *Diseño de software*); el resto queda formalizado matemáticamente, pero sin construirse ni probarse en campo.

No se implementó por dos razones concretas, no por falta de desarrollo matemático: primero, cada uno de los tres componentes exige su propio protocolo de validación experimental independiente —un banco de pruebas con carga controlada para calibrar y verificar el factor K, y sesiones de conducción dedicadas para identificar $\eta_v(N)$ en línea sin reutilizar los datos ya empleados en la calibración estática del Capítulo 4—, lo que excede el tiempo disponible de un proyecto de grado si se suma al protocolo de validación en lazo abierto ya reportado en ese capítulo. Segundo, se priorizó deliberadamente contar con un sistema simple e íntegramente verificable, coherente con lo efectivamente probado en campo, en vez de uno más sofisticado pero sin evidencia experimental propia que respalde reportarlo como validado; el Capítulo 6 retoma los tres componentes como trabajo futuro concreto. Esta distinción entre lo formalizado y lo implementado se mantiene explícita a lo largo de toda la sección, para que el lector no infiera validación experimental de componentes que aún no se han puesto a prueba.

El método Speed-Density desarrollado en la sección anterior entrega una estimación *en lazo abierto* del flujo de aire y, por consiguiente, del consumo de combustible. Esta sección formaliza por qué dicha estimación es insuficiente por sí sola y cómo se la corrige incorporando la realimentación que el propio ECU del vehículo expone a través del puerto OBD-II. El resultado es un **estimador en lazo cerrado** cuyo modelo matemático final, con todas sus variables de entrada y de salida, se consolida al cierre de la sección.

3.5.1. Justificación del enfoque en lazo cerrado

Limitaciones de la estimación en lazo abierto

La estimación Speed-Density reposa sobre la ecuación de llenado del motor, en la que la eficiencia volumétrica η_v y los sensores de presión (P_{MAP}) y temperatura (T_{IAT}) determinan por completo el flujo de aire estimado. En un sistema embebido de bajo costo, sin acceso a banco dinamométrico, las principales fuentes de error son:

- **Incertidumbre de η_v.** La eficiencia volumétrica no es un dato de fábrica y varía con el régimen del motor, el envejecimiento mecánico y la apertura de válvulas. Un error del 10 % en η_v se traslada directamente como un error del 10 % en el caudal de combustible (Heywood, 2018).
- **Ruido y cuantización de sensores.** Las PID OBD-II entregan P_{MAP} con resolución de 1 kPa y T_{IAT} con resolución de 1 °C; ambos propagan ruido a la densidad ρ del modelo.
- **Condición no estequiométrica.** En enriquecimiento (aceleración a plena carga, arranque en frío) la relación aire–combustible se aparta de la estequiométrica, por lo que asumir $\lambda = 1$ introduce un sesgo sistemático.
- **Deriva de η_v con el envejecimiento.** A diferencia del efecto de altitud (ya resuelto por construcción al usar el MAP real en cada ciclo, como se explicó en la Subsección 3.4.3), la eficiencia volumétrica sí puede derivar con el tiempo (desgaste de válvulas, depósitos de carbón, calidad variable del combustible), y esa deriva no la corrige ningún sensor que el vehículo ya reporte por sí solo.

Al integrarse en el tiempo para obtener el consumo acumulado (L totales), estos errores no se cancelan, sino que se acumulan como deriva. Se requiere, por tanto, una señal de referencia que corrija la estimación de forma continua.

El ECU como lazo cerrado físico

El ECU del vehículo opera un lazo de control de combustible cerrado: mediante la sonda de oxígeno (lambda) mide la relación aire–combustible real y ajusta el tiempo de inyección para sostener la relación objetivo. Este ajuste se publica en el bus como ajustes de combustible(fuel trims): el ajuste de corto plazo STFT (PID `0x06`) y el de largo plazo LTFT (PID `0x07`). En consecuencia, el lazo cerrado físico ya existe en el vehículo; nuestra contribución es observarlo y emplear su señal de corrección para realimentar la estimación propia (Guzzella and Onder, 2010).

Estimador, no controlador: el marco del observador

Es necesaria una precisión conceptual que sostiene la defensa de este trabajo. El módulo desarrollado no actúa sobre los inyectores; es un dispositivo de monitoreo. Por ello no constituye un controlador en el sentido clásico, sino un estimador en lazo cerrado (sensor virtual). Su arquitectura reproduce la estructura predicción–corrección de un observador de estados (Åström and Murray, 2008):

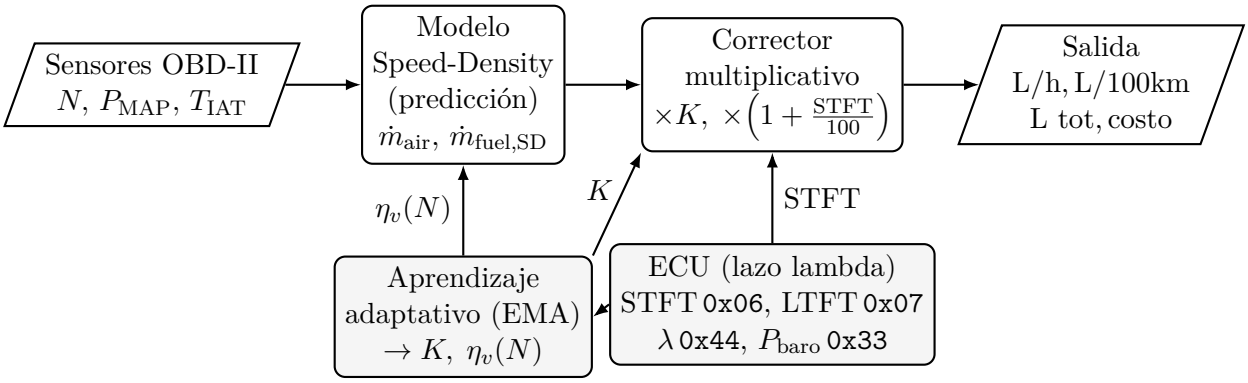
- *Predicción*: el modelo Speed-Density estima en lazo abierto el flujo de aire y de combustible a partir de N, P_{MAP} e T_{IAT}.
- *Corrección (innovación)*: los fuel trims del ECU, que codifican la medición real de la sonda lambda, corrigen la predicción.

De este modo, la planta es el motor; el controlador físico es el lazo de lambda del ECU; y el algoritmo aquí propuesto es el observador que se realimenta de la señal de corrección de ese controlador. Esta interpretación es directamente análoga al paso de actualización de un observador de Luenberger o de un filtro de Kalman, y responde de forma rigurosa a la pregunta de dónde reside el lazo cerrado del sistema.

3.5.2. Arquitectura del lazo cerrado

La Figura 3.40 presenta el diagrama de bloques del estimador. Se distinguen cinco bloques funcionales: (1) adquisición de señales OBD-II; (2) modelo base Speed-Density (predicción en lazo abierto); (3) corrector multiplicativo, que aplica la realimentación rápida del ECU; (4) bloque de aprendizaje adaptativo, que ajusta lentamente el factor de calibración K (y, opcionalmente, la curva η_v); y (5) cálculo de métricas de salida.

Figura 3.40
Diagrama de bloques del estimador de consumo en lazo cerrado.



Nota. Elaboración propia. El modelo Speed-Density opera como predictor en lazo abierto; los ajustes de combustible de la ECU realimentan el corrector multiplicativo (lazo rápido) y el bloque de aprendizaje adaptativo (lazo lento).

3.5.3. Modelo base Speed-Density (lazo abierto)

El bloque predictor reproduce la cadena de cálculo Speed-Density. La densidad del aire en el múltiple se obtiene de la ley de gases ideales:

$$\rho = \frac{P_{\mathrm{MAP}}}{R\,T_{\mathrm{IAT}}}, \qquad R = 287\ \mathrm{J/(kg\cdot K)}. \tag{3.19}$$

Tabla 3.20

Nomenclatura de las variables del modelo.

Símbolo	Descripción	Unidad
N	Régimen del motor	rpm
P_{MAP}	Presión absoluta del múltiple	kPa
T_{IAT}	Temperatura del aire de admisión	K
ρ	Densidad del aire en el múltiple	kg/m^3
V_d	Cilindrada total del motor	m^3
η_v	Eficiencia volumétrica	–
$\dot{m}_{\mathrm{air}}$	Flujo másico de aire	kg/s
$\dot{m}_{\mathrm{fuel}}$	Flujo másico de combustible	kg/s
λ	Relación de equivalencia comandada	–
AFR_s	Relación aire–combustible estequiom.	–
ρ_{fuel}	Densidad de la gasolina	kg/m^3
R	Constante específica del aire	J/(kg·K)
STFT	Ajuste de combustible de corto plazo	%
LTFT	Ajuste de combustible de largo plazo	%
K	Factor de calibración adaptativo	–
α	Coeficiente de suavizado (EMA)	–

Nota. Elaboración propia.

Para un motor de cuatro tiempos, que completa una admisión cada dos revoluciones, el flujo másico de aire teórico ponderado por la eficiencia volumétrica es:

$$\dot{m}_{\mathrm{air}} = \eta_v(N)\,\frac{P_{\mathrm{MAP}}\,V_d\,N}{120\,R\,T_{\mathrm{IAT}}} \qquad [\mathrm{kg/s}], \tag{3.20}$$

donde el factor $120 = 2 \times 60$ combina las dos revoluciones por ciclo y la conversión de minutos a segundos. La relación aire–combustible efectiva se forma a partir de la relación de equivalencia comandada λ (PID `0x44`):

$$\mathrm{AFR} = \mathrm{AFR}_s\,\lambda, \qquad \mathrm{AFR}_s = 14{,}7, \tag{3.21}$$

de donde el flujo de combustible estimado en lazo abierto resulta:

$$\dot{m}_{\mathrm{fuel,SD}} = \frac{\dot{m}_{\mathrm{air}}}{\mathrm{AFR}} = \frac{\dot{m}_{\mathrm{air}}}{\mathrm{AFR}_s\,\lambda} \qquad [\mathrm{kg/s}]. \tag{3.22}$$

La Ecuación (3.22) es la predicción; las subsecciones siguientes construyen el término de corrección.

3.5.4. Señales de error del ECU

La realimentación se compone de cuatro señales OBD-II (Tabla 3.21). Los ajustes STFT y LTFT son desviaciones porcentuales respecto del cálculo base de combustible del ECU: un valor positivo indica que el ECU está añadiendo combustible (mezcla pobre, mayor masa de aire que la estimada), y uno negativo, que lo está restando (mezcla rica). La presión barométrica P_{baro} (PID `0x33`) habilita la corrección de altitud, y λ (PID `0x44`) define la relación de equivalencia comandada.

Tabla 3.21
Señales OBD-II de realimentación empleadas por el estimador.

PID	Señal	Función en el lazo	Rango
`0x06`	STFT Banco 1	Corrección rápida (transitoria)	$\pm 100\,\%$
`0x07`	LTFT Banco 1	Corrección lenta (aprendida)	$\pm 100\,\%$
`0x44`	λ comandada	Relación de equivalencia	0–2
`0x33`	P_{baro}	Compensación de altitud	0–255 kPa

Nota. Elaboración propia.

Separación de escalas temporales

Una decisión de diseño central es asignar a cada ajuste un papel distinto según su escala temporal, evitando aplicarlos de forma redundante:

- El STFT oscila rápidamente en torno a cero conforme la sonda lambda conmuta; representa la corrección de transitorios y se aplica de forma instantánea en el corrector multiplicativo.
- El LTFT evoluciona lentamente y captura sesgos persistentes (altitud, desgaste, deriva de sensores); representa la corrección de régimen y alimenta la adaptación del factor K.

Esta partición reproduce la forma en que los ECU reales reparten los ajustes y, en términos prácticos, evita el doble conteo del LTFT que de otro modo introduciría una sobrecorrección sistemática.

3.5.5. Corrector multiplicativo

El corrector convierte la predicción Speed-Density en una estimación corregida aplicando el factor de calibración K y la corrección rápida del STFT:

$$\boxed{\dot{m}_{\text{fuel}} = K\,\dot{m}_{\text{fuel,SD}}\left(1 + \frac{\text{STFT}}{100}\right)} \qquad [\text{kg/s}]. \tag{3.23}$$

Análisis dimensional

El término K es adimensional, al igual que el factor $(1 + \text{STFT}/100)$, pues STFT es un porcentaje. Por tanto ambos factores son ganancias puras y la Ecuación (3.23) conserva las unidades de $\dot{m}_{\text{fuel,SD}}$, esto es, kg/s. La consistencia dimensional queda verificada.

Interpretación física

Cada factor de la Ecuación (3.23) tiene un significado físico definido:

- $\dot{m}_{\text{fuel,SD}}$ es la mejor estimación a priori obtenida del modelo de llenado del motor.
- $(1 + \text{STFT}/100)$ corrige la discrepancia instantánea entre la mezcla comandada y la medida real por la sonda lambda: si la mezcla es pobre (STFT > 0), el factor amplifica la estimación.
- K absorbe el sesgo sistemático de régimen que el modelo no captura (calidad del combustible, deriva, error residual de η_v), y se actualiza lentamente según la subsección 3.5.6.

3.5.6. Aprendizaje adaptativo — Factor K

El factor K se ajusta mediante un filtro de media móvil exponencial (EMA, por sus siglas en inglés) accionado por el LTFT. Sea K_n el valor en el instante de actualización n y α_K el coeficiente de suavizado:

$$K_n = (1 - \alpha_K)\, K_{n-1} + \alpha_K \underbrace{K_{n-1}\left(1 + \frac{\text{LTFT}}{100}\right)}_{\text{objetivo}}, \qquad \alpha_K = 0{,}20. \tag{3.24}$$

Reagrupando, la actualización se reduce a un crecimiento proporcional al ajuste de largo plazo:

$$K_n = K_{n-1}\left(1 + \alpha_K \frac{\text{LTFT}}{100}\right). \tag{3.25}$$

A medida que K corrige el sesgo, el LTFT tiende a cero y K se estabiliza, lo que confiere consistencia interna al lazo. Para garantizar robustez ante valores espurios del LTFT, el factor se acota a un rango físico admisible:

$$K_n \leftarrow \text{mín}\Big(\,\text{máx}(K_n,\, 0{,}70),\, 1{,}30\Big), \tag{3.26}$$

es decir, $K \in [0{,}70,\, 1{,}30]$ ($\pm 30\,\%$). El valor inicial es $K_0 = 1{,}00$. Sustituyendo la Ecuación (3.23), el caudal de combustible adaptado queda completamente definido por la predicción Speed-Density y las dos correcciones del ECU.

3.5.7. Análisis de estabilidad del filtro EMA

Para respaldar ingenierilmente el bloque adaptativo se analiza el EMA como un sistema lineal e invariante en el tiempo de primer orden. Considerando una entrada genérica u_n y salida y_n, la ecuación en diferencias del filtro es $y_n = (1-\alpha)\, y_{n-1} + \alpha\, u_n$. Aplicando la transformada Z (Oppenheim and Schafer, 2010):

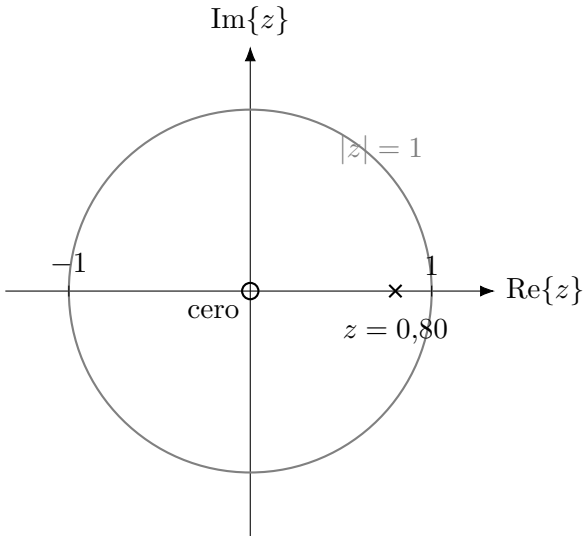
$$H(z) = \frac{Y(z)}{U(z)} = \frac{\alpha}{1-(1-\alpha)\, z^{-1}} = \frac{\alpha\, z}{z-(1-\alpha)}. \tag{3.27}$$

Polos, ceros y estabilidad

La función de transferencia presenta un cero en $z = 0$ y un polo en $z = 1-\alpha$. Para $\alpha_K = 0{,}20$ el polo se ubica en $z = 0{,}80$. Dado que $|0{,}80| < 1$, el polo está dentro del círculo unitario y el filtro es estable en sentido BIBO. La Figura 3.41 muestra el mapa de polos y ceros.

Figura 3.41

Mapa de polos y ceros del filtro EMA para $\alpha = 0{,}20$. El polo en $z = 0{,}80$ se encuentra dentro del círculo unitario, lo que garantiza estabilidad.



Nota. Elaboración propia.

Ganancia, constante de tiempo y ancho de banda

La ganancia en continua se obtiene evaluando $z = 1$ en la Ecuación (3.27): $H(1) = \alpha/\alpha = 1$, es decir, ganancia unitaria: una entrada constante se transmite sin atenuación, como corresponde a un filtro promediador. La constante de tiempo, en número de muestras, se deriva del decaimiento de la respuesta al impulso $(1-\alpha)^n$:

$$\tau = -\frac{1}{\ln(1-\alpha)} \text{ muestras}, \qquad \tau\Big|_{\alpha=0{,}20} = -\frac{1}{\ln 0{,}80} \approx 4{,}48 \text{ muestras}. \tag{3.28}$$

Con un periodo de actualización $T_s = 1\,\text{s}$, esto equivale a $\tau \approx 4{,}5\,\text{s}$. La frecuencia de corte aproximada del filtro pasa-bajos resulta $f_c \approx \alpha/(2\pi T_s) \approx 0{,}032\,\text{Hz}$, de modo que el lazo rechaza las fluctuaciones rápidas del LTFT y conserva su deriva lenta, que es justamente la información de interés.

3.5.8. Criterios de sintonización de α

La elección de α es un compromiso entre velocidad de aprendizaje y rechazo de ruido. Para un filtro EMA excitado por ruido blanco, el factor de reducción de varianza es $\alpha/(2-\alpha)$, por lo que la desviación estándar de salida relativa a la de entrada es $\sqrt{\alpha/(2-\alpha)}$. La Tabla 3.22 resume el efecto de α sobre el polo, la constante de tiempo τ, el tiempo de establecimiento al 2 % ($t_s = \ln(0{,}02)/\ln(1-\alpha)$) y la atenuación de ruido.

Tabla 3.22
Sintonización del coeficiente α del filtro EMA.

$\boldsymbol{\alpha}$	**Polo** $(1-\alpha)$	$\boldsymbol{\tau}$ **[muestras]**	$\boldsymbol{t_s}$ **(2%) [muestras]**	**Ruido de salida**
0,05	0,95	19,5	76,3	$0{,}16\,\sigma$
0,10	0,90	9,5	37,1	$0{,}23\,\sigma$
0,20	**0,80**	**4,5**	**17,5**	$\mathbf{0{,}33\,\sigma}$
0,30	0,70	2,8	11,0	$0{,}42\,\sigma$
0,50	0,50	1,4	5,6	$0{,}58\,\sigma$

Nota. Elaboración propia.

Elección de diseño

Se adopta $\alpha_K = 0{,}20$ para el factor K. Este valor sitúa el polo en $z = 0{,}80$, con una constante de tiempo de $\approx 4{,}5$ muestras y un rechazo de ruido que reduce la fluctuación a un tercio de la de entrada ($0{,}33\,\sigma$). Representa el compromiso buscado: suficientemente rápido para seguir la deriva del LTFT en pocos segundos, pero suficientemente lento para no propagar las oscilaciones de la conmutación de la sonda lambda. Para el refinamiento en línea de la curva η_v (sección 3.5.9) se emplea un valor más conservador, $\alpha_{\eta_v} = 0{,}05$, que prioriza la estabilidad de la curva por encima de la velocidad de convergencia.

3.5.9. Identificación de la curva de eficiencia volumétrica

La eficiencia volumétrica es el parámetro de mayor incidencia sobre la exactitud del modelo y, a la vez, el más difícil de conocer sin banco de pruebas. Esta subsección formaliza cómo obtener η_v como una curva suave en función del régimen, en lugar de valores aislados que fluctúan de un punto a otro.

Por qué η_v es una curva del régimen

La eficiencia volumétrica admite distintas referencias de densidad. Cuando se la refiere a las condiciones del múltiple (es decir, cuando ρ se calcula con P_{MAP} y T_{IAT}, como

hace Speed-Density en la Ecuación (3.19)) la eficiencia volumétrica deja de capturar el efecto de estrangulamiento y refleja únicamente la capacidad de respiración del motor: efectos de inercia y acústicos de los conductos, sincronización de válvulas y flujo en los puertos (Heywood, 1988). Estos fenómenos están gobernados por la velocidad media del gas y, por tanto, por el régimen N. La dependencia respecto del MAP es secundaria (asociada a la fracción de gases residuales). En consecuencia, a primer orden:

$$\eta_v \approx \eta_v(N), \tag{3.29}$$

lo que justifica representar la eficiencia volumétrica como una curva y no como un campo disperso. La curva presenta la forma característica de un único máximo: crece con el régimen, alcanza un máximo cerca del par máximo y decae a alto régimen por restricción de flujo.

Modelo paramétrico (garantía de suavidad)

Para garantizar suavidad por construcción, $\eta_v(N)$ no se almacena como una tabla de valores independientes, sino que se modela mediante un polinomio cúbico, capaz de reproducir la curva de un solo máximo con cuatro coeficientes:

$$\eta_v(N) = c_0 + c_1 N + c_2 N^2 + c_3 N^3. \tag{3.30}$$

El carácter polinómico elimina los saltos punto a punto: cualquier régimen intermedio se evalúa sobre una función continua y derivable. Para aplicaciones que exijan mayor fidelidad puede sustituirse el polinomio por un *spline* cúbico sin alterar el resto del modelo. Si se requiere capturar la dependencia secundaria del MAP, se añade una modulación lineal pequeña en torno a una presión de referencia P_{ref}:

$$\eta_v(N, P) = \eta_v(N) \Big[1 + k_P \left(P_{\text{MAP}} - P_{\text{ref}} \right) \Big], \qquad |k_P| \ll 1. \tag{3.31}$$

Obtención de la curva a partir de datos

Los coeficientes de la Ecuación (3.30) se identifican a partir de datos de operación del vehículo. El procedimiento separa explícitamente los estimados ruidosos (que se promedian o se descartan) de la curva suave (salida de la regresión). Consta de cuatro etapas, sintetizadas en el Algoritmo 3:

1. **Compuerta de régimen permanente.** Solo se aceptan muestras con el lazo del ECU en modo cerrado, con variación acotada del régimen y de la presión ($|\mathrm{d}N/\mathrm{d}t| < \varepsilon_N$, $|\mathrm{d}P/\mathrm{d}t| < \varepsilon_P$) y fuera de enriquecimiento ($\lambda \approx 1$). Esto descarta los transitorios, donde la hipótesis cuasiestática del modelo no es válida.
2. **Estimación por muestra.** En cada muestra aceptada se calcula el factor de corrección implícito en los ajustes del ECU y se obtiene un estimado puntual de

la eficiencia volumétrica:

$$\hat{\eta}_v = \eta_v(N)\left(1 + \frac{\mathrm{STFT} + \mathrm{LTFT}}{100}\right). \tag{3.32}$$

3. **Promediado por bin (rechazo de ruido).** El eje de régimen se divide en intervalos (*bins*) de, por ejemplo, 250 rpm. Dentro de cada bin se promedian los estimados $\hat{\eta}_v$ (mediante el mismo filtro EMA con $\alpha_{\eta_v} = 0{,}05$), de modo que el ruido aleatorio se cancela. Los bins con muestras insuficientes se marcan como de baja confianza.

4. **Regresión polinómica.** Se ajusta el polinomio cúbico de la Ecuación (3.30) a las medias por bin, ponderadas por el número de muestras. El coeficiente de determinación R^2 y el análisis de residuos verifican la calidad del ajuste. El resultado es la curva suave $\eta_v(N)$.

Algorithm 3: Identificación de la curva $\eta_v(N)$

Input: flujo de muestras $(N, P_{\mathrm{MAP}}, T_{\mathrm{IAT}}, \mathrm{STFT}, \mathrm{LTFT}, \lambda)$; curva semilla $\eta_v^0(N)$; umbrales $\varepsilon_N, \varepsilon_P$; $\alpha_{\eta_v} = 0{,}05$; mínimo de muestras por bin $N_{\mathrm{mín}}$.

Output: curva ajustada $\eta_v(N)$.

```
Inicializar bins η_v[k] ← η_v^0(N_k), cont[k] ← 0
foreach muestra do
    if lazo cerrado y |Ṅ| < ε_N y |Ṗ| < ε_P y λ ≈ 1 then
        ρ ← P_MAP/(R T_IAT)
        f ← 1 + (STFT + LTFT)/100
        η̂_v ← η_v(N) · f
        k ← bin(N)
        η_v[k] ← (1 − α_ηv) η_v[k] + α_ηv η̂_v
        cont[k] ← cont[k] + 1
Ajustar polinomio cúbico a {η_v[k] : cont[k] ≥ N_mín} ponderado por cont[k]
return η_v(N) ajustada
```

Estrategia de despliegue y problema de identificabilidad

Coexisten dos mecanismos adaptativos: el factor global K y la curva $\eta_v(N)$. Como una ganancia global puede ser absorbida por cualquiera de los dos, se presenta un problema de identificabilidad. Se resuelve separando autoridades y escalas temporales:

- **Opción A (línea base, recomendada).** La curva $\eta_v(N)$ se identifica *fuera de línea* a partir de un trayecto de calibración (compuerta, binning y regresión) y se congela. En operación, únicamente el factor K se adapta en línea, acotado a $\pm 30\,\%$, para corregir deriva residual, calidad del combustible y transitorios. No hay interacción entre lazos.

- **Opción B (extensión).** La curva $\eta_v(N)$ se refina además en línea mediante el EMA lento ($\alpha_{\eta_v} = 0{,}05$), mientras K permanece rápido ($\alpha_K = 0{,}20$) y acotado. La condición $\alpha_{\eta_v} \ll \alpha_K$ y el acotamiento de K mitigan cualquier conflicto entre ambos.

Sembrado y validación de la escala absoluta

Los ajustes del ECU informan la *forma* de la curva, pero no su escala absoluta, pues son magnitudes relativas. Por ello la curva se siembra con valores típicos de literatura para un motor atmosférico de gasolina, y su escala absoluta se *ancla* comparando el consumo acumulado estimado contra una medición tanque-a-tanque (*fill-to-fill*) sobre una ruta de prueba, el método de validación estándar para sistemas de consumo (Guzzella and Onder, 2010). La Tabla 3.23 presenta valores ilustrativos de las medias por bin y la Figura 3.42 muestra la curva resultante.

Tabla 3.23

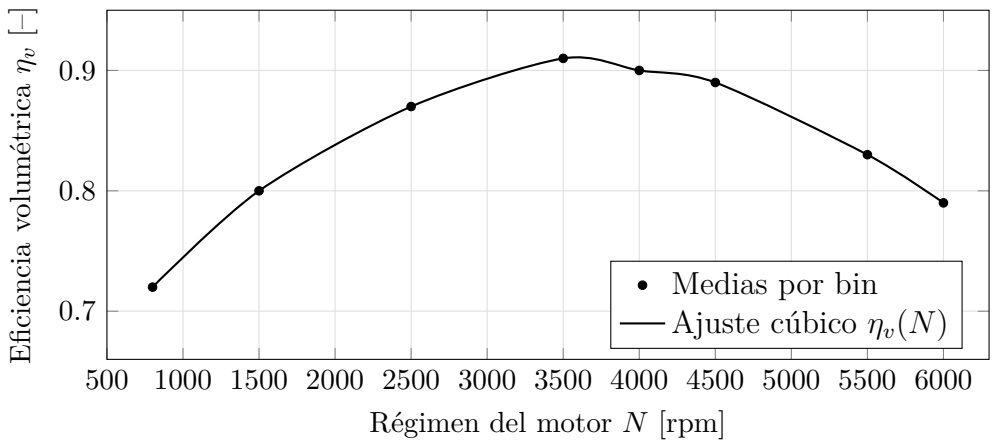
Valores ilustrativos de η_v por bin de régimen (medias por bin). Los valores definitivos se identifican para el vehículo de ensayo mediante el Algoritmo 3.

N [rpm]	η_v	N [rpm]	η_v
800	0,72	4 000	0,90
1 500	0,80	4 500	0,89
2 500	0,87	5 500	0,83
3 500	0,91	6 000	0,79

Nota. Elaboración propia.

Figura 3.42

Curva de eficiencia volumétrica identificada en función del régimen. Los marcadores corresponden a las medias por bin (datos ilustrativos) y la línea, al ajuste polinómico cúbico que constituye la curva $\eta_v(N)$ del modelo.



Nota. Elaboración propia. Datos ilustrativos; los valores definitivos se obtienen del trayecto de calibración del vehículo de ensayo.

3.5.10. Modelo matemático final y variables de entrada/salida

Reuniendo las ecuaciones anteriores, el modelo completo del estimador en lazo cerrado se expresa como la siguiente cadena de cálculo. A partir de las entradas OBD-II y de los parámetros del vehículo, el sistema produce el caudal instantáneo de combustible y las métricas derivadas:

$$\rho = \frac{P_{\mathrm{MAP}}}{R\,T_{\mathrm{IAT}}}, \tag{3.33}$$

$$\dot{m}_{\mathrm{air}} = \eta_v(N)\,\frac{P_{\mathrm{MAP}}\,V_d\,N}{120\,R\,T_{\mathrm{IAT}}}, \tag{3.34}$$

$$\dot{m}_{\mathrm{fuel,SD}} = \frac{\dot{m}_{\mathrm{air}}}{\mathrm{AFR}_s\,\lambda}, \tag{3.35}$$

$$\dot{m}_{\mathrm{fuel}} = K\,\dot{m}_{\mathrm{fuel,SD}}\left(1 + \frac{\mathrm{STFT}}{100}\right), \tag{3.36}$$

$$\dot{V}_{\mathrm{fuel}} = \frac{\dot{m}_{\mathrm{fuel}}}{\rho_{\mathrm{fuel}}}, \tag{3.37}$$

con $\mathrm{AFR}_s = 14{,}7$, $R = 287\ \mathrm{J/(kg\cdot K)}$ y $\rho_{\mathrm{fuel}} \approx 745\ \mathrm{kg/m^3}$. Las métricas de salida se obtienen por conversión de unidades e integración temporal:

$$Q_{\mathrm{inst}}\,[\mathrm{L/h}] = \dot{V}_{\mathrm{fuel}} \times 3{,}6 \times 10^6, \tag{3.38}$$

$$Q_{\mathrm{dist}}\,[\mathrm{L/100\,km}] = \frac{Q_{\mathrm{inst}}}{v} \times 100, \tag{3.39}$$

$$V_{\mathrm{total}}\,[\mathrm{L}] = \int_0^t \dot{V}_{\mathrm{fuel}}\,\mathrm{d}t, \tag{3.40}$$

$$\mathrm{Costo}\,[\mathrm{Bs}] = V_{\mathrm{total}} \times \mathrm{precio}\,[\mathrm{Bs/L}], \tag{3.41}$$

donde v es la velocidad del vehículo (PID `0x0D`). El factor $3{,}6 \times 10^6$ convierte $\mathrm{m^3/s}$ a L/h ($\times 10^3\,\mathrm{L/m^3} \times 3600\,\mathrm{s/h}$). La Tabla 3.24 consolida todas las variables de entrada, los parámetros y las salidas del modelo.

En síntesis, el modelo final propuesto integra la predicción física del método Speed-Density (Ecuaciones (3.33)–(3.35)) con dos mecanismos de realimentación de distinta escala temporal: la corrección rápida del STFT y la adaptación lenta del factor K (Ecuación (3.36)), apoyados en una curva de eficiencia volumétrica $\eta_v(N)$ identificada de los propios datos del vehículo. Esta arquitectura de estimador en lazo cerrado está diseñada para convertir una estimación abierta sensible al error de η_v en un sistema robusto, autocalibrable y trazable frente a una referencia física. Como se precisó al inicio de la sección, el prototipo evaluado en el Capítulo 4 implementa el subconjunto de esta arquitectura correspondiente a la corrección directa por STFT + LTFT; el factor K adaptativo y la identificación en línea de $\eta_v(N)$ quedan formalizados como la vía de

Tabla 3.24

Variables de entrada, parámetros y salidas del modelo en lazo cerrado.

Categoría	Variable	Origen / valor	Unidad
Entradas (OBD-II)	N	PID `0x0C`	rpm
	P_{MAP}	PID `0x0B`	kPa
	T_{IAT}	PID `0x0F`	°C $\rightarrow$ K
	λ	PID `0x44`	–
	STFT	PID `0x06`	%
	LTFT	PID `0x07`	%
	v	PID `0x0D`	km/h
Parámetros	V_d	Ficha del vehículo	m^3
	AFR_s	14,7 (gasolina)	–
	ρ_{fuel}	≈ 745	kg/m^3
	R	287	J/(kg·K)
	$\eta_v(N)$	Identificada (Alg. 3)	–
	α_K, α_{η_v}	0,20 / 0,05	–
Estados adaptativos	K	EMA, $\in [0{,}70, 1{,}30]$	–
	$\eta_v[k]$	Bins por régimen	–
Salidas	$\dot{m}_{\text{air}}$	Ec. (3.34)	kg/s
	$\dot{m}_{\text{fuel}}$	Ec. (3.36)	kg/s
	Q_{inst}	Ec. (3.38)	L/h
	Q_{dist}	Ec. (3.39)	L/100 km
	V_{total}	Ec. (3.40)	L
	Costo	Ec. (3.41)	Bs

Nota. Elaboración propia.

mejora que el Capítulo 6 recomienda para satisfacer, en una versión futura del firmware, los requisitos de exactitud y confiabilidad de largo plazo del proyecto.

3.6. Diseño de software

El código completo del firmware descrito en esta sección está disponible en el repositorio de GitHub del proyecto (Calle, 2026), carpeta `CODIGOS/esp32s3_obd2_speeddensity/`.

Esta sección retoma el capítulo de diseño de software y describe, parte por parte, el firmware que corre en el ESP32-S3: las herramientas con las que se construyó, cómo reparte el trabajo entre tareas, el protocolo con el que habla con el vehículo, el método de cálculo del consumo, cómo persiste los datos, cómo expone un panel de control, y el criterio detrás de cada elección de componente y de librería.

3.6.1. Herramientas y entorno de desarrollo

Visual Studio Code + PlatformIO

El firmware se desarrolla en Visual Studio Code con la extensión PlatformIO IDE, en vez del IDE oficial de Arduino. La diferencia no es cosmética:

- **Toolchain embebido integrado.** PlatformIO gestiona la descarga y versión exacta del framework (`framework-arduinoespressif32`) (Espressif Systems, 2024a), el compilador cruzado (`toolchain-xtensa-esp32s3`) y `esptool` para programar, todo declarado y fijado en `platformio.ini` (PlatformIO, 2024). Esto hace que el *build* sea reproducible entre máquinas (un requisito real una vez que el proyecto tiene múltiples archivos fuente y dependencias externas), algo que el modelo de "sketch único" del IDE de Arduino maneja peor.
- **Autocompletado real de C++.** Con un servidor de lenguaje indexando todo el árbol de `src//include/`, moverse entre los siete archivos fuente del proyecto y sus cabeceras compartidas (`config.h`, `types.h`) es inmediato.
- **Terminal integrado.** Compilar, subir y abrir el monitor serie desde el mismo terminal donde se edita el código evita el cambio de contexto entre editor y herramienta externa, y deja el registro de errores de compilación con enlaces directos a archivo y línea.

Por qué FreeRTOS

Un sistema que a la vez sondea un bus CAN sensible a tiempos, refresca una pantalla, escribe en una tarjeta SD y sirve un panel web por WiFi es, por naturaleza, un problema **concurrente**. Hay dos formas razonables de resolverlo: un único `loop()` con máquinas de estado manuales y sondeo no bloqueante en cada punto donde se podría esperar (viable en proyectos pequeños, pero frágil a medida que crecen), o un sistema

operativo de tiempo real con prioridades y primitivas de sincronización reales. Este proyecto usa la segunda opción.

FreeRTOS no es, en rigor, una dependencia externa que se agregó, es el planificador que el propio núcleo Arduino-ESP32 ya usa por debajo (Amazon Web Services, 2024); usarlo explícitamente es aprovechar la plataforma en vez de pelear contra ella con un `loop()` monolítico encima. Concretamente, esto permite:

- Darle al sondeo CAN la **prioridad más alta** de todas las tareas, garantizando que una transacción I2C lenta hacia el OLED o una descarga de CSV por WiFi nunca retrase la lectura del vehículo.
- Proteger el estado compartido con mutex reales en vez de banderas improvisadas.
- Que agregar una funcionalidad nueva sea, casi siempre, agregar una tarea nueva con una prioridad clara, en vez de insertar más lógica condicional dentro de un bucle ya complejo: el modo de bajo consumo de la Sección 3.6.7 solo requirió tocar el bucle de la tarea CAN y agregar una bandera de estado que las demás tareas ya sabían consultar.

Figura 3.43
Creación de tareas FreeRTOS en ***src/main.cpp****.*

```
statusLedsInit();
xTaskCreatePinnedToCore(statusLedsTask, "leds", STACK_LED_TASK, nullptr,
PRIO_LED_TASK, nullptr, CORE_LED_TASK);

bool oledOk = displayInit();
bool sdOk   = sdLoggerInit();
bool canOk  = canObd2Init();

webServerInit(); // reporta ErrorCode::WIFI_FAIL si el softAP falla

if (canOk) {
    xTaskCreatePinnedToCore(canObd2Task, "canObd2", STACK_CAN_TASK, nullptr,
    PRIO_CAN_TASK, nullptr, CORE_CAN_TASK);
}
if (oledOk) {
    xTaskCreatePinnedToCore(displayTask, "display", STACK_DISPLAY_TASK, nullptr,
    PRIO_DISPLAY_TASK, nullptr, CORE_DISPLAY_TASK);
}
if (sdOk) {
    xTaskCreatePinnedToCore(sdLoggerTask, "sdLogger", STACK_SD_TASK, nullptr,
    PRIO_SD_TASK, nullptr, CORE_SD_TASK);
}

```

Nota. Inicialización condicional de las tareas asignadas a los núcleos del ESP32 según el estado de la verificación del hardware.

3.6.2. Interfaz con el hardware

El software está organizado alrededor de las interfaces físicas del ESP32-S3 (Espressif Systems, 2024b,c). Cada periférico se eligió por una razón concreta, no por ser "lo que venía en el kit":

Tabla 3.25
Mapa de pines definido en `include/config.h`.

Bus	Señal	GPIO ESP32-S3
TWAI (CAN)	TX → SN65HVD230 CTX	GPIO4
TWAI (CAN)	RX ← SN65HVD230 CRX	GPIO5
I2C	SDA → OLED	GPIO8
I2C	SCL → OLED	GPIO9
SPI (SD)	SCK	GPIO12
SPI (SD)	MISO	GPIO13
SPI (SD)	MOSI	GPIO11
SPI (SD)	CS	GPIO10
LED	CAN activo	GPIO15
LED	Actividad SD	GPIO16
LED	Servidor activo	GPIO18
LED	Error	GPIO17

Nota. Asignación de pines del microcontrolador ESP32-S3 para los buses de comunicación CAN, I2C, SPI y los indicadores LED de estado.

Nota. Pines reservados (no usar): GPIO 26-37 (bus interno hacia el flash y la PSRAM octal del módulo N16R8) y GPIO 0/3/19/20/43-46 (pines de *strapping*, USB nativo y consola UART0 del chip).

Controlador CAN: TWAI integrado, no un chip externo

El ESP32-S3 trae un controlador CAN (TWAI) integrado en el propio chip (Espressif Systems, 2024c). Esa es la razón por la que el diseño no usa un controlador CAN externo por SPI como el MCP2515, común en proyectos sobre Arduino Uno/Nano que no tienen esta capacidad de fábrica: usar el controlador nativo ahorra un dispositivo, una interrupción y un bus SPI que ya está ocupado por la microSD, y evita la latencia adicional de tener que serializar cada trama CAN a través de otro periférico intermediario.

Transceptor CAN: SN65HVD230

El TWAI del ESP32-S3 solo genera niveles lógicos TTL; convertirlos a las señales diferenciales CAN_H/CAN_L del bus físico requiere un transceptor. Se eligió el SN65HVD230 (Texas Instruments, 2015) en concreto porque opera nativamente a 3,3 V, el mismo nivel lógico del ESP32-S3; así se evita un *level-shifter* que sí haría falta con

transceptores clásicos de 5 V como el MCP2551. CAN_H y CAN_L se conectan a los pines 6 y 14 del conector OBD2 (J1962).

Pantalla: SSD1306 OLED por I2C

Se eligió un OLED monocromático de 128 × 64 px con controlador SSD1306 (Solomon Systech, 2008) sobre I2C en vez de, por ejemplo, una pantalla TFT a color por SPI, por tres razones: consume solo 2 pines (I2C) dejando el SPI libre para la SD; su consumo (10-20 mA encendida, y se puede apagar por software, Sección 3.6.7) es bajo comparado con un TFT retroiluminado; y la resolución alcanza de sobra para los pocos números grandes que hace falta mostrar al conductor (no se necesita color ni gráficos complejos en el tablero físico: eso ya lo cubre el panel web).

Almacenamiento: microSD por SPI dedicado

La tarjeta microSD usa su propio bus SPI, sin compartirlo con ningún otro periférico. La alternativa, reutilizar un bus SPI ya en uso, se descartó porque el registro en la SD ocurre una vez por segundo de forma predecible mientras que una eventual escritura de mayor tamaño (por ejemplo, servir una descarga de CSV completa) no debe competir por el bus con otra responsabilidad no relacionada; un bus dedicado simplifica el análisis de tiempos y evita tener que coordinar dos periféricos distintos sobre el mismo SPI.

3.6.3. Arquitectura de tareas

Tabla 3.26

Tareas FreeRTOS, definidas en `include/config.h` *y creadas en* `src/main.cpp`.

Tarea	Función	Núcleo	Prioridad	Pila (bytes)
CAN/OBD2	`canObd2Task`	0	5	4096
SD logger	`sdLoggerTask`	1	4	6144
WebSocket push	`webSocketPushTask`	0	3	4096
OLED display	`displayTask`	1	2	4096
LEDs de estado	`statusLedsTask`	1	1	2048

Nota. Distribución de tareas en el sistema operativo en tiempo real FreeRTOS, especificando la asignación de núcleos del ESP32, prioridades de ejecución y tamaño de pila asignado a cada hilo.

El núcleo 0 concentra lo sensible a tiempos (sondeo CAN de alta prioridad + la pila WiFi/AsyncTCP, que en Arduino-ESP32 corre en ese mismo núcleo). El núcleo 1 queda con las tareas de interfaz/almacenamiento, que toleran más *jitter* sin afectar la lectura del vehículo. Las prioridades no son arbitrarias: reflejan directamente qué tan caro es que cada tarea llegue tarde (perder un ciclo de sondeo CAN corrompe el cálculo de consumo; llegar tarde a refrescar un LED es imperceptible).

Sincronización entre tareas

Dos mutex protegen el estado compartido: `g_dataMutex` protege `g_vehicleData` (la tarea CAN arma cada ciclo en una copia local y solo toca la estructura global, bajo mutex, en el paso final de fusión, ver Figura 3.44); `g_sdMutex` protege el bus SPI de la microSD, compartido entre el *logger* periódico y los endpoints de descarga/gráfica del servidor web. El criterio para usar mutex en vez de, por ejemplo, deshabilitar interrupciones o copiar datos sin protección, es que ambos recursos (la estructura de datos y el bus SPI) tienen escritores y lectores en tareas de distinta prioridad corriendo en núcleos distintos: un mutex es la primitiva correcta quando el acceso puede, en principio, bloquear brevemente sin romper los tiempos críticos del sistema.

Figura 3.44

Fusión bajo mutex al final de cada ciclo, ***src/can_obd2.cpp****.*

```
if (xSemaphoreTake(g_dataMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
    VehicleData &g = g_vehicleData;
    g.rpm = scratch.rpm;
    g.speed_kmh = scratch.speed_kmh;
    // ... resto de los campos ...
    g.can_status = status;
    g.data_valid = criticalOk;
    g.last_update_ms = now;

    if (criticalOk && dt > 0.0f && dt < 5.0f) {
        fuelCalcUpdate(g, dt);
    }
    xSemaphoreGive(g_dataMutex);
}

```

Nota. Sincronización de variables del vehículo mediante semáforo mutex para garantizar la integridad de los datos compartidos entre tareas en FreeRTOS.

Manejo de errores

`systemReportError(ErrorCode)` centraliza el reporte de fallos: guarda el último código en `g_systemStatus.error_code` (dispara el parpadeo del LED de error) y lo imprime por `Serial` con el detalle completo. El criterio es separar dos audiencias distintas: el LED es para el conductor, que solo necesita saber *que algo falló*; el log serie es para depuración en banco, donde sí importa el detalle completo y el historial.

Tabla 3.27
Códigos de `ErrorCode` *(*`include/types.h`*).*

Código	Significado
1	Fallo al iniciar el driver TWAI/CAN
2	Fallo al iniciar la microSD
3	Fallo al iniciar la pantalla OLED
4	Fallo al iniciar el punto de acceso Wi-Fi

Nota. Identificación numérica de errores de inicialización del hardware y periféricos empleada por el sistema para el diagnóstico.

3.6.4. Protocolo CAN / OBD2

El controlador TWAI del ESP32-S3 se configura a **500 kbit/s**, el estándar de la mayoría de vehículos livianos con OBD2 sobre CAN (ISO 15765-4, (International Organization for Standardization, 2016)). Todas las peticiones usadas caben en un único *frame* CAN de 8 bytes (*single frame*), así que no hace falta implementar el mecanismo multi-*frame* de control de flujo ISO-TP (una simplificación deliberada: ninguno de los PIDs que este proyecto necesita supera ese tamaño, y soportar multi-*frame* sin necesitarlo solo agrega superficie de error).

PIDs consultados (Modo 01)

El conjunto de PIDs se eligió por ser el mínimo necesario para el cálculo speed-density (Sección 3.6.5) más los que enriquecen el registro sin costo relevante de tiempo de bus, siguiendo el estándar SAE J1979 (SAE International, 2014):

Tabla 3.28
PIDs Mode 01 consultados por `canObd2Task`*.*

PID	Dato	Uso
`0x0C`	RPM	Speed-density (crítico)
`0x0B`	MAP (kPa)	Speed-density (crítico)
`0x0F`	IAT (°C)	Speed-density (crítico)
`0x0D`	Velocidad (km/h)	L/100km, distancia (crítico)
`0x05`	Temperatura refrigerante	Mostrado en pantalla/CSV
`0x04`	Carga calculada (%)	Mostrado en pantalla/CSV
`0x06`	Fuel trim corto plazo (%)	Corrige el AFR estimado
`0x07`	Fuel trim largo plazo (%)	Corrige el AFR estimado

Nota. Identificadores de parámetros (PIDs) solicitados mediante el protocolo OBD-II para el cálculo del consumo instantáneo de combustible y monitoreo de sensores.

Nota. Los cuatro PIDs marcados “crítico” son indispensables para el cálculo

speed-density; si la ECU no responde alguno, el ciclo completo se marca inválido y no se integra al viaje.

Formato de la petición

Figura 3.45
Trama de petición Mode 01, PID genérico.

```
ID = 0x7DF
Data[0] = 0x02   ; SF, 2 bytes de payload (modo + pid)
Data[1] = 0x01   ; Modo 01: datos actuales
Data[2] = <PID>
Data[3..7] = 0x00 (relleno)

```

Nota. Estructura de la trama CAN empleada para la solicitud de parámetros OBD-II en el vehículo.

Las ECUs responden en el rango `0x7E8-0x7EF`; se acepta la primera respuesta cuyo `Data[1] == 0x41` y `Data[2]` coincida con el PID pedido.

Recuperación de bus-off

Si el controlador entra en estado `BUS_OFF`, la recuperación se sondea en cada vuelta del bucle sin bloquear con una espera fija: según el driver TWAI de ESP-IDF (Espressif Systems, 2024c), la recuperación solo llega a `STOPPED` tras contar 128 ocurrencias de 11 bits recesivos consecutivos en el bus, un tiempo que depende del tráfico real y no se puede predecir de antemano; sondear en vez de esperar un tiempo fijo evita tanto reintentar demasiado pronto (fallar en vano) como esperar de más innecesariamente.

3.6.5. Método de cálculo: Speed-Density

El método speed-density estima el flujo másico de aire admitido por el motor a partir de la ley de gases ideales, sin medirlo con un sensor MAF (la razón de fondo para elegir este método sobre uno basado en MAF real es que la mayoría de los vehículos con OBD2 reportan MAP, no flujo de aire directo, así que speed-density es el único método viable sin instalar un sensor adicional):

$$\dot{m}_{aire}\ [\mathrm{g/s}] = \frac{\dfrac{VE}{100} \cdot RPM \cdot MAP\,[\mathrm{kPa}] \cdot V_d\,[\mathrm{L}] \cdot 28{,}97}{2 \cdot 8{,}314 \cdot T_{IAT}\,[\mathrm{K}] \cdot 60} \tag{3.42}$$

donde VE es la eficiencia volumétrica (%) interpolada de una tabla calibrable (Sección 3.6.5); V_d la cilindrada en litros; 28,97 la masa molar del aire (g/mol); 8,314 la constante universal de los gases (L·kPa/(mol·K)); el factor 2 corresponde a que un motor de 4 tiempos completa una admisión cada 2 vueltas de cigüeñal; y T_{IAT} la temperatura de admisión en Kelvin, con guarda de rango $[-40, 100]$ °C.

Figura 3.46
Núcleo del cálculo speed-density, ***src/fuel_calc.cpp****.*

```
float ve = fuelCalcVeForRpm(vd.rpm);
float iat_K = vd.iat_c + 273.15f;
if (iat_K < 233.15f || iat_K > 373.15f) iat_K = 288.15f;

const float MM_AIR = 28.97f; // g/mol
const float R = 8.314f;      // L*kPa/(mol*K)

float maf = (ve / 100.0f) * (float)vd.rpm * vd.map_kpa *
ENGINE_DISPLACEMENT_L * MM_AIR /(2.0f * R * iat_K * 60.0f);

```

Nota. Algoritmo para la estimación del flujo de masa de aire (MAF) a partir de la eficiencia volumétrica (VE), presión MAP y temperatura IAT.

De aire a combustible

$$\dot{m}_{comb}\,[\mathrm{g/s}] = \frac{\dot{m}_{aire}}{AFR} \cdot k_{cal} \tag{3.43}$$

AFR es la relación aire/combustible, por defecto el valor estequiométrico de la gasolina ($AFR_{stoich} = 14{,}7$), opcionalmente corregido con los *fuel trims* que reporta la ECU:

$$AFR_{efectivo} = \frac{AFR_{stoich}}{1 + \frac{STFT\% + LTFT\%}{100}} \tag{3.44}$$

acotado a $[8, 25]$. k_{cal} es `FUEL_CALIBRATION_FACTOR` (Sección 3.6.5), 1,0 por defecto. El criterio para incluir la corrección por *fuel trim* es que, sin ella, el cálculo asume que el motor siempre corre exactamente en estequiométrico, cosa que en la práctica no ocurre; usar los *trims* que la propia ECU ya reporta es una corrección "gratis" (no requiere sensores adicionales) frente a un error sistemático conocido.

De masa a volumen

$$\dot{V}_{comb}\,[\mathrm{L/h}] = \frac{\dot{m}_{comb} \cdot 3600}{\rho_{comb}} \tag{3.45}$$

$$L/100km = \begin{cases} \dfrac{\dot{V}_{comb}}{v} \cdot 100 & \text{si } v > 2\,\mathrm{km/h} \\ 0 & \text{vehículo detenido} \end{cases} \tag{3.46}$$

con $\rho_{comb} = 745\,\mathrm{g/L}$ y v la velocidad en km/h.

Integración del viaje

$$L_{viaje} \mathrel{+}= \dot{m}_{comb} \cdot dt \,/\, \rho_{comb} \tag{3.47}$$

$$km_{viaje} \mathrel{+}= v \cdot dt/3600 \tag{3.48}$$

$$Bs_{viaje} = L_{viaje} \cdot P_L \tag{3.49}$$

con dt el paso de tiempo real entre ciclos (no un valor fijo, precisamente para no perder precisión si un ciclo de sondeo tarda más o menos de lo típico) y $P_L = 6{,}96$ (`FUEL_PRICE_BS_PER_L`) el precio de referencia del litro.

Tabla de eficiencia volumétrica (VE)

Tabla 3.29
`kVeTable`, `src/fuel_calc.cpp`. Interpolación lineal entre puntos.

RPM	VE (%)	Origen
940	82,7	Calibrado (Nissan Vanette)
1500	79,7	Calibrado (Nissan Vanette)
2500	79,7	Calibrado (Nissan Vanette)
3500	72,7	Calibrado (Nissan Vanette)
4000	88,0	Genérico, sin calibrar
5500	83,0	Genérico, sin calibrar
7000	74,0	Genérico, sin calibrar

Nota. Tabla de valores de eficiencia volumétrica (VE) caracterizados según el régimen de rotación del motor (RPM) para la estimación del flujo de masa de aire mediante el método speed-density.

Nota. Los cuatro puntos marcados como "Calibrado" se obtuvieron con la prueba estática de pasos de RPM del Nissan Vanette contra su PID 0x10/MAF real (Sección 4.3.3 del Capítulo 4); el procedimiento de calibración general se documenta en la Subsección 3.6.5. Los puntos restantes (4000/5500/7000 rpm) son un punto de partida genérico, no un dato de fábrica del vehículo: sin calibrarlos, el consumo calculado en ese rango tendrá un error sistemático.

Cálculo general de la eficiencia volumétrica para un vehículo arbitrario

Los cuatro puntos calibrados de la Tabla 3.29 son específicos del Nissan Vanette: no existen dos motores con la misma curva η_v, así que no es razonable pedirle a un código un número universal de eficiencia volumétrica. Lo que sí es general (y se documenta en esta subsección) es el *procedimiento* para calcularla a partir de datos reales de **cualquier** vehículo, implementado en `CODIGOS/ve_curve_calibrator.py` y disponible en el repositorio de GitHub del proyecto (Calle, 2026).

Teoría. La eficiencia volumétrica se define como el cociente entre la masa de aire realmente admitida y la masa teórica que ocuparía la cilindrada a la densidad de admisión (Ecuación (3.16)). No es un dato de fábrica del vehículo (Heywood, 2018): depende de la geometría de válvulas, el árbol de levas, la resonancia de los múltiples de admisión y escape, y el estado mecánico del motor: parámetros de diseño que ningún PID OBD-II expone y que no admiten una fórmula cerrada válida para cualquier motor. Por esto, en la práctica de la puesta a punto de motores (por ejemplo, en sistemas de gestión programable tipo *speed-density*) la curva $\eta_v(N)$ siempre se obtiene de forma empírica, nunca analítica.

Ingeniería. Lo que sí generaliza es la propia ecuación de speed-density (Ecuación (3.15)): dado un conjunto real de muestras (RPM, P_{MAP}, T_{IAT}) y una referencia real de flujo de aire MAF_{ref} (el PID 0x10, cuando el vehículo lo soporta), se despeja algebraicamente la única incógnita:

$$VE_{\text{requerida}} = \frac{MAF_{ref} \cdot 100 \cdot 2 \cdot 8{,}314 \cdot T_{IAT} \cdot 60}{RPM \cdot P_{MAP} \cdot V_d \cdot 28{,}97} \tag{3.50}$$

Esta es la misma inversión de la Ecuación (3.15) usada para calibrar la Tabla 3.29 (Sección 4.3.3 del Capítulo 4), pero aquí generalizada: el código no asume ningún vehículo, régimen objetivo ni número fijo de puntos: solo necesita la cilindrada V_d y un registro CSV con columnas `rpm`, `map_kpa`, `iat_c` y `maf_referencia_gs`, en el mismo formato que produce `speed_density_bench.py`.

Figura 3.47
Inversión general de la ecuación speed-density, ***`ve_curve_calibrator.py`****.*

```
def estimate_ve(rpm, map_kpa, iat_c, maf_ref_gs, displacement_l):
    iat_k = iat_c + 273.15
    if iat_k < 233.15 or iat_k > 373.15:
        iat_k = 288.15
    return maf_ref_gs * 100.0 * 2.0 * R * iat_k * 60.0 \
           / (rpm * map_kpa * displacement_l * MM_AIR)

```

Nota. `displacement_l` (V_d) es el único parámetro que cambia entre vehículos; el resto de la función es idéntico a la Ecuación (3.50) para cualquier motor.

Criterio del programa. Una sola muestra invertida es ruidosa (cuantización de los PID, transitorios de conducción), así que el script no reporta la VE muestra por muestra sino que sigue el mismo criterio de agrupamiento y ajuste que el Algoritmo 3 del lazo cerrado (Sección 3.5), con una diferencia deliberada en la fuente del estimado puntual:

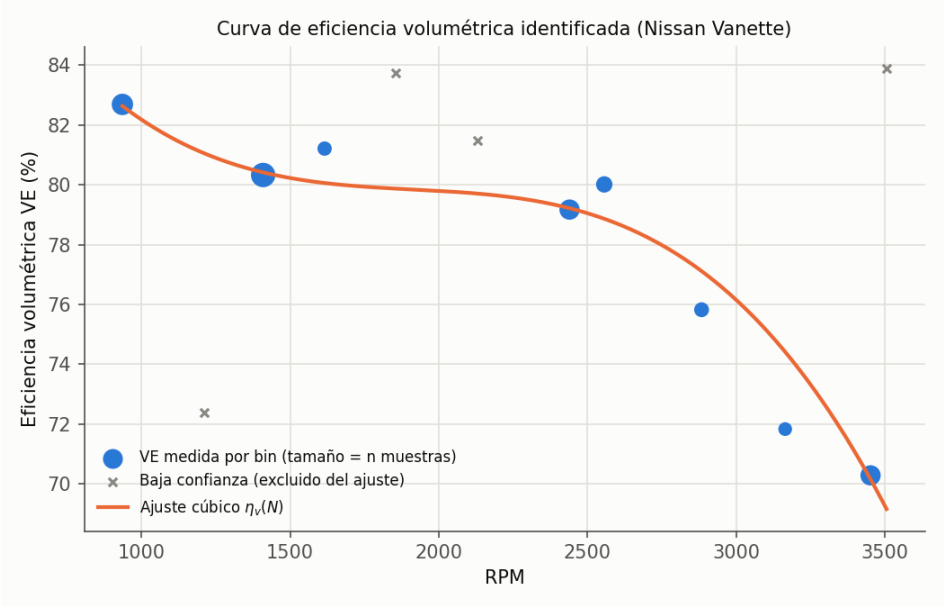
- **Agrupamiento por bin de régimen.** Las muestras se agrupan en intervalos de RPM de ancho configurable (250 rpm por defecto), igual que la Ecuación (3.30), en vez de exigir puntos de régimen fijos como `calibrate_ve_table.py`: esto permite calibrar con cualquier registro de conducción, no solo con la prueba estática de pasos de RPM.

- **Umbral mínimo de confianza.** Un bin con menos de `min_samples` muestras (5 por defecto) se marca como de baja confianza y se excluye del ajuste en vez de participar con un promedio poco confiable; no se rellena ningún bin sin datos suficientes con un valor supuesto.
- **Regresión cúbica ponderada.** Sobre los bins de confianza se ajusta el mismo polinomio cúbico $\eta_v(N) = c_0 + c_1 N + c_2 N^2 + c_3 N^3$ de la Ecuación (3.30), ponderado por el número de muestras de cada bin, junto con su coeficiente de determinación R^2 para poder juzgar la calidad del ajuste en vez de aceptarlo a ciegas.

La diferencia con el Algoritmo 3 original es la fuente del estimado puntual: aquel, pensado para ejecutarse en línea dentro del propio firmware, infiere $\hat{\eta}_v$ de forma indirecta a partir del STFT/LTFT (por lo que requiere ya conocer una curva semilla $\eta_v^0(N)$ para funcionar, Ecuación (3.32)), mientras que `ve_curve_calibrator.py` lo calcula de forma directa a partir de un MAF_{ref} real, sin depender de ninguna curva previa. Por esta misma razón, este script offline es además una forma práctica de generar esa curva semilla $\eta_v^0(N)$ que el Algoritmo 3 necesita como punto de partida, en cualquier vehículo que soporte el PID 0x10.

La Figura 3.48 muestra el resultado de correr el script sobre el mismo registro del Nissan Vanette (Sección 4.3.3 del Capítulo 4, $V_d = 1{,}626\,\mathrm{L}$): a diferencia de la calibración de cuatro puntos fijos, aquí se aprovechan también las muestras tomadas durante las transiciones entre regímenes, lo que produce ocho bins de confianza en vez de cuatro y permite juzgar la calidad del ajuste cúbico con un $R^2 = 0{,}968$.

Figura 3.48
Curva de eficiencia volumétrica identificada por `ve_curve_calibrator.py` *sobre datos reales del Nissan Vanette*



Nota. Círculos: VE promedio por bin (tamaño proporcional al número de muestras). Cruces: bins de baja confianza, excluidos del ajuste. Línea: polinomio cúbico ponderado, Ecuación (3.30). Elaboración propia.

Caso sin MAF: el Changan Honor. La Ecuación (3.50) requiere una referencia real de flujo de aire (MAF_{ref}) para despejar la VE de forma directa; sin ella, RPM, MAP e IAT por sí solos no alcanzan para aislar esa incógnita. El Changan Honor no soporta el PID 0x10 (Tabla 4.8, Capítulo 4), así que `ve_curve_calibrator.py` no es aplicable a este vehículo: ninguna manipulación algebraica sustituye a la referencia real que falta.

El escaneo de PIDs del Changan (Apéndice O) confirma, sin embargo, que sí soporta el PID 0x06 (STFT), el PID 0x07 (LTFT) y el PID 0x44 (relación de equivalencia aire-combustible comandada, usada aquí como indicador de lazo cerrado). Esa combinación es exactamente la que requiere el Algoritmo 3 ya formalizado en la Sección 3.5: en vez de una referencia absoluta de aire, usa el propio ajuste de mezcla que la ECU ya reporta como señal indirecta de que la VE asumida está mal. `CODIGOS/ve_curve_calibrator_indirect.py` implementa una versión offline de ese algoritmo, análoga en estructura a `ve_curve_calibrator.py` (mismo agrupamiento por bin, mismo umbral mínimo de confianza, mismo ajuste cúbico ponderado) pero con un estimador puntual distinto, el de la Ecuación (3.32):

$$\hat{\eta}_v(N) = \eta_v^0(N)\left(1 + \frac{\text{STFT\%} + \text{LTFT\%}}{100}\right) \tag{3.51}$$

donde $\eta_v^0(N)$ es una curva semilla (por defecto, la misma tabla genérica de literatura con la que este proyecto arrancó antes de calibrar la del Vanette, Tabla 3.29, valores originales) y solo se aceptan muestras con el motor en lazo cerrado ($|\lambda_{cmd} - 1|$ dentro de una tolerancia, PID 0x44), descartando los tramos de enriquecimiento donde el STFT/LTFT no reflejan un error de VE sino una estrategia distinta de la ECU. Nótese que esta corrección es multiplicativa y relativa a la curva semilla (no involucra V_d, P_{MAP} ni T_{IAT}), a diferencia de la Ecuación (3.50), que sí es una inversión absoluta de la ecuación de gas ideal.

Advertencia epistemológica. El resultado de este procedimiento **no** debe reportarse como una VE "calibrada" del Changan en el mismo sentido que la del Vanette: es una *corrección relativa a una curva semilla asumida*, no una medición absoluta. Si la curva semilla generica está lejos de la realidad de este motor en particular, la corrección heredará ese sesgo: el método solo garantiza mayor consistencia con los STFT/LTFT observados, no exactitud absoluta, precisamente por la ausencia de una referencia real de aire o combustible en este vehículo. Queda, junto con la recolección misma de los datos con `CODIGOS/changan_trim_logger.py`, como trabajo pendiente para una eventual continuación de este proyecto (Capítulo 6): no se generó una curva para el Changan dentro del alcance de este trabajo por no haberse completado esa recolección de campo.

Calibración contra un tanque real

No existe forma de que el ESP32 sepa, por sí solo, si el número que calcula es correcto (no hay con qué comparar dentro del propio microcontrolador). El procedimiento recomendado: llenar el tanque y anotar el odómetro, manejar con normalidad hasta la próxima recarga, comparar los litros reales cargados contra `trip_fuel_L` acumulado en esa misma distancia, y ajustar

$$\texttt{FUEL_CALIBRATION_FACTOR} = \frac{L_{real}}{L_{calculado}} \tag{3.52}$$

en `config.h`. El criterio de aplicar un único factor multiplicativo sobre `fuel_gs`, en vez de reajustar toda la tabla VE a mano, es que corrige de forma consistente el instantáneo, el acumulado del viaje y el costo con un solo número, y es reversible/auditable, a diferencia de retocar seis puntos de la tabla VE por prueba y error.

3.6.6. Registro en microSD

Cada arranque crea un archivo nuevo `/logs/trip_NNN.csv` (índice autoincremental calculado escaneando los archivos existentes, en vez de un contador separado que podría desincronizarse tras un corte de energía). Se escribe una fila por segundo, solo después del primer ciclo CAN válido, y se pausa por completo durante el modo de bajo consumo (Sección 3.6.7).

Por qué CSV

Se eligió CSV en vez de un formato binario compacto o JSON por fila por tres razones: es directamente legible/abrible en cualquier hoja de cálculo sin escribir un parser; el tamaño de fila (unos 100 bytes) es intrascendente frente a la capacidad de una microSD moderna incluso a una fila por segundo durante horas; y es el formato que el propio panel web ya necesita poder descargar y volver a interpretar (Sección 3.6.8), así que un mismo formato sirve para ambos usos sin conversión intermedia.

Figura 3.49
Fila de ejemplo del CSV (encabezado y fila de datos).

```
uptime_ms,rpm,speed_kmh,map_kpa,iat_c,coolant_c,ve_pct,maf_gs,
fuel_gs,instant_Lh,instant_L100km,trip_km,trip_fuel_L,
trip_avg_L100km,can_status,trip_cost_bs

184532,2150,62.0,58.0,29.0,89.0,81.4,7.82,0.532,1.91,3.09,
12.400,0.384,3.10,1,2.67

```

Nota. Estructura del registro de datos almacenado en la tarjeta microSD durante la ejecución del sistema.

Tabla 3.30
Columnas escritas por `sdLoggerTask` (`src/sd_logger.cpp`).

Columna	Descripción
`uptime_ms`	Milisegundos desde el arranque
`rpm`	Revoluciones por minuto
`speed_kmh`	Velocidad, km/h
`map_kpa`	Presión absoluta del múltiple, kPa
`iat_c`	Temperatura del aire de admisión, °C
`coolant_c`	Temperatura del refrigerante, °C
`ve_pct`	Eficiencia volumétrica usada, %
`maf_gs`	Flujo másico de aire calculated, g/s
`fuel_gs`	Flujo másico de combustible calculado, g/s
`instant_Lh`	Consumo instantáneo, L/h
`instant_L100km`	Consumo instantáneo, L/100km
`trip_km`	Distancia acumulada del viaje, km
`trip_fuel_L`	Combustible acumulado del viaje, L
`trip_avg_L100km`	Promedio del viaje, L/100km
`can_status`	Estado del bus CAN (enum `CanStatus`)
`trip_cost_bs`	Costo acumulado del viaje, Bs

Nota. Especificación de la estructura del registro de datos guardado en el archivo CSV de la memoria microSD.

3.6.7. Modo de bajo consumo (“sin contacto”)

Motivación

El dispositivo se alimenta del pin 16 del conector OBD2, que entrega 12 V **constantes** sin importar la posición de la llave. Si el firmware siguiera sondeando el bus a ritmo completo, con el panel web y todos los periféricos encendidos las 24 horas, se convertiría en una carga parásita permanente sobre la batería del vehículo. La primera versión de este modo se basaba en detectar RPM= 0 y solo bajaba el ritmo de sondeo (una mejora real pero modesta, porque el ESP32-S3 seguía activo con el WiFi encendido). La versión actual va más lejos: separa la alimentación en dos rieles y usa **deep sleep** real del ESP32-S3 mientras no hay contacto.

Dos rieles de alimentación

La PCB entrega dos salidas de 3,3 V independientes desde el 12 V constante del OBD2:

- **Riel 1** (siempre encendido): ESP32-S3 + SN65HVD230. Tiene que quedar siempre alimentado porque el transceptor es lo único capaz de “escuchar” el bus CAN para saber cuándo volvió el contacto.
- **Riel 2** (conmutado por software): LEDs + OLED, y por separado la microSD. Se corta por completo mientras no hay contacto.

Circuito de *power-gating*: MOSFET y pines

Cada riel 2 se controla con un MOSFET de **canal P, como interruptor de alto lado** (corta el 3,3 V que entra al periférico, no su GND). Se prefiere sobre un N-MOSFET de bajo lado porque con este último el GND del periférico puede quedar ligeramente flotante mientras conduce, lo que genera problemas de integridad de señal en buses referenciados a GND como I2C (OLED) y SPI (microSD).

Tabla 3.31
Pines de power-gating, `include/config.h`.

Señal	GPIO ESP32-S3	Controla
`PIN_PWR_LED_OLED`	GPIO6	LEDs + OLED
`PIN_PWR_SD`	GPIO7	microSD

Nota. Asignación de los pines de salida del microcontrolador utilizados para conmutar la alimentación de los periféricos durante los modos de bajo consumo.

Ambos GPIO son de dominio RTC en el ESP32-S3 (rango GPIO0-21), requisito para que `gpio_hold_en()` pueda retener su nivel lógico durante el *deep sleep* (Espressif Systems, 2024c). Topología de cada MOSFET (misma para los dos rieles):

Figura 3.50
Interruptor de alto lado con P-MOSFET, un canal por riel.

```
Riel 2 (3.3V) ---+---------------------- Source (P-MOSFET)
|
[R_pullup ~10k]
|
GPIO --[R_gate ~1k]------------------- Gate
|
Drain --- carga (OLED+LED, o microSD)

```

Nota. Esquema básico del circuito de conmutación en el lado alto para el control de energía de periféricos.

El criterio detrás de cada componente: como el GPIO y el riel 2 comparten el mismo nivel lógico (3,3 V), no hace falta un transistor adicional para manejar la compuerta: GPIO en alto da $V_{gs} = 0$ (MOSFET apagado); GPIO en bajo da $V_{gs} \approx -3{,}3\,\text{V}$, muy por debajo del umbral típico de un P-MOSFET de nivel lógico (-1 a $-2\,\text{V}$ aprox., p. ej. un AO3401A o equivalente). La resistencia de *pull-up* en la compuerta asegura que el MOSFET arranque apagado por defecto durante el breve instante antes de que el firmware configure el GPIO como salida; la resistencia en serie hacia el GPIO limita la corriente de carga de la compuerta.

Cadena de regulación

La alimentación completa, del 12 V del OBD2 a los dos rieles de 3,3 V, pasa por tres reguladores en cascada:

1. **TPS54302**: convertidor *buck* (conmutado) que baja el 12 V del pin 16 a 5 V. Es la única etapa de conversión conmutada del sistema; las dos siguientes son lineales.
2. **AMS1117** (5 V $\rightarrow$ 3,3 V): alimenta el **riel 1** (ESP32-S3 + SN65HVD230). Nunca se apaga.
3. **AP2112** (5 V $\rightarrow$ 3,3 V): alimenta el **riel 2**; su salida de 3,3 V entra a los dos MOSFET de la Sección 3.6.7, que gatean OLED+LED y microSD por separado. El AP2112 en sí mismo nunca se apaga (solo se corta lo que hay *después* de los MOSFET), pero al ser una LDO de muy baja corriente de reposo su aporte al consumo en espera es pequeño de todos modos (Sección 3.6.7).

Esta cadena (*buck* + 2 LDO en paralelo desde el mismo 5 V) es una elección de diseño válida y común, pero tiene una consecuencia directa para el consumo "sin contacto" que conviene tener presente: a diferencia del AP2112, el **AMS1117 es una LDO de propósito general, no de bajo consumo**: su corriente de reposo (*ground current*) es comparativamente alta, típicamente 5-8 mA casi sin importar la carga en su salida, con variación notable entre fabricantes clon del mismo chip. Como el AMS1117 alimenta justo el riel que nunca se apaga, esa corriente de reposo se sostiene las 24 horas del día, se detalla en la Sección 3.6.7.

Implementación en firmware

`canObd2CheckContact()` manda un único *ping* (PID `0x0C`, RPM, obligatorio en todo ECU OBD2) y reporta si algo respondió, sin importar el valor. Se usa en dos puntos:

- Al arrancar (`setup()`, tanto en un encendido normal como al despertar del *deep sleep*): si no hay contacto, el chip vuelve a dormir antes de tocar el riel 2 o el WiFi.
- Ya despierto, dentro de `canObd2Task`: si el bus deja de responder durante `CONTACT_LOST_DEBOUNCE_CYCLES` ciclos seguidos (3, para no dormirse por un solo *frame* perdido en un bus ocupado), se considera que se fue el contacto.

Figura 3.51
Entrada a deep sleep, `src/power_sleep.cpp`.

```
void enterDeepSleepUntilContact() {
    sdLoggerShutdown();  // cierra el archivo antes de cortarle la
    energia
    WiFi.mode(WIFI_OFF);

    powerGateSet(false);
    gpio_hold_en((gpio_num_t)PIN_PWR_LED_OLED);
    gpio_hold_en((gpio_num_t)PIN_PWR_SD);
    gpio_deep_sleep_hold_en(); // sin esto el hold no sobrevive al
    deep sleep

    esp_sleep_enable_timer_wakeup(DEEP_SLEEP_WAKE_INTERVAL_US);
    esp_deep_sleep_start();  // no retorna
}

```

Nota. Procedimiento para el apague seguro de perifericos, mantenimiento del estado GPIO (hold) y entrada al modo de bajo consumo deep sleep en el ESP32.

Cerrar el archivo de la SD **antes** de cortar su MOSFET es obligatorio, no una formalidad: cortar la alimentación con un archivo abierto arriesga corromper el sistema de archivos completo, no solo perder la última fila. El *deep sleep* borra toda la RAM salvo el dominio RTC, así que al despertar `setup()` vuelve a correr desde el principio, exactamente como un arranque en frío.

Consumo estimado y autonomía de la batería

Los siguientes números son estimaciones a partir de valores típicos de hoja de datos, no mediciones sobre la placa real (sirven para tener una idea de orden de magnitud, no un valor exacto).

Tabla 3.32

Contribuyentes al consumo promedio "sin contacto", cadena real (TPS54302 + AMS1117 + AP2112).

Nodo	Fuente	Aporte típico	Nota
3,3 V (riel 1)	ESP32-S3 en deep sleep	∼0,02 mA	10–25 μA (Espressif Systems, 2024b)
3,3 V (riel 1)	SN65HVD230, siempre activo	∼5–10 mA	Modo normal, no standby (Texas Instruments, 2015)
3,3 V (riel 1)	Pulsos de despertar (cada 20 s)	∼1,5–2 mA	Promediado: ∼150 mA por ∼0,2–0,3 s
5 V	**AMS1117**, corriente de reposo propia	**∼5–8 mA**	Casi independiente de la carga
5 V	AP2112, corriente de reposo propia	∼0,06 mA	Sin carga (MOSFET de salida abiertos)
12 V	TPS54302, corriente de reposo propia	∼0,1–0,3 mA	
Total aproximado, lado 12 V		**∼7–9 mA**	Ver conversión abajo

Nota. Estimación del consumo de corriente del sistema en estado de reposo (sin contacto de encendido), detallando la contribución de cada componente regulador y periférico en la línea de alimentación.

El resultado es contraintuitivo pero importante: **el transceptor CAN y la LDO del riel 1, no el ESP32-S3, dominan el consumo mientras el dispositivo "duerme"**: ese riel no se puede cortar sin perder la capacidad de detectar cuándo vuelve el contacto (Sección 3.6.7). El *deep sleep* resuelve la parte del ESP32-S3 casi por completo, pero no puede resolver la del transceptor ni la de su LDO con esta arquitectura de dos rieles fija.

Convertir la corriente del lado 3,3 V/5 V al lado 12 V: el AMS1117 y el AP2112 son LDO (no reducen corriente, solo voltaje, disipando la diferencia como calor), así que la corriente que el TPS54302 debe entregar en su salida de 5 V es aproximadamente la suma de lo que consumen el riel 1 completo (∼8,5 mA de ESP32+SN65+pulsos) más la corriente de reposo propia del AMS1117 (∼6,5 mA punto medio) y del AP2112 (∼0,06 mA): ≈15 mA a 5 V. El TPS54302, al ser un *buck* conmutado, sí convierte potencia de forma más eficiente (∼80 %en carga liviana, estimado):

$$I_{12V} \approx \frac{5\,\mathrm{V} \times 15\,\mathrm{mA}}{12\,\mathrm{V} \times 0{,}80} + 0{,}2\,\mathrm{mA}\ (\mathrm{I}_q\ \text{TPS54302}) \approx 8\,\mathrm{mA} \tag{3.53}$$

Tomando esos ∼8 mA y una batería típica de auto de 50 Ah (de la cual, por buena práctica, no conviene usar más del 50 %para no acortar su vida útil, es decir ∼25 000 mAh utilizables):

$$\frac{25\,000\,\text{mAh}}{8\,\text{mA}} \approx 3\,125\,\text{horas} \approx 130\,\text{días} \tag{3.54}$$

Es decir, **el dispositivo por sí solo** tardaría del orden de 4-5 meses en consumir la mitad de una batería sana (menos que si el riel 1 usara una LDO de baja corriente de reposo en vez del AMS1117, pero todavía un margen amplio frente al uso normal del vehículo). En la práctica esto importa menos de lo que parece, porque el propio vehículo ya tiene su propio consumo parásito de fábrica (alarma, memorias de radio/ECU, reloj, etc.), típicamente ∼20-50 mA con todo "apagado" (del mismo orden o mayor que el aporte de este dispositivo), y ese consumo de base es el que realmente determina cuánto aguanta la batería estacionada, con o sin el ESP32-S3 conectado.

Decisión de diseño: se evaluó reemplazar el AMS1117 del riel 1 por una LDO de baja corriente de reposo (el propio AP2112, un MCP1700, o una TPS7A02 con I_q en el orden de nanoamperios bajarían el consumo "sin contacto"varios mA más), pero se optó por mantener el AMS1117 y el AP2112 tal como estaban ya elegidos en la placa: es el contribuyente individual más grande después del SN65HVD230, y queda documentado aquí como una compensación consciente entre simplicidad/costo de la lista de materiales y la autonomía teórica de la batería, no un descuido.

Consumo estimado en modo activo

El mismo procedimiento de conversión de la Ecuación (3.53) se aplica a dos escenarios activos, con la diferencia de que en modo activo la corriente que domina en cada riel ya no es la de reposo de las LDO sino la carga real que atraviesan (el AMS1117 y el AP2112 no reducen corriente, solo la dejan pasar con caída de tensión). Los aportes por componente:

- **ESP32-S3, activo con WiFi asociado (modo *modem-sleep*, tráfico intermitente cada 500 ms, no transmisión continua):** ∼40–48 mA a 240 MHz (Espressif Systems, 2024b). Es el escenario realista del panel web: el WiFi permanece asociado pero la radio pasa la mayor parte del tiempo en reposo entre los envíos periódicos del WebSocket, muy distinto del pico de >283 mA que exige una transmisión continua a máxima tasa (Espressif Systems, 2024b).
- **ESP32-S3, activo sin WiFi (CPU corriendo, radio apagada):** no tabulado directamente en la hoja de datos; se estima ∼25 mA, por debajo del modo *modem-sleep* al no incluir el consumo periódico de la banda base Wi-Fi.
- **SN65HVD230, activo:** ∼8 mA, punto medio del rango ya usado en la Tabla 3.32 (Texas Instruments, 2015).
- **OLED SSD1306, encendida:** ∼15 mA, punto medio de 10–20 mA (Solomon Systech, 2008).
- **MicroSD, activa (lectura/escritura):** ∼25 mA, valor típico de fabricante para tarjetas microSD en operación (no corresponde a una hoja de datos de un modelo específico, a diferencia de los aportes anteriores).

Tabla 3.33
Consumo estimado en modo activo, dos escenarios (misma cadena TPS54302+AMS1117+AP2112)

Escenario	Riel 1+2 a 5 V	I_{12V} estimada
Activo, WiFi + OLED + microSD	∼100 mA	∼52 mA
Activo, solo CAN + microSD (sin WiFi ni OLED)	∼65 mA	∼34 mA

Nota. I_{12V} calculada con la Ecuación (3.53) (eficiencia del TPS54302 ∼80 %), sumando los aportes de componente listados arriba más la corriente de reposo propia de cada LDO. Son estimaciones de orden de magnitud, no mediciones, pendientes de validar una vez ensamblada la PCB (Sección 4.3.8 del Capítulo 4).

Recomendación de mantenimiento

Con esos números, la recomendación práctica no cambia mucho respecto al cuidado normal de cualquier batería de auto: **arrancar/circular el vehículo al menos cada 2-3 semanas**, o usar un cargador de mantenimiento (*battery tender*) si va a quedar estacionado más tiempo que eso. Con la cadena de reguladores actual (AMS1117 incluido) el dispositivo agrega una fracción del consumo parásito que el auto ya tiene de fábrica (notable, pero no dominante), así que este intervalo sigue siendo una recomendación razonable sin necesitar ser más estricta.

¿Afecta consultar tan seguido?

Con la arquitectura actual hay dos frecuencias distintas que vale la pena separar:

- **Sondeo CAN mientras hay contacto** (un ciclo completo de 8 PIDs cada ∼0,2-1,4 s): el bus CAN a 500 kbit/s tiene capacidad de sobra para esto (cada trama de petición/respuesta son unos 100 bits, así que incluso varias por segundo representan bien menos del 1 %del ancho de banda del bus). Una ECU real está diseñada para atender consultas de diagnóstico bastante más frecuentes que esto sin ningún problema; no hay riesgo de saturar el bus ni de sobrecargar la ECU.
- **Escritura a la microSD, una fila por segundo** (`SD_LOG_INTERVAL_MS`): las escrituras son secuenciales (se van agregando al final del archivo, no se reescribe una y otra vez el mismo bloque), que es el mejor caso posible para el desgaste de la memoria flash. Una fila ronda los 100 bytes; a una por segundo son unos 360 KB por hora de viaje, insignificante frente a la resistencia típica de una microSD moderna (miles de ciclos de escritura por bloque, con *wear leveling* integrado). No es un factor de desgaste relevante en la práctica.

3.6.8. Panel web

Punto de acceso WiFi: por qué SoftAP en vez de solo STA

El dispositivo siempre levanta su propio punto de acceso (SSID `ESP32_OBD2`, IP típica `192.168.4.1`) en vez de depender únicamente de unirse a una red WiFi conocida. El criterio es el contexto de uso real: dentro de un vehículo casi nunca hay una red WiFi conocida en rango, así que un diseño solo-STA dejaría el panel inaccesible la mayor parte del tiempo. El SoftAP garantiza acceso local sin esa dependencia; unirse además como cliente a una red conocida queda como algo opcional, únicamente para sincronizar hora por NTP cuando hay una disponible.

Por qué ESPAsyncWebServer en vez del servidor síncrono de Arduino

Se eligió ESPAsyncWebServer (ESP32Async contributors, 2024) sobre el `WebServer` síncrono incluido en el core de Arduino-ESP32 porque atiende las conexiones de forma asíncrona, basada en eventos: una petición lenta (por ejemplo, una descarga de CSV grande) no bloquea el hilo mientras espera que el cliente reciba los datos. Dado que el servidor corre en el mismo núcleo que la pila WiFi (Sección 3.6.1), un modelo síncrono habría significado que una descarga lenta retrasara también el resto del tráfico de red, incluyendo el WebSocket que empuja los datos en vivo.

Empuje en vivo: WebSocket con respaldo por *polling*

El estado se empuja a los clientes por WebSocket cada 500 ms (la Figura 3.44 ya mostró de dónde sale ese dato), con un mecanismo de respaldo: si el navegador detecta que el WebSocket no está abierto, recurre a pedir `/api/live` por HTTP cada 1,5 s. El criterio es la robustez: un WebSocket puede caerse por muchas razones ajenas al firmware (el navegador durmiendo la pestaña, un cambio de red en el celular), y el panel no debería quedar congelado solo porque ese canal se cortó.

JSON con ArduinoJson

Las respuestas HTTP y los mensajes WebSocket se serializan con la librería ArduinoJson (Blanchon, 2024) en vez de concatenar *strings* a mano. El criterio: construir JSON por concatenación de texto es una fuente común de errores de escape/formato difíciles de depurar en un microcontrolador, mientras que ArduinoJson gestiona el documento en un búfer de tamaño acotado y conocido, algo relevante en un sistema con 320 KB de RAM total repartidos entre cinco tareas.

Tabla 3.34
Endpoints servidos por ESPAsyncWebServer (`src/web_server.cpp`).

Método	Ruta	Descripción
GET	`/`	Panel del dashboard (HTML embebido en el firmware)
WS	`/ws`	WebSocket, empuja el estado en vivo cada 500 ms
GET	`/api/live`	Snapshot JSON del estado actual
GET	`/api/files`	Lista de archivos en `/logs`
GET	`/api/download`	Descarga un CSV completo
GET	`/api/chartdata`	Puntos decimados para graficar
POST	`/api/reset`	Reinicia el contador de viaje

Nota. Definición de la interfaz de programación de aplicaciones (API) REST y punto de conexión WebSocket implementados para el servidor web incrustado del microcontrolador.

Interfaz

El HTML/CSS/JS está embebido por completo en el firmware, sin dependencias de CDN externo (criterio directo del contexto de uso: el SoftAP no tiene salida a internet, así que cualquier librería externa de gráficas simplemente no cargaría). El panel tiene dos pestañas (*En vivo* e *Historial*), tarjetas destacadas con el consumo instantáneo, litros y costo del viaje, casillas para elegir qué variables graficar, y un *tooltip* interactivo con línea guía. Cuando hay 2 o más variables seleccionadas a la vez, cada una se normaliza a su propio rango (0-100 %) para poder comparar formas de curva con unidades muy distintas (RPM 0-7000 frente a L/100km 0-20) sin que una aplaste visualmente a la otra.

3.6.9. Compilación y programación

PlatformIO

Figura 3.52
Configuración relevante de `platformio.ini`.

```
board = esp32-s3-devkitc-1
board_build.flash_mode = qio
board_build.arduino.memory_type = qio_opi
board_build.psram_type = opi
board_upload.flash_size = 16MB

```

Nota. Parámetros de compilación y definición del tipo de memoria Flash y PSRAM OPI para el módulo ESP32-S3.

`platformio.ini` define un entorno de compilación por vehículo (`[env:vanette]` / `[env:changan]`) en vez de un único firmware genérico: cada uno agrega un *build flag* (`-DVEHICLE_VANETTE` / `-DVEHICLE_CHANGAN`) que selecciona en tiempo de compilación, mediante bloques `#if`/`#elif` en `config.h` y `fuel_calc.cpp`, la cilindrada real y la

tabla VE correspondientes a cada motor (Sección 3.6.5). El criterio de resolver esto en tiempo de compilación, y no dejarlo como una constante a editar a mano antes de cada *flasheo*, es que un error de este tipo ya ocurrió una vez en este proyecto (la cilindrada del Vanette quedó en 1,6 L en vez de 1,626 L) y un *build* que no selecciona ningún vehículo simplemente no compila (`#error`), en vez de compilar en silencio con un valor incorrecto. Para compilar y subir: `pio run -e vanette -t upload` o `pio run -e changan -t upload`, y luego `pio device monitor -b 115200`.

USB nativo frente a UART externo

El *devkit* de desarrollo solo expone el **USB nativo** del ESP32-S3 (el periférico USB-Serial-JTAG integrado, sin un chip puente CP2102/CH340 aparte). Esto tiene dos implicancias para una PCB propia:

1. Programar por UART **siempre funciona**, sin importar la configuración del firmware: `esptool` habla directo con el *bootloader* de ROM del chip, independiente de cualquier opción de compilación de la aplicación.
2. El firmware define `-DARDUINO_USB_CDC_ON_BOOT=1`. Con ese *flag*, el objeto `Serial` de Arduino se enruta al USB nativo en vez del UART0 físico (GPIO43/44). Si la PCB propia usa un adaptador UART externo y no tiene el USB nativo conectado a nada, ese *flag* debe quitarse: de lo contrario el monitor serie del adaptador UART no mostrará nada, aunque el firmware esté corriendo correctamente.

Circuito mínimo para programar/arrancar

Cualquier placa propia con el ESP32-S3 necesita, como mínimo: `EN` (reset) con *pull-up* a 3,3 V y un capacitor de $\sim$1 μF a GND; `GPIO0` (BOOT) con *pull-up* a 3,3 V y un botón a GND para forzar el modo *bootloader* manualmente; y, opcionalmente, un circuito de auto-reset (2 transistores desde DTR/RTS del adaptador UART hacia EN y GPIO0) para que PlatformIO entre solo al modo de programación sin botones.

3.6.10. Solución de problemas comunes

- **La subida se queda en "Connecting..." indefinidamente.** Entrar manualmente en modo *bootloader* (mantener BOOT, pulsar y soltar RESET/EN, soltar BOOT, y recién ahí lanzar la subida). Verificar también que ningún otro programa tenga el puerto COM abierto.
- **El monitor serie no muestra nada tras programar.** Revisar que `ARDUINO_USB_CDC_ON_BOOT` coincida con el hardware real (Sección 3.6.9).
- **`sdSelectCard(): Select Failed / SD=0` en el arranque.** Casi siempre es cableado/alimentación: revisar tarjeta insertada, tensión correcta del módulo lector, continuidad de los 4 pines SPI + GND, formato FAT32, y el *pull-up* de 10 kΩ en CS si se soldó directo sin módulo.

- **El panel web no carga en el celular.** Escribir `http://192.168.4.1` con el esquema explícito (varios navegadores intentan `https://` por defecto y el dispositivo no tiene TLS). Algunos celulares además desconectan una red WiFi sin salida a internet a menos que se les indique quedarse conectados igual.

Capítulo 4

IMPLEMENTACIÓN Y PRUEBAS

Este capítulo describe el proceso de implementación física del sistema de monitoreo de consumo de combustible, desde el prototipado en protoboard hasta pruebas con la placa de circuito impreso (PCB) definitiva, y expone la metodología experimental empleada para validar su desempeño en condiciones reales de operación. La validación se realiza sobre dos vehículos de características distintas y se contrasta contra un adaptador ELM327 combinado con una aplicación comercial de cálculo de consumo. Esta comparación permite evaluar tanto la exactitud del método de estimación como la fiabilidad del sistema desarrollado frente a una alternativa disponible en el mercado.

4.1. Montaje del sistema

El montaje del sistema se organiza en dos etapas complementarias. En la primera, se ensambla un prototipo funcional sobre protoboard que permite verificar la viabilidad eléctrica y de firmware de la arquitectura propuesta antes de comprometer recursos en la fabricación de la PCB. En la segunda etapa, una vez validado el prototipo, se envía el diseño definitivo a fabricación bajo el proceso de JLCPCB. Este orden de trabajo reduce el riesgo de errores de diseño irreversibles en la placa final y permite depurar el firmware con mayor libertad de modificación del cableado.

4.1.1. Prototipado en protoboard

El prototipo se construye alrededor de la placa de desarrollo ESP32-S3-DevKitC-1 (módulo ESP32-S3-WROOM-1, variante N16R8), que integra el controlador CAN nativo TWAI, evitando así la necesidad del controlador externo MCP2515 empleado en la arquitectura inicial del proyecto. La Tabla 4.1 resume los componentes utilizados en esta etapa.

Siguiendo el proceso de armado:

- Colocar primeramente el ESP32-S3 en una ubicación fácil de programar y de alimentar por su puerto de comunicación.

- Colocar estratégicamente el SN65HDV230 junto con el OLED, ya que estos componentes comparten una alimentación de 3,3 V.
- Después para no arruinar un micro SD se usa un adaptador SD, el cual soldaremos para una mejor conexión al protoboard.
- Y como ultimo se conecta una configuración de 4 leds para ayudar la visualización de los periféricos.

Tabla 4.1
Componentes empleados en el prototipo sobre protoboard

Componente	Función en el prototipo	Cantidad
ESP32-S3-DevKitC-1 (N16R8)	Unidad de procesamiento principal, TWAI nativo	1
Módulo transceptor SN65HVD230	Adaptación de niveles para el bus CAN	1
Módulo lector de tarjeta MicroSD (SPI)	Registro local de datos	1
Pantalla OLED SSD1306 (I2C)	Visualización de parámetros en tiempo real	1
Módulo regulador step-down 12 V a 5 V	Alimentación desde el conector OBD-II	1
Conector OBD-II tipo A con cable	Interfaz física con el vehículo	1
Protoboard, cables jumper macho-macho y macho-hembra	Interconexión temporal de los módulos	Variable

Nota. Relación de elementos de hardware integrados para el montaje preliminar y validación de comunicaciones del sistema.

La Figura 4.1 muestra la disposición física del prototipo, referenciada a lo largo de esta sección.

Figura 4.1
Componentes del prototipo y elementos principales del sistema.

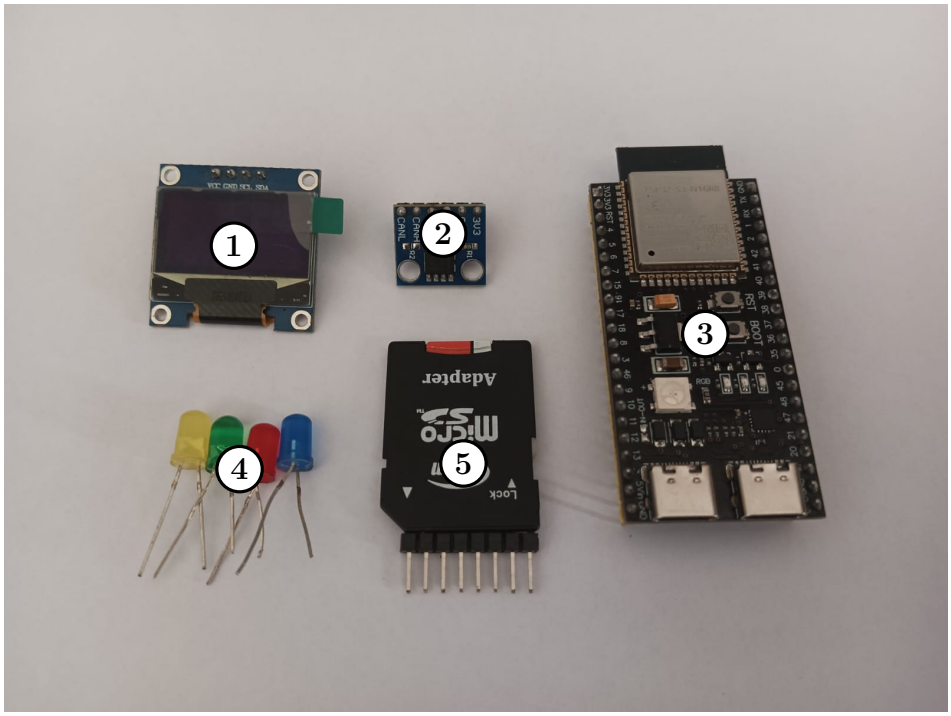


Tabla 4.2
Descripción de los componentes numerados en la Figura 4.1.

#	Componente	Qué es y qué hace
1	Pantalla OLED SSD1306	Pantalla de 128×64 px por bus I2C; muestra en tiempo real los parámetros del vehículo (RPM, velocidad, consumo instantáneo) directamente sobre el dispositivo.
2	Módulo transceptor SN65HVD230	Transceptor CAN de 3,3 V (pines CANL, CANH, CRX, CTX, GND y 3V3 visibles); adapta la señal diferencial del bus CAN del vehículo a los niveles lógicos TTL que el ESP32-S3 puede leer.
3	Módulo de desarrollo ESP32-S3-DevKitC-1 (N16R8)	Microcontrolador principal del sistema; ejecuta el firmware completo: adquisición de datos por OBD-II, cálculo de consumo (Speed-Density), registro en microSD y servidor web.
4	LEDs indicadores (amarillo, verde, rojo, azul)	Cuatro LEDs de estado que señalan de forma visual, sin necesidad de una computadora, si hay comunicación CAN activa, si se está escribiendo en la microSD y si el servidor web está disponible.
5	Adaptador de tarjeta microSD	Adaptador con pines soldados para conexión directa al protoboard por bus SPI; permite el registro del historial de cada viaje en un archivo CSV.

Nota. Componentes individuales del prototipo, antes de su montaje sobre protoboard, numerados en la Figura 4.1. Elaboración propia.

4.1.2. Integración del módulo ESP32-S3 y periféricos en protoboard

La asignación de pines para el prototipo se resume en la Tabla 4.3. Tal como se presentó en el esquemático de conexión dentro de la sección de diseño de software, esta misma distribución se mantiene para el prototipado físico (Tabla 3.25).

Tabla 4.3

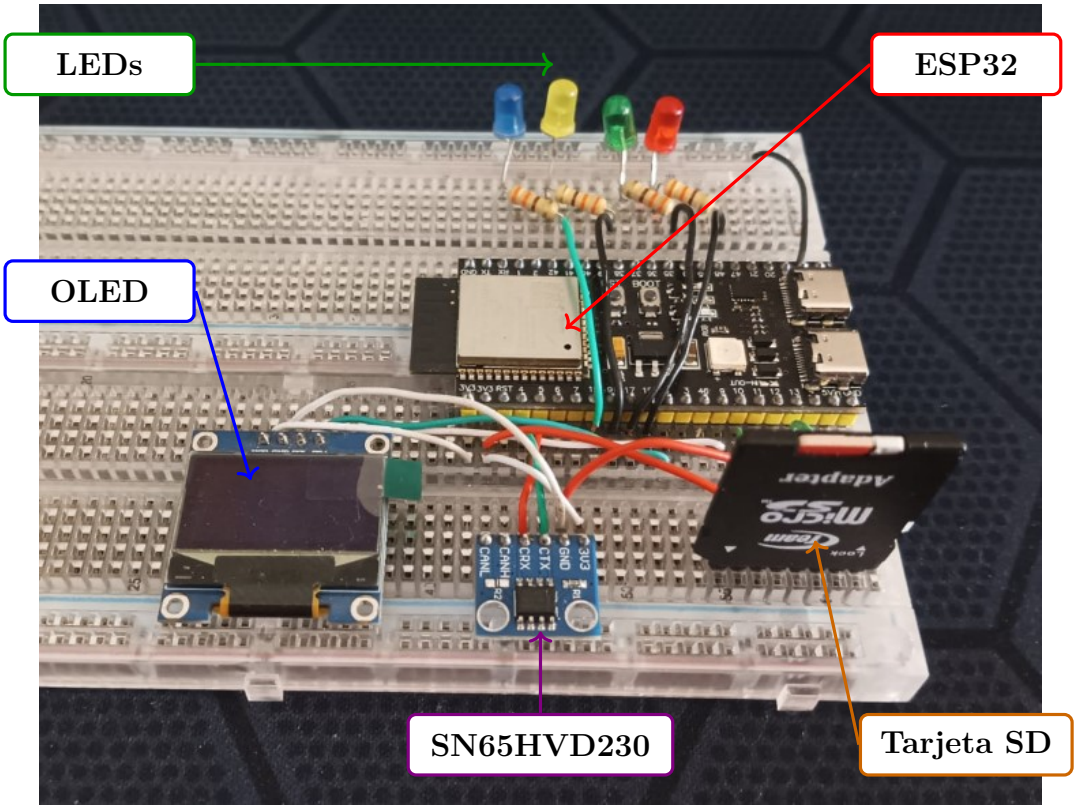
Asignación de pines GPIO del ESP32-S3 en el prototipo

Periférico	Pines GPIO	Observaciones
Bus CAN (TWAI)	TX: GPIO4, RX: GPIO5	Modo listen-only durante pruebas comparativas
Bus I2C (pantalla OLED)	SDA: GPIO8, SCL: GPIO9	Dirección I2C del controlador SSD1306
Bus SPI (MicroSD)	MOSI/MISO/SCK/CS: GPIO11-GPIO14	Sistema de archivos FAT32
Control de load switches (TPS22918DBVR)	GPIO6,GPIO10	Habilitación activa en alto (HIGH)
UART de programación	TXD0/RXD0	Uso exclusivo durante flasheo y depuración

Nota. Mapeo de la distribución física de señales digitales y buses de comunicación configurados en el microcontrolador principal.

Figura 4.2

Diagrama de interconexión de periféricos en el prototipo



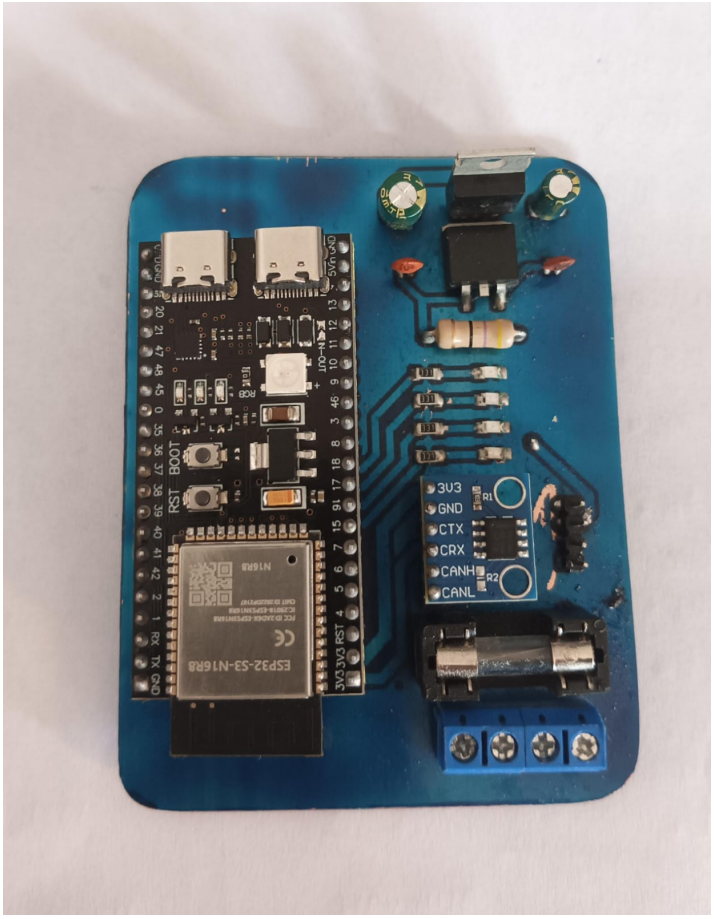
Nota. Vista general del prototipo sobre protoboard, con etiquetas identificativas para cada módulo. El **ESP32-S3** (rojo) es el microcontrolador principal: sondea el bus OBD-II, ejecuta el cálculo de consumo por el método Speed-Density y sirve el panel web local. Los **LEDs** (verde) son indicadores de estado de montaje superficial que señalan de forma visual, sin necesidad de conectar una computadora, si hay comunicación CAN activa, si se está escribiendo en la microSD y si el servidor web está disponible. La **OLED** (azul) es una pantalla de 128×64 px por bus I2C que muestra en tiempo real los parámetros principales del vehículo (RPM, velocidad, consumo instantáneo) directamente sobre el dispositivo. La **Tarjeta SD** (naranja), mediante su módulo lector por bus SPI, registra en un archivo CSV el historial completo de cada viaje para su análisis posterior. El **SN65HVD230** (violeta) es el transceptor CAN: adapta la señal diferencial de 2 V pico a pico del bus CAN del vehículo a los niveles lógicos TTL de 3,3 V que el controlador TWAI integrado del ESP32-S3 puede interpretar directamente. Elaboración propia.

La integración se verifica de manera incremental: primero se confirma la comunicación I2C con la pantalla OLED mediante un barrido de direcciones (*I2C scanner*), luego se valida la escritura y lectura en la tarjeta MicroSD, y finalmente se habilita el controlador TWAI para la recepción de tramas CAN, como se detalla en la Figura 4.2.

4.1.3. Placa preliminar en Altium Designer

Una vez concluidas las pruebas de campo sobre el prototipo en protoboard, se diseñó en Altium Designer una placa preliminar que replica el mismo circuito funcional ya validado (mismo módulo ESP32-S3, mismo transceptor CAN, misma pantalla OLED y microSD), con el único fin de contar con una versión más robusta y cómoda de manejar que las conexiones sueltas de una protoboard, previa a la placa final documentada en el Capítulo 3. A diferencia de esa placa final, la preliminar resuelve la alimentación con un regulador lineal simple en vez de la conversión conmutada de dos rieles, suficiente para las pruebas de campo dado que no corre con la misma restricción de autonomía en reposo; el esquemático completo y la simulación 3D se presentan en el Capítulo 3 (Sección 3.3.5). La fabricación de esta placa preliminar se encargó a Robokit (La Paz) por 150 Bs (Tabla 5.6, Capítulo 5), y el ensamblaje de los componentes se realizó manualmente, reutilizando el mismo módulo ESP32-S3, el transceptor SN65HVD230 y la pantalla OLED ya validados en la etapa de protoboard. La Figura 4.3 muestra la placa resultante tras este ensamblaje.

Figura 4.3
Ensamblaje de la placa preliminar



Nota. Placa preliminar ensamblada: el **módulo ESP32-S3-N16R8** actúa como unidad de procesamiento principal; el **regulador lineal** (7805 más el transistor de paso TIP42C) entrega los 5 V de alimentación a partir de los 12 V constantes del vehículo; el **módulo transceptor SN65HVD230** adapta la señal diferencial del bus CAN a los niveles lógicos TTL que el microcontrolador puede leer; y el **conector de entrada de 4 pines** (GND, 12 V, CAN_L, CAN_H) conecta la placa al puerto OBD-II del vehículo mediante el cable correspondiente. La descripción funcional completa de cada bloque de esta placa se presenta en el Capítulo 3 (Sección 3.3.5). Elaboración propia.

Al tratarse del mismo circuito ya probado, esta placa no requirió una nueva ronda de pruebas de campo: su función es exclusivamente de robustez física y comodidad de manejo, no de validación técnica adicional sobre lo ya reportado en este capítulo.

4.1.4. Conexión al bus CAN del vehículo

La conexión física al vehículo se realiza a través del conector estandarizado de 16 pines J1962, cuya asignación depende del protocolo de comunicación soportado; desde 2008 la práctica totalidad de los vehículos livianos emplea el protocolo ISO 15765 sobre bus CAN (ISO, 2021). La Tabla 4.4 resume los pines relevantes para este proyecto.

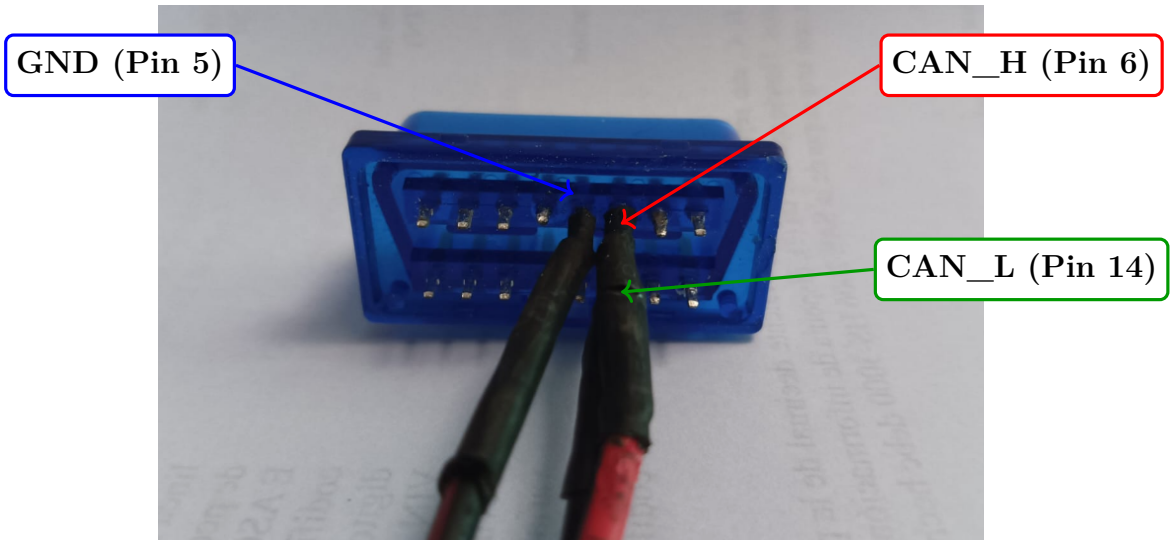
Tabla 4.4
Pines relevantes del conector OBD-II (J1962) utilizados en el proyecto

Pin	Señal	Uso en el sistema
6	CAN High (CAN_H)	Línea diferencial positiva del bus CAN
14	CAN Low (CAN_L)	Línea diferencial negativa del bus CAN
16	Alimentación +12 V	Entrada al regulador TPS54302DDCR
4 y 5	Tierra (chasis y señal)	Referencia común del sistema

Nota. Identificación de las líneas físicas de alimentación y comunicación del estándar J1962 requeridas para la interfaz con el vehículo.

Con el fin de evitar que el sistema propio introduzca tramas de solicitud adicionales que puedan interferir con el adaptador comercial, el controlador TWAI del ESP32-S3 se configura en modo de solo escucha (*listen-only*), de manera que únicamente recibe y decodifica las tramas sin transmitir reconocimientos ni solicitudes propias. En la Figura 4.4

Figura 4.4
Conexión del sistema propuesto mediante OBD-II



Nota. Fotografía de la interfaz OBD-II del vehículo mostrando la derivación del bus CAN. Elaboración propia.

4.1.5. Instalación física en el vehículo de pruebas

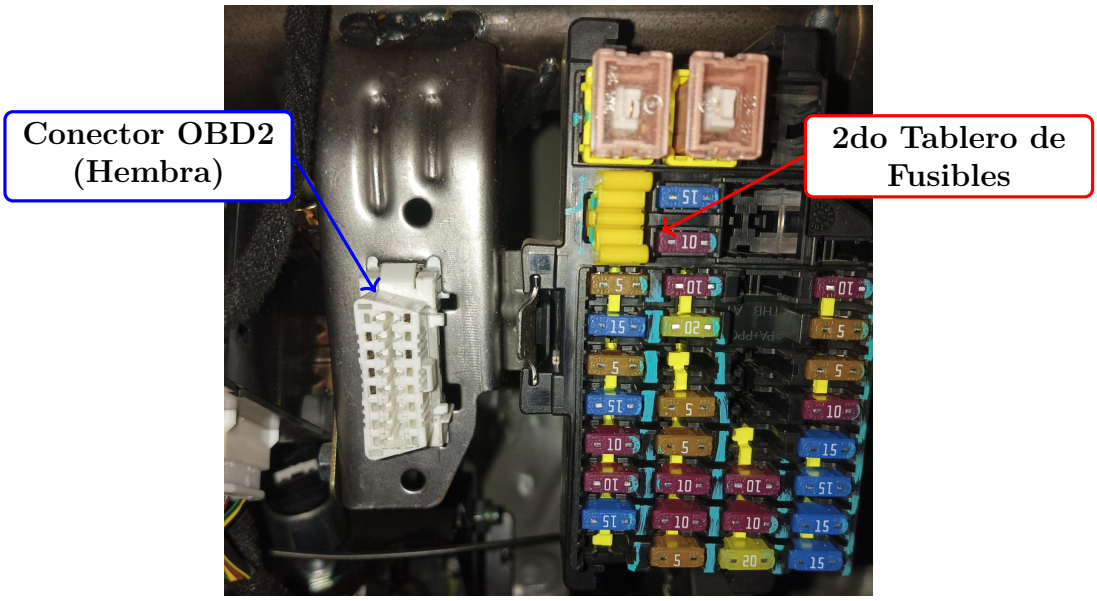
Buscamos para hacer las pruebas en campo , primero ubicar el puerto OBD2 del vehiculo el cual se ecuentra casi siempre abajo del volante arriba de los pedales, algunos a primera vista otros hay que desarmar unas piezas para llegar a su ubicacion.

En la Figura 4.5 mostramos la ubicacion de uno de los vehiculos a ponerlo a

prueba el cual se desarmo una parte para acceder este se encontro junto alos fusibles de los tableros.

Figura 4.5

Identificación física del OBD2 en el vehículo antes de las pruebas



Nota. Vista general del dispositivo dentro del habitáculo del vehículo. Elaboración propia.

Con el objetivo de que el dispositivo se instale de manera que no obstruye los controles del conductor y permite el acceso visual a la pantalla OLED durante la conducción.

4.2. Configuración del entorno de pruebas

4.2.1. Vehículos utilizados para la validación

La validación experimental se realiza sobre dos vehículos con arquitecturas de motor distintas. Esto permite evaluar la capacidad de generalización del método Speed-Density ante diferentes configuraciones de inyección y gestión electrónica. La Tabla 4.5 resume sus características principales.

Tabla 4.5
Vehículos utilizados en la fase de validación experimental

Característica	Changan Honor	Nissan Vanette
Año de fabricación	2023	2018
Cilindrada	1298 mL	1626 mL
Capacidad de tanque	40 L	60 L
Tipo de inyección	Electrónica	Electrónica
Combustible	Gasolina	Gasolina
Protocolo OBD-II	ISO 15765-4	ISO 15765-4
Kilometraje aproximado	5210	98765
Uso habitual	Particular y/o comercial	Cargamento

Nota. Ficha técnica comparativa de las unidades vehiculares empleadas para las pruebas de lectura de datos en el bus CAN y la recolección en campo.

La diferencia de antigüedad entre ambos vehículos (2023 frente a 2018) permite además evaluar el comportamiento del sistema frente a posibles variaciones en la calibración de fábrica de los sensores MAP y en el desgaste mecánico propio de un motor con mayor kilometraje.

Figura 4.6
Vehículos de prueba utilizados en la validación experimental

(a) *Changan Honor (2023)* **(b)** *Nissan Vanette (2018)*

Nota. Placa del Changan Honor cubierta por privacidad. Elaboración propia.

4.2.2. Configuración del adaptador ELM327 como referencia de validación

El adaptador comercial ELM327 (Figura 4.7) se emplea como dispositivo de referencia para validar el correcto funcionamiento y la adquisición de datos del sistema desarrollado. Debido a su amplia difusión en la decodificación de protocolos OBD-II,

este adaptador permite contrastar las lecturas de los parámetros del motor en tiempo real obtenidos por el prototipo propuesto. Para las pruebas de campo, el ELM327 se configura en modo de comunicación Bluetooth, forzando la selección del protocolo ISO 15765-4 (CAN a 500 kbps, ID de 11 bits) de manera manual para optimizar los tiempos de inicialización y evitar el proceso de autodetección en cada conexión.

Como herramienta comercial de evaluación se utiliza la aplicación *Car Scanner ELM OBD2*, la cual procesa la información enviada por el ELM327 para calcular el consumo instantáneo y acumulado a partir de las mismas variables del bus CAN (como las RPM, la presión absoluta del colector de admisión MAP y la velocidad del vehículo).

Conviene precisar que tanto el sistema propuesto como la solución basada en el ELM327 calculan el consumo de combustible mediante estimaciones indirectas (método *Speed-Density*) basadas en las señales recopiladas por el bus CAN, y no mediante una medición de flujo directo de gasolina. Así, el cotejo entre ambos equipos representa una prueba de concordancia entre dos plataformas de decodificación.

Limitación de conexión simultánea. En las pruebas de campo se verificó que el sistema propio y el adaptador ELM327 no pueden mantener ambos una comunicación estable con la ECU del vehículo al conectarse al mismo tiempo al puerto OBD-II: únicamente uno de los dos dispositivos logra establecer y sostener la sesión de diagnóstico. En consecuencia, la comparación entre ambos no se realiza sobre una única sesión simultánea, sino recorriendo el mismo trayecto dos veces de forma consecutiva (primero con el sistema propio conectado en solitario y, luego, inmediatamente después, con el ELM327 en solitario), manteniendo en lo posible las mismas condiciones de tráfico, ruta y estilo de conducción entre ambas pasadas. Esto introduce una fuente adicional de variabilidad entre las dos lecturas que no proviene de los instrumentos en sí, sino de la imposibilidad de repetir exactamente el mismo tránsito dos veces; se documenta aquí como una limitación metodológica explícita del estudio. Para obtener una referencia de consumo que no dependa de esta limitación ni de ningún método de estimación electrónica, las lecturas de ambos dispositivos se complementan con mediciones volumétricas directas mediante el método de tanque lleno a tanque lleno (Sección 4.3.5), realizadas con el sistema propio conectado de forma continua durante todo el trayecto real, tal como se detalla en la Sección 4.5.

4.2.3. Puesta en marcha del firmware y verificación inicial

Antes de cada sesión de pruebas, se ejecuta una rutina de verificación inicial que confirma la correcta inicialización del controlador TWAI (sin errores de bus-off), el montaje del sistema de archivos en la tarjeta MicroSD, la sincronización horaria del registro de datos y la visualización de al menos un parámetro en tiempo real (RPM o velocidad) en la pantalla OLED. La confirmación del correcto funcionamiento de cada uno de estos subsistemas se realiza mediante indicadores LED integrados en el módulo (Tabla 4.6); así se descarta en segundos una falla de montaje antes de invertir tiempo de trayecto en una sesión de prueba inválida.

Figura 4.7
Adaptador comercial ELM327 Bluetooth utilizado para la validación de lecturas OBD-II.

Nota. Scanner comercial ELM327.

Tabla 4.6
Lista de verificación previa a cada sesión de pruebas

Verificación	Indicador esperado	Resultado
Bus CAN inicializado, sin bus-off	LED CAN encendido fijo tras conectar el contacto	✓
Tarjeta MicroSD montada	LED SD parpadea al escribir (una vez por segundo)	✓
Servidor web activo	LED Server encendido; SSID `ESP32_OBD2` visible al buscar redes	✓
Ningún error activo	LED Error apagado (sin patrón de parpadeo)	✓
Parámetro en vivo visible	RPM o velocidad actualizándose en el OLED	✓

Nota. Se completa con ✓ (correcto) o `X` (falla, con la causa anotada aparte) al inicio de cada sesión, antes de arrancar el vehículo. Si el LED de error está encendido, el código de parpadeo (Sección 4.3 del Capítulo 3, `status_leds.cpp`) indica el subsistema que falló.

4.2.4. Herramientas y procesamiento de datos para el análisis

Cada sesión de prueba produce datos de tres fuentes distintas, que deben combinarse fuera del dispositivo para calcular las métricas de la Sección 4.5:

1. **CSV del sistema propio**, descargado desde el panel web (`/api/download`) al finalizar cada trayecto, o directamente de la tarjeta MicroSD. Contiene una fila por segundo con las columnas descritas en el Capítulo 3 (`trip_km`, `trip_fuel_L`,

`instant_L100km`, `baro_kpa`, entre otras).

2. **Registro del ELM327 + *Car Scanner***, exportado como CSV o KML/GPX desde la propia aplicación (opción de exportación en el historial de sesión), con marcas de tiempo y consumo instantáneo/acumulado calculado por la app.
3. **Bitácora manual de aforo**, un cuaderno o planilla con hora, odómetro y litros cargados en cada llenado de tanque (Sección 4.3.5).

El análisis se centraliza en un script de Python (`pandas` para leer y alinear los CSV por marca de tiempo, `numpy`/`scipy.stats` para las métricas estadísticas) en vez de calcularse a mano: de esta forma el proceso es reproducible, permite reprocesar todos los trayectos si se detecta un error, y deja un registro auditable de exactamente cómo se obtuvo cada número de la Tabla 4.21. Este y el resto de los scripts de análisis usados en el capítulo se encuentran en la carpeta `CODIGOS/` del repositorio de GitHub del proyecto (Calle, 2026), junto con el firmware del dispositivo. Las funciones utilizadas para cada métrica son las siguientes:

- RMSE y MAE: `numpy.sqrt(numpy.mean((est-ref)**2))` y `numpy.mean(numpy.abs(est-ref))` respectivamente, sobre los pares (estimado, referencia) de cada trayecto o punto de muestreo.
- MAPE: `numpy.mean(numpy.abs((est-ref)/ref))*100`, evitando dividir entre valores de referencia cercanos a cero (por ejemplo, excluyendo muestras con el vehículo detenido).
- Pearson r y R^2: `scipy.stats.pearsonr(est, ref)`; R^2 es el cuadrado del coeficiente devuelto, o directamente el de una regresión lineal con `scipy.stats.linregress`.
- Bland-Altman: calcular por muestra la diferencia $d = est - ref$ y el promedio $m = (est + ref)/2$; el sesgo medio es $\bar{d}$ y los límites de concordancia son $\bar{d} \pm 1{,}96\, s_d$, con s_d la desviación estándar de d. Un gráfico de dispersión de d contra m, con esas tres líneas horizontales, es la figura estándar de este análisis.
- Repetibilidad: coeficiente de variación $CV = s/\bar{x}$ del sistema propio sobre las repeticiones del mismo trayecto (Sección 4.3.5), expresado en porcentaje.

4.3. Pruebas experimentales

Esta sección describe la metodología seguida para cada tipo de prueba. En todos los casos se recomienda repetir cada prueba al menos 3 veces para poder estimar la variabilidad de la medición y no depender de una sola observación.

4.3.1. Validación de la comunicación CAN (ESP32 vs. ELM327)

Esta prueba verifica que el sistema propuesto decodifica correctamente las tramas de la ECU, con una tasa de éxito comparable a la del adaptador comercial. Dado que, como se explicó en la Sección 4.2.2, ambos dispositivos no pueden mantener una sesión OBD-II estable conectados simultáneamente al mismo puerto, esta verificación se realiza en dos sesiones separadas y consecutivas —motor encendido en ralentí, ventana de tiempo fija (por ejemplo, 5 minutos) para cada dispositivo—, registrando el número de tramas recibidas, la tasa de tramas perdidas o corruptas, y la latencia entre solicitud y respuesta de cada uno por separado. La Tabla 4.7 propone el formato de registro; la columna "Diferencia" compara el desempeño de cada dispositivo en su propia sesión, no una medición conjunta.

Tabla 4.7
Formato de registro para la validación de la comunicación CAN (sesiones separadas)

Parámetro	ESP32-S3 (sesión propia)	ELM327 (sesión separada)	Diferencia
Tramas recibidas en 5 min	2936	1712	1224
Tramas con error de CRC	0	34	34
Latencia promedio (ms)	2,2	231,4	229,2

Nota. Cada columna de dispositivo corresponde a una sesión de conexión independiente (Sección 4.2.2), con el vehículo en ralentí para minimizar la variabilidad entre ambas ventanas de medición.

Herramientas de medición

Los valores de la Tabla 4.7 se obtuvieron con dos programas de referencia construidos específicamente para esta prueba, uno por dispositivo, ya que ninguno de los dos expone estas métricas de forma nativa:

- **ESP32-S3** (`CODIGOS/can_bench_tool/`): un proyecto PlatformIO independiente del firmware de producción que, durante 5 minutos, solicita repetidamente el PID 0x0C (RPM), el único PID obligatorio en todo ECU OBD-II (así se aísla el desempeño de la comunicación en sí de la disponibilidad de un PID particular), y mide el tiempo entre cada solicitud y su respuesta directamente sobre el controlador TWAI nativo.
- **ELM327** (`CODIGOS/elm327_bench.py`): un script en Python que se conecta al adaptador por Bluetooth y le envía el mismo PID mediante comandos AT estándar, sin pasar por la aplicación comercial, para medir la misma ventana de 5 minutos bajo las mismas condiciones de solicitud.

Figura 4.8
Núcleo de medición de latencia en el benchmark del ESP32-S3, `can_bench_tool/src/main.cpp`.

```
uint32_t reqStartUs = micros();
requestsSent++;

if (sendPidRequest(PID_TO_TEST) && waitForResponse(PID_TO_TEST,
    RESPONSE_TIMEOUT_MS)) {
  responsesOk++;
  latencySumUs += (uint64_t)(micros() - reqStartUs);
}

uint32_t alerts = 0;
if (twai_read_alerts(&alerts, 0) == ESP_OK && alerts != 0) {
  if (alerts & (TWAI_ALERT_BUS_ERROR | TWAI_ALERT_ERR_PASS |
    TWAI_ALERT_BUS_OFF)) {
    errorFrames++;
  }
}

```

Nota. La latencia se mide con `micros()` entre el envío de la solicitud y la recepción de la trama de respuesta válida; los errores se cuentan a partir de las alertas del propio controlador TWAI (`ESP-IDF`), no de una re-implementación manual de detección de errores de bus.

Figura 4.9
Núcleo de medición y clasificación de respuestas en el benchmark del ELM327, `elm327_bench.py`.

```
t0 = time.time()
ser.reset_input_buffer()
ser.write(b"010C\r")
requests_sent += 1

resp = ser.read(64).decode(errors="ignore")
elapsed = time.time() - t0
resp_clean = resp.replace("\r", " ").upper()

if "NO DATA" in resp_clean or "ERROR" in resp_clean or "BUS" in resp_clean:
    error_frames += 1
elif "41 0C" in resp_clean:
    responses_ok += 1
    latencies.append(elapsed)

```

Nota. La latencia incluye, a diferencia de la medición del ESP32-S3, el tiempo de procesamiento interno del ELM327 (traducción de comandos AT a tramas CAN) y de la pila Bluetooth del sistema operativo, no solo el tiempo de bus.

4.3.2. Verificación de comunicación OBD-II mediante PIDs

La verificación de la comunicación con cada vehículo se realizó mediante la solicitud y recepción de parámetros estándar definidos por el protocolo OBD-II (SAE J1979), conocidos como *Parameter IDs* (PIDs), utilizando el modo de servicio `0x01` (datos en tiempo real).

El procedimiento consistió en enviar tramas CAN con el formato estándar OBD-II (`0x7DF` como ID de solicitud broadcast) y capturar la respuesta de la ECU de cada vehículo (`0x7E8`). Los PIDs reportados como disponibles por cada vehículo fueron contrastados contra la tabla de referencia del modo 01 (Apéndice A) y el análisis matemático de resolución por PID disponible, obteniendo los resultados de disponibilidad resumidos en la Tabla 4.8, un detalle mas completo se presenta en el Apéndice O.

Tabla 4.8
PIDs consultados y disponibilidad por vehículo.

PID	Parámetro	Unidad	Changan Honor	Nissan Vanette
`0x0C`	Régimen del motor (RPM)	rpm	✓	✓
`0x0D`	Velocidad del vehículo	km/h	✓	✓
`0x10`	Flujo másico de aire (MAF)	g/s		✓
`0x05`	Temperatura del refrigerante	°C	✓	✓
`0x11`	Posición del acelerador	%	✓	✓

Nota. PIDs verificados mediante el escaneo de la Figura 3.11.

La disponibilidad del PID `0x10` (MAF) en ambos vehículos es crítica para el sistema, ya que es el parámetro base para el cálculo del consumo de combustible, conforme a la fórmula de decodificación definida por el estándar SAE J1979 (SAE International, 2014).

4.3.3. Pruebas del modelo Speed-Density

Se realizan pruebas estáticas a régimen de motor controlado (en ralentí y en pasos de 1500, 2500 y 3500 rpm, sin carga en las ruedas) con el fin de aislar el comportamiento del modelo Speed-Density de las variables dinámicas de conducción. El primer punto no se fija a un número redondo arbitrario (por ejemplo, 800 rpm): el regulador de ralentí de cada motor tiene su propio piso, por debajo del cual el motor no puede sostenerse sin calarse —en el Nissan Vanette utilizado, ese piso se ubicó experimentalmente entre 960 y 980 rpm—, así que el primer punto se toma directamente como el ralentí real medido al inicio de la prueba. En cada punto se registra el caudal de aire estimado por el sistema propio y se contrasta contra el valor de referencia reportado por la ECU a través del PID estándar de MAF, cuando el vehículo lo soporta.

Esta comparación directa punto a punto solo es posible en el **Nissan Vanette**, que sí reporta el PID 0x10 (Tabla 4.8). El **Changan Honor** no dispone de sensor MAF (es, de hecho, la razón por la que este proyecto necesita el método Speed-Density en primer lugar, Sección 3.4.5 del Capítulo 3), de modo que no existe una referencia directa de caudal de aire para ese vehículo en esta prueba estática. Su exactitud se evalúa entonces de forma indirecta, aguas abajo, a través de la comparación de consumo acumulado contra el aforo real por tanque lleno (Sección 4.3.5), que sí aplica a ambos vehículos por igual.

Herramienta de medición

A diferencia de la validación de comunicación CAN de la sección anterior, esta prueba no compara un dispositivo contra otro, sino una *estimación* contra una *referencia*, y ambas se calculan a partir de las mismas lecturas de RPM, MAP e IAT tomadas en el mismo instante. Por eso aquí basta con el **ELM327 solo**, hablado directamente por un script en Python (`CODIGOS/speed_density_bench.py` en el repositorio del proyecto (Calle, 2026)): en cada ciclo solicita RPM (PID 0x0C), MAP (PID 0x0B), IAT (PID 0x0F) y MAF real (PID 0x10), y con las tres primeras calcula el MAF estimado usando exactamente la misma fórmula y tabla de eficiencia volumétrica (VE) que el firmware de producción (`fuel_calc.cpp`). Al no requerir una segunda sesión OBD-II en paralelo, esta prueba queda al margen de la limitación de conexión simultánea documentada en la Sección 4.2.2.

Figura 4.10
Cálculo del MAF estimado en ***speed_density_bench.py****, réplica en Python de la fórmula de* ***fuel_calc.cpp****.*

```python
def maf_estimado_gs(rpm, map_kpa, iat_c):
    ve = ve_for_rpm(rpm)          # misma tabla VE que kVeTable en fuel_calc.
    cpp
    iat_k = iat_c + 273.15
    if iat_k < 233.15 or iat_k > 373.15:
        iat_k = 288.15             # misma guarda que el firmware
    maf = (ve / 100.0) * rpm * map_kpa * ENGINE_DISPLACEMENT_L * MM_AIR \
          / (2.0 * R * iat_k * 60.0)
    return max(maf, 0.0)

```

Nota. `ENGINE_DISPLACEMENT_L`, la tabla `VE_TABLE` y las constantes `MM_AIR`/`R` son una copia literal de los valores de `config.h` y `fuel_calc.cpp`, de modo que la estimación calculada en la prueba es la misma que produciría el equipo embarcado ante las mismas lecturas.

Al iniciar, el script mide el ralentí real promediando varias lecturas de RPM antes de comenzar a clasificar muestras, y lo antepone a los otros tres puntos fijos (1500, 2500 y 3500 rpm) como el primer objetivo de la prueba. Cada muestra se clasifica según el punto de régimen objetivo más cercano, con tolerancia de ±100 rpm, y se descarta si no cae dentro de esa tolerancia; al finalizar la prueba el script promedia el MAF estimado y el MAF de referencia dentro de cada punto y calcula el error porcentual entre ambos.

Tabla 4.9

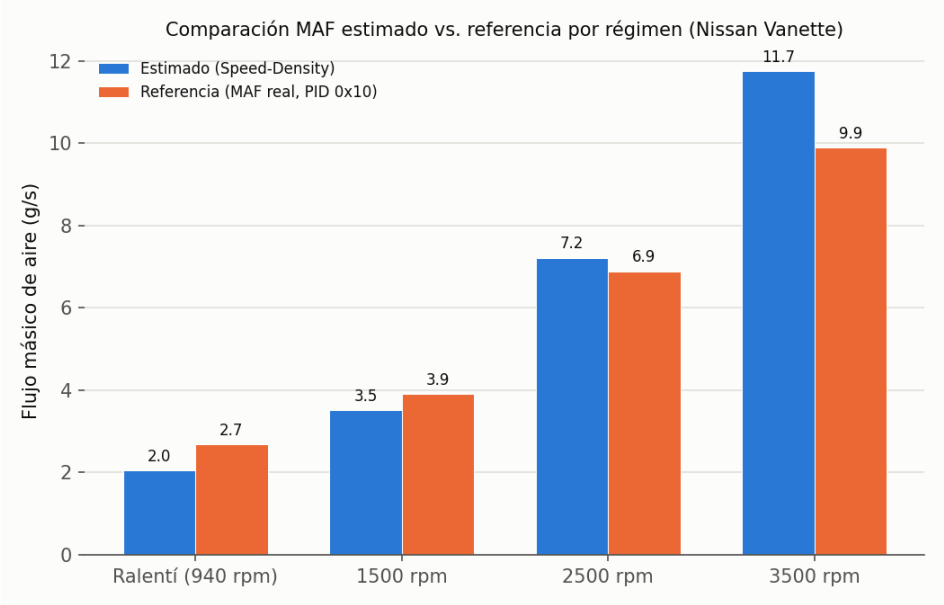
Comparación del MAF estimado por el modelo Speed-Density frente al MAF de referencia de la ECU (PID 0x10), Nissan Vanette

RPM	n muestras	MAF estimado (g/s)	MAF ref. (g/s)	Error (%)
Ralentí (940 rpm)	22	2,04	2,68	−23,7
1500	25	3,52	3,90	−9,8
2500	22	7,21	6,89	+4,7
3500	20	11,74	9,90	+18,7

Nota. Valores promedio por punto de régimen, obtenidos con `speed_density_bench.py` y registrados en `speed_density_log.csv`. Error (%) = (MAF estimado − MAF referencia) / MAF referencia × 100.

Figura 4.11

Comparación del caudal de aire estimado por el modelo Speed-Density frente al valor de referencia de la ECU, por régimen de motor



Nota. Gráfico de barras agrupadas, un par de barras (estimado / referencia) por cada uno de los cuatro regímenes evaluados. Se prefiere sobre un diagrama de dispersión porque el diseño experimental produce cuatro promedios discretos por punto de régimen, no una nube de muestras continuas cuya correlación interese visualizar: precisamente el caso de uso de las barras agrupadas frente a la dispersión. Elaboración propia.

Calibración del modelo Speed-Density

Los resultados de la Tabla 4.9 muestran un error grande y, más importante, de *signo variable* entre regímenes: el modelo subestima el caudal de aire en ralentí y a 1500 rpm (−23,7 % y −9,8 %), pero lo sobrestima a 2500 y 3500 rpm (+4,7 % y +18,7 %). Un sesgo de signo constante podría explicarse por un solo factor de escala mal calibrado

(por ejemplo, la cilindrada); un sesgo que cambia de signo con el régimen, en cambio, apunta a que la *forma* de la curva de eficiencia volumétrica (VE) usada por el modelo no corresponde a la de este motor en particular. Al notar un error de esta magnitud, se procede a calibrar el sistema con los mismos datos de la Tabla 4.9, en vez de aceptar la tabla VE genérica con la que partió el diseño.

La revisión encontró además un segundo problema, independiente del anterior: la constante `ENGINE_DISPLACEMENT_L` del firmware (`config.h`) estaba fijada en 1,6 L, mientras que la cilindrada real del Nissan Vanette, según su ficha técnica (Tabla 4.5), es de 1626 mL. Ambas correcciones —cilindrada real y tabla VE calibrada— se aplican juntas antes de recalcular.

Método de calibración. La ecuación de `fuel_calc.cpp` (Sección 3.6.5 del Capítulo 3) se invierte algebraicamente para despejar la VE que, dada una muestra real de RPM, MAP e IAT, hace que el MAF estimado coincida exactamente con el MAF de referencia:

$$VE_{\text{requerida}} = \frac{MAF_{\text{ref}} \cdot 100 \cdot 2 \cdot R \cdot T_{IAT} \cdot 60}{RPM \cdot P_{MAP} \cdot V_d \cdot 28{,}97} \tag{4.1}$$

Esta inversión se aplica muestra por muestra sobre `speed_density_log.csv` y se promedia por punto de régimen mediante `CODIGOS/calibrate_ve_table.py`, un script nuevo e independiente de `speed_density_bench.py` (no requiere el vehículo ni el ELM327, solo el CSV ya grabado). La Tabla 4.10 resume la VE que exigía la tabla genérica original frente a la VE real requerida en cada punto.

Tabla 4.10
Eficiencia volumétrica (VE) genérica frente a la VE real requerida, por punto de régimen

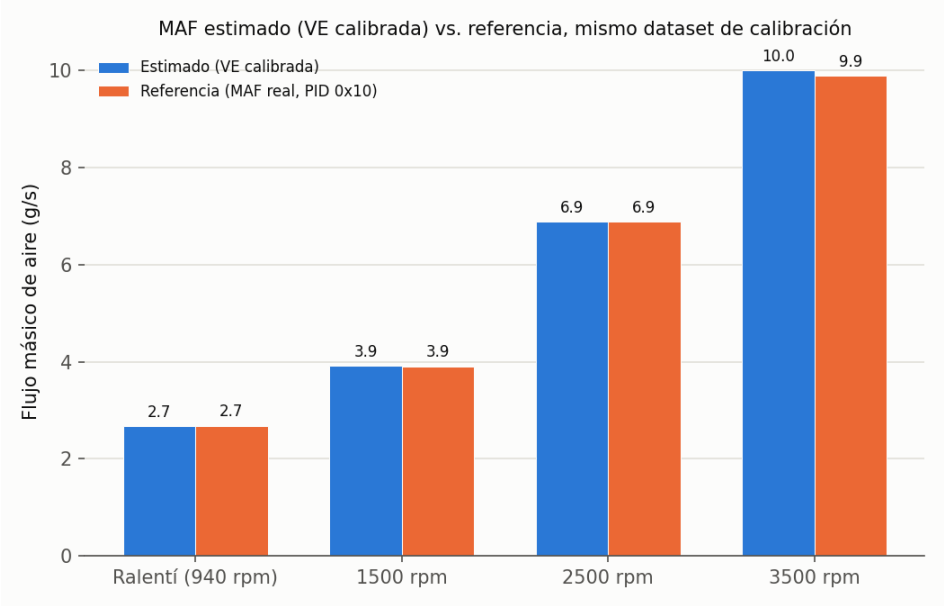
Punto	RPM promedio real	VE genérica (%)	VE requerida (%)
Ralentí	936,3	64,1	82,7
1500	1446,7	73,1	79,7
2500	2491,6	84,9	79,7
3500	3483,0	87,0	72,7

Nota. Calculado con `calibrate_ve_table.py` sobre `speed_density_log.csv`, usando $V_d = 1{,}626$ L. Los puntos 4000/5500/7000 rpm de `kVeTable` no se modifican por no contar con datos de prueba en ese rango; quedan documentados como sin calibrar.

La tabla `kVeTable` de `fuel_calc.cpp` y su copia en `speed_density_bench.py` se actualizan con estos cuatro puntos calibrados, conservando sin cambios los tres puntos superiores (4000/5500/7000 rpm) por no contarse con mediciones reales en ese rango.

Figura 4.12

Validación de la tabla VE calibrada frente al MAF de referencia, mismo conjunto de datos de calibración



Nota. Generado por `calibrate_ve_table.py` recalculando el MAF estimado de cada muestra con la tabla ya calibrada. El error residual (entre −0,02 % y +1,03 % según el punto) confirma que la inversión algebraica se aplicó correctamente, pero **no** es una validación independiente del modelo: al calibrar y verificar sobre el mismo conjunto de datos, un error cercano a cero es el resultado esperado por construcción, no evidencia de que el modelo generalice a condiciones dinámicas de conducción o a otro rango de régimen. Elaboración propia.

Esta limitación metodológica —validar sobre el mismo conjunto de datos con el que se calibró— se resuelve con una segunda prueba, dinámica, descrita a continuación.

Prueba dinámica (ruta real)

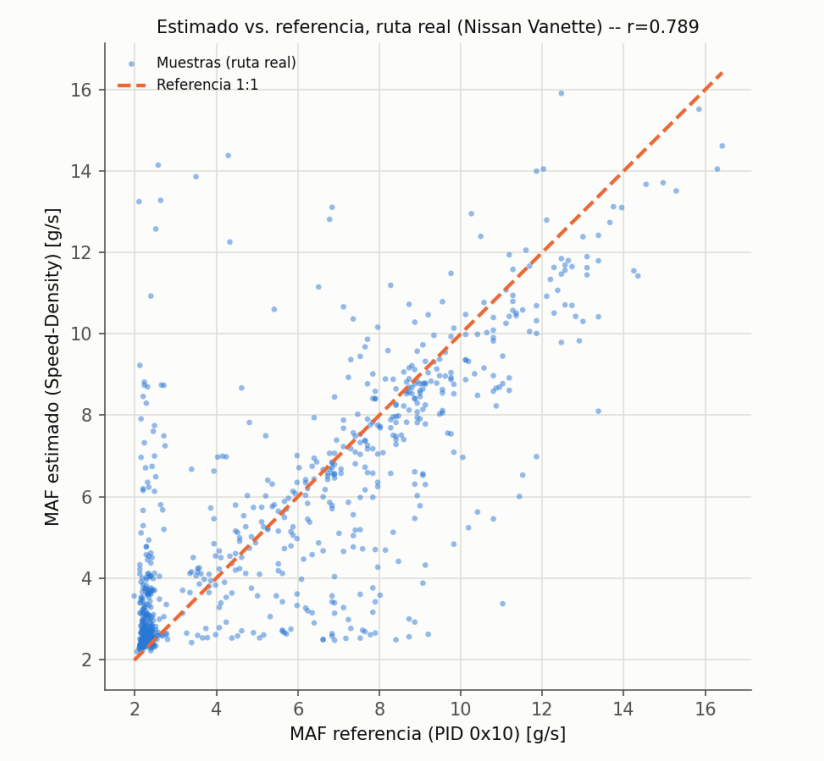
A diferencia de la prueba estática anterior, aquí no se sostiene el motor en puntos fijos de régimen: se conduce una ruta real (arranque, aceleración, crucero a distintas velocidades, frenado) mientras el sistema registra continuamente RPM, MAP, IAT y el MAF real de referencia. El objetivo no es calibrar nada —la tabla VE ya quedó fija en la prueba estática—, sino verificar qué tan bien predice el modelo *ya calibrado* sobre datos que nunca participaron en esa calibración, resolviendo directamente la limitación señalada en la Figura 4.12. Al igual que la prueba estática, esta solo aplica al **Nissan Vanette** (único vehículo con PID 0x10).

`CODIGOS/speed_density_drive_log.py` realiza este registro: a diferencia de `speed_density_bench.py`, no agrupa las muestras por punto de régimen objetivo ni corta automáticamente (ese criterio, pensado para la prueba estática, cortaría una ruta real apenas el RPM de manejo normal pasara cerca de cualquiera de los 4 puntos fijos), así que aquí simplemente registra cada muestra tal como

llega, sin requerir ninguna interacción mientras se maneja. Al finalizar (Ctrl+C), `CODIGOS/speed_density_drive_validation.py` calcula sobre el registro completo las mismas métricas ya definidas en la Sección 4.2.4 (RMSE, MAE, MAPE, coeficiente de Pearson) y genera un diagrama de dispersión del MAF estimado frente al de referencia. A diferencia de la Figura 4.11 (barras, 4 promedios discretos), aquí sí corresponde un diagrama de dispersión: el registro de una ruta produce una nube continua de muchas muestras correlacionadas, el caso de uso exacto de ese tipo de gráfico.

Figura 4.13

Dispersión del MAF estimado frente al MAF de referencia durante una ruta real



Nota. Cada punto es una muestra del registro de ruta; la línea discontinua es la recta 1:1 (predicción perfecta). Elaboración propia.

4.3.4. Validación del sensor MAP frente a la altitud

A diferencia de una implementación que asumiera una presión de referencia fija de nivel del mar, el método speed-density de este proyecto usa en cada ciclo la presión absoluta real del colector (MAP, PID 0x0B) reportada por la propia ECU, por lo que el cálculo de consumo ya queda corregido por altitud sin necesitar ningún factor adicional (Sección 3.6.5 y Subsección 3.4.3 del Capítulo 3). Lo que sí conviene verificar en campo es que el *sensor* MAP del vehículo esté bien calibrado para operar a los $\approx 3\,600$ msnm de La Paz, ya que un sesgo de fábrica en ese sensor sí se propagaría al consumo calculado.

La prueba consiste en comparar, con el contacto puesto y el motor apagado (*key-on, engine-off*, sin vacío de admisión), la lectura de MAP contra la presión barométrica

real. El firmware registra esta última como `baro_kpa` en el CSV y en el panel web (PID 0x33 cuando el vehículo lo soporta; en su defecto, la estimación del modelo ISA evaluado en `SITE_ALTITUDE_M`, marcada como tal mediante `baro_is_estimated`). La Tabla 4.11 propone el formato de registro.

Herramienta de medición

Al ser una prueba puntual (motor apagado, sin trayecto de por medio), esta fila de la Tabla 4.11 se llena con un script independiente, `CODIGOS/map_altitude_report.py`, que se conecta directamente al ELM327 por Bluetooth y no requiere el sistema propio conectado a la vez: solicita el PID 0x0B (MAP) y el PID 0x33 (presión barométrica) un número fijo de veces, promedia ambas lecturas y calcula la diferencia porcentual entre ellas. Si el vehículo no soporta el PID 0x33 —caso que se decide en tiempo de ejecución, no de antemano—, el script recurre al mismo modelo ISA evaluado en `SITE_ALTITUDE_M` que usa el firmware como respaldo (`isaPressureKpaAt()` en `can_obd2.cpp`), replicado con la fórmula idéntica en Python. Cada corrida se ejecuta una vez por vehículo y se anexa a `map_altitude_report.csv`, listo para copiar a la Tabla 4.11.

Tabla 4.11
Validación del sensor MAP en key-on, engine-off frente a la presión barométrica

Vehículo	MAP en KOEO (kPa)	`baro_kpa` (kPa)	Diferencia (%)
Changan Honor	63,0	63,0	0,0
Nissan Vanette	63,0	64,6	-2,5

Nota. Una diferencia pequeña (idealmente de pocos kPa) confirma que el sensor MAP reporta la presión atmosférica real de La Paz, y con ello que el cálculo speed-density parte de una base correcta; una diferencia grande señalaría un sesgo de calibración del sensor MAP que ninguna corrección de software puede resolver por sí sola.

Como referencia teórica de contraste, el modelo ISA evaluado a 3 600 msnm predice $P(3\,600) \approx 64{,}9\,\text{kPa}$ (Capítulo 3, Ecuación de la Atmósfera Estándar Internacional).

4.3.5. Protocolo de trayectos de referencia (tanque lleno a tanque lleno)

Esta es la prueba central de la validación (Sección 4.5), ya que produce el único dato de consumo que no depende de ningún método de estimación electrónica: el volumen de combustible efectivamente repuesto. Su valor como referencia depende por completo de la disciplina con la que se ejecute el protocolo de llenado, por lo que se fijan las siguientes reglas antes de iniciar cada trayecto:

- **Llenado de referencia.** Cargar combustible hasta el corte automático de la pistola del surtidor (*primer clic*), sin continuar el llenado manualmente más allá de

ese punto; repetir el mismo criterio al cerrar el trayecto. Siempre que sea posible, usar el mismo surtidor o al menos la misma estación de servicio en ambos extremos, para minimizar diferencias de calibración entre bombas.

- **Registro previo al arranque.** Anotar en la bitácora (Sección 4.2.4): fecha, hora, odómetro del vehículo, y confirmar que el sistema propio pasó la lista de verificación de la Tabla 4.6.
- **Selección de rutas.** Definir de antemano tres categorías de trayecto por vehículo (corto, medio y largo) para cubrir un rango amplio de distancias y condiciones de manejo (urbano de La Paz y, cuando sea posible, ruta más abierta) en vez de validar el modelo en una única condición. El corto y el medio se repiten 2 veces cada uno, registrando ida y vuelta como tramos separados (4 tramos por categoría); el largo se recorre una sola vez, sin repetición ni separación ida/vuelta.
- **Control del conductor.** Mantener, en la medida de lo posible, el mismo conductor y un estilo de conducción habitual (sin aceleraciones ni frenadas deliberadamente extremas) entre trayectos, de forma que la variabilidad medida refleje principalmente al vehículo y al sistema, no al estilo de manejo.
- **Cierre de cada tramo.** Al final de cada uno de los nueve tramos (corto ida/vuelta $\times 2$, medio ida/vuelta $\times 2$, largo), registrar odómetro y descargar el CSV del trayecto (`trip_km`, `trip_fuel_L`) antes de reiniciar el contador de viaje para el siguiente, *sin* volver a llenar el tanque todavía.

A diferencia del planteamiento original de repetir el llenado en cada trayecto, en la práctica no es viable reabastecer en cada uno de los nueve tramos en una sola sesión de pruebas: el llenado de referencia se hace **una sola vez**, antes del primer tramo (ida del corto) y una vez más al cerrar el último (el largo), de modo que los litros repuestos en ese segundo llenado son el consumo real de referencia de los **nueve tramos combinados**, no de cada uno por separado. El sistema propio permanece conectado de forma continua durante todos, por lo que sí se puede leer su `trip_fuel_L` individual de cada tramo además del acumulado; la lectura del ELM327 + Car Scanner para cada tramo se obtiene, como en las demás pruebas, en una sesión separada e inmediatamente posterior sobre la misma ruta (Sección 4.2.2). Las Tablas 4.19 y 4.20 (una por vehículo) y las métricas de la Tabla 4.21 comparan el sistema propio contra el ELM327 tramo por tramo, y contra el aforo real solo a nivel del total combinado —única granularidad en la que existe ese dato—, con la variabilidad entre pasadas documentada como limitación metodológica.

4.3.6. Pruebas de registro de datos en MicroSD

Se verifica la integridad de los archivos generados (formato, ausencia de líneas corruptas o incompletas) tras sesiones de registro de distinta duración, así como el comportamiento del sistema ante una desconexión abrupta de la alimentación durante la escritura, situación habitual al apagar el vehículo.

La prueba consta de dos partes. Primero, se registran al menos dos sesiones de

duración claramente distinta (una corta y una larga) para caracterizar el tamaño de archivo y la tasa de muestreo real bajo uso normal; lo que importa no es un número exacto de minutos, sino confirmar que ambos valores se mantienen estables independientemente de la duración de la sesión. Segundo, durante una sesión de registro activa se desconecta la alimentación del sistema de forma abrupta, sin pasar por el apagado ordenado, y se examina el archivo resultante: dado que `sdLoggerTask()` llama `flush()` después de cada fila (`sd_logger.cpp`), en el peor caso se espera perder únicamente la fila que estaba a medio escribir en el momento del corte, no el archivo completo ni sesiones anteriores. Esta segunda parte es independiente de las duraciones elegidas para la primera.

Herramienta de medición

Ambas partes de la prueba se evalúan con `validate_sd_log.py`, un script que se corre directamente sobre los archivos `trip_NNN.csv` ya descargados (desde `/api/download` o copiados de la tarjeta), sin necesitar ninguna conexión al vehículo ni al ELM327. Por cada archivo, verifica que el encabezado coincida con las 18 columnas de `sd_logger.cpp`, cuenta las líneas corruptas o incompletas (número de columnas incorrecto o campos no numéricos) y los huecos de muestreo (saltos en `uptime_ms` mayores a 2,5 veces el intervalo nominal de `SD_LOG_INTERVAL_MS`), y calcula la duración cubierta, la tasa de muestreo real y el tamaño de archivo extrapolado a una hora de registro. Cada archivo analizado se anexa a `sd_log_report.csv`, listo para copiar a la Tabla 4.12.

Tabla 4.12
Integridad y tasa de registro de los archivos CSV generados en MicroSD, por sesión

Sesión	Duración	Filas válidas	Filas corruptas	Tasa (Hz)	Tamaño / hora
5 min (nominal)	5,2 min	292	0	1,004	290,3 KB
60 min (nominal)	60,8 min	3649	0	1,000	315,8 KB
Corte abrupto de alimentación	60,8 min	3649	0	1,000	—

Nota. Valores obtenidos con `validate_sd_log.py` (Changan Honor). Las filas de 60 min y corte abrupto provienen del mismo archivo (`trip_003.csv`: sesión en ralentí, cortada de forma abrupta al final). La tasa y el tamaño por hora varían solo 8,8 % entre la sesión corta y la larga: no dependen, entonces, de la duración de la sesión. Las 0 filas corruptas en ambas confirman que el `flush()` por fila evita corrupción, salvo la última fila, que ni llega a escribirse.

4.3.7. Pruebas del panel de control web (WebSocket/HMI)

Se evalúan tres aspectos del panel: la latencia de actualización (qué tan seguido llegan datos nuevos frente al periodo nominal de envío), la estabilidad de la conexión

WebSocket durante sesiones prolongadas, y el comportamiento ante la pérdida temporal de la señal WiFi al alejarse del punto de acceso generado por el ESP32-S3.

Herramienta de medición

El propio panel web se instrumentó para esta prueba (`web_server.cpp`), sin alterar su funcionamiento normal: cada vez que llega un mensaje por el WebSocket se registra, en el propio navegador, el intervalo respecto al mensaje anterior; cada reconexión también se cuenta. Dos funciones quedan disponibles desde la consola del navegador (F12) mientras el panel está abierto:

- `wsStatsReport()`: imprime un resumen en el momento (n.° de muestras, media, desviación estándar, mínimo, máximo, reconexiones y duración de la sesión).
- `wsStatsDownload()`: descarga `ws_intervals.csv` con todos los intervalos individuales, para analizarlos con `CODIGOS/ws_latency_report.py`, que calcula el resumen estadístico y genera un histograma frente al valor nominal (`WS_PUSH_PERIOD_MS` = 500 ms en `config.h`).

El protocolo tiene tres partes: (1) una sesión con el panel abierto y el vehículo en ralentí, leyendo tanto la latencia de actualización (`wsStatsDownload()`) como el número de reconexiones acumulado (`wsStatsReport()`) al final (una sola sesión sirve para ambas, ya que no hay razón para separarlas en dos corridas); (2) alejarse físicamente del vehículo con el dispositivo que muestra el panel, en pasos de distancia crecientes, anotando a qué distancia se pierde la conexión (el indicador "Enlace: WS" cambia a "Enlace: polling") y si reconecta solo al volver a acercarse.

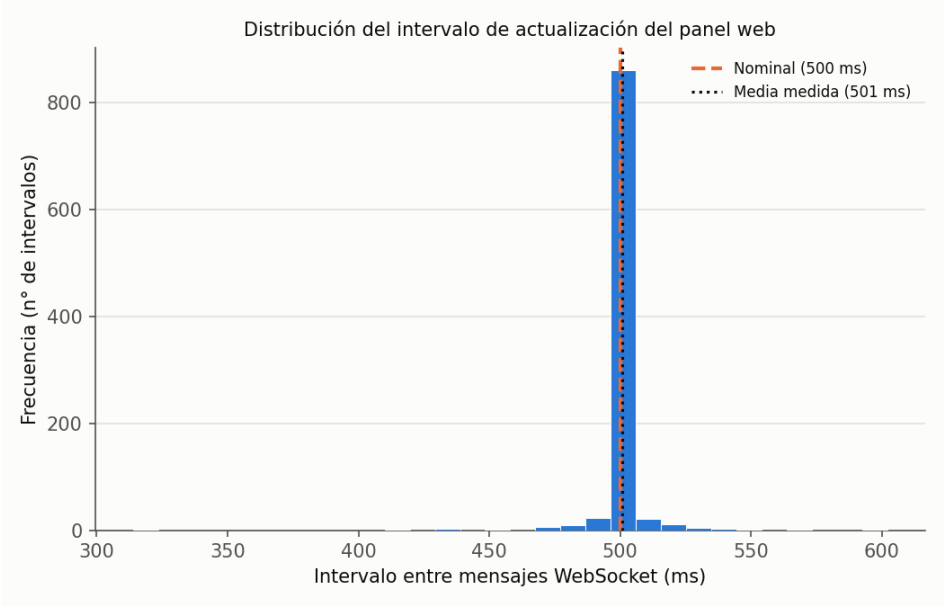
Tabla 4.13
Latencia de actualización del panel web (intervalos entre mensajes WebSocket)

Nominal (ms)	n muestras	Media (ms)	Desv. (ms)	Mín. (ms)	Máx. (ms)
500	945	500,0	10,0	314,9	602,0

Nota. Valores calculados por `ws_latency_report.py` sobre `ws_intervals.csv`.

Figura 4.14

Distribución del intervalo de actualización del panel web frente al valor nominal



Nota. Histograma de los intervalos individuales, con líneas verticales marcando el valor nominal (500 ms) y la media medida. Se prefiere el histograma sobre otros tipos de gráfico porque la pregunta es "qué tan dispersos están los intervalos alrededor del valor esperado", una distribución de una sola variable continua. Elaboración propia.

Tabla 4.14

Estabilidad de la conexión WebSocket en sesión prolongada

Duración de la sesión	N.° de reconexiones	Observaciones
8,1 min	0	Misma sesión de la Tabla 4.13; sin caídas en todo el registro

Nota. Reconexiones acumuladas reportadas por `wsStatsReport()` al cierre de la sesión. El reconector automático (`setTimeout(connectWs, 2000)`) hace que una caída puntual no requiera intervención manual; esta tabla registra cuántas veces ocurrió, no si el panel dejó de funcionar. La sesión registrada (8,1 min) es más corta que una sesión típica de uso, pero al no presentar ninguna reconexión aporta evidencia consistente con una conexión estable; ampliarla queda como posible verificación adicional.

Tabla 4.15
Alcance del punto de acceso WiFi generado por el ESP32-S3

Distancia (m)	Obstáculos	¿Conectado?	Reconectó al acercarse
5	Si (pared)	✓	-
10	Si (pared)	✓	-
20	Si (cuartos)	✓	-

Nota. Distancia aproximada entre el punto de acceso (ESP32-S3, dentro del vehículo) y el dispositivo que muestra el panel; agregar una fila por cada distancia probada.

4.3.8. Pruebas de consumo energético y modo de bajo consumo

Esta prueba requiere medir corriente directamente sobre la placa (un multímetro o pinza en serie con la alimentación de 12 V), algo que todavía no es posible con el prototipo actual sobre protoboard sin alterar el cableado de forma poco representativa. Queda pendiente hasta contar con la PCB ensamblada (Capítulo 5); mientras tanto, la Tabla 4.16 ya incorpora la corriente *teórica* de cada modo, derivada en la Sección 3.6.7 (modos activos) y la Ecuación (3.53) (*deep sleep*) del Capítulo 3, y deja la columna de corriente medida en blanco para completarla en cuanto la PCB esté disponible.

Tabla 4.16
Consumo de corriente en los distintos modos de operación, teórico y medido

Modo de operación	Corriente teórica (mA)	Corriente medida (mA)
Activo, pantalla y WiFi encendidos	∼52	[COMPLETAR, pendiente de PCB]
Activo, solo registro CAN/MicroSD	∼34	[COMPLETAR, pendiente de PCB]
Deep sleep	∼8	[COMPLETAR, pendiente de PCB]

Nota. Corriente teórica calculada a la entrada de 12 V (`OBD2`), sumando el aporte de cada componente a su riel nativo y convirtiendo a través de la eficiencia estimada del TPS54302 (∼80 %), igual que en la Sección 3.6.7 del Capítulo 3. Son estimaciones de orden de magnitud, no mediciones.

4.4. Resultados obtenidos

4.4.1. Resultados de la validación de la comunicación CAN

Los resultados de la Tabla 4.7, obtenidos con las herramientas de la Sección 3.1, muestran una diferencia sistemática y no marginal entre ambos dispositivos en los tres indicadores medidos:

- **Tramas recibidas.** El sistema propio registró 2 936 respuestas válidas en la ventana de 5 minutos frente a 1 712 del ELM327, un 71,5 %más de tramas procesadas en el mismo intervalo.
- **Tasa de error.** El sistema propio no presentó ninguna trama con error de bus (0 de 2 936), mientras que el ELM327 registró 34 respuestas de error o *NO DATA* sobre 1 746 intentos clasificados, una tasa de error de $\approx 1{,}9\,\%$.
- **Latencia.** La latencia promedio del sistema propio (2,2 ms) fue del orden de 100 veces menor que la del ELM327 (231,4 ms).

Esta diferencia es atribuible a la arquitectura de cada dispositivo y no a una falla del ELM327: el sistema propio accede al controlador TWAI del ESP32-S3 de forma nativa, con la trama de respuesta disponible en la propia pila de firmware apenas el hardware la recibe del bus. El ELM327, en cambio, resuelve cada solicitud a través de una capa adicional de comandos AT en texto ASCII, que el propio adaptador debe traducir hacia y desde tramas CAN, y transporta el resultado por un enlace Bluetooth serie. Son dos fuentes adicionales de retardo y de posibles pérdidas de trama, ausentes en una implementación con acceso directo al controlador CAN.

Para la finalidad específica de este proyecto (adquirir parámetros del motor a alta frecuencia y con baja latencia para integrar el consumo instantáneo), el sistema desarrollado demuestra un desempeño de comunicación superior al de un adaptador comercial genérico como el ELM327, no en la precisión del cálculo de consumo en sí (que se evalúa por separado, contra el aforo real, en la Sección 4.5), sino específicamente en la capa de adquisición de datos de la que depende esa precisión: una mayor tasa de muestreo efectiva y una latencia baja y estable reducen el error de integración temporal del consumo acumulado, y una tasa de error nula evita que datos corruptos o ausentes se propaguen al cálculo de `fuel_calc.cpp`.

4.4.2. Resultados de precisión del modelo Speed-Density

La prueba estática (Sección 4.3.3) ya arrojó resultados concretos: antes de calibrar, el modelo subestimaba el caudal de aire en ralentí y 1500 rpm ($-23{,}7\,\%$ y $-9{,}8\,\%$) y lo sobrestimaba a 2500 y 3500 rpm ($+4{,}7\,\%$ y $+18{,}7\,\%$); tras calibrar la tabla VE con esos mismos datos, el error residual bajó a un rango de $-0{,}02\,\%$ a $+1{,}03\,\%$ (Figura 4.12, Capítulo 3), una validación que, como se explicó ahí, es circular por construcción y no basta por sí sola. La Tabla 4.17 cierra ese punto con la prueba dinámica (Sección 4.3.3): métricas calculadas sobre una ruta real que nunca participó en la calibración.

Tabla 4.17

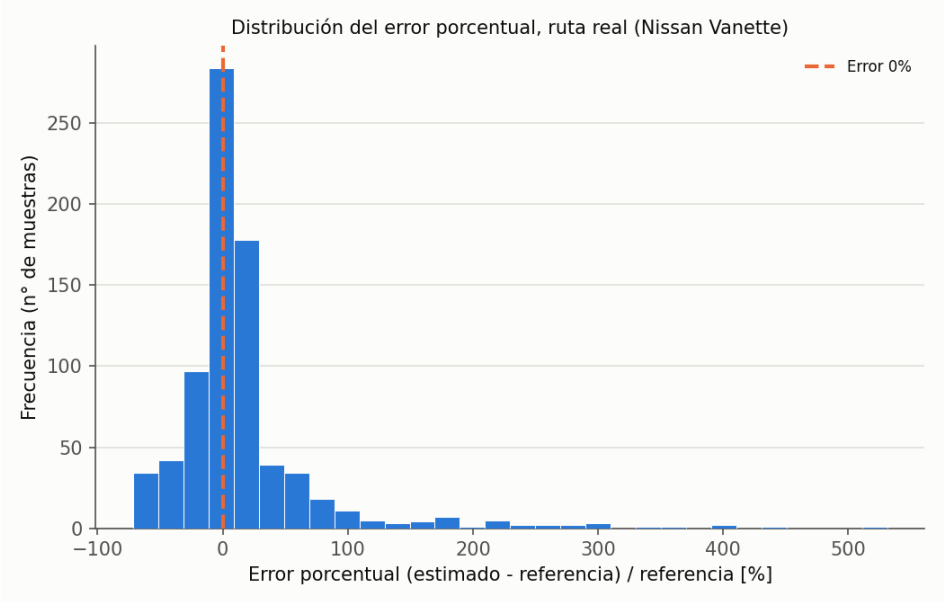
Métricas de validación dinámica del modelo Speed-Density (ruta real, Nissan Vanette)

n muestras	RMSE (g/s)	MAE (g/s)	MAPE (%)	Pearson r
777	2,159	1,343	32,24	0,789

Nota. Calculado por `speed_density_drive_validation.py` sobre el registro completo de `speed_density_drive_log.csv` (777 muestras válidas), sin ningún dato compartido con la calibración de la Sección 4.3.3. $R^2 = 0{,}622$.

Figura 4.15

Distribución del error porcentual del modelo Speed-Density, ruta real (Nissan Vanette)



Nota. Distribución del error porcentual muestra a muestra durante la ruta real; complementa la dispersión estimado-vs-referencia de la Figura 4.13. Elaboración propia.

La prueba dinámica confirma lo que la validación circular de la Figura 4.12 no podía garantizar: sobre 777 muestras de una ruta real, ajenas a la calibración, el modelo mantiene una correlación fuerte con la referencia ($r = 0{,}789$, $R^2 = 0{,}622$) y un error absoluto moderado (RMSE 2,16 g/s, MAE 1,34 g/s sobre un rango de referencia de 2 a 16 g/s), pero el MAPE (32,2 %) es considerablemente mayor que el ±1 % observado en la calibración estática, la brecha esperable entre calibrar y generalizar. La Figura 4.13 muestra por qué: la mayoría de los puntos se agrupan cerca de la recta 1:1, pero hay dispersión notable a caudal de referencia bajo (cerca de 2 g/s, correspondiente a ralentí y desaceleración), donde el estimado se dispara hasta 14 g/s en algunas muestras. El histograma de la Figura 4.15 confirma el mismo patrón: el grueso de las muestras cae cerca de 0 % de error, con una cola larga de errores porcentuales muy grandes que solo puede venir de esas condiciones de flujo bajo, donde dividir entre una referencia pequeña amplifica cualquier discrepancia absoluta modesta. Dos causas probables, no

excluyentes entre sí: (1) el modelo asume un régimen cuasiestático (Sección 3.6.5 del Capítulo 3), hipótesis que se degrada durante transitorios de aceleración/desaceleración reales, ausentes en la prueba estática por diseño; y (2) las cuatro lecturas de PID (RPM, MAP, IAT, MAF) se consultan de forma secuencial, no simultánea, así que en un transitorio rápido cada una corresponde a un instante ligeramente distinto. De ahí que el MAPE agregado deba leerse con cautela: está dominado por unas pocas muestras de bajo caudal, mientras que RMSE y MAE, menos sensibles a denominadores pequeños, son el indicador más representativo del error típico en conducción real.

Tabla 4.18
Validación del modelo Speed-Density en el Changan Honor

Vehículo	Validación disponible
Changan Honor	No soporta el PID 0x10 (MAF), por lo que no existe una referencia directa de caudal de aire contra la cual calcular RMSE/MAE/MAPE, ni en la prueba estática ni en la dinámica (Sección 3.6.5 del Capítulo 3, "Caso sin MAF"). Su exactitud se evalúa en cambio de forma indirecta, a nivel de consumo acumulado, en la Tabla 4.20 (Sección 4.4.5) frente al aforo real de tanque lleno.

Nota. A diferencia del Vanette, aquí no aplica ninguna de las métricas punto a punto de la Tabla 4.17 por ausencia de sensor de referencia, no por falta de la prueba.

4.4.3. Resultados de la validación del sensor MAP frente a la altitud

Los resultados de la Tabla 4.11 son favorables en ambos vehículos, pero no del mismo tipo de evidencia. El **Changan Honor** sí soporta el PID 0x33, así que su fila compara dos sensores reales de la propia ECU entre sí (MAP contra presión barométrica real) y da una diferencia de 0,0 % (63,0 frente a 63,0 kPa): una confirmación directa de que el sensor MAP reporta la presión atmosférica real de La Paz. El **Nissan Vanette**, en cambio, no soporta el PID 0x33, así que su `baro_kpa` cae al respaldo del modelo ISA evaluado en `SITE_ALTITUDE_M` (Subsección 3.4.3 del Capítulo 3); su diferencia de $-2{,}5\,\%$ (63,0 frente a 64,6 kPa) es entonces una comparación del MAP contra un *modelo* de la atmósfera estándar, no contra un segundo sensor real: una validación más débil, aunque igualmente favorable. El valor ISA de 64,6 kPa es además consistente con la predicción teórica ya calculada en el Capítulo 3 ($P(3600) \approx 64{,}9$ kPa, Ecuación (3.17)), la pequeña diferencia atribuible a la altitud exacta configurada (3640 frente a 3600 m).

En ambos casos la diferencia es pequeña (menos de 2 kPa). Esto descarta un sesgo de calibración de fábrica en el sensor MAP de cualquiera de los dos vehículos y confirma que el método speed-density parte, en ambos, de una lectura de presión correctamente calibrada para la altitud de operación, sin necesitar ningún factor de corrección adicional, como se justificó en el Capítulo 3.

4.4.4. Resultados de las pruebas del panel web (WebSocket/HMI)

Los resultados de las Tablas 4.13–4.15 (Changan Honor) confirman que el panel es apto para monitoreo en tiempo real: el intervalo de actualización promedió 500,0 ms con una desviación de solo 10,0 ms frente al periodo nominal configurado (`WS_PUSH_PERIOD_MS`), sin ninguna reconexión en el registro de 8,1 min, y sin pérdida de enlace hasta 20 m de distancia con paredes de por medio, más que suficiente para el uso previsto, con el dispositivo dentro del mismo vehículo. Estos resultados son evidencia objetiva de la confiabilidad del panel y complementan, desde la capa de comunicación, la percepción subjetiva relevada en la encuesta de usabilidad de la Sección 4.5.2.

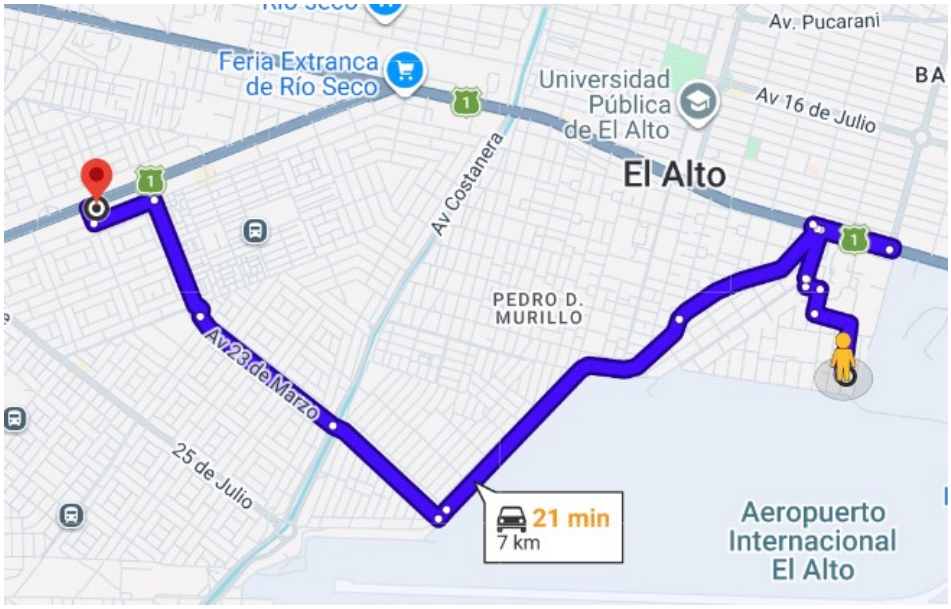
4.4.5. Comparación de consumo estimado vs. consumo real

Esta sección constituye el resultado central de la validación, ya que contrasta tres cantidades sobre nueve tramos de referencia (el corto y el medio, cada uno con 2 repeticiones completas: ida y vuelta por repetición, 4 tramos por categoría; y el largo una sola vez, sin ida/vuelta, Sección 4.3.5): el consumo estimado por el sistema propio (medido de forma continua durante cada tramo), el consumo estimado por el adaptador ELM327 junto con la aplicación comercial (medido en una sesión separada e inmediatamente posterior sobre la misma ruta, por la limitación de conexión simultánea explicada en la Sección 4.2.2), y el consumo real determinado por el método de aforo de tanque lleno a tanque lleno.

Repetir los trayectos corto y medio, y registrar ida y vuelta por separado en vez de como un solo tramo de ida-y-vuelta, no es solo para acumular más distancia: de ahí sale la **repetibilidad** del sistema propio (coeficiente de variación entre pasadas del mismo trayecto y del mismo sentido) de la Sección 4.5, que de otro modo no tendría con qué calcularse; separar ida de vuelta además evita mezclar dos perfiles de tráfico/pendiente potencialmente distintos bajo un mismo número. El aforo real solo existe a nivel del **total combinado** de los nueve tramos (un único llenado de referencia cubre a todos, no se repitió por tramo, Sección 4.3.5), así que la fila de "Total" es la única que se puede comparar directamente contra el aforo real; las filas individuales sirven para comparar el sistema propio contra el ELM327 tramo por tramo. Las Tablas 4.19 y 4.20 (una por vehículo) y la Figura 4.19 presentan estos valores en forma descriptiva; el análisis estadístico formal (RMSE, MAPE, concordancia de Bland-Altman) se desarrolla por separado en la Sección 4.5. Al interpretar la columna del ELM327 conviene tener presente que su comparación contra el sistema propio está sujeta a la variabilidad adicional de tratarse de una pasada distinta del mismo trayecto, y no de la misma sesión de manejo.

Figura 4.16

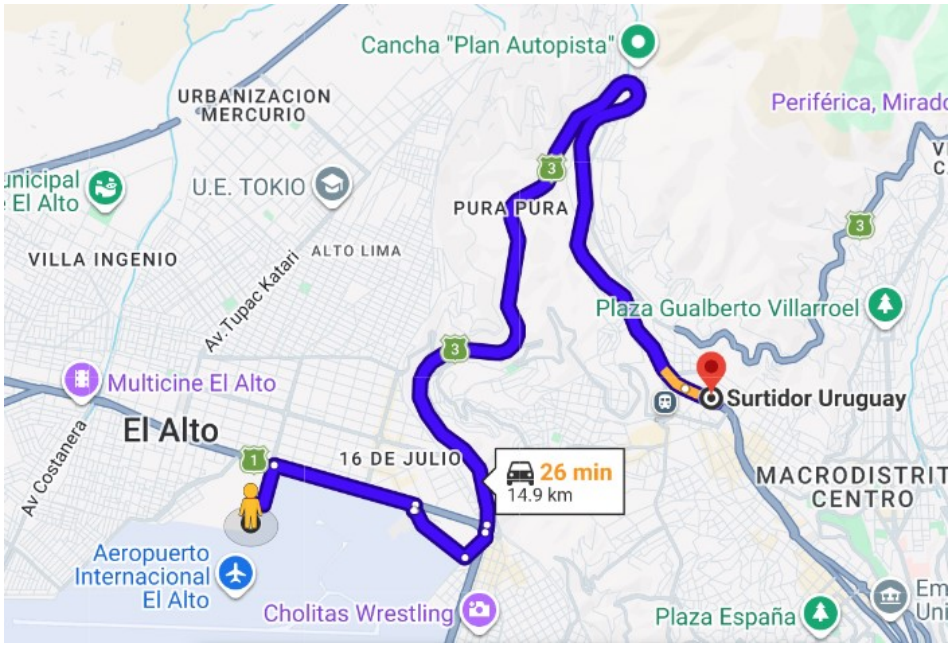
Ejemplo del trayecto corto



Nota. Recorrido de referencia del trayecto corto, con 2 repeticiones completas (ida y vuelta cada una, Tablas 4.19/4.20); la ruta real puede variar levemente entre repeticiones. Elaboración propia.

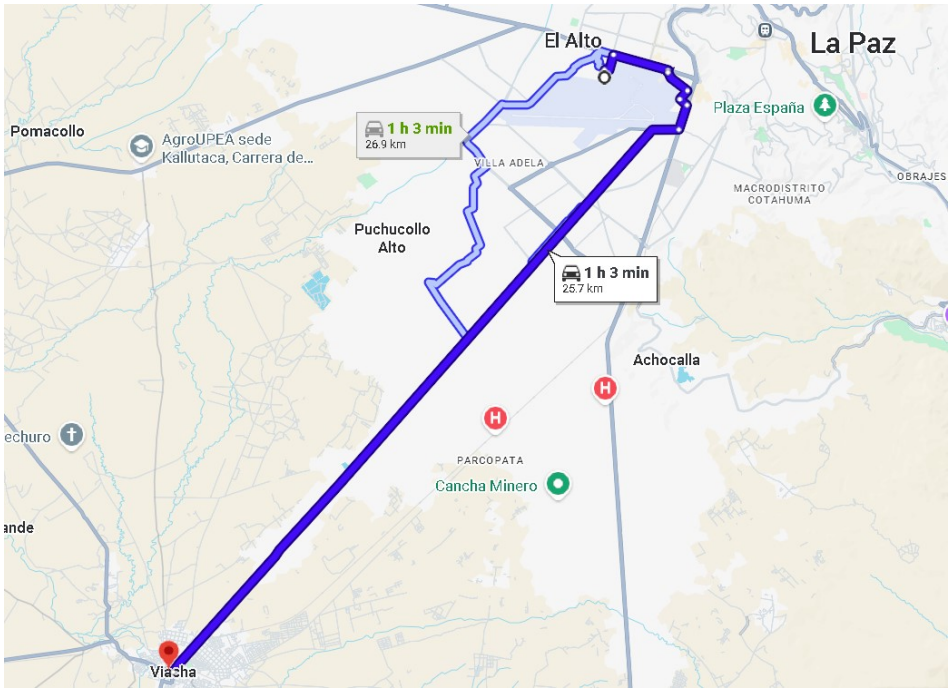
Figura 4.17

Ejemplo del trayecto medio



Nota. Recorrido de referencia del trayecto medio, con 2 repeticiones completas (ida y vuelta cada una). Elaboración propia.

Figura 4.18
Trayecto del viaje largo



Nota. Recorrido único (sin repetición) del trayecto largo. Elaboración propia.

Las tres rutas de la Figura 4.16–4.18 se recorrieron con **ambos vehículos** por separado, cada uno con su propio par de llenados de referencia; la Tabla 4.19 y la Tabla 4.20 presentan los resultados de cada uno.

Tabla 4.19

Comparación de consumo de combustible entre el sistema propuesto, la referencia comercial y el consumo real, por trayecto (Nissan Vanette)

Trayecto	Distancia (km)	Sistema propuesto (L)	ELM327 + app (L)[a]	Real, tanque lleno (L)
Viaje corto 1 (ida)	5,72	0,52	0,50	—
Viaje corto 1 (vuelta)	5,53	0,46	0,49	—
Viaje corto 2 (ida)	10,56	1,12	1,15	—
Viaje corto 2 (vuelta)	10,49	1,24	1,28	—
Viaje medio 1 (ida)	14,31	1,23	1,22	—
Viaje medio 1 (vuelta)	15,10	1,57	1,61	—
Viaje medio 2 (ida)	18,01	1,66	1,68	—
Viaje medio 2 (vuelta)	18,24	1,86	1,89	—
Viaje largo	26,3	2,43	2,44	—
Total (suma de los 9)	**124,26**	**12,09**	**12,26**	**12,96**

Nota. Prototipo frente a herramienta comercial y reabastecimiento real. [a]Sesión separada, misma ruta (Sección 4.2.2). Aforo real solo en el total (Sección 4.3.5). Viajes medio y largo: litros = gasto en Bs / 6,96 Bs/L (`FUEL_PRICE_BS_PER_L`).

Tabla 4.20

Comparación de consumo de combustible entre el sistema propuesto, la referencia comercial y el consumo real, por trayecto (Changan Honor)

Trayecto	Distancia (km)	Sistema propuesto (L)	ELM327 + app (L)[a]	Real, tanque lleno (L)
Viaje corto 1 (ida)	6,28	0,52	0,53	—
Viaje corto 1 (vuelta)	6,14	0,58	0,57	—
Viaje corto 2 (ida)	6,74	0,60	0,56	—
Viaje corto 2 (vuelta)	4,02	0,40	0,42	—
Viaje medio 1 (ida)	14,25	1,00	0,99	—
Viaje medio 1 (vuelta)	15,04	1,39	1,31	—
Viaje medio 2 (ida)	18,01	1,30	1,33	—
Viaje medio 2 (vuelta)	17,78	1,71	1,68	—
Viaje largo	25,99	1,839	1,825	—
Total (suma de los 9)	**114,25**	**9,34**	**9,21**	**9,07**

Nota. Igual estructura que la Tabla 4.19. Sin PID 0x10, "sistema propuesto" corre en Speed-Density puro; esta tabla es la validación aguas abajo de la Tabla 4.18. [a]Ídem nota anterior.

Figura 4.19

Comparación gráfica del consumo de combustible por trayecto y vehículo según los tres métodos evaluados

Nota. Comparativa del volumen de combustible estimado frente a las lecturas obtenidas por cada método, tramo por tramo, para cada vehículo. Elaboración propia.

Con los nueve tramos completos en ambos vehículos, el acuerdo entre el sistema propio y el ELM327 + Car Scanner vuelve a ser bueno en el total combinado ($-1{,}4\,\%$ en el Vanette: 12,090 frente a 12,260 L en 124,26 km; y $+1{,}4\,\%$ en el Changan: 9,338 frente a 9,212 L en 114,25 km), aunque, a diferencia de la comparación con solo tres trayectos, ahora el error por tramo individual oscila en ambos sentidos y con más dispersión: de $-6{,}1\,\%$ a $+4{,}0\,\%$ en el Vanette, y de $-4{,}8\,\%$ a $+7{,}1\,\%$ en el Changan, sin que el signo se mantenga constante ni siquiera entre ida y vuelta del mismo trayecto. Esto es consistente con lo ya discutido en la Figura 4.20: el error por tramo depende de la mezcla de regímenes de motor de ese tramo en particular, no es una propiedad fija del "trayecto corto" o "trayecto medio" como categoría.

Al tener ahora dos repeticiones de cada categoría, se puede estimar por primera

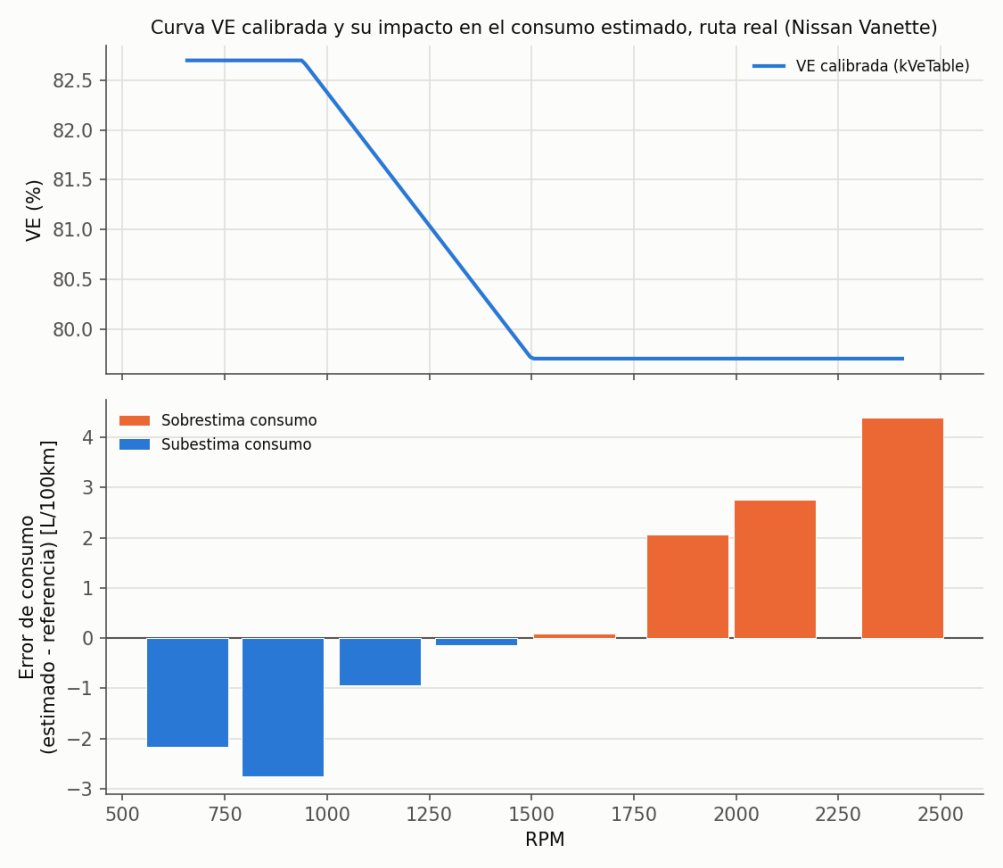
vez la **repetibilidad** del sistema propio (Sección 4.5): el coeficiente de variación del consumo normalizado (L/100 km) entre las 4 mediciones de cada categoría es de 13,6 % (corto) y 7,6 % (medio) en el Vanette, y de 6,9 % (corto) y 14,1 % (medio) en el Changan. Esta variabilidad, de un solo dígito alto a poco más de 13 %, es del orden de magnitud esperable para conducción real en tráfico urbano de La Paz, donde el mismo trayecto nominal rara vez se recorre dos veces en condiciones idénticas (semáforos, tráfico, pendiente efectiva según el sentido); no debe leerse como ruido propio del sistema de medición, ya que la Tabla 4.17 ya mostró que el modelo, sobre las mismas condiciones de manejo, correlaciona fuertemente con el MAF real ($r = 0{,}789$).

De dónde viene la diferencia: impacto de la curva VE en el consumo

Las Tablas 4.9 y 4.17 ya mostraron que la tabla VE calibrada no es igual de precisa en todo el rango de régimen; lo que no muestran es cuánto vale eso en litros. La Figura 4.20, generada por `CODIGOS/ve_consumo_impacto.py` sobre el mismo registro de la prueba dinámica (Sección 4.3.3), conecta ambas cosas: agrupa las muestras por bin de RPM y, en cada uno, calcula tanto el consumo instantáneo que resulta de la VE calibrada como el que resultaría del MAF real, expresando la diferencia directamente en L/100km en vez de en g/s de aire.

Figura 4.20

Curva VE calibrada y su impacto en el consumo estimado, por bin de RPM (ruta real, Nissan Vanette)



Nota. Panel superior: curva VE calibrada (`kVeTable`) en el rango de RPM cubierto por la ruta. Panel inferior: error de consumo promedio por bin de RPM (estimado − referencia), con barras coloreadas por signo. Ambos paneles comparten el eje de RPM para leer directamente en qué régimen se origina cada tramo de la diferencia. Elaboración propia.

El patrón es claro y consistente con la Figura 4.15: por debajo de ∼1500 rpm el sistema **subestima** el consumo (hasta −2,7 L/100km en el bin de ∼890 rpm), y por encima de ∼1600 rpm lo **sobrestima**, con una brecha creciente que llega a +4,4 L/100km cerca de 2400 rpm. Esto es consecuencia directa de la forma de la tabla VE calibrada (Tabla 4.10, Capítulo 3): con solo cuatro puntos calibrados contra régimen sostenido y sin datos por encima de 3500 rpm, la curva no captura del todo la variación real de la eficiencia volumétrica del motor durante los transitorios de una ruta: la misma limitación ya discutida en la Sección 4.3.3. En términos prácticos, esto explica por qué el error total de un trayecto del Vanette (Tabla 4.19) no es simplemente la suma de errores aleatorios: depende de qué proporción del viaje se pasó en cada rango de régimen, de modo que una ruta con más tramos por encima de 1600 rpm tenderá a mostrar una sobrestimación neta mayor que una de manejo predominantemente urbano a bajas revoluciones.

4.4.6. Desempeño del sistema en condiciones reales de conducción

Con los nueve tramos ya medidos en ambos vehículos (Tablas 4.19 y 4.20), el sistema propio se mantiene cerca del ELM327 + Car Scanner en el total combinado: $-1{,}4\,\%$ en el Vanette, $+1{,}4\,\%$ en el Changan (Sección 4.5 desarrolla las métricas formales sobre estos mismos datos). Vale la pena notar la aparente contradicción con el $32\,\%$ de MAPE de la Sección 4.4.2: esas cifras no son incompatibles, sino que miden cosas distintas. El MAPE se calcula muestra a muestra, así que un error grande y de signo variable en cada instante (Figura 4.20) puede compensarse casi por completo al integrarlo en litros a lo largo de todo un trayecto, si el tiempo pasado sobrestimando y subestimando es comparable. En otras palabras: el modelo es menos preciso instante a instante de lo que sugiere el litraje total de un viaje, una distinción relevante para cualquier uso que dependa de la lectura instantánea (por ejemplo, retroalimentar el estilo de conducción en tiempo real) y no solo del acumulado al final del trayecto.

Los tramos cortos fueron viajes de no más de 15 minutos circulando en la ciudad de El Alto: el terreno fue en general plano, con la excepción del tráfico denso en los distintos puntos céntricos recorridos. Los tramos medios cubrieron mayor distancia y se realizaron por dos zonas distintas de la urbe paceña, cada una con su propia condición particular. Uno de ellos aprovechó la autopista La Paz-El Alto, donde se observa una asimetría clara entre ida y vuelta: en la bajada el acelerador no se pisa de forma continua, lo que se refleja en el menor consumo registrado por el sistema (Medio 1 ida, $8{,}60\,\mathrm{L}/100\,\mathrm{km}$, Tabla 4.19); en la subida de vuelta, en cambio, el acelerador se mantiene pisado por más tiempo, y el consumo sube de forma correspondiente ($10{,}40\,\mathrm{L}/100\,\mathrm{km}$). El otro tramo medio se realizó por las avenidas principales de la ciudad de La Paz, esta vez con mayor peso a bordo del vehículo, factor que también influye en el consumo. En todos los tramos cortos y medios la temperatura ambiente fue fría, ya que las pruebas se realizaron de madrugada o por la noche.

En el viaje largo a Viacha, el sistema reaccionó de forma más lenta de lo esperado al inicio del trayecto, pero se estabilizó conforme avanzó el recorrido, resultado consistente con la calibración de la curva de eficiencia volumétrica del Nissan Vanette (Sección 4.3.3). Comparado con el Changan Honor, se esperaba, y se observó en el total combinado (Tablas 4.19/4.20), un consumo menor en el Changan, coherente con su cilindrada más pequeña frente a la del Vanette.

4.5. Validación del sistema

4.5.1. Validación cuantitativa (error porcentual y métricas estadísticas)

Tomando como insumo los mismos trayectos ya presentados de forma descriptiva en las Tablas 4.19 y 4.20 (Sección 4.4), se recomienda calcular, para cada conjunto de datos comparado (sistema propio frente a ELM327 y app, y sistema propio frente al

consumo real de tanque lleno), las siguientes métricas (con las herramientas y fórmulas ya detalladas en la Sección 4.2.4):

- **Error porcentual relativo**, punto a punto, definido como:

$$\text{Error}_{\%} = \frac{|V_{\text{estimado}} - V_{\text{referencia}}|}{V_{\text{referencia}}} \times 100 \tag{4.2}$$

- **Error cuadrático medio (RMSE)** y **error absoluto medio (MAE)**, que expresan la magnitud típica del error en las mismas unidades que la variable medida (litros o litros por cada 100 km), sobre los n tramos comparados:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(V_{\text{estimado},i} - V_{\text{referencia},i}\right)^2} \tag{4.3}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|V_{\text{estimado},i} - V_{\text{referencia},i}\right| \tag{4.4}$$

- **Error porcentual absoluto medio (MAPE)**, útil para comparar el desempeño entre trayectos de distinta longitud; es el promedio del error porcentual relativo (Ecuación 4.2) sobre los n tramos:

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{|V_{\text{estimado},i} - V_{\text{referencia},i}|}{V_{\text{referencia},i}} \times 100 \tag{4.5}$$

- **Coeficiente de correlación de Pearson** y **coeficiente de determinación** (R^2) de una regresión lineal entre los valores estimados y los de referencia, que indican qué tan bien se explica la variación de un método a partir del otro:

$$r = \frac{\sum_{i=1}^{n}\left(V_{\text{estimado},i} - \overline{V}_{\text{estimado}}\right)\left(V_{\text{referencia},i} - \overline{V}_{\text{referencia}}\right)}{\sqrt{\sum_{i=1}^{n}\left(V_{\text{estimado},i} - \overline{V}_{\text{estimado}}\right)^2}\sqrt{\sum_{i=1}^{n}\left(V_{\text{referencia},i} - \overline{V}_{\text{referencia}}\right)^2}} \tag{4.6}$$

  con $R^2 = r^2$ para el caso de una regresión lineal simple entre las dos series.

- **Análisis de concordancia de Bland-Altman**, recomendado específicamente para comparar dos métodos de medición que estiman una misma magnitud, como es el caso del sistema propuesto frente al adaptador ELM327 con su aplicación comercial. Este método grafica la diferencia entre ambas mediciones contra su promedio, y permite estimar el sesgo medio y los límites de concordancia, evidenciando si existe un sesgo sistemático entre los dos sistemas y no solo una correlación:

$$d_i = V_{\text{estimado},i} - V_{\text{referencia},i} \tag{4.7}$$

$$\overline{d} = \frac{1}{n}\sum_{i=1}^{n} d_i \qquad \text{(sesgo medio)} \tag{4.8}$$

$$\text{LoA} = \overline{d} \pm 1{,}96\, s_d \tag{4.9}$$

  donde s_d es la desviación estándar (muestral) de las diferencias d_i.

- **Repetibilidad**, expresada como el coeficiente de variación del sistema propio al repetir el mismo trayecto varias veces bajo condiciones similares; esto separa el error atribuible al método de estimación del error atribuible a la variabilidad de la conducción. Se calcula sobre las $n = 4$ mediciones de consumo normalizado (L/100 km) de una misma categoría de trayecto (ida y vuelta $\times$ 2 repeticiones) y vehículo:

$$\mathrm{CV} = \frac{s}{\overline{x}} \times 100 \tag{4.10}$$

con $\overline{x}$ la media y s la desviación estándar de esas $n = 4$ mediciones.

La Tabla 4.21 propone el formato de resumen de estas métricas.

Tabla 4.21
Resumen de métricas de validación cuantitativa

Métrica	Sistema propio vs. ELM327+app	Sistema propio vs. tanque lleno
RMSE (L)	0,0312	0,64[c]
MAE (L)	0,0262	0,57[c]
MAPE (%)	2,75	4,84[c]
R^2	0,9971	—[c]
Sesgo medio, Bland-Altman (L)	$-0,0024$	—[c]
Límites de concordancia (L)	$[-0,0635,\ 0,0586]$	—[c]
Repetibilidad[b] (CV, %): Vanette corto	13,6	—
Repetibilidad[b] (CV, %): Vanette medio	7,6	—
Repetibilidad[b] (CV, %): Changan corto	6,9	—
Repetibilidad[b] (CV, %): Changan medio	14,1	—

Nota. Estadísticos de precisión, error global y concordancia evaluados para validar el ajuste numérico del modelo desarrollado frente a las referencias. Sistema propio vs. ELM327+app calculado sobre los 18 tramos combinados de ambos vehículos (Tablas 4.19/4.20). [b]La repetibilidad no es una comparación *vs.* una referencia externa, sino la variabilidad del propio sistema (coeficiente de variación del consumo en L/100 km) entre las 4 mediciones de cada categoría de trayecto por vehículo; se deja en la primera columna por consistencia visual con el resto de filas, y el guion en la segunda porque no aplica una comparación contra el tanque lleno. [c]Sistema propio vs. tanque lleno tiene $n = 2$ (el total del Vanette y el del Changan, Tablas 4.19/4.20: único nivel en el que existe aforo real, Sección 4.3.5); RMSE, MAE y MAPE se reportan igual sobre esos dos puntos, pero deben leerse como una referencia gruesa y no como una estimación estadísticamente robusta. R^2 y el análisis de Bland-Altman no se reportan en esta columna porque con $n = 2$ el coeficiente de correlación es trivialmente ± 1 (una recta pasa exactamente por cualquier par de puntos) y no refleja una bondad de ajuste real.

4.5.2. Validación cualitativa (usabilidad de la interfaz y confiabilidad)

Se recomienda complementar la validación numérica con una evaluación cualitativa breve de usabilidad, por ejemplo mediante una escala de Likert de 5 puntos aplicada a

un pequeño grupo de usuarios (conductores de los vehículos de prueba o compañeros del programa), sobre aspectos como la claridad de la información mostrada en la pantalla OLED, la facilidad de acceso al panel web, y la percepción general de confiabilidad del dispositivo durante el uso cotidiano. Esta encuesta mide la *percepción* del usuario; como contraparte objetiva de la afirmación "confiaría en el valor mostrado", la Sección 4.4.4 ya reporta datos medidos de la estabilidad y latencia reales de la conexión con el panel.

Tabla 4.22
Formato de encuesta de usabilidad (escala de Likert, 1 = muy en desacuerdo, 5 = muy de acuerdo)

Afirmación	1	2	3	4	5
La información en la pantalla OLED es clara y legible			✓		
El panel web es fácil de interpretar			✓		
El dispositivo no interfiere con la conducción					✓
Confiaría en el valor de consumo mostrado				✓	

Nota. Cuestionario cualitativo diseñado para medir la percepción de usuario en aspectos de legibilidad, ergonomía e interfaz del prototipo desarrollado.

4.5.3. Discusión de resultados frente a los objetivos del proyecto

Retomando los seis objetivos específicos planteados en el Capítulo 1, a continuación se discute, uno por uno, en qué medida los resultados de este capítulo los satisfacen:

- **Determinar los parámetros del sistema OBD-II disponibles.** Se cumple: la Tabla 4.8 confirma que ambos vehículos exponen RPM (`0x0C`) y velocidad (`0x0D`), pero difieren justo en el parámetro crítico del modelo: el Nissan Vanette reporta MAF (`0x10`) y el Changan Honor no. Esta asimetría, lejos de ser un obstáculo, es la que motivó desarrollar la ruta alternativa sin MAF (VE genérica y STFT/LTFT, Sección 3.6.5 del Capítulo 3) que exige implícitamente el objetivo al hablar de MAF *y/o* MAP.
- **Desarrollar un modelo matemático de estimación basado en la relación aire-combustible.** Cumplido mediante el modelo Speed-Density calibrado en el Capítulo 3 y puesto a prueba en este: la calibración de la tabla VE con datos reales del Vanette (Sección 4.3.3) y su aplicación en la prueba dinámica de ruta real (Tabla 4.17, $r = 0{,}789$) muestran que el modelo captura la tendencia del caudal de aire real, aunque con una dispersión instantánea considerable (MAPE del 32 % punto a punto) que debe leerse como una limitación del modelo en sí, no solo de su calibración.
- **Recopilar datos del motor mediante OBD-II.** Este objetivo se cumple: el firmware adquiere y registra RPM, MAP o MAF, IAT y velocidad en cada ciclo,

con una tasa de integridad de registro en MicroSD del 100 % (0 archivos corruptos en las tres sesiones de la Tabla 4.12, incluida una prueba deliberada de corte abrupto de energía).

- **Diseñar un sistema embebido de adquisición para validar el modelo.** Cumplido también: el prototipo basado en ESP32-S3 (Capítulo 3) fue el instrumento con el que se generaron todos los datos de este capítulo, desde la validación estática (Tabla 4.9) hasta los nueve tramos de ruta real (Tablas 4.19/4.20).
- **Integrar una plataforma telemétrica IoT.** Cumplido de forma parcial. El ESP32-S3 sí actúa como servidor local, transmite la telemetría por WebSocket con latencia estable ($500{,}0 \pm 10{,}0$ ms sobre 945 muestras, Tabla 4.13) y sin reconexiones en la sesión prolongada registrada (Tabla 4.14), y el punto de acceso WiFi se mantuvo conectado hasta los 20 m, la mayor distancia probada (Tabla 4.15), suficiente para el caso de uso dentro de un vehículo. Sin embargo, el objetivo específico planteaba explícitamente el almacenamiento en una *base de datos estructurada*; lo efectivamente implementado es un registro en archivos CSV sobre MicroSD, sin una base de datos relacional o documental de por medio. Esta es una limitación real frente al objetivo original, que queda como trabajo futuro (por ejemplo, una capa de sincronización periódica del CSV hacia una base de datos en el servidor local o en la nube).
- **Evaluar el sistema integral bajo condiciones reales.** Cumplido con matices. El acuerdo global frente al ELM327 + Car Scanner es bueno sobre los nueve tramos ($-1{,}4$ % Vanette, $+1{,}4$ % Changan) y, frente al aforo real de tanque lleno, se mantiene en el mismo orden de magnitud (MAPE $\approx 4{,}8$ %, Sección 4.5); pero dos limitaciones deben señalarse explícitamente. Primero, el tamaño de muestra es reducido —nueve tramos por vehículo y un único par de vehículos—, insuficiente para generalizar sin pruebas adicionales al resto de la flota de transporte público de La Paz. Segundo, los dos vehículos de prueba no se comportaron de forma equivalente frente al sistema: el Changan Honor, al no soportar el PID de MAF, solo pudo validarse de forma indirecta (a nivel de consumo acumulado) y no cuenta con la validación dinámica punto a punto que sí tiene el Vanette (Tabla 4.17). Esta asimetría entre vehículos es en sí misma un hallazgo relevante, y amerita, como trabajo futuro, repetir la prueba dinámica en un vehículo adicional con sensor MAF para confirmar que el patrón de error observado no es específico de un solo motor.

Como cierre metodológico, conviene recordar que el modelo Speed-Density es, en sí mismo, una técnica ampliamente utilizada en unidades de control electrónico comerciales para la estimación del flujo de aire cuando no se dispone de un sensor MAF dedicado, y que su exactitud depende críticamente de una correcta caracterización de la eficiencia volumétrica del motor y de las condiciones ambientales; de ahí la pertinencia de las pruebas de corrección por altitud realizadas en este capítulo.

Capítulo 5

ANÁLISIS DE COSTOS Y PRESUPUESTO

Este capítulo cuantifica el costo de desarrollar, fabricar y validar el sistema descrito en los Capítulos 3 y 4, y evalúa su viabilidad económica frente a alternativas comerciales existentes. El desglose sigue el mismo criterio de agrupación funcional ya establecido en la lista de materiales del Capítulo 3 (Tabla 3.15) y en la tabla de componentes del prototipo del Capítulo 4 (Tabla 4.1). Así, cada partida de costo puede rastrearse hasta el componente físico concreto que la origina, en lugar de quedar como una cifra agregada sin respaldo.

Los montos se expresan en bolivianos (Bs), moneda de referencia ya utilizada en el resto del proyecto de grado (por ejemplo, `FUEL_PRICE_BS_PER_L` en el firmware), con una columna equivalente en dólares (USD) para los componentes que se cotizan internacionalmente (LCSC, JLCPCB, AliExpress). Dado que el tipo de cambio BOB/USD ha tenido variaciones significativas en el mercado boliviano en los últimos años, las conversiones de este capítulo usan el tipo de cambio paralelo vigente al momento de la cotización, 11,60 Bs/USD (agosto de 2026), y no un valor fijo asumido de antemano ni el tipo de cambio oficial, que en este periodo no refleja el costo real de acceso a dólares para la importación de componentes.

5.1. Costos de componentes electrónicos

Esta sección desglosa el costo de los componentes electrónicos en los mismos cuatro bloques funcionales de la Tabla 3.15: procesamiento, alimentación, interfaz CAN y periféricos. Las cantidades de cada fila provienen directamente de esa tabla; aquí solo se añaden el precio unitario y el subtotal por bloque.

5.1.1. Costos del módulo de procesamiento (ESP32-S3)

Tabla 5.1
Costo del módulo de procesamiento

Componente	Cant.	Precio unit. (USD)	Precio unit. (Bs)	Subtotal (Bs)
Módulo ESP32-S3-WROOM-1-N16R8 (DevKitC-1, prototipo)	1	9,48	110	110

Nota. Precio del módulo de desarrollo completo (*DevKitC-1*); la versión final en PCB propia podría usar el módulo suelto a menor costo, a cambio de requerir un programador externo para el primer flasheo.

5.1.2. Costos de la etapa de alimentación (buck y LDOs)

Tabla 5.2
Costo de la etapa de alimentación de dos rieles

Componente	Cant.	Precio unit. (USD)	Precio unit. (Bs)	Subtotal (Bs)
Convertidor buck síncrono TPS54302DDCR	1	0,2538	2,94	2,94
LDO AMS1117-3,3 (riel 1)	1	0,2039	2,37	2,37
LDO AP2112K-3,3TRG1 (riel 2)	1	0,2043	2,37	2,37
Diodo Schottky SS34 (protección polaridad)	1	0,039	0,45	0,45
Fusible PTC reseteable 1812L150/33GR	1	0,0575	0,67	0,67
Inductor de potencia 10 µH	1	1,3769	15,97	15,97
Condensadores y resistencias pasivas (Tabla 3.15)	16	0,7323	8,49	135,91

Nota. Los pasivos se cotizan por cinta/carrete completo, por lo que a cantidad de prototipo su costo unitario es mayor que en producción en volumen; se diluye al escalar (Subsección 5.6.2).

5.1.3. Costos de la interfaz CAN (transceptor y protección)

Tabla 5.3
Costo de la interfaz de bus CAN

Componente	Cant.	Precio unit. (USD)	Precio unit. (Bs)	Subtotal (Bs)
Transceptor CAN SN65HVD230	1	1,5216	17,65	17,65
Resistor control de pendiente (10 kΩ)	1	0,0912	1,06	1,06
Resistor de terminación 120 Ω (*footprint* opcional, no poblado)	0	—	—	0,00
Condensador de desacople	1	0,4296	4,98	4,98

Nota. El resistor de terminación no se puebla por defecto (Tabla 3.15, nota [a]): el dispositivo se conecta en derivación a un bus CAN que el vehículo ya termina, por eso su costo es cero y no una omisión.

5.1.4. Costos de periféricos (MicroSD, OLED, conectores)

Tabla 5.4
Costo de periféricos de almacenamiento, visualización y conexión

Componente	Cant.	Precio unit. (USD)	Precio unit. (Bs)	Subtotal (Bs)
Zócalo MicroSD (*push-push*)	1	0,1318	1,53	1,53
Tarjeta MicroSD (capacidad de referencia)	1	3,2141	37,28	37,28
Pantalla OLED SSD1306 I^2C 0,96	1	2,686	31,16	31,16
Conector OBD-II tipo A con cable	1	0,8	9,28	9,28
LEDs indicadores + resistencias limitadoras	4	0,2120	2,46	9,84
Resistencias *pull-up* SPI/I^2C (Tabla 3.15)	6	0,0460	0,53	3,20

Nota. La capacidad de la tarjeta MicroSD es más una decisión de autonomía de registro que de costo: dado el tamaño de fila estimado en el Capítulo 4 (≈100 bytes/fila a 1 muestra/s), incluso una tarjeta de 8 GB excede ampliamente los requisitos de años de uso continuo, así que pagar por mayor capacidad no tiene sentido.

5.2. Costos de desarrollo y software

El conjunto de herramientas de desarrollo de firmware empleado (Visual Studio Code, la extensión PlatformIO IDE y el *framework* `arduino-esp32`) es de uso libre y sin costo de licenciamiento, criterio de selección ya justificado en la Sección de herramientas del Capítulo 3. El costo real de esta partida no está en licencias, entonces, sino en el tiempo de ingeniería invertido en el diseño de hardware, el firmware y la interfaz web. La tarifa de la Tabla 5.5 se deja deliberadamente en 0 Bs/h: el objetivo de este capítulo es el costo de materiales y fabricación, no el valor de mercado de las horas de un proyecto de grado, así que el total de horas queda documentado como referencia de esfuerzo, no como partida monetaria.

Tabla 5.5
Costo estimado de horas de ingeniería por etapa de desarrollo

Etapa	Horas estimadas	Tarifa (Bs/h)	Subtotal (Bs)
Diseño de esquemático y PCB	8	0	0
Desarrollo de firmware (adquisición, cálculo, almacenamiento)	28	0	0
Desarrollo del panel web (HMI)	28	0	0
Pruebas, calibración y depuración	120	0	0
Total	**184**	—	**0**

Nota. Horas estimadas a partir de la dedicación real del autor (≈4 h/día): 2 días para la PCB, 1 semana para firmware y panel web cada uno, y 1 mes para la calibración.

5.3. Costos de fabricación y ensamblaje

5.3.1. Costos de la placa preliminar (Robokit)

La placa preliminar en Altium Designer descrita en la Sección 4.1.3 (Capítulo 4) es, hasta el momento, la única placa de este proyecto efectivamente fabricada y ensamblada; por eso es la que se usa como **costo real del prototipo** en el presupuesto consolidado de este capítulo (Tabla 5.12). Se encargó localmente a Robokit (La Paz), lo que evita el envío internacional y la aduana que sí aplican a una fabricación en JLCPCB (Subsección 5.3.3).

Tabla 5.6
Costo de fabricación de la placa preliminar en Robokit

Concepto	Cantidad	Subtotal (Bs)
Fabricación de PCB (Robokit, La Paz)	1	150,00
Ensamblaje (soldadura manual, propia)	—	0,00
Total	—	**150,00**

Nota. Cotización y fabricación locales (La Paz), sin envío internacional ni aduana; el ensamblaje manual, hecho por el propio autor, no tiene costo adicional (mismo criterio de tarifa 0 Bs/h de la Tabla 5.5).

La lista de materiales real de esta placa (16 referencias, exportada desde Altium Designer) muestra que su etapa de alimentación es una regulación lineal simple (7805 + transistor de paso TIP42C) y no la conversión conmutada de dos rieles de la Tabla 5.2 —esa etapa es específica de la placa final—; el módulo ESP32-S3, el transceptor CAN, la pantalla OLED y la tarjeta microSD son los mismos ya presupuestados en las Tablas 5.1, 5.3 y 5.4, reutilizados desde la etapa de protoboard y conectados a esta placa mediante los conectores de la Tabla 5.7 en vez de cableado suelto. Los indicadores LED y sus resistencias limitadoras también coinciden con los ya contabilizados en la Tabla 5.4, por lo que no se repiten aquí. Los precios de la Tabla 5.7 se tomaron de LCSC a partir de las referencias de fabricante exportadas desde Altium Designer; las tres partidas sin coincidencia exacta en el catálogo (capacitor de 22 µF, header de 8 pines y conector de 4 pines) se estiman por interpolación directa desde variantes de la misma familia con precio confirmado, no por un valor arbitrario.

Tabla 5.7
Componentes adicionales de la placa preliminar (etapa de alimentación y conectores)

Componente	Cant.	Precio unit. (USD)	Precio unit. (Bs)	Subtotal (Bs)
Regulador lineal 7805 (TO-252)	1	0,0979	1,14	1,14
Transistor de paso TIP42C (TO-220)	1	0,1777	2,06	2,06
Capacitor electrolítico 220 µF	1	0,1400	1,62	1,62
Capacitor electrolítico 22 µF	1	0,1200	1,39	1,39
Capacitor cerámico 100 nF	2	0,0200	0,23	0,46
Resistor THT 47 Ω 1 W	1	0,0112	0,13	0,13
Conector de entrada de alimentación, 4 pines (WJ301V-5.0)	1	0,1700	1,97	1,97
Header 4 pines (pantalla OLED)	1	0,0198	0,23	0,23
Header 6 pines (transceptor SN65HVD230)	1	0,0298	0,35	0,35
Header 8 pines (módulo microSD)	1	0,0400	0,46	0,46
Total	—	—	—	**9,81**

Nota. Precios de referencia LCSC (por ejemplo, `KS104M050C07RR0VH2FP0` para el capacitor de 100 nF), convertidos a bolivianos al tipo de cambio de este capítulo (11,60 Bs/USD).

5.3.2. Costo de la carcasa del prototipo

La carcasa impresa en 3D para la placa preliminar (Capítulo 3, Sección 3.3.6) se costea por peso de material: PLA a 130 Bs el rollo de 1 kg, impresión con impresora propia (sin costo de servicio de impresión).

Tabla 5.8
Costo de material de la carcasa del prototipo

Pieza	Peso (g)	Subtotal (Bs)
Base (impresa)	21	2,73
Tapa (impresa)	16	2,08
Total	**37**	**4,81**

Nota. PLA a 130 Bs/kg (0,13 Bs/g); no incluye costo de mano de obra propia (Tabla 5.5) ni desgaste de la impresora.

5.3.3. Costos de manufactura de la placa final (JLCPCB)

A diferencia de la placa preliminar, la placa final documentada en el Capítulo 3 (diseñada en Altium Designer) todavía no se ha mandado a fabricar; esta subsección presenta su cotización de referencia, relevante si el proyecto avanza hacia esa versión más profesional o hacia una serie piloto para varios vehículos de transporte público. Se cotiza sobre el lote mínimo estándar de JLCPCB para placas de dos capas (Tabla 3.16, Capítulo 3), habitualmente de 5 unidades, que es también el tamaño de lote recomendado para tener margen ante fallas de soldadura o defectos de una unidad individual durante las pruebas iniciales.

Tabla 5.9

Costo de fabricación de PCB en JLCPCB

Concepto	Cantidad	Precio unit. (USD)	Precio unit. (Bs)	Subtotal (Bs)
Fabricación de PCB (lote mínimo, 2 capas, 1 oz)	1	4,00	46,40	46,40
Envío internacional (DHL/courier)	1	38,00	440,80	440,80
Impuestos/aduana de importación (si aplica)	1	-	-	-

Nota. El envío y la aduana suelen pesar más que la fabricación misma en lotes de prototipo pequeños importados a Bolivia; conviene cotizarlos por separado.

5.3.4. Costos de ensamblaje SMD de la placa final

Para la placa final, el ensamblaje de los componentes de montaje superficial puede resolverse por dos vías con una disyuntiva clara de costo contra riesgo: el servicio de ensamblaje (PCBA) del propio fabricante, o el soldado manual con estación de aire caliente/soldadora de precisión. La placa preliminar (Sección 5.3.1), en cambio, ya se ensambló manualmente por el propio autor, sin necesidad de esta disyuntiva.

Tabla 5.10
Comparación de estrategias de ensamblaje SMD

Criterio	Ensamblaje PCBA (JLCPCB)	Soldadura manual
Costo	1 440,60 Bs (124,19 USD; incluye *setup* + componentes + mano de obra)	Solo costo de componentes; tiempo propio no facturado
Riesgo técnico	Bajo: máquina *pick-and-place* coloca el TPS54302DDCR (SOT-23-6) sin riesgo de puente de soldadura	Medio-alto en los pasivos 0603 y el regulador *buck*, componentes de paso fino
Tiempo de entrega	Sujeto al lote de fabricación + envío	Inmediato una vez recibida la PCB desnuda
Recomendado cuando	Se fabrica más de una unidad, o no se dispone de estación de retrabajo SMD	Se ensambla una sola unidad de prototipo y ya se cuenta con el equipo de soldadura

Nota. Comparación de referencia para la futura placa final; la placa preliminar ya construida (Tabla 5.6) usó soldadura manual (columna derecha). Elaboración propia.

5.4. Costos de pruebas y validación

El precio del litro de gasolina (6,96 Bs) es el mismo valor de referencia (`FUEL_PRICE_BS_PER_L`) que usa el firmware para calcular el costo del viaje (Capítulo 3), y los 22,03 L de combustible corresponden a la suma de los totales reales de aforo tanque lleno del Capítulo 4 (Nissan Vanette 12,96 L + Changan Honor 9,07 L, Tablas 4.19/4.20): el consumo efectivamente gastado en los nueve tramos de prueba, no la capacidad de los tanques.

Tabla 5.11
Costos asociados a la fase de pruebas y validación (Capítulo 4)

Concepto	Cantidad	Precio unit. (Bs)	Subtotal (Bs)
Adaptador ELM327 Bluetooth (referencia de validación)	1	85,00	85,00
Suscripción *Car Scanner* Premium (3 meses)	1	22,99	22,99
Combustible para trayectos de prueba tanque a tanque (Sección 4.3.5, Cap. 4)	22,03 L	6,96	153,33
Instrumento de medición de corriente (tester)	1	55,00	55,00
Total	—	—	**316,32**

Nota. Elaboración propia.

5.5. Presupuesto total del proyecto

La Tabla 5.12 consolida los subtotales de las secciones anteriores, usando la placa preliminar de Robokit (Sección 5.3.1) como fabricación real del prototipo. El presupuesto se presenta también en su forma de *costo marginal por unidad adicional* (Tabla 5.12, última fila), escalando esta vez hacia la placa final en Altium Designer y su fabricación en JLCPCB (Sección 5.3.3), relevante si el proyecto avanza hacia esa versión más profesional o hacia una pequeña serie de dispositivos para varios vehículos de transporte público, escenario directamente ligado a la motivación planteada en el Capítulo 1.

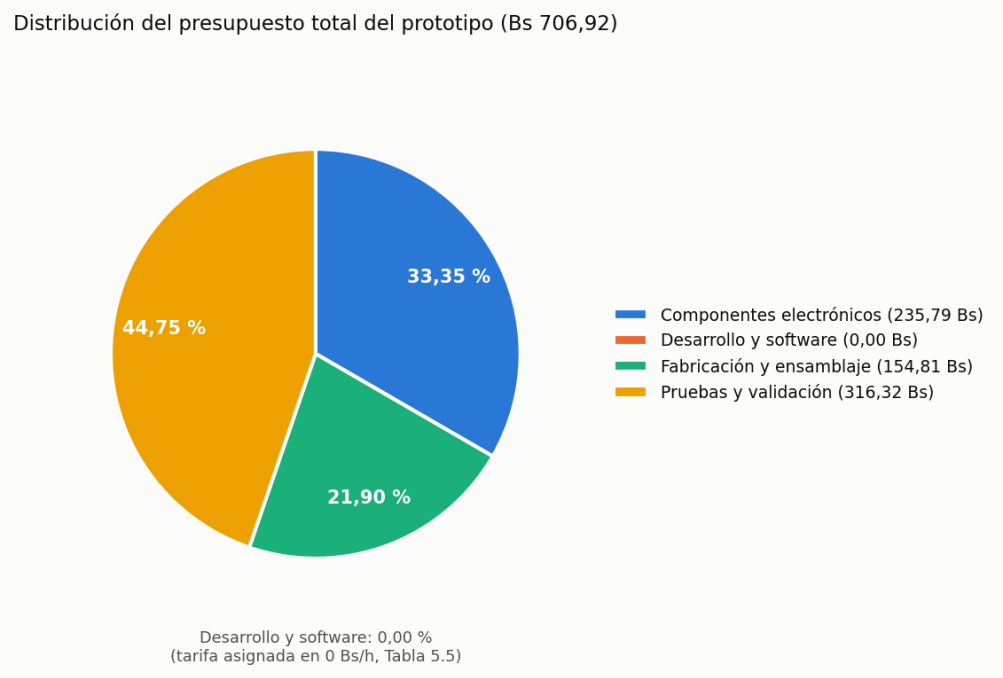
Tabla 5.12
Presupuesto total del proyecto, por categoría

Categoría	Subtotal (Bs)	% del total
Componentes electrónicos (Tablas 5.1, 5.3, 5.4 y 5.7)	235,79	33,35 %
Desarrollo y software (Tabla 5.5)	0,00	0,00 %
Fabricación y ensamblaje, placa preliminar (Tablas 5.6 y 5.8)	154,81	21,90 %
Pruebas y validación (Tabla 5.11)	316,32	44,75 %
Total del prototipo	**706,92**	**100 %**
Costo marginal estimado por unidad adicional, placa final[a]	772,22	—

Nota. [a]Estimado para la placa final (JLCPCB), no para el total del prototipo (placa preliminar de Robokit); detalle del cálculo a continuación.

El costo marginal excluye las horas de diseño de hardware/firmware (Tabla 5.5), un costo de desarrollo único (*non-recurring engineering*) que no se repite por unidad fabricada, e incluye los componentes (386,66 Bs) más la fabricación de PCB y el ensamblaje PCBA a prorrata del lote mínimo de 5 unidades de JLCPCB (Tablas 5.9/5.10: $(487{,}20 + 1\,440{,}60)/5 = 385{,}56$ Bs), es decir $386{,}66 + 385{,}56 = 772{,}22$ Bs por unidad adicional dentro del mismo lote.

Figura 5.1
Distribución porcentual del presupuesto total del proyecto por categoría



Nota. Gráfico de pastel: las cuatro categorías son mutuamente excluyentes y suman el 100 % del presupuesto, lo que comunica de forma más directa que una tabla de números. Elaboración propia a partir de `CODIGOS/presupuesto_pie.py` (Calle, 2026).

5.5.1. Precio de venta estimado (utilidad e IVA)

Los costos anteriores son costos directos, sin margen de ganancia ni impuestos. La Tabla 5.13 aplica una utilidad del 20 %, valor de referencia habitual en la formulación de proyectos de la carrera, y luego el Impuesto al Valor Agregado (IVA) vigente en Bolivia (13 %) sobre el precio ya con utilidad, sobre tres bases de costo distintas. La primera y la tercera parten del prototipo tal como se construyó (706,92 Bs, fila "Total del prototipo" de la Tabla 5.12); la tercera, además, resta el combustible de las pruebas tanque a tanque (153,33 Bs, Tabla 5.11), bajo el supuesto de que esas pruebas ya se realizaron y ese gasto no se traslada a un comprador de esta misma unidad. La segunda es el costo marginal por unidad adicional si se escalara a la placa final (772,22 Bs, fila "Costo marginal estimado por unidad adicional" de la misma tabla), que ya excluye por completo la Tabla 5.11 (no solo el combustible, también ELM327 y el tester), al no ser un gasto que se repita al producir una unidad adicional.

Tabla 5.13
Precio de venta estimado, prototipo actual vs. unidad adicional

Concepto	Este prototipo (Bs)	Unidad adicional, placa final (Bs)	Este prototipo, sin combustible de prueba[a] (Bs)
Costo directo (Tabla 5.12)	706,92	772,22	553,59
Utilidad (20 %)	141,38	154,44	110,72
Subtotal (costo + utilidad)	848,30	926,66	664,31
IVA (13 %)	110,28	120,47	86,36
Precio de venta estimado	**958,58**	**1 047,13**	**750,67**

Nota. Utilidad y IVA aplicados de forma secuencial (utilidad sobre el costo directo, IVA sobre el precio ya con utilidad), no simultánea sobre el mismo costo base. El 20 % de utilidad es un valor de referencia, no una cotización de mercado; puede ajustarse sin afectar el resto del capítulo. [a]706,92 Bs − 153,33 Bs de combustible (Tabla 5.11) = 553,59 Bs; a diferencia de la columna del centro, aquí sí se conservan el ELM327 y el tester dentro del costo, ya que son instrumentos físicos y no un insumo consumido.

5.6. Análisis de viabilidad económica

5.6.1. Comparación con soluciones comerciales existentes

El sistema propuesto se contrasta contra dos categorías de alternativas ya mencionadas en los Antecedentes del Capítulo 1: los adaptadores OBD-II genéricos de bajo costo combinados con una aplicación móvil (por ejemplo, ELM327 + *Car Scanner*, ya usado como referencia de validación en el Capítulo 4), y las plataformas de telemetría de flota comerciales orientadas a empresas de transporte.

Tabla 5.14
Comparación funcional y de costo frente a alternativas comerciales

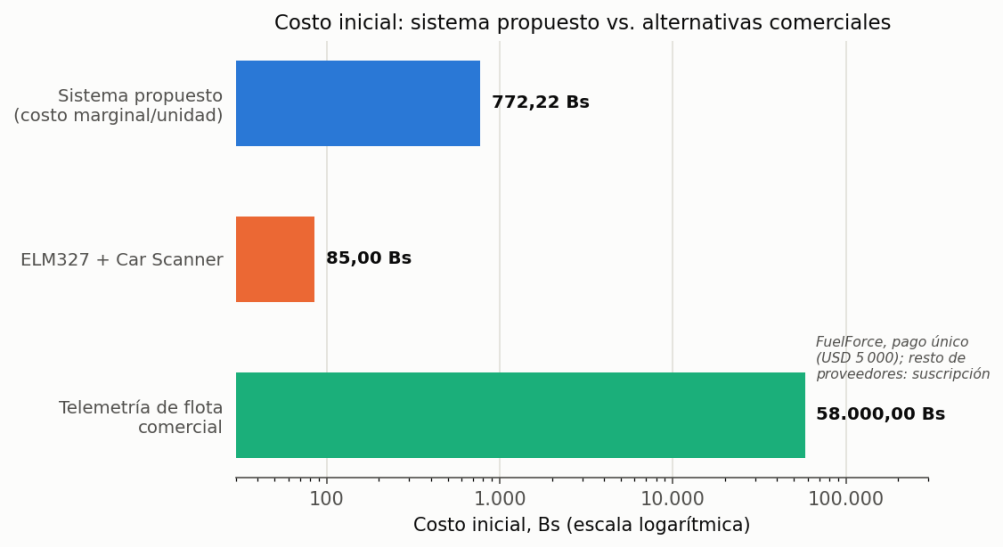
Criterio	Sistema propuesto	ELM327 + app comercial	Telemetría de flota comercial
Costo inicial	772,22 Bs[a] (Tabla 5.12)	85,00 Bs (Tabla 5.11)	Desde 58 000 Bs[b] (USD 5 000, *FuelForce*, pago único); el resto de proveedores cobra el hardware por separado, sin monto público
Costo recurrente	Ninguno (sin suscripción)	Suscripción *Car Scanner* Premium: 22,99 Bs cada 3 meses	USD 4 a 100 por vehículo/mes según proveedor[b] (≈46 a 1 160 Bs/mes)
Registro local sin conexión a internet	Sí (microSD)	Depende de la app	Generalmente requiere conectividad
Panel propio, sin dependencia de un tercero	Sí (SoftAP local)	No (depende del proveedor de la app)	No (plataforma del proveedor)
Código y diseño abiertos, personalizables	Sí	No	No
Corrección/ calibración específica del vehículo	Sí (`FUEL_CALIBRATION_FACTOR`, tabla VE)	Limitada o inexistente	Variable según proveedor

Nota. Elaboración propia. [a]Costo marginal escalando a la placa final (Tabla 5.12), no el total del prototipo, que aún incluye instrumentos de validación ajenos a una unidad desplegable. [b]Precios internacionales de referencia; fuentes detalladas a continuación.

Los precios de telemetría de flota no tienen cotización pública para el mercado boliviano; se tomaron de *Fleetio*, "6 Best Fuel Monitoring Systems: 2025 Comparison Guide" (`fleetio.com/blog/fuel-monitoring-systems`, consultado en agosto de 2026): Fleetio USD 4–10/vehículo/mes; Motive desde USD 25/vehículo/mes más costo de dispositivos ELD; Samsara USD 27–33/vehículo/mes (software; hardware aparte); Geotab USD 35–100/vehículo/mes (software; hardware aparte); y FuelForce desde USD 5 000 en pago único por funcionalidad, en vez de suscripción. La conversión a bolivianos usa el mismo tipo de cambio de este capítulo (11,60 Bs/USD).

Figura 5.2

Comparación de costo inicial entre el sistema propuesto y alternativas comerciales



Nota. Barras horizontales: los nombres de las tres alternativas se leen mejor como etiquetas de eje. Escala logarítmica en el costo porque los valores abarcan casi tres órdenes de magnitud (85 a 58 000 Bs), con el valor exacto en cada barra. El dato de telemetría de flota es de un único proveedor (*FuelForce*, pago único; Tabla 5.14), no un promedio del rubro. Elaboración propia.

5.6.2. Análisis costo-beneficio

El beneficio económico del sistema no proviene de evitar un gasto de compra (como en la comparación anterior), sino de la reducción de consumo de combustible que su uso puede inducir en el conductor al hacer visible, en tiempo real, un comportamiento de manejo ineficiente. El mecanismo exacto por el que un sistema de monitoreo reduce el consumo es de naturaleza conductual (retroalimentación al conductor) y no una modificación mecánica del vehículo, así que su magnitud solo puede estimarse una vez completada la comparación de consumo del Capítulo 4 (Tablas 4.19 y 4.20) entre trayectos con y sin atención activa al panel del dispositivo.

El periodo de recuperación de la inversión (*payback period*) se estima como:

$$T_{recuperación}\,[\text{meses}] = \frac{\text{Costo total del prototipo (Tabla 5.12)}}{\text{Ahorro mensual estimado}\,[\text{Bs/mes}]} \tag{5.1}$$

donde el ahorro mensual se obtiene de multiplicar la reducción porcentual de consumo por retroalimentación al conductor (valor de literatura, ya que no se midió en este proyecto) por el gasto mensual de combustible típico del vehículo evaluado, a su vez estimado a partir del kilometraje mensual habitual (también de literatura) y el consumo promedio real observado en el Capítulo 4 (totales de aforo de las Tablas 4.19/4.20).

El kilometraje mensual no se midió para estos vehículos: se usa un valor representativo (200 km/día × 30 días) dentro del rango de 150 a 300 km/día que reporta

la literatura de operación de buses urbanos en servicio de todo el día (Public-Private Infrastructure Advisory Facility (PPIAF) / World Bank, 2026), igual para ambos vehículos al no existir un dato diferenciado por unidad. El consumo promedio sí es un dato real: el total de aforo de tanque lleno entre la distancia total recorrida en los nueve tramos de prueba del Capítulo 4. La reducción de consumo atribuible al monitoreo tampoco se midió en este proyecto —requeriría un diseño experimental adicional, por ejemplo contrastar el mismo trayecto con el panel visible frente al panel oculto para el conductor—; se usa en su lugar el efecto promedio reportado por un metaanálisis de 17 estudios sobre sistemas de retroalimentación *eco-driving* a bordo, 6,6 % de mejora en economía de combustible (Sanguinetti et al., 2020).

Tabla 5.15

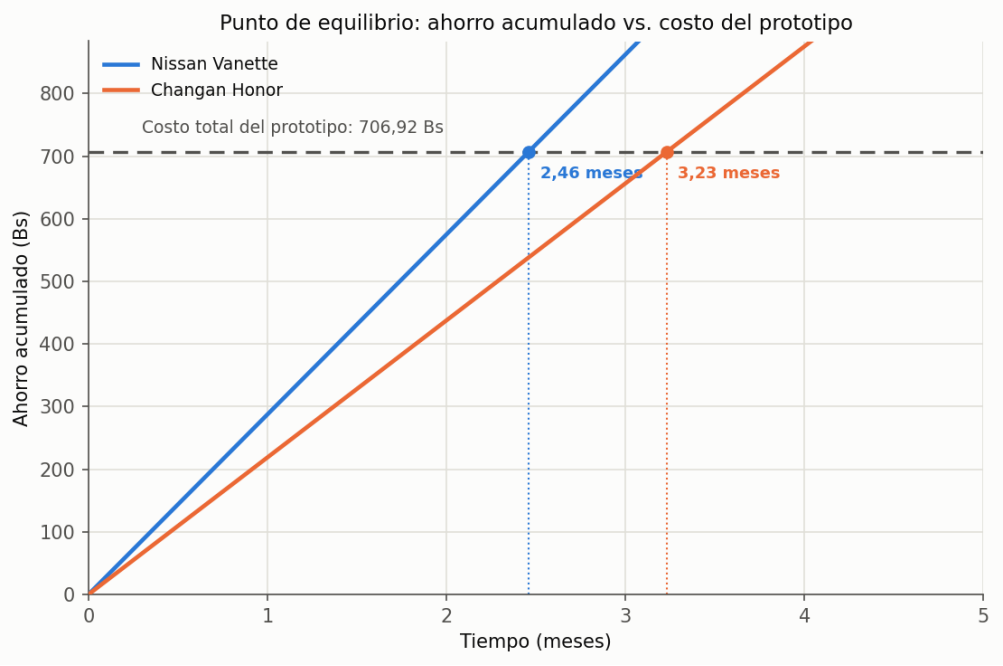
Estimación del periodo de recuperación de la inversión

Variable	Changan Honor	Nissan Vanette
Kilometraje mensual estimado (km)[a]	6 000	6 000
Consumo promedio (L/100 km, Cap. 4)[b]	7,94	10,43
Gasto mensual de combustible (Bs)	3 315,74	4 355,57
Reducción de consumo atribuible al monitoreo (%)[c]	6,6	6,6
Ahorro mensual estimado (Bs/mes)	218,84	287,47
Periodo de recuperación (meses)	3,23	2,46

Nota. [a]Valor de literatura, no medido. [b]Dato real: Nissan Vanette $12{,}96/124{,}26 \times 100$; Changan Honor $9{,}07/114{,}25 \times 100$ (Tablas 4.19/4.20). [c]Valor de literatura, no medido en este proyecto; ver detalle en el texto anterior.

Figura 5.3

Punto de equilibrio (breakeven) entre el costo del dispositivo y el ahorro acumulado de combustible



Nota. Gráfico de líneas: costo fijo y ahorro acumulado son funciones continuas del tiempo, y lo relevante es el punto de cruce entre ambas. Los periodos de recuperación (2,46 y 3,23 meses) dependen de los valores de literatura de la Tabla 5.15 (kilometraje y reducción de consumo), así que deben leerse como una estimación, no como una garantía. Elaboración propia.

La viabilidad económica de este proyecto, para cerrar el capítulo, no depende únicamente de recuperar el costo de un prototipo individual: depende de su potencial de escalar a una pequeña serie de unidades para el transporte público de La Paz. En ese escenario, el costo marginal por unidad (Tabla 5.12), ya sin las horas de diseño amortizadas por este primer desarrollo, es la cifra económicamente relevante, y no el presupuesto total del prototipo aquí calculado.

Capítulo 6

CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

6.1.1. Conclusiones respecto a los objetivos planteados

Se lograron los objetivos planteados en el Capítulo 1, con los siguientes puntos:

- **Determinar los parámetros OBD-II disponibles.** El escaneo de PIDs en ambos vehículos confirmó que el Changan Honor no soporta el PID 0x10 de MAF y que el Nissan Vanette sí; este hallazgo justificó adoptar Speed-Density como método principal de estimación.
- **Desarrollar el modelo matemático de estimación del consumo.** El modelo Speed-Density, corregido con los ajustes de combustible del ECU, quedó formalizado e implementado en firmware. Su componente de lazo cerrado con aprendizaje adaptativo se formalizó matemáticamente por completo; su implementación en firmware queda como trabajo futuro, detallado en la Sección 6.2.3.
- **Recopilar datos del funcionamiento del motor.** El firmware sondea MAP, RPM, velocidad, IAT y los ajustes de combustible por CAN, y los registra junto con las variables derivadas en un archivo CSV por viaje en la tarjeta microSD.
- **Diseñar el sistema embebido de adquisición.** El hardware se diseñó y validó mediante un prototipo en protoboard, incluyendo la gestión de energía por dos rieles conmutados y un modo de bajo consumo activado por ausencia de contacto.
- **Integrar una plataforma telemétrica IoT.** El ESP32-S3 expone un panel web propio vía WiFi SoftAP, sin depender de conectividad externa ni de un servidor de terceros, con datos en vivo por WebSocket y descarga del histórico en CSV.
- **Evaluar el sistema en condiciones reales.** El sistema propio concuerda con el ELM327 dentro de un MAPE de 2,75 % y con el aforo real de tanque lleno

dentro de un MAPE de 4,84 %, un nivel de exactitud adecuado para aplicaciones prácticas derivadas de la estimación de consumo, como el cálculo de autonomía por tanque lleno presentado en la Subsección 6.1.6; el detalle completo de esta validación se presenta en la Subsección 6.1.5.

6.1.2. Conclusiones sobre el diseño del hardware

El diseño de hardware adoptó, desde su primera decisión de arquitectura, el módulo ESP32-S3-WROOM-1-N16R8 con controlador TWAI nativo en lugar de la combinación inicialmente considerada de un microcontrolador más simple con el controlador CAN externo MCP2515, con lo cual se redujo el conteo de componentes y se eliminó una fuente adicional de fallas en la interfaz SPI entre ambos circuitos integrados.

La etapa de alimentación de dos rieles, compuesta por el TPS54302DDCR como conversión conmutada de 12 V a 5 V seguido de los LDO AMS1117-3,3 y AP2112K-3,3TRG1 en paralelo para separar la alimentación del microcontrolador de la de los periféricos, demostró ser una decisión de diseño acertada para la robustez del sistema frente a los picos de corriente del transmisor WiFi. Sin embargo, como se documentó en la Sección de consumo del Capítulo 3, introduce un costo de corriente de reposo no trivial en el riel permanentemente activo, atribuible a que el AMS1117 no es una LDO de bajo consumo. Esta tensión entre simplicidad de la lista de materiales y autonomía de batería en reposo quedó documentada como una decisión consciente y no como un descuido, y se retoma como recomendación de mejora en la Sección 6.2.1.

Gracias al circuito de *power-gating* de dos MOSFET de canal P para las cargas conmutadas (LEDs/OLED y microSD) fue posible implementar un modo de bajo consumo real basado en *deep sleep* del ESP32-S3, en vez de solamente reducir el ritmo de sondeo del bus CAN como en la primera aproximación del proyecto. Esto representa una mejora sustancial en la autonomía del dispositivo frente a la batería del vehículo cuando este permanece estacionado por periodos prolongados.

6.1.3. Conclusiones sobre el desarrollo del firmware

La arquitectura de firmware, basada en FreeRTOS con tareas repartidas explícitamente entre los dos núcleos del ESP32-S3 —adquisición CAN y pila WiFi en el núcleo 0; pantalla, registro en microSD y LEDs de estado en el núcleo 1—, permitió que el sondeo del bus OBD-II, el proceso más sensible a interrupciones de tiempo real, no compitiera por CPU con el servidor web ni con la escritura a la tarjeta microSD. El patrón de fusión mediante una copia local (*scratch*) del estado del vehículo, sincronizada con el estado global solo bajo mutex al final de cada ciclo de sondeo, resultó ser el mecanismo central que evita que una escritura concurrente, como el reinicio de viaje desde el panel web, corrompa un ciclo de lectura en curso.

Implementar únicamente el método Speed-Density con corrección directa por ajustes de combustible del ECU, en vez de la arquitectura completa de lazo cerrado con aprendizaje adaptativo formalizada en el Capítulo 3, fue la decisión de alcance más

relevante del desarrollo de firmware: privilegió tener un sistema simple, íntegramente verificable y coherente con lo efectivamente probado en campo, sobre un sistema más sofisticado pero sin validar experimentalmente dentro del plazo del proyecto de grado.

6.1.4. Conclusiones sobre la interfaz web HMI

El uso de un punto de acceso WiFi propio (SoftAP) en vez de depender de una red conocida resultó coherente con el contexto real de uso del dispositivo, pues dentro de un vehículo rara vez hay una red WiFi doméstica en rango, y garantiza acceso al panel de control en cualquier circunstancia de despliegue. La elección de ESPAsyncWebServer sobre el servidor síncrono estándar del núcleo Arduino-ESP32, junto con un renderizador de gráficas en *canvas* desarrollado a medida en vez de una librería externa, evitó introducir una dependencia de conectividad a internet que el propio punto de acceso local no puede satisfacer.

6.1.5. Conclusiones sobre las pruebas y validación del sistema

La exactitud global del sistema propio, evaluada sobre los nueve tramos de prueba combinados de ambos vehículos (Tablas 4.19 y 4.20), resultó buena a nivel de consumo acumulado por trayecto: frente al ELM327 + *Car Scanner*, RMSE = 0,0312 L, MAE = 0,0262 L, MAPE = 2,75 % y $R^2 = 0{,}9971$ (Tabla 4.21); frente al aforo real de tanque lleno —disponible solo como total combinado por vehículo, $n = 2$— RMSE = 0,64 L, MAE = 0,57 L y MAPE = 4,84 %. El sesgo medio de Bland-Altman frente al ELM327 fue de apenas −0,0024 L, con límites de concordancia de [−0,0635, 0,0586] L: esto descarta un sesgo sistemático relevante entre ambos sistemas. Esta exactitud contrasta con el MAPE del 32,2 % obtenido en la validación dinámica muestra a muestra (Tabla 4.17); ambos resultados son consistentes entre sí, no contradictorios, porque un error instantáneo de signo variable se compensa al integrarse en litros a lo largo de un trayecto completo (Sección 4.4.6, Capítulo 4).

El desempeño sí difirió entre vehículos, pero por profundidad de validación disponible más que por exactitud del resultado final. El Nissan Vanette, al soportar el PID 0x10 (MAF), cuenta con una cadena de validación completa: estática (Tabla 4.9), dinámica frente a una referencia de flujo de aire real ($r = 0{,}789$, Tabla 4.17) y de consumo acumulado. El Changan Honor, al carecer de sensor MAF, solo pudo validarse de forma indirecta a nivel de consumo acumulado (Tabla 4.18), pese a lo cual su acuerdo agregado con el ELM327 fue comparable al del Vanette (+1,4 % frente a −1,4 % en el total combinado). La repetibilidad (coeficiente de variación entre repeticiones del mismo trayecto, Tabla 4.21) tampoco mostró un patrón sistemático a favor de un vehículo: osciló entre 6,9 % y 14,1 % según la combinación de vehículo y categoría de trayecto, un rango atribuible a la variabilidad normal del tráfico urbano de La Paz y no a un defecto propio de alguno de los dos vehículos.

Un tercer punto de validación tiene que ver con el sensor MAP: la comparación frente a la altitud (Tabla 4.11) respalda el supuesto de autocorrección por altitud discutido en el Capítulo 3. En la prueba *key-on, engine-off*, la lectura de MAP coincidió

exactamente con la presión barométrica real en el Changan Honor (0,0 % de diferencia) y se apartó apenas −2,5 % en el Nissan Vanette, ambas cifras cercanas al valor teórico del modelo ISA a 3 600 msnm (≈ 64,9 kPa). Esto confirma que el sensor MAP de ambos vehículos reporta la presión atmosférica real de La Paz (y no un valor de fábrica calibrado para el nivel del mar), de modo que el modelo Speed-Density parte de una base correcta sin requerir una corrección adicional por altitud en el firmware, tal como se anticipó conceptualmente en el Capítulo 3.

6.1.6. Aplicación derivada: autonomía estimada por tanque lleno

Dividiendo la capacidad de tanque de cada vehículo, según la Tabla 4.5 del Capítulo 4, entre la tasa de consumo medida sobre los nueve tramos combinados de las Tablas 4.19 y 4.20, el consumo estimado se traduce en una autonomía práctica por tanque lleno. El sistema propio estima 617 km para el Nissan Vanette, de tanque de 60 L, y 489 km para el Changan Honor, de tanque de 40 L. La autonomía realmente alcanzada, calculada igual pero a partir del aforo real de tanque lleno, fue de 575 km y 504 km respectivamente. La diferencia frente a la estimación del sistema, de 42 km y 15 km, es del mismo orden que el MAPE de consumo acumulado ya discutido en la Subsección 6.1.5: la exactitud del sistema en la estimación del consumo se traslada de forma consistente a la autonomía calculada, una magnitud derivada de interés práctico directo para el conductor.

6.2. Recomendaciones

6.2.1. Mejoras propuestas para el hardware

- **LDO de baja corriente de reposo en el riel 1.** Sustituir el AMS1117-3,3 por una LDO de baja I_q (por ejemplo, una TPS7A02 o similar, con corriente de reposo del orden de nanoamperios) reduciría de forma directa el consumo del dispositivo mientras el vehículo permanece estacionado, sin necesitar ningún cambio de firmware, ya que es una sustitución de componente sobre el mismo riel.
- **Recubrimiento conformado (*conformal coating*).** Dado que la PCB se instala bajo el tablero del vehículo, expuesta a humedad y polvo, se recomienda aplicar un recubrimiento conformado sobre la placa ensamblada antes de la instalación definitiva, especialmente sobre el transceptor CAN y el conector OBD-II.
- **Alivio de tensión mecánica en el conector OBD-II.** Se recomienda anclar mecánicamente el cable del conector OBD-II al gabinete del dispositivo (por ejemplo, con una brida o prensaestopas), de forma que los tirones accidentales del cable no transmitan esfuerzo directamente a las soldaduras del conector J1 en la PCB.
- **Unificación de la documentación de diseño.** La documentación de hardware del Capítulo 3 alterna referencias a Altium Designer y a EasyEDA como herramien-

ta de captura esquemática y ruteado; dado que el servicio de fabricación JLCPCB se integra nativamente con EasyEDA (Sección de capacidades de fabricación, Capítulo 3), se recomienda unificar la documentación y las capturas de pantalla hacia una sola herramienta antes de la entrega final, para evitar dar la impresión de que se usaron dos flujos de diseño distintos.

6.2.2. Mejoras propuestas para el firmware

- **Parámetros configurables desde el panel web.** Actualmente, valores como `FUEL_PRICE_BS_PER_L`, `ENGINE_DISPLACEMENT_L` o `FUEL_CALIBRATION_FACTOR` solo pueden cambiarse recompilando el firmware (`config.h`). Exponerlos como una página de configuración persistida en la propia microSD permitiría recalibrar el dispositivo para un vehículo distinto sin reprogramar el ESP32-S3, lo cual es particularmente relevante si el sistema se replica en varias unidades del transporte público.
- **Actualización de firmware por WiFi (OTA).** Añadir soporte de actualización *over-the-air* evitaría desmontar el dispositivo del vehículo cada vez que se corrija o mejore el firmware, algo especialmente valioso si el proyecto escala a varias unidades instaladas simultáneamente.
- **Vigilancia por *watchdog*.** Incorporar un temporizador *watchdog* de hardware que reinicie el ESP32-S3 ante un bloqueo no capturado de alguna tarea de FreeRTOS aumentaría la tolerancia a fallos del dispositivo en operación desatendida durante viajes largos.
- **Paginación del historial en el panel web.** Para sesiones de registro muy largas, descargar y parsear el CSV completo en el navegador (`/api/chartdata`) puede volverse lento; se recomienda explorar una paginación o resumen por tramos en el propio firmware para historiales de varios días.

6.2.3. Escalabilidad y trabajo futuro

El Capítulo 3 (Sección 3.5) formalizó una arquitectura de estimador en lazo cerrado más amplia que la evaluada en el Capítulo 4. Se recomienda como línea de continuidad directa del proyecto implementar en firmware los siguientes tres componentes, ya diseñados y justificados matemáticamente, pero pendientes de codificación y validación experimental:

- **Factor de calibración adaptativo K (filtro EMA).** Sustituir el `FUEL_CALIBRATION_FACTOR` estático de `config.h` por la actualización en línea de la Ecuación (3.25) ($\alpha_K = 0{,}20$), acotada a [0,70, 1,30], accionada por el LTFT ya disponible en `g_vehicleData`. Es el cambio de menor costo de implementación de los tres, ya que no requiere PIDs adicionales a los que el firmware actual ya consulta.
- **Identificación en línea de la curva $\eta_v(N)$.** Reemplazar la tabla fija `kVeTable`

(`fuel_calc.cpp`) por el procedimiento de compuerta, promediado por bin y regresión polinómica cúbica del Algoritmo 3, lo que eliminaría la necesidad de recalibrar manualmente la tabla por vehículo.

- **Conmutación automática MAF/Speed-Density.** Agregar el sondeo del PID 0x10 en la inicialización (análogo al ya implementado para el PID 0x33) y, cuando el vehículo lo soporte, usar el flujo de aire medido directamente en vez de estimarlo, conforme al Algoritmo 2. Esto beneficiaría en particular a vehículos como el Nissan Vanette de este estudio, que sí soporta MAF pero se evaluó en modo speed-density para mantener un único método comparable entre ambas unidades de prueba.

Los tres componentes son independientes entre sí y pueden implementarse y validarse por separado, de modo que pueden priorizarse según el tiempo disponible en una eventual continuación del proyecto.

En un plano más inmediato, la tabla `kVeTable` calibrada en la Sección 4.3.3 del Capítulo 4 se validó sobre el mismo conjunto de datos con el que se calibró, lo que confirma que la inversión algebraica se aplicó correctamente pero no constituye evidencia de que el modelo generalice fuera de esas condiciones. Se recomienda, como trabajo futuro de corto plazo, repetir la prueba de pasos de RPM en el Nissan Vanette en una sesión independiente, extendiéndola además a los regímenes de 4000 rpm en adelante (donde `kVeTable` conserva todavía sus valores genéricos sin calibrar), para validar la tabla contra datos que no participaron en su propia calibración.

En el caso del Changan Honor, que no soporta el PID 0x10 y queda por ello fuera del método de calibración directa anterior, la Sección "Caso sin MAF" del Capítulo 3 documenta un procedimiento alternativo basado en STFT/LTFT (`CODIGOS/changan_trim_logger.py` y `ve_curve_calibrator_indirect.py`) que no llegó a ejecutarse dentro del alcance de este trabajo. Queda como trabajo futuro recolectar ese registro de campo en el Changan Honor y correr el ajuste indirecto, teniendo presente que su resultado sería una corrección relativa a una curva semilla genérica, no una calibración absoluta como la del Vanette.

Más allá del firmware de una sola unidad, uno de los contextos de aplicación planteados en el Capítulo 1 —el transporte público urbano de La Paz, como ejemplo de flota de vehículos a gasolina con inyección electrónica— sugiere una línea de escalabilidad natural hacia una flota: agregación de los históricos de varias unidades en un servidor central (por ejemplo, sincronizando el CSV cuando el vehículo se conecta a una red WiFi conocida al final del día), y un panel comparativo que permita a un operador de transporte identificar qué unidades de su flota presentan un consumo sistemáticamente por encima del resto.

El acceso al bus CAN que ya provee el hardware desarrollado, actualmente empleado solo para lectura de parámetros de consumo, habilita además dos líneas de extensión funcional que exceden el alcance de este proyecto pero que se documentan aquí como trabajo futuro:

- **Escáner de fallas (lectura de DTC).** El firmware ya implementa el ciclo de

solicitud/respuesta del Modo 01 (datos en tiempo real); extenderlo a los Modos 03 (códigos de falla almacenados), 07 (códigos pendientes) y 02 (*freeze frame*) del estándar OBD-II reutilizaría la misma infraestructura de tramas CAN sobre 0x7DF/0x7E8–0x7EF, añadiendo únicamente la decodificación específica de DTC. Esto convertiría al dispositivo en un escáner de diagnóstico general, no limitado a consumo de combustible.

- **Comunicación bidireccional con la ECU.** La Sección de configuración de SavvyCAN del Capítulo 3 ya documenta un puente bidireccional funcional (`handleSavvyCANTransmit()`) que permite a una PC no solo escuchar sino también inyectar tramas hacia el bus CAN a través del ESP32, usado allí con fines de ingeniería inversa del protocolo del vehículo. Formalizar esta capacidad en el propio firmware —por ejemplo, para accionar funciones del vehículo mediante los mismos identificadores CAN ya identificados— es técnicamente alcanzable con el hardware actual, pero constituye un cambio de naturaleza del dispositivo: pasa de ser un instrumento de monitoreo pasivo a uno de control activo. El bus CAN de un vehículo no incorpora autenticación a nivel de trama, por lo que cualquier implementación de esta capacidad debería incorporar, como requisito de diseño y no como añadido posterior, control de acceso a la función de transmisión, límites por lista blanca de qué tramas puede emitir el dispositivo, y una revisión de seguridad específica antes de instalarse en un vehículo en uso real. Por esa misma razón, esta capacidad queda fuera del alcance de un despliegue de campo dentro de este proyecto de grado.

6.2.4. Recomendaciones para la etapa de producción en PCB

- **Escalado del lote de fabricación.** El diseño ya cumple con margen las reglas de diseño del proceso estándar de dos capas de JLCPCB (Tabla 3.16, Capítulo 3), por lo que no requiere cambios de *layout* para escalar del lote mínimo de prototipo (5 unidades) a un lote mayor si el proyecto avanza a una serie piloto para varios vehículos de transporte público.
- **Puntos de prueba (*test points*).** Se recomienda añadir puntos de prueba accesibles para las señales críticas (salida de 5 V del TPS54302, salidas de 3,3 V de ambos LDO, líneas CAN_H/CAN_L) en futuras revisiones de la PCB, para agilizar el diagnóstico de fallas de ensamblaje sin necesidad de desoldar componentes.
- **Paso de un lote ensamblado manualmente a servicio PCBA.** Como se discutió en el Capítulo 5, el ensamblaje manual es adecuado para una unidad de prototipo, pero para una serie piloto se recomienda migrar al servicio de ensamblaje del propio fabricante, dado el riesgo de defectos de soldadura en el regulador *buck* de paso fino (SOT-23-6) al escalar la cantidad de placas ensambladas a mano.

6.3. Aportes del proyecto

6.3.1. Aporte tecnológico

El proyecto entrega un dispositivo de monitoreo de consumo de combustible que no depende de un proveedor externo ni de una suscripción recurrente: el registro histórico se almacena localmente en microSD y el panel de control se sirve desde el propio dispositivo, características que lo diferencian de las soluciones comerciales de telemetría de flota comparadas en el Capítulo 5. La caracterización experimental de los PIDs OBD-II realmente soportados por dos vehículos representativos del parque automotor de La Paz, incluida la ausencia confirmada de sensor MAF en un vehículo de fabricación 2023, aporta un dato de campo concreto y poco documentado localmente, útil más allá de este proyecto para cualquier desarrollo posterior de sistemas de diagnóstico OBD-II orientados al contexto boliviano.

6.3.2. Aporte académico

Más allá del sistema efectivamente implementado y evaluado en el Capítulo 4, el desarrollo matemático del estimador en lazo cerrado (Capítulo 3, Sección 3.5) —incluyendo el análisis de estabilidad del filtro EMA por transformada Z y el procedimiento de identificación de la curva de eficiencia volumétrica por regresión— constituye un aporte formal que excede el alcance mínimo de un proyecto de grado y queda disponible como base directa para un trabajo de continuación. El análisis cuantitativo del efecto de la altitud de La Paz sobre los métodos de estimación de consumo basados en OBD-II, con el hallazgo de que el método Speed-Density se autocorrige por altitud al usar la presión real del colector, corrige además una simplificación conceptual frecuente en la literatura de sistemas similares evaluados a nivel del mar, y queda documentado en este trabajo con su justificación física completa (Sección 3.4.3, Capítulo 3).

Bibliografía

(2002). Determinación del consumo de combustible en un vehículo automotor. https://ve.scielo.org/scielo.php?script=sci_arttext&pid=S0254-07702002000300004.

(2003). *TJA1050 High Speed CAN Transceiver Product Specification*. Philips Semiconductors. Document order number: 9397 750 12157.

(2004). Estimación del consumo energético del transporte en el ámbito urbano. http://www.adingor.es/Documentacion/CIO/cio2004/comunicaciones/401-410.pdf. Comunicación presentada en CIO 2004.

(2008). *Automotive embedded systems handbook*. CRC Press / Taylor & Francis.

(2008). *SSD1306 Advance Information: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*. Solomon Systech Limited. Rev 1.1.

(2011). Road vehicles – Electrical disturbances from conduction and coupling. Technical Report ISO 7637-2, International Organization for Standardization.

(2016). Consumo de combustible en vehículos para transporte por carretera: Modelos predictivos. *Revista Ingenierías Universidad de Medellín*.

(2018). Estimating fuel consumption using regression and machine learning. http://www.diva-portal.org/smash/get/diva2:1255660/FULLTEXT01.pdf.

(2018). Research article on OBD-based fuel consumption estimation. https://pdfs.semanticscholar.org/78ad/818ce19c97d4db6fb01a78624ae1fc63944b.pdf.

(2019). *Desarrollo de un algoritmo para la estimación del consumo de combustible*. PhD thesis, Universidad Politécnica Salesiana. Verificar autor del trabajo de grado.

(2021). A robust method for collecting and processing the on-road instantaneous data of fuel consumption and speed for motorcycles. *International Journal of Sustainable Transportation*.

(2021). Qué es bifuel o bicombustible y cuáles son sus características. https://www.motor.es/que-es/bifuel-bicombustible.

(2021). *RH850/U2A microcontroller user's manual*. Renesas Electronics.

(2022). *AURIX TC3xx user's manual*. Infineon Technologies AG. Rev. 2.0.

(2022). *S32K3 automotive general-purpose microcontrollers reference manual*. NXP Semiconductors.

(2022). ¿Merece la pena más que nunca un coche bifuel de GLP? https://es.motor1.com/news/591717/coche-bifuel-ventajas-ahorro-precio/.

(2023). Fuels – Densities and specific volumes. https://www.engineeringtoolbox.com/fuels-densities-specific-volumes-d_166.html.

(2023). Road vehicles – Environmental conditions and testing for electrical and electronic equipment – Part 2: Electrical loads. Technical Report ISO 16750-2, International Organization for Standardization.

(2024). Estimated fuel consumption for a vehicle with manual transmission using the MAF sensor. https://laccei.org/LACCEI2024-CostaRica/papers/Contribution_1002_final_a.pdf.

(2024). Explicación de las relaciones aire-combustible, Lambda y estequiometría. https://www.hptuners.com/es/articles/air-fuel-ratios-lambda-and-stoichiometry-explained/.

(2024). ¿Qué efecto tienen los biocombustibles en el motor de un automóvil? https://fedebiocombustibles.com/que-efecto-tienen-los-biocombustibles-en-el-motor-de-un-automovil/.

(2026). 10 tipos de combustibles: Usos y diferencias clave. https://www.webfleet.com/es_mx/webfleet/blog/tipos-de-combustible/.

(2026). D.R Bifuel: Guía sobre el sistema GLP, ahorro y conducción. https://lorenzogonzalezautos.com/d-r-bifuel-guia-sobre-el-sistema-glp-ahorro-y-conduccion/.

(s.f.). AFR: Relación aire-combustible. https://www.ansedsoftware.com/doc_get.aspx?DocID=42.

(s.f.). La presión barométrica en la ciudad de La Paz. http://www.scielo.org.bo/pdf/chc/v63n2/v63n2_a09.pdf.

(s.f.). Sistema bi-fuel vehicular: Una ventaja eficiente y sostenible. https://www.gasnaturalgnu.com/sistema-bi-fuel-vehicular-una-ventaja-eficiente-y-sostenible/.

AcademiaLab (2024). Unidad de control del motor. https://academia-lab.com/enciclopedia/unidad-de-control-del-motor/. Recuperado el 6 de mayo de 2026.

Agencia Nacional de Hidrocarburos (2014). Especificaciones técnicas. https://www.anh.gob.bo/InsideFiles/Referencia/DafExep/ANH-14-2014.pdf.

Agencia Nacional de Hidrocarburos (2019). Gasolina especial plus de 87 octanos. https://www.energypress.com.bo/2019/05/22/anh-gasolina-especial-plus-de-87-octanos-ya-se-vende-en-la-paz/. Accedido: 2026-05-08.

Agencia Nacional de Hidrocarburos (2024). Especificaciones de calidad de combustibles en Bolivia. https://www.anh.gob.bo/w2019/contenido.php?s=13. Consultado el 3 de mayo de 2026.

Al-Nuaimi, A. et al. (2020). Fuel consumption using OBD-II and support vector machine model. *International Journal of Advanced Computer Science and Applications*, 11(1).

All3DP (2023). PLA vs. ABS vs. PETG: The Main Differences. https://all3dp.com/2/pla-vs-abs-vs-petg-differences-compared/. Consultado en octubre de 2026.

Alternative Fuels Data Center (2026). How do bi-fuel natural gas vehicles work? https://afdc.energy.gov/vehicles/how-do-bifuel-natural-gas-cars-work. Consultado en febrero de 2026.

Amazon Web Services (2024). *FreeRTOS Kernel Developer Docs*. FreeRTOS.

Aprendiendo Arduino (2016). Microcontroladores. https://aprendiendoarduino.wordpress.com/2016/06/26/microcontroladores-2/. Accedido: 25-may-2026.

Åström, K. J. and Murray, R. M. (2008). *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, Princeton.

Autoavance (2018). Manual OBDII y manejo scanner virtual. https://www.autoavance.co/wp-content/uploads/2018/01/MANUAL-OBDII-Y-MANEJO-SCANNER-VIRTUAL.pdf. Recuperado el 8 de mayo de 2026.

Autoavance (2020). ECU automotriz: Funcionamiento. https://www.autoavance.co/blog-tecnico-automotriz/ecu-automotriz-funcionamiento/. Recuperado el 6 de mayo de 2026.

Autoavance (2025). Red FlexRay: ¿Qué es el protocolo FlexRay? https://www.autoavance.co/blog-tecnico-automotriz/red-flexray/. Recuperado el 7 de mayo de 2026.

Autoavance (2026). OBD2: Qué es, cómo funciona y el futuro del diagnóstico automotriz. https://www.autoavance.co/blog-tecnico-automotriz/obd2/. Recuperado el 8 de mayo de 2026.

Autodoc (2024). Funcionamiento de la bomba de combustible. https://www.autodoc.es. Accedido: 2026-05-08.

Autosoporte (2021). Protocolo de diagnóstico en sistemas OBD II. https://autosoporte.com/diagnostico-en-sistemas-obd-ii/. Recuperado el 8 de mayo de 2026.

Autosoporte Capacitación (2024). Sistema de inyección monopunto y multipunto. https://www.autosoporte.com. Accedido: 2026-05-08.

Blanchon, B. (2024). ArduinoJson. Software.

Bosch Mobility (2024). The E/E architecture of the future. https://www.bosch-mobility.com/en/mobility-topics/ee-architecture/. Accedido: 2025.

Bourns, Inc. (2022). *MF-SM Series: Multifuse® PPTC Resettable Fuses, SMD 1812 Package*. Bourns, Inc., Riverside, CA, USA. Series MF-SM, Document No. MF-SMHV.

Buendía, R. (2017). Inyección multipunto vs. inyección directa. ¿En realidad son tan diferentes? https://www.motorpasion.com.mx/tecnologia/inyeccion-multipunto-vs-inyeccion-directa-en-realidad-son-tan-diferentes. Accedido: 2026-05-08.

Calle, E. (2026). Monitoreo del consumo de combustible – La Paz, Bolivia (repositorio de código). https://github.com/ErickCalle/Monitoreo-Consumo-Combustible-La-Paz-Bolivia. Repositorio del proyecto: firmware del ESP32-S3, herramientas de análisis en Python (carpeta `CODIGOS/`) y código del panel web utilizados a lo largo de este trabajo.

CarLightVision (2025). CANBUS vs non-CANBUS LED: La guía definitiva para el comprador. https://carlightvision.com/es/blog/canbus-vs-non-canbus-led/. Recuperado el 8 de mayo de 2026.

Çengel, Y. A. and Boles, M. A. (2019). *Thermodynamics: An Engineering Approach*. McGraw-Hill Education, Nueva York, NY, Estados Unidos, 9th edition.

Centralita BSI (2025). El papel de la ECU en la eficiencia del combustible y el rendimiento del motor. https://centralita-bsi.es/el-papel-de-la-ecu-en-la-eficiencia-del-combustible-y-el-rendimiento-del-motor/. Recuperado el 7 de mayo de 2026.

Chawda, D. et al. (2025). Design of sliding mode model predictive dual-loop control through self-learning strategy to mitigate the torque ripple in BLDC motor for electric vehicles. *Scientific Reports*, 15.

CISE Electronics (s.f.). Protocolo de comunicación CAN. https://www.cise.com/portal/notas-tecnicas/item/166-protocolo-de-comunicaci%C3%B3n-can.html. Recuperado el 8 de mayo de 2026.

Club AutoDoc (2022). ¿Qué es la ECU y cómo funciona? https://club.autodoc.es/magazin/que-es-la-ecu-y-como-funciona. Recuperado el 6 de mayo de 2026.

ClubJapo Forum (2023). ¿Cómo funciona una ECU? https://foro.clubjapo.com/t/como-funciona-una-ecu/2330. Recuperado el 7 de mayo de 2026.

CSS Electronics (2025). OBD2 explained – A simple intro. https://www.csselectronics.com/pages/obd2-explained-simple-intro. Accedido: 2026-05-08.

Dojo Five (2025). CI/CD for firmware teams: How to streamline your workflow. https://dojofive.com/blog/ci-cd-for-firmware-teams-how-to-streamline-your-workflow/. Accedido: 2025.

Embien Technologies (2024). Automotive EE architecture: The backbone of vehicle electronics. https://www.embien.com/automotive-insights/automotive-ee-architecture-the-backbone-of-vehicle-electronics. Accedido: 2025.

Equipo Automotriz JAVAZ (2010). Alimentación de 5 protocolos de OBD2. https://equipoautomotrizjavaz.com/datos_tecnicos/alimentacion-de-5-protocolos-de-obd2.pdf. Recuperado el 8 de mayo de 2026.

Escuela Politécnica Nacional (2025). Sistemas de inyección automotriz: Evolución y diagnóstico 2025. https://www.studocu.com/ec/document/escuela-politecnica-nacional/quimica-general-i/sistemas-inyeccion-automotriz-2025/117671057. Accedido: 2026-05-08.

ESP32Async contributors (2024). ESPAsyncWebServer. Software, GitHub.

Espressif Systems (2023a). *ESP32 series datasheet*. Espressif Systems. v5.2. Consultado: mayo 2025.

Espressif Systems (2023b). *ESP32 Technical Reference Manual: Two-Wire Automotive Interface (TWAI)*. Espressif Systems, ver. 5.1 edition. [Online].

Espressif Systems (2024a). Arduino-ESP32 core documentation. Software.

Espressif Systems (2024b). *ESP32-S3 Series Datasheet*. Espressif Systems.

Espressif Systems (2024c). *ESP32-S3 Technical Reference Manual*. Espressif Systems.

EVTV (2026). Savvycan documentation - serial/slcan connection. Consultado el 5 de junio de 2026.

Facultad de Ingeniería, Instituto de Computación (s.f.). Introducción a los sistemas embebidos. https://www.fing.edu.uy/inco/proyectos/butia/files/clase03-embebidos.pdf.

Fette, I. and Melnikov, A. (2011). The WebSocket Protocol. RFC 6455, Internet Engineering Task Force (IETF). https://www.rfc-editor.org/rfc/rfc6455.

FlexiHub (2024). Conectores OBD2 explicados. Conectores de las principales marcas de automóviles. https://www.flexihub.com/es/oobd2-pinout/. Recuperado el 8 de mayo de 2026.

Geotab (2020). ¿Qué es el puerto OBDII? Historia del diagnóstico a bordo. https://www.geotab.com/es/blog/obd-ii/. Recuperado el 8 de mayo de 2026.

Global Market Insights (2025). Automotive E-E architecture market size, growth forecasts 2034. https://www.gminsights.com/industry-analysis/automotive-e-e-architecture-market. Accedido: 2025.

Guzzella, L. and Onder, C. H. (2010). *Introduction to Modeling and Control of Internal Combustion Engine Systems*. Springer, Berlin, Heidelberg, 2nd edition.

Hackaday.io (2017). OBD2 fuel consumption gauge. https://hackaday.io/project/20715-obd2-fuel-consumption-gauge. Accedido: 2025.

HELLA Tech World (2024a). Sensor MAF: Funcionamiento y diagnóstico. https://www.hella.com/techworld. Accedido: 2026-05-08.

HELLA Tech World (2024b). Sensores del sistema de gestión del motor. https://www.hella.com/techworld. Accedido: 2026-05-08.

Heywood, J. B. (1988). *Internal combustion engine fundamentals*. McGraw-Hill.

Heywood, J. B. (2018). *Internal combustion engine fundamentals*. McGraw-Hill Education, 2 edition.

Industrial Monitor Direct (2026). Terminación de red CAN bus: Especificaciones de resistencias y aplicación práctica. https://industrialmonitordirect.com/es/blogs/knowledgebase/can-bus-network-termination-resistor-specifications. Recuperado el 8 de mayo de 2026.

Infolitz (2025). CI/CD for embedded systems: Modern DevOps for firmware delivery. https://www.infolitz.com/blog-post/ci-cd-for-embedded-systems-modern-devops-for-firmware-delivery. Accedido: 2025.

Instituto Nacional de Estadística (INE) de Bolivia (2026). El Índice de Precios al Consumidor registró una variación positiva de 1,31%en enero 2026. https://www.ine.gob.bo/index.php/el-indice-de-precios-al-consumidor-registro-una-variacion-positiva-de-131-en-enero-2026/. Consultado en febrero de 2026.

International Council on Clean Transportation (2013). ICCT fuel economy pilot study – Project report. Technical report, International Council on Clean Transportation.

International Electrotechnical Commission (2010). IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems.

International Organization for Standardization (1975). *ISO 2533: Standard Atmosphere*. International Organization for Standardization (ISO), Ginebra, Suiza.

International Organization for Standardization (2015). ISO 11898: Road vehicles – Controller area network (CAN).

International Organization for Standardization (2016). Road vehicles — diagnostic communication over controller area network (DoCAN). ISO 15765-4:2016.

International Organization for Standardization (2018). ISO 26262: Road vehicles – Functional safety. Segunda edición.

IPC (2009). *IPC-2152: Standard for Determining Current Carrying Capacity in Printed Board Design*. IPC — Association Connecting Electronics Industries, Bannockburn, IL, Estados Unidos.

IPC – Association Connecting Electronics Industries (2012). IPC-2221B: Generic Standard on Printed Board Design. Norma técnica IPC-2221B, IPC – Association Connecting Electronics Industries, Bannockburn, Illinois, EE.UU.

ipXchange (2026). Future-proofing embedded designs: Firmware and hardware upgrade strategies. https://ipxchange.tech/industry-insights/firmware-hardware-upgradeability/. Accedido: 2026.

ISO (2021). ISO 15765-4: Road vehicles – Diagnostics on CAN – Requirements for emissions-related systems. Technical report, International Organization for Standardization. Consultado: mayo 2025.

ITESO (2024). Sistemas embebidos: Una guía práctica. https://cruce.iteso.mx/sistemas-embebidos-una-guia-practica/.

Jeong, K. W. (s.f.). Ch7 vehicle network. https://web.chungbuk.ac.kr/~kwjeong/lectures/green_car_system/Ch7_vehicle_network.pdf. Chungbuk National University. Recuperado el 8 de mayo de 2026.

Jidoshaseibishi (s.f.). Jidoshaseibishi: Material de referencia automotriz. https://jidoshaseibishi.com/2G/2019_10/19/19.html. Recuperado el 8 de mayo de 2026.

JLCPCB (2025). *PCB Manufacturing & Assembly Capabilities*. JLCPCB, Shenzhen, China.

Kaiser, M. (2014). Electronic control unit (ECU). In *Gasoline engine management: Systems and components*, pages 254–259. Springer.

Kaschel, H. and Pinto, E. (2005). Análisis protocolar del bus de campo CAN. *Revista Facultad de Ingeniería, Universidad de Santiago de Chile*.

Lamby, L., Venturi, L., and Ignacio, J. (2026). Un giro económico para Bolivia: Reactivando el sector energético. Documento de análisis / Reporte electrónico.

Lifeder (2024). Topología de bus: Qué es, características, ventajas y desventajas. https://www.lifeder.com/topologia-de-bus/. Recuperado el 8 de mayo de 2026.

López Ruano, E. (2018). *Efectos producidos por el cambio de presión atmosférica debido a la altitud y su relación con la pérdida de potencia en los motores de combustión interna ciclo Otto Mitsubishi 4G94*. PhD thesis, Universidad de San Carlos de Guatemala.

Machado, B. G., Montoto, F. A., and Baes, L. F. (2021). Análise de dados veiculares automotores através de interface OBD2. Trabalho de conclusão de curso, Universidade Presbiteriana Mackenzie.

Mamani Ticona, E. et al. (2024). *Fundamentos de los sistemas de inyección a gasolina y autotrónica automotriz*. Universidad Mayor de San Andrés.

McCord, K. (2011). *Automotive diagnostic systems: Understanding OBD I and OBD II*. CarTech.

Mecánica Fácil (2024). Tipos de sistemas de inyección electrónica. https://www.mecanicafacil.info. Accedido: 2026-05-08.

MikroElektronika (2018). Introducción al mundo de los microcontroladores. https://www.mikroe.com/ebooks/microcontroladores-pic-programacion-en-c-con-ejemplos/introduccion-al-mundo-de-los-microcontroladores.

MotoresUno (2023). ¿Qué es el sistema OBD2, cómo funciona y por qué es importante? https://motoresuno.com/que-es-el-sistema-obd2-como-funciona-y-por-que-es-importante/. Recuperado el 8 de mayo de 2026.

Motorpasión (2024). Qué es la inyección directa de gasolina. https://www.motorpasion.com. Accedido: 2026-05-08.

MSG Equipment (2025). Acerca de los buses de comunicación en los vehículos: FlexRay, CAN, LIN. https://msg.equipment/es/blog/articles/vehicle-communication-buses-flexray-can-li/. Recuperado el 7 de mayo de 2026.

National Cheng Kung University, Department of Computer Science and Information Engineering (s.f.). Embedded system overview. https://wiki.csie.ncku.edu.tw/embedded/. Recuperado el 1 de mayo de 2026.

Navixy (2024). Comunicación intra-vehicular: CAN, FlexRay y MOST. https://navixy.com/docs/expert-center/es/vehicle-telematics-technology/can-and-obdii/intra-vehicle-communication-can-flexray-and-most/. Recuperado el 7 de mayo de 2026.

NVIDIA Networking (2024). ConnectX-6 adapter cards firmware release notes v20.39.3004 LTS (2023 LTS U2). https://docs.nvidia.com/networking/display/ConnectX6Firmwarev20393004lts. Referencia de concepto LTS en firmware embebido. Accedido: 2025.

OBDTester (2015). OBD2 connector. https://www.obdtester.com/obd2_connector. Recuperado el 8 de mayo de 2026.

Ocsaly (2022). Diy 10: Can bus fuzzer. https://ocsaly.com/diy-10-can-bus-fuzzer/. Consultado el 5 de junio de 2026.

onsemi (2023). *SS3x Series: 3.0 A Schottky Barrier Rectifiers*. Semiconductor Components Industries, LLC (onsemi), Scottsdale, AZ, USA. Publication Order Number: SS36/D, Rev. 8.

Oppenheim, A. V. and Schafer, R. W. (2010). *Discrete-Time Signal Processing*. Pearson, Upper Saddle River, 3rd edition.

Pedal Commander (2025). Guía de ajuste de ECU: Información esencial. https://pedalcommander.mx/blogs/garage/ecu-tuning-what-you-need-to-know. Recuperado el 7 de mayo de 2026.

PlatformIO (2024). *PlatformIO Documentation*.

Poudel, B. and Munir, A. (2018). Design and evaluation of a reconfigurable ECU architecture for secure and dependable automotive CPS. *IEEE Transactions on Dependable and Secure Computing*, 18(1):235–252.

Preston-Werner, T. (2013). Semantic versioning 2.0.0. https://semver.org/. Accedido: 2025.

Promwad (2025). Future of ECUs: Central compute vs. zonal – Where automotive architecture is headed. https://promwad.com/news/future-of-ecus-central-compute-vs-zonal-automotive-architecture. Accedido: 2025.

Public-Private Infrastructure Advisory Facility (PPIAF) / World Bank (2026). Kilometers per Vehicle per Day (kpvpd) – Urban Bus Toolkit. https://www.ppiaf.org/sites/ppiaf.org/files/documents/toolkits/UrbanBusToolkit/assets/1/1c/1c11.html. Consultado en agosto de 2026.

Pulkrabek, W. W. (2014). *Engineering Fundamentals of the Internal Combustion Engine*. Pearson, Harlow, Reino Unido, 2nd edition.

Putra, F. A. A. W., De Nozyra, N., Ramadhan, M. F., Nasution, S. M., and Septiawan, R. R. (2025). Real-time vehicle monitoring system using ESP32 and OBD-II. In *Proceedings of the 2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, pages 186–191. IEEE.

Reddit - CarHacking (2023). Open source CAN reverse engineering tool. https://www.reddit.com/r/CarHacking/comments/186jjz6/i_developed_an_open_source_can_reverse/. Accedido: 2026-05-08.

Reddit - Embedded (2022). Getting started with automotive controller reverse engineering. https://www.reddit.com/r/embedded/comments/u5okpn. Accedido: 2026-05-08.

Reif, K. (2015). *Gasoline engine management: Systems and components*. Springer Vieweg.

Revista Autocrash (2016). Evolución de los sistemas de inyección de combustible. https://www.revistaautocrash.com/evolucion-de-los-sistemas-de-inyeccion-de-combustible/. Accedido: 2026-05-08.

Rimpas, D., Papadakis, A., and Samarakou, M. (2020). OBD-II sensor diagnostics for monitoring vehicle operation and consumption. *Energy Reports*, 6:55–63.

Robert Bosch GmbH (2014a). *Automotive electrics and automotive electronics*. Springer Vieweg, 5 edition.

Robert Bosch GmbH (2014b). *Gasoline engine management*. Robert Bosch GmbH.

Robert Bosch GmbH (2018a). *Automotive electrics and automotive electronics*. Springer, 6 edition.

Robert Bosch GmbH (2018b). *Automotive handbook*. Robert Bosch GmbH, 10 edition.

Rodi (2025). ¿Qué es el sistema OBD2 y por qué es importante para tu coche? https://www.rodi.es/blog/sistema-obd2-importancia-coche/. Recuperado el 8 de mayo de 2026.

SAE International (2014). SAE J1979: E/E diagnostic test modes. Technical report, SAE International. Consultado: mayo 2025.

Sanguinetti, A., Queen, E., Yee, C., and Akanesuvan, K. (2020). Average impact and important features of onboard eco-driving feedback: A meta-analysis. *Transportation Research Part F: Traffic Psychology and Behaviour*, 70:265–279.

Scribd (2025). Protocolos de comunicación automotriz. https://www.scribd.com/presentation/461192891/Protocolos-de-comunicacion-automotriz. Recuperado el 7 de mayo de 2026.

SERNAUTO (2024). La ECU, clave para optimizar el rendimiento, la eficiencia y las emisiones del motor. https://www.sernauto.es/blog/la-ecu-clave-para-optimizar-el-rendimiento-la-eficiencia-y-las-emisiones-del-motor/. Recuperado el 7 de mayo de 2026.

Shaw, S., Hou, Y., Zhong, W., and Sun, Q. (2019). Instantaneous fuel consumption estimation using smartphones. In *Proceedings of the IEEE Vehicular Technology Conference (VTC-Fall)*.

Smith, J. A. and Johnson, R. L. (2015). Engine control unit strategies for modern gasoline engines. *SAE International Journal of Engines*, 8(2):1120–1130.

Solomon Systech (2008). *SSD1306 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*. Solomon Systech. Datasheet.

Swedish Embedded (2024). Mastering version control in CI/CD for firmware development: Strategies for success. https://swedishembedded.com/project/devops/version-control. Accedido: 2025.

Tarter, R. E. (1991). *Solid-state power conversion handbook*. Wiley.

Tech Briefs (2024). Zonal electrical architectures cut vehicle wiring-system cost, complexity. https://www.techbriefs.com/component/content/article/49578-zonal-electrical-architectures-cut-vehicle-wiring-system-cost-complexity. Accedido: 2025.

Tecnología del Automóvil (2025). Alimentación de gasolina y su historia. https://tecnologia-automovil.com/alimentacion-de-gasolina-su-historia/. Accedido: 2026-05-08.

Texas Instruments (2015). *SN65HVD230, SN65HVD231, SN65HVD232 3.3-V CAN bus transceivers*. Texas Instruments Incorporated. SLOS346G. Consultado: mayo 2025.

Tope Parts (s.f.). What eBike protocol is for LCD display? https://topeparts.com/what-ebike-protocol-is-for-lcd-display/. Recuperado el 8 de mayo de 2026.

Transcend Information Inc. (2006). *microSD Memory Card Datasheet (TS256M–2GUSD)*. Transcend Information Inc., Taipei, Taiwan. Especificaciones técnicas de operación en Modo SD y Modo SPI (Voltaje: 2.7V–3.6V).

Universidad Mayor de San Andrés (s.f.). Sistema de diagnóstico OBD-II e interfaz de comunicación. https://repositorio.umsa.bo/bitstream/handle/123456789/32627/PG-7097.pdf?sequence=1&isAllowed=y. Recuperado el 8 de mayo de 2026.

Uvarov, K. and Ponomarev, A. (2021). Driver identification with OBD-II public data. In *Proceedings of the 2021 28th Conference of Open Innovations Association (FRUCT)*, pages 495–501. IEEE.

Verma, M. E., Bridges, R. A., Sosnowski, J. J., Hollifield, S. C., and Iannacone, M. D. (2020). CAN-D: A modular four-step pipeline for comprehensively decoding controller area network data. arXiv. arXiv:2006.05993. https://arxiv.org/abs/2006.05993.

Vishay General Semiconductor (2023). *TRANSZORB® SMAJ Series: 400 W Surface Mount Transient Voltage Suppressors*. Vishay Intertechnology, Inc., Malvern, PA, USA. Document Number: 88369.

VVDN Technologies (2024). Comprehensive validation strategies for automotive ECUs: MIL, SIL, and HIL. https://www.vvdntech.com/en-us/blog/comprehensive-validation-strategies-for-automotive-ecus-mil-sil-and-hil/. Accedido: 2025.

Wang, J. et al. (2020). Effects of air-fuel ratio on combustion characteristics and emissions of gasoline engines. *Journal of the Air & Waste Management Association*, 71(2):145–160. Verificar DOI: el asignado originalmente pertenece a otro artículo.

Webfleet (2000). Una guía para el diagnóstico a bordo del vehículo (OBD). https://www.webfleet.com/es_mx/webfleet/fleet-management/glossary/on-board-diagnostics/. Recuperado el 8 de mayo de 2026.

Wevolver (2023). PLA vs ABS vs PETG: A Comparison of 3D Printing Filaments. https://www.wevolver.com/article/comparison-of-pla-abs-and-petg-filaments-for-3d-printing. Consultado en octubre de 2026.

Wikibooks (2021a). El OBDII completo/Los PIDs/PID Modo 01. https://es.wikibooks.org/wiki/El_OBDII_Completo/Los_PIDs/PID_Modo_01. Recuperado el 8 de mayo de 2026.

Wikibooks (2021b). El OBDII completo/OBD en breve/OBD, PID y DTC en breve. https://es.wikibooks.org/wiki/El_OBDII_Completo/OBD_En_Breve/OBD,_PID_y_DTC_en_Breve. Recuperado el 8 de mayo de 2026.

Wikipedia contributors (2004). Inyección de combustible. https://es.wikipedia.org/wiki/Inyecci%C3%B3n_de_combustible. Accedido: 2026-05-08.

Wu, W. et al. (2024). Review of electrical and electronic architectures for autonomous vehicles: Topologies, networking and simulators. *Automotive Innovation*.

YouTube (2024). Funcionamiento del sistema de inyección electrónica. https://www.youtube.com/watch?v=1wJQamTIBB8. Video. Accedido: 2026-05-08.

Zimmermann, W. and Schmidgall, R. (2014). *Bussysteme in der Fahrzeugtechnik*. Springer Vieweg.

Apéndice A
PIDs OBD-II modo 01 — datos en tiempo real

Tabla A1 *Códigos de Parámetro (PIDs) del Modo 01 del estándar OBD-II.*

PID (hex)	Bytes	Descripción	Mín.	Máx.	Unidad
00	4	PIDs implementados [01–20]	–	–	bits
01	4	Estado de monitores de diagnóstico (MIL y DTC)	–	–	bits
02	2	Código de falla DTC almacenado	–	–	bits
03	2	Estado del sistema de combustible	–	–	bits
04	1	Carga calculada del motor	0	100	%
05	1	Temperatura del líquido de enfriamiento	-40	215	°C
06	1	Ajuste de combustible corto plazo — Banco 1	-100	99,2	%
07	1	Ajuste de combustible largo plazo — Banco 1	-100	99,2	%
08	1	Ajuste de combustible corto plazo — Banco 2	-100	99,2	%
09	1	Ajuste de combustible largo plazo — Banco 2	-100	99,2	%
0A	1	Presión del combustible	0	765	kPa
0B	1	Presión absoluta del colector de admisión	0	255	kPa
0C	2	RPM del motor	0	16383,75	rpm
0D	1	Velocidad del vehículo	0	255	km/h
0E	1	Avance del tiempo de encendido	-64	63,5	°
0F	1	Temperatura del aire de admisión	-40	215	°C
10	2	Velocidad de flujo de aire MAF	0	655,35	g/s
11	1	Posición del acelerador	0	100	%
12	1	Estado del aire secundario	–	–	bits
1C	1	Estándar OBD implementado	–	–	bits
1F	2	Tiempo desde arranque del motor	0	65535	s
21	2	Distancia recorrida con MIL encendido	0	65535	km
2C	1	EGR comandado	0	100	%
2F	1	Nivel del tanque de combustible	0	100	%
31	2	Distancia recorrida desde borrado de fallas	0	65535	km

Continúa en la siguiente página...

Continúa de la página anterior...

PID (hex)	Bytes	Descripción	Mín.	Máx.	Unidad
33	1	Presión barométrica absoluta	0	255	kPa
3C	2	Temperatura del catalizador: Banco 1, Sensor 1	-40	6513,5	°C
42	2	Voltaje del módulo de control	0	65,535	V
45	1	Posición relativa del acelerador	0	100	%
46	1	Temperatura del aire ambiental	-40	215	°C
4D	2	Tiempo con MIL encendido	0	65535	min
5A	1	Posición relativa del pedal del acelerador	0	100	%
5C	1	Temperatura del aceite del motor	-40	210	°C
5E	2	Velocidad del combustible del motor	0	3276,75	L/h
61	1	Porcentaje de torque solicitado por conductor	-125	125	%
62	1	Porcentaje de torque actual del motor	-125	125	%
63	2	Torque de referencia del motor	0	65535	Nm

Nota. Selección de los PIDs más relevantes del Modo 01 del estándar OBD-II, correspondientes a datos en tiempo real provistos por la ECU del vehículo. La lista completa incluye PIDs adicionales hasta el código `C4`. Adaptado de *El OBDII Completo/Los PIDs/PID Modo 01*, por Wikilibros, 2024 (https://es.wikibooks.org/wiki/El_OBDII_Completo/Los_PIDs/PID_Modo_01). Contenido bajo licencia Creative Commons Atribución-CompartirIgual.

Apéndice B

Hoja de datos técnicos del transceptor CAN TJA1050

El transceptor TJA1050 es la interfaz física entre el controlador del protocolo Area Network (CAN) y el bus físico diferencial de datos (tja, 2003). Está diseñado específicamente para aplicaciones automotrices de alta velocidad que requieren tasas de transferencia de hasta 1 Mbaud.

Características principales y funcionalidades

De acuerdo con las especificaciones del fabricante, el dispositivo destaca por las siguientes prestaciones de ingeniería:

- **Compatibilidad estándar:** Totalmente compatible con la norma internacional ISO 11898.
- **Inmunidad electromagnética:** Emisión electromagnética (EME) extremadamente baja gracias al emparejamiento óptimo de las señales de salida CANH y CANL, combinada con una alta inmunidad a interferencias (EMI).
- **Compatibilidad lógica:** Niveles de entrada tolerantes y compatibles con dispositivos de lógica de 5 V.
- **Protección automotriz:** Pines de bus protegidos contra transitorios eléctricos en entornos automotrices de acuerdo con la norma ISO 7637, además de contar con protección térmica y salvaguarda contra cortocircuitos a batería y a masa.
- **Temporizador de seguridad:** Circuito de tiempo muerto (*Dominant Time-out*) en la línea TXD que previene el bloqueo permanente del bus por fallas de hardware.

Configuración de pines (pinout)

El TJA1050 se presenta en un encapsulado plástico de contorno reducido SO8 con 8 terminales. La función de cada pin se detalla a continuación en la Tabla B1.

Tabla B1 *Descripción de pines del transceptor TJA1050.*

Símbolo	Pin	Descripción Técnica
TXD	1	Entrada de datos de transmisión; lee los datos lógicos provenientes del controlador CAN.
GND	2	Conexión de referencia a tierra (0 V).
VCC	3	Voltaje de alimentación principal (4,75 V a 5,25 V).
RXD	4	Salida de datos de recepción; envía los datos leídos desde el bus hacia el controlador CAN.

Continúa en la siguiente página...

Tabla B1. Continúa de la página anterior...

Símbolo	Pin	Descripción Técnica
Vref	5	Salida de voltaje de referencia del circuito ($0{,}5 \cdot V_{CC}$ nominal).
CANL	6	Línea de nivel bajo del bus CAN físico (*LOW-level CAN bus line*).
CANH	7	Línea de nivel alto del bus CAN físico (*HIGH-level CAN bus line*).
S	8	Entrada de selección de modo: conectar a GND para modo de alta velocidad o a VCC para modo silencioso.

Nota. Datos técnicos adaptados de la especificación de producto de Philips Semiconductors para el encapsulado plástico SO8. Los pines lógicos de entrada (TXD y S) cuentan con tolerancia de nivel tanto para 3,3 V como para 5 V, facilitando el acoplamiento directo con el microcontrolador.

Especificaciones eléctricas de referencia

La Tabla B2 resume las magnitudes límites y de operación indispensables para el correcto acoplamiento eléctrico del componente dentro del hardware diseñado.

Tabla B2 *Valores eléctricos fundamentales y límites absolutos del TJA1050.*

Símbolo	Parámetro	Mínimo	Máximo	Unidad
V_{CC}	Voltaje de alimentación de operación	4,75	5,25	V
$V_{CC(abs)}$	Voltaje de alimentación límite absoluto	-0,3	+6,0	V
V_{CANH}	Voltaje continuo en pin CANH	-27	+40	V
V_{CANL}	Voltaje continuo en pin CANL	-27	+40	V
$V_{i(dif)(bus)}$	Voltaje de entrada diferencial (Dominante)	1,5	3,0	V
$t_{d(TXD-RXD)}$	Retraso de propagación total (TXD a RXD)	—	250	ns
T_{vj}	Temperatura de unión virtual de operación	-40	+150	°C
V_{esd}	Voltaje de descarga electroestática (HBM)	-4000	+4000	V

Nota. Los parámetros eléctricos delimitan las condiciones nominales de funcionamiento frente a los rangos límites absolutos del fabricante. Destaca la alta tolerancia a voltajes continuos inversos y transitorios en las líneas del bus, así como el nivel de protección contra descargas electrostáticas (ESD), fundamentales para preservar la integridad del circuito ante las perturbaciones eléctricas del vehículo.

Apéndice C
Hoja de datos transceptor SN65HVD230

Transceptor CAN de 3,3 V de Texas Instruments, interfaz física entre el controlador TWAI del ESP32-S3 y el bus diferencial CAN (Texas Instruments, 2015). Se reproducen la página de características y la de dimensiones mecánicas de la hoja de datos original.

Product Folder | Order Now | Technical Documents | Tools & Software | Support & Community

SN65HVD23x 3.3-V CAN Bus Transceivers

1 Features

- Operates with a single 3.3 V Supply
- Compatible With ISO 11898-2 Standard
- Low Power Replacement for the PCA82C250 Footprint
- Bus Pin ESD Protection Exceeds ±16 kV HBM
- High Input Impedance Allows for Up to 120 Nodes on a Bus
- Adjustable Driver Transition Times for Improved Emissions Performance
  - SN65HVD230 and SN65HVD231
- SN65HVD230: Low Current Standby Mode
  - 370 μA Typical
- SN65HVD231: Ultra Low Current Sleep Mode
  - 40 nA Typical
- Designed for Data Rates(1) up to 1 Mbps
- Thermal Shutdown Protection
- Open Circuit Fail-Safe Design
- Glitch Free Power Up and Power Down Protection for Hot Plugging Applications

(1) The signaling rate of a line is the number of voltage transitions that are made per second expressed in the units bps (bits per second).

2 Applications

- Industrial Automation, Control, Sensors and Drive Systems
- Motor and Robotic Control
- Building and Climate Control (HVAC)
- Telecom and Basestation Control and Status
- CAN Bus Standards Such as CANopen, DeviceNet, and CAN Kingdom

3 Description

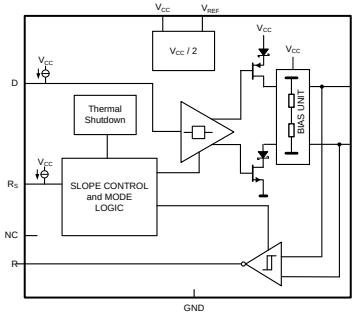
The SN65HVD230, SN65HVD231, and SN65HVD232 controller area network (CAN) transceivers are compatible to the specifications of the ISO 11898-2 High Speed CAN Physical Layer standard (transceiver). These devices are designed for data rates up to 1 megabit per second (Mbps), and include many protection features providing device and CAN network robustness. The SN65HVD23x transceivers are designed for use with the Texas Instruments 3.3 V μPs, MCUs and DSPs with CAN controllers, or with equivalent protocol controller devices. The devices are intended for use in applications employing the CAN serial communication physical layer in accordance with the ISO 11898 standard.

Device Information(1)

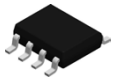
PART NUMBER	PACKAGE	BODY SIZE (NOM)
SN65HVD230	SOIC (8)	4.90 mm × 3.91 mm
SN65HVD231		
SN65HVD232		

(1) For all available packages, see the orderable addendum at the end of the datasheet.

Equivalent Input and Output Schematic Diagrams



An IMPORTANT NOTICE at the end of this data sheet addresses availability, warranty, changes, use in safety-critical applications, intellectual property matters and other important disclaimers. PRODUCTION DATA.

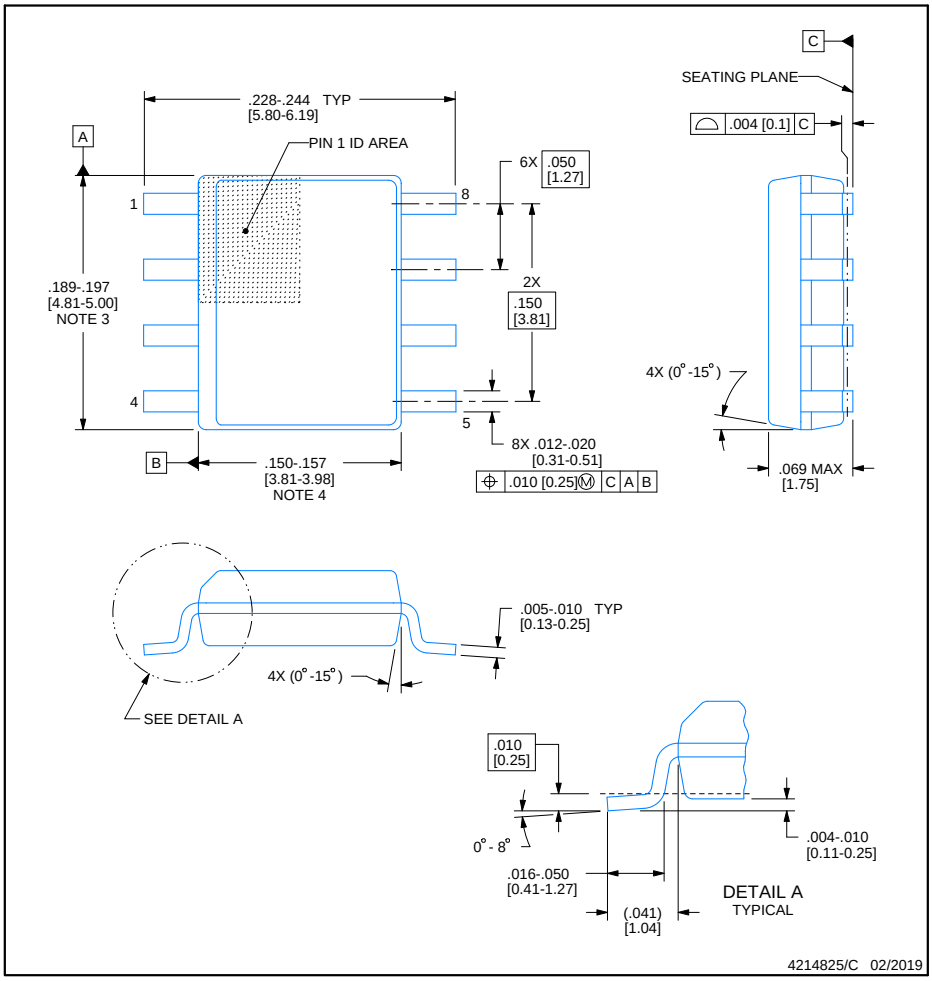


PACKAGE OUTLINE

D0008A

SOIC - 1.75 mm max height

SMALL OUTLINE INTEGRATED CIRCUIT



NOTES:

1. Linear dimensions are in inches [millimeters]. Dimensions in parenthesis are for reference only. Controlling dimensions are in inches. Dimensioning and tolerancing per ASME Y14.5M.
2. This drawing is subject to change without notice.
3. This dimension does not include mold flash, protrusions, or gate burrs. Mold flash, protrusions, or gate burrs shall not exceed .006 [0.15] per side.
4. This dimension does not include interlead flash.
5. Reference JEDEC registration MS-012, variation AA.

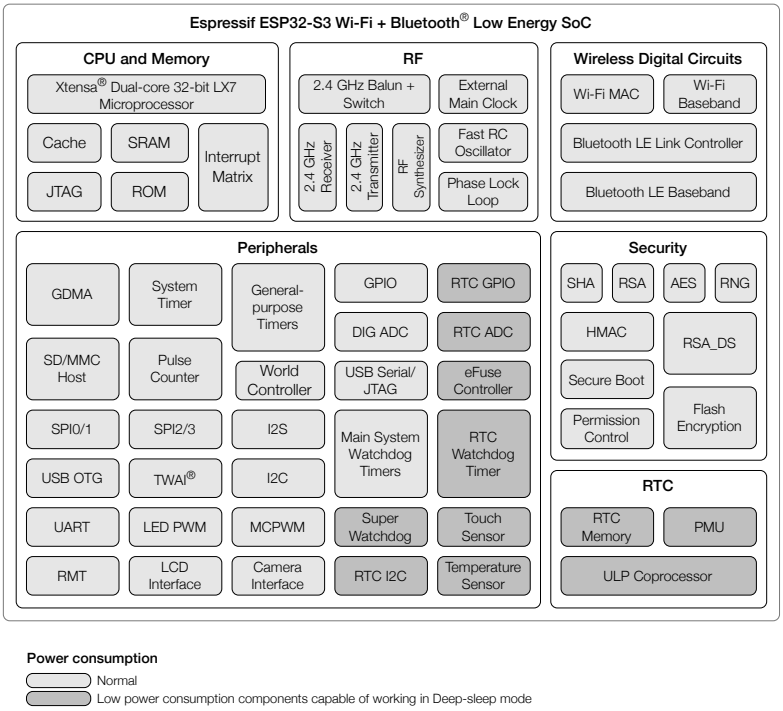
Apéndice D
Hoja de datos técnicos del módulo ESP32-S3-WROOM-1-N16R8

Módulo Espressif ESP32-S3-WROOM-1-N16R8 (16 MB flash, 8 MB PSRAM octal), con controlador TWAI.

Product Overview

ESP32-S3 is a low-power MCU-based system on a chip (SoC) with integrated 2.4 GHz Wi-Fi and Bluetooth® Low Energy (Bluetooth LE). It consists of high-performance dual-core microprocessor (Xtensa® 32-bit LX7), a ULP coprocessor, a Wi-Fi baseband, a Bluetooth LE baseband, RF module, and numerous peripherals.

The functional block diagram of the SoC is shown below.



ESP32-S3 Functional Block Diagram

For more information on power consumption, see Section 4.1.3.5 *Power Management Unit (PMU)*.

2 Pins

2.1 Pin Layout

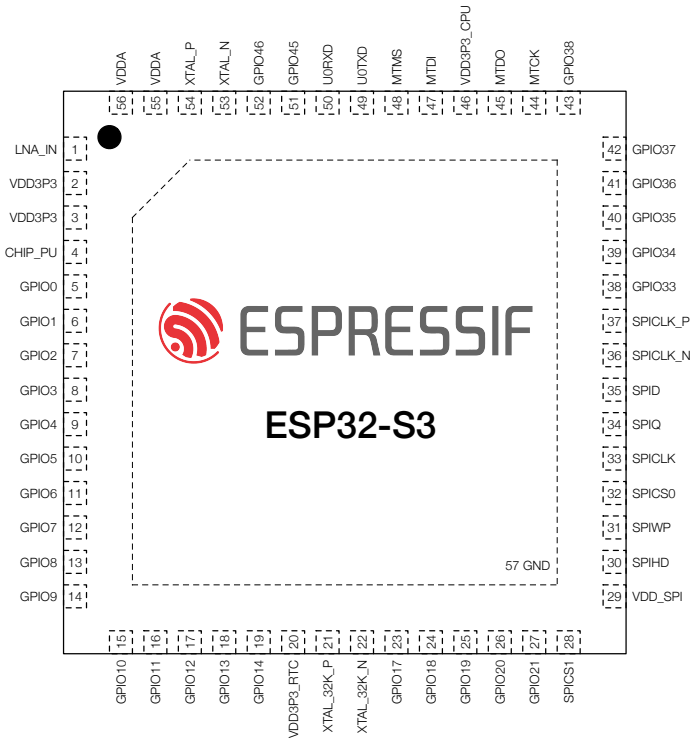


Figure 2-1. ESP32-S3 Pin Layout (Top View)

Apéndice E

Hoja de datos técnicos del controlador de pantalla OLED SSD1306

Módulo de pantalla OLED I2C de 0,96", 128 × 64 píxeles, basado en el controlador SSD1306 (ssd, 2008), con dirección I2C configurable entre `0x3C` y `0x3D`. Se reproducen las páginas de especificaciones básicas y el plano de dimensiones mecánicas del módulo.

---

1. Basic Specifications

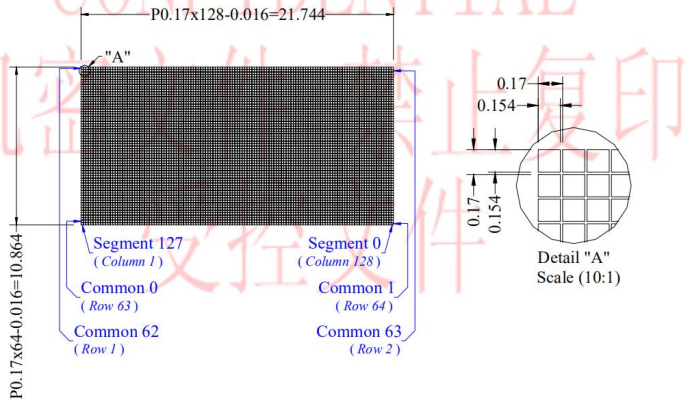
1.1 Display Specifications

1)	Display Mode:	Passive Matrix
2)	Display Color:	Monochrome (White)
3)	Drive Duty:	1/64 Duty

1.2 Mechanical Specifications

1)	Outline Drawing:	According to the annexed outline drawing
2)	Number of Pixels:	128 × 64
3)	PCB Size:	27.3×27.8× 2.65 (mm)
4)	Active Area:	21.744 × 10.864 (mm)
5)	Pixel Pitch:	0.17 × 0.17 (mm)
6)	Pixel Size:	0.154 × 0.154 (mm)

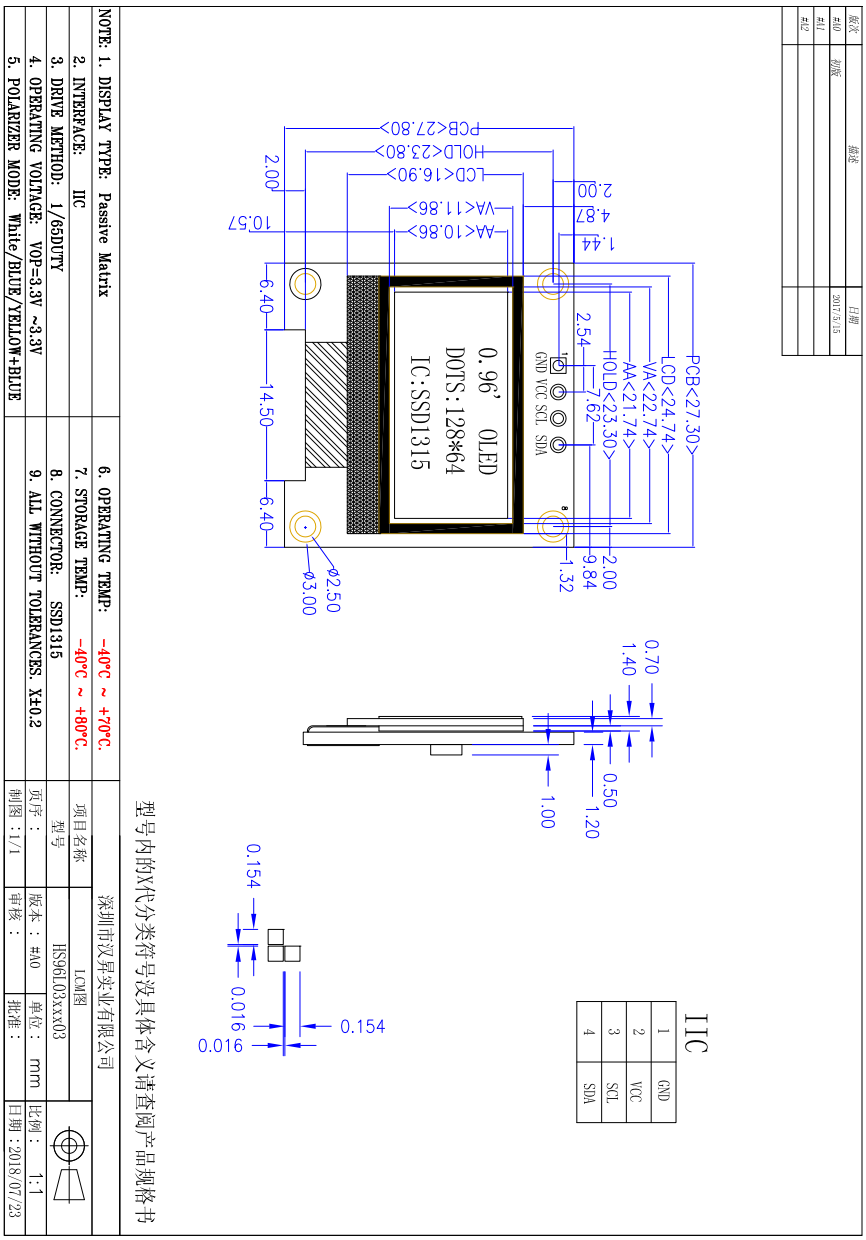
1.3 Active Area / Memory Mapping & Pixel Construction



CONFIDENTIAL
機密文件 禁止複印

5

1.4MechanicalDrawing



Apéndice F

Hoja de datos técnicos del fusible PTC reseteable, serie BSMD1812

Fusible autorrecuperable PPTC de la serie BSMD1812 (encapsulado SMD 1812), con corriente de mantenimiento (I_{hold}) de 1,5 A y de disparo (I_{trip}) de 3,0 A (Bourns, Inc., 2022), primera línea de protección contra sobrecorriente en la línea de 12 V del conector OBD-II.

BHFUSE
伯宏保险丝

BSMD1812 Series Surface Mount PTC Device

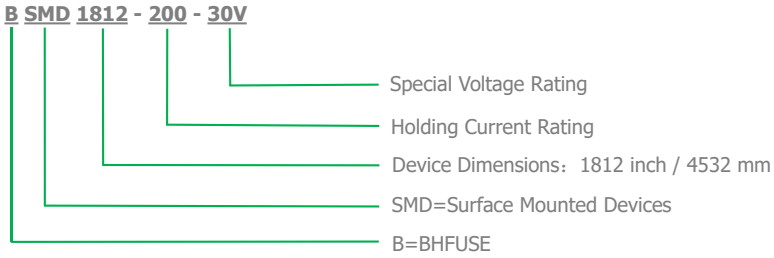
➢ Features

- Size 0.18*0.12 inch /4.5*3.2 mm
- RoHS compliant, lead-free and halogen-free
- Fast response to fault current
- Low resistance
- Low profile
- Compatible with high temperature solders

➢ Applications

- Computer, Mobile phones, Multimedia
- Automotive, Industrial controls, Telephony and broadband
- Game machines, Portable electronics, Battery

➢ Part Number System



➢ Electrical Characteristics（25°C）

Part Number	I_{hold}	I_{trip}	V_{max}	I_{max}	$P_{d\ typ}$	Time to trip		R_{min}	$R1_{max}$
	(A)	(A)	(V_{dc})	(A)	(W)	(A)	(Sec)	(Ω)	(Ω)
BSMD1812-010-30V	0.10	0.30	30	40	0.8	0.50	1.50	0.750	15.00
BSMD1812-010-60V	0.10	0.30	60	40	0.8	0.50	1.50	0.750	15.00
BSMD1812-014-60V	0.14	0.34	60	40	0.8	1.50	0.15	0.650	6.000
BSMD1812-020-30V	0.20	0.40	30	40	0.8	8.00	0.04	0.350	5.000
BSMD1812-020-60V	0.20	0.40	60	40	0.8	8.00	0.04	0.350	5.000
BSMD1812-020-72V	0.20	0.40	72	40	0.8	8.00	0.04	0.350	5.000
BSMD1812-030-30V	0.30	0.60	30	40	0.8	8.00	0.10	0.250	3.000
BSMD1812-030-60V	0.30	0.60	60	40	0.8	8.00	0.10	0.250	3.000

Apéndice G

Diodo Schottky SS34 — especificaciones técnicas

Diodo Schottky SS34 (familia SS32 a SS3200, MDD/China Base Electronics), encapsulado DO-214AC (SMA), con tensión inversa repetitiva máxima (V_{RRM}) de 40 V y corriente media rectificada (I_O) de 3 A (onsemi, 2023), usado como protección de polaridad inversa en la línea de 12 V del OBD-II. La fila correspondiente a SS34 es la aplicable a este proyecto.

SS32 THRU SS3200

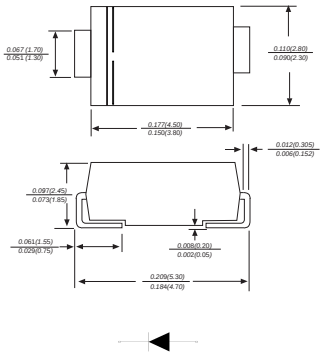
Reverse Voltage - 20 to 200 Volts Forward Current - 3.0 Ampere

SURFACE MOUNT SCHOTTKY BARRIER RECTIFIER

Features

- The plastic package carries Underwriters Laboratory Flammability Classification 94V-0
- For surface mounted applications
- Metal silicon junction,majority carrier conduction
- Low power loss,high efficiency
- Built-in strain relief,ideal for automated placement
- High forward surge current capability
- High temperature soldering guaranteed: 250 °C/10 seconds at terminals

DO-214AC/SMA RoHS COMPLIANT



Dimensions in inches and (millimeters)

Mechanical Data

Case : JEDEC DO-214AC/SMA molded plastic body
Terminals : Solderable per MIL-STD-750,Method 2026
Polarity : Color band denotes cathode end Mounting Position : Any
Weight : 0.0018 ounce, 0.064 grams

Maximum Ratings And Electrical Characteristics

Ratings at 25℃ ambient temperature unless otherwise specified.
Single phase half-wave 60Hz,resistive or inductive load,for capacitive load current derate by 20%.

Parameter / Marking Code	SYMBOLS	MDD SS32	MDD SS33	MDD SS34	MDD SS35	MDD SS36	MDD SS38	MDD SS310	MDD SS3150	MDD SS3200	UNITS
Maximum repetitive peak reverse voltage	V_{RRM}	20	30	40	50	60	80	100	150	200	V
Maximum RMS voltage	V_{RMS}	14	21	28	35	42	56	70	105	140	V
Maximum DC blocking voltage	V_{DC}	20	30	40	50	60	80	100	150	200	V
Maximum average forward rectified current at TL(see fig.1)	$I_{(AV)}$	3.0									A
Peak forward surge current 8.3ms single half sine-wave superimposed onrated load (JEDEC Method)	I_{FSM}	80									A
Maximum instantaneous forward voltage at 3.0A	V_F	0.55			0.70		0.85			0.95	V
Maximum DC reverse current T_A=25℃	I_R	0.5					0.3				mA
at rated DC blocking voltage T_A=125℃		5					3.0				
Typical junction capacitance (NOTE 1)	C_J	250			180						pF
Typical thermal resistance (NOTE 2)	$R_{\theta JA}$	70									℃/W
Operating junction temperature range	T_J	-55 to +125									℃
Storage temperature range	T_{STG}	-55 to +150									℃

Note:1.Measured at 1MHz and applied reverse voltage of 4.0V D.C.
2.P.C.B. mounted with 0.2x0.2"(5.0x5.0mm) copper pad areas

Apéndice H

Hoja de datos técnicos del diodo supresor de tensión transitoria (TVS) SMAJ18A

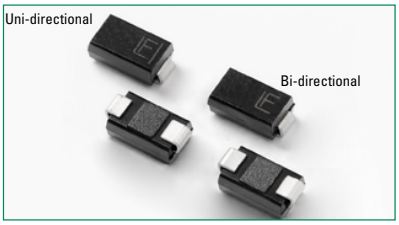
Diodo TVS Littelfuse serie SMAJ, modelo SMAJ18A (DO-214AC/SMA, 400 W pico), con tensión de trabajo (*stand-off*) de 18 V y tensión de sujeción máxima de 29,2 V a 13,7 A (Vishay General Semiconductor, 2023), usado como protección contra transitorios de *load dump* y ESD en la línea de 12 V.

Littelfuse
Expertise Applied | Answers Delivered

Transient Voltage Suppression Diodes
Surface Mount – 400W > SMAJ series

SMAJ Series

HF RoHS



Agency Approvals

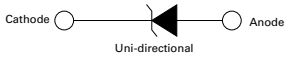
AGENCY	AGENCY FILE NUMBER
	E230531

Maximum Ratings and Thermal Characteristics (T_A=25°C unless otherwise noted)

Parameter	Symbol	Value	Unit
Peak Pulse Power Dissipation at T_A=25°C by 10/1000µs Waveform (Fig.2)(Note 1), (Note 2), (Note 5)	P_{PPM}	400	W
Power Dissipation on Infinite Heat Sink at T_L=50°C	P_D	3.3	W
Peak Forward Surge Current, 8.3ms Single Half Sine Wave (Note 3)	I_{FSM}	60	A
Maximum Instantaneous Forward Voltage at 25A for Unidirectional Only (Note 4)	V_F	3.5/5.0	V
Operating Temperature Range	T_J	-65 to 150	°C
Storage Temperature Range	T_{STG}	-65 to 175	°C
Typical Thermal Resistance Junction to Lead	$R_{\theta JL}$	30	°C/W
Typical Thermal Resistance Junction to Ambient	$R_{\theta JA}$	120	°C/W

Notes:
1. Non-repetitive current pulse, per Fig.4 and derated above T_J (initial) =25°C per Fig. 3.
2. Mounted on 5.0x5.0mm copper pad to each terminal.
3. Measured on 8.3ms single half sine wave or equivalent square wave for unidirectional device only.
4. V_F < 3.5V for single die parts and V_F< 5.0V for stacked-die parts.
5. The P_{PPM} of stacked-die parts is 600W and please contact littelfuse for the detail stacked-die parts.

Functional Diagram



Description

The SMAJ series is designed specifically to protect sensitive electronic equipment from voltage transients induced by lightning and other transient voltage events.

Features

- 400W Peak pulse power capability at 10/1000µs waveform, repetition rate (duty cycle): 0.01%
- Excellent clamping capability
- Typical I_R less than 1µA when V_{BR} min>12V
- For surface mounted applications to optimize board space
- Low profile package
- Typical failure mode is short from over-specified voltage or current
- Whisker test is conducted based on JEDEC JESD201A per its table 4a and 4c
- IEC-61000-4-2 ESD 30kV(Air), 30kV (Contact)
- ESD protection of data lines in accordance with IEC 61000-4-2
- EFT protection of data lines in accordance with IEC 61000-4-4
- Built-in strain relief
- Fast response time: typically less than 1.0ps from 0 Volts to V_{BR} min
- Glass passivated junction
- Low inductance
- High temperature to reflow soldering guaranteed: 260°C/40sec
- V_{BR} @ T_J= V_{BR}@25°C x (1+α T x (T_J - 25)) (α T:Temperature Coefficient, typical value is 0.1%)
- Plastic package is flammability rated V-0 per Underwriters Laboratories
- Meet MSL level1, per J-STD-020, LF maximun peak of 260°C
- Matte tin lead–free Plated
- Halogen free and RoHS compliant
- Pb-free E3 means 2nd level interconnect is Pb-free and the terminal finish material is tin(Sn) (IPC/JEDEC J-STD-609A.01)

Applications

TVS devices are ideal for the protection of I/O Interfaces, V_{CC} bus and other vulnerable circuits used in Telecom, Computer, Industrial and Consumer electronic applications.

Additional Infomarion

Datasheet | Resources | Samples

© 2015 Littelfuse, Inc.
Specifications are subject to change without notice.
Revised: 11/20/15

Apéndice I

Hoja de datos técnicos del convertidor buck síncrono TPS54302DDCR

Convertidor *buck* síncrono TPS54302DDCR (Texas Instruments), primera etapa de la cadena de regulación de dos rieles (Capítulo 3): 12 V → 5 V, entrada de 4,5 V a 28 V, hasta 3 A continuos a 400 kHz, encapsulado SOT-23-6. Se reproducen las páginas de características y de dimensiones del encapsulado.

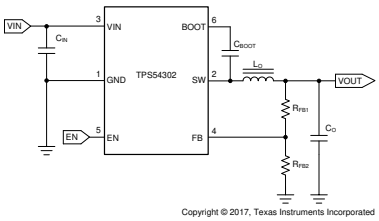
TPS54302 4.5V to 28V Input, 3A Output, EMI-Friendly Synchronous Step-Down Converter

1 Features

- 4.5V to 28V wide input voltage range
- Integrated 85mΩ and 40mΩ MOSFETs for 3A, continuous output current
- Low 2μA shutdown, 45μA quiescent current
- Internal 5ms soft start
- Fixed 400kHz switching frequency
- Frequency spread spectrum to reduce EMI
- Advanced Eco-mode pulse skip
- Peak current mode control
- Internal loop compensation
- Overcurrent protection for both MOSFETs with hiccup mode protection
- Overvoltage protection
- Thermal shutdown
- SOT-23 (6) package

2 Applications

- 12V, 24V distributed power-bus supply
- Industry application
  - White goods
- Consumer application
  - Audio
  - STB, DTV
  - Printer



Copyright © 2017, Texas Instruments Incorporated

Simplified Schematic

3 Description

The TPS54302 device is a 4.5V to 28V input voltage range, 3A synchronous buck converter. The device includes two integrated switching FETs, internal loop compensation, and 5ms internal soft start to reduce component count.

By integrating the MOSFETs and employing the SOT-23 package, the TPS54302 device achieves the high power density and offers a small footprint on the PCB.

Advanced Eco-mode implementation maximizes the light load efficiency and reduces the power loss.

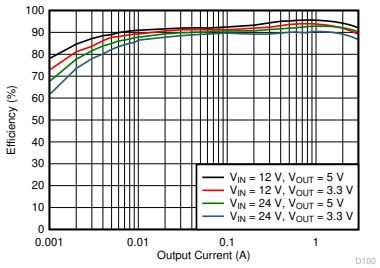
The frequency spread spectrum operation is introduced for EMI reduction.

Cycle-by-cycle current limit in both high-side MOSFET protects the converter in an overload condition and is enhanced by a low-side MOSFET freewheeling current limit which prevents current runaway. Hiccup mode protection is triggered if the overcurrent condition has persisted for longer than the present time.

Package Information

PART NUMBER	PACKAGE(1)	PACKAGE SIZE(2)
TPS54302	DDC (SOT-23-THIN, 6)	2.9mm × 2.8mm

(1) For more information, see Section 10.
(2) The package size (length × width) is a nominal value and includes pins, where applicable.



Efficiency vs Output Current

An IMPORTANT NOTICE at the end of this data sheet addresses availability, warranty, changes, use in safety-critical applications, intellectual property matters and other important disclaimers. PRODUCTION DATA.

7 Application and Implementation

Note

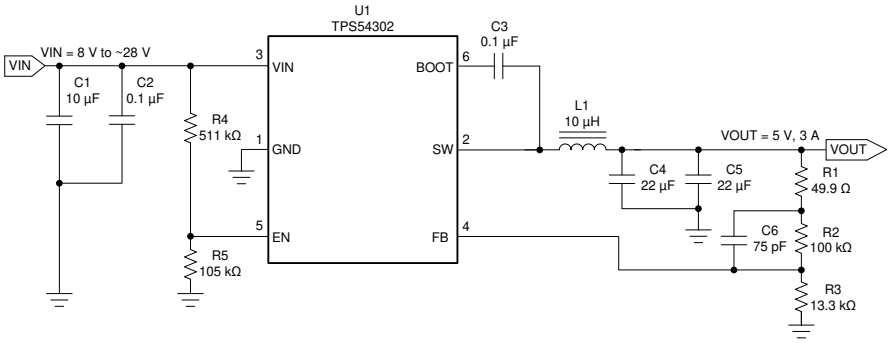
Information in the following applications sections is not part of the TI component specification, and TI does not warrant its accuracy or completeness. TI's customers are responsible for determining suitability of components for their purposes, as well as validating and testing their design implementation to confirm system functionality.

7.1 Application Information

The TPS54302 device is typically used as a step-down converter, which converts an input voltage from 8V to 28V to a fixed output voltage of 5V.

7.2 Typical Application

7.2.1 TPS54302 8V to 28V Input, 5V Output Converter



Copyright © 2017, Texas Instruments Incorporated

Figure 7-1. 5V, 3A Reference Design

7.2.2 Design Requirements

For this design example, use the parameters in Table 7-1.

Table 7-1. Design Parameters

PARAMETER	VALUE
Input voltage range	8V to 28V
Output voltage	5V
Output current	3A
Transient response, 1.5-A load step	ΔV_{OUT} = ±5 %
Input ripple voltage	400mV
Output voltage ripple	30mV_{PP}
Switching frequency	400kHz

Copyright © 2026 Texas Instruments Incorporated

Apéndice J

Hoja de datos técnicos del regulador lineal AMS1117-3,3

LDO AMS1117-3,3, alimenta el riel 1 permanentemente activo (ESP32-S3-WROOM-1-N16R8 y SN65HVD230) a partir de los 5 V del TPS54302DDCR (Apéndice I), con capacidad de hasta 1 A continuo.

Advanced Monolithic Systems

AMS1117

1A LOW DROPOUT VOLTAGE REGULATOR

RoHs Compliant

FEATURES

- **Three Terminal Adjustable or Fixed Voltages* 1.5V, 1.8V, 2.5V, 2.85V, 3.3V and 5.0V**
- **Output Current of 1A**
- **Operates Down to 1V Dropout**
- **Line Regulation: 0.2% Max.**
- **Load Regulation: 0.4% Max.**
- **SOT-223, TO-252 and SO-8 package available**

APPLICATIONS

- **High Efficiency Linear Regulators**
- **Post Regulators for Switching Supplies**
- **5V to 3.3V Linear Regulator**
- **Battery Chargers**
- **Active SCSI Terminators**
- **Power Management for Notebook**
- **Battery Powered Instrumentation**

GENERAL DESCRIPTION

The AMS1117 series of adjustable and fixed voltage regulators are designed to provide up to1A output current and to operate down to 1V input-to-output differential. The dropout voltage of the device is guaranteed maximum 1.3V, decreasing at lower load currents.

On-chip trimming adjusts the reference voltage to 1.5%. Current limit is set to minimize the stress under overload conditions on both the regulator and power source circuitry.

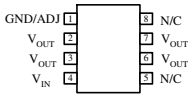
The AMS1117 devices are pin compatible with other three-terminal SCSI regulators and are offered in the low profile surface mount SOT-223 package, in the 8L SOIC package and in the TO-252 (DPAK) plastic package.

ORDERING INFORMATION:

PACKAGE TYPE			OPERATING JUNCTION
TO-252	SOT-223	8L SOIC	TEMPERATURE RANGE
AMS1117CD	AMS1117	AMS1117CS	-40 to 125° C
AMS1117CD-1.5	AMS1117-1.5	AMS1117CS-1.5	-40 to 125° C
AMS1117CD-1.8	AMS1117-1.8	AMS1117CS-1.8	-40 to 125° C
AMS1117CD-2.5	AMS1117-2.5	AMS1117CS-2.5	-40 to 125° C
AMS1117CD-2.85	AMS1117-2.85	AMS1117CS-2.85	-40 to 125° C
AMS1117CD-3.3	AMS1117-3.3	AMS1117CS-3.3	-40 to 125° C
AMS1117CD-5.0	AMS1117-5.0	AMS1117CS-5.0	-40 to 125° C

*For additional available fixed voltages contact factory.

8L SOIC Top View

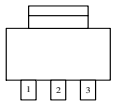


PIN CONNECTIONS

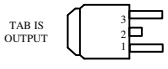
3 PIN FIXED/ADJUSTABLE VERSION

1- Ground/Adjust
2- V_{OUT}
3- V_{IN}

SOT-223 Top View



TO-252 FRONT VIEW



Advanced Monolithic Systems, Inc. www.advanced-monolithic.com Phone (925) 443-0722 Fax (925) 443-0723

Apéndice K

Hoja de datos técnicos del regulador lineal AP2112K-3,3TRG1

LDO AP2112K-3,3TRG1 (Diodes Incorporated), alimenta el riel 2 conmutado (OLED, microSD) vía el interruptor de carga TPS22918 (Apéndice L), con capacidad de hasta 600 mA y bajo nivel de ruido.

TECH PUBLIC
台舟电子

AP2112K Series

600mA,10uA, Higt PSRR Voltage Reaulator

www.sot23.com.tw

Features

- 10μA Ground Current at no Load
- ±2% Output Accuracy
- 600mA Output Current
- 10nA Disable Current (by option)
- Wide Operating Input Voltage Range: 1.2V to 5.5V
- Dropout Voltage: 0.32V at 600mA/ V_{OUT} 3.3V
- Support Fixed Output Voltage 1.2V, 1.5V, 1.6V, 1.8V, 2.5V, 2.8V, 3.0V, 3.3V
- Stable with Ceramic or Tantalum Capacitor
- Current Limit Protection
  Over Temperature Protection
- SOT23-5 Packages

Applications

- Portable, Battery Powered Equipment
- Low Power Microcontrollers
- Laptop, Palmtops and PDAs
- Wireless Communication Equipment
- Audio/Video Equipment
- Car Navigation Systems

General Descri tion

This production is group of low -dropout (LDO) voltage regulators offering the benefits of wide input voltage range from 1.2V to 5.5V, low dropout voltage, low power consumption, and miniaturized packaging. Quiescent current of only 10μA makes these devices ideal for powering the battery-powered, always-on systems that require very little idle-state power dissipation to a longer service life. There is an option of shutdown mode by selecting the parts with the EN pin and pulling it low. The shutdown current in this mode goes down to only 10nA (typical).

This production is of linear regulators are stable with the ceramic output capacitor over its wide input range from 1.2V to 5.5V and the entire range of output load current (0mA to 600mA).

Ordering Information

AP2112K-3.3TRG1

Output voltage: 1.2=1.2V
1.8=1.8V
3.0=3.0V
3.3=3.3V
3.6=3.6V

Apéndice L
Hoja de datos técnicos del interruptor de carga TPS22918

Interruptor de carga (*load switch*) TPS22918 (Texas Instruments), conecta/desconecta el riel 2 de 3,3 V (OLED, microSD) bajo control de un GPIO del ESP32-S3, implementando el modo de bajo consumo del Capítulo 3. $R_{ON} \approx 52\,\text{m}\Omega$, control configurable del tiempo de subida.

TEXAS INSTRUMENTS
TPS22918
SLVSD76C – FEBRUARY 2016 – REVISED JULY 2017

TPS22918 5.5-V, 2-A, 52-mΩ On-Resistance Load Switch

1 Features

- Integrated Single Channel Load Switch
- Ambient Operating Temperature: –40°C to +105°C
- Input Voltage Range: 1 V to 5.5 V
- On-Resistance (R_{ON})
  - R_{ON} = 52 mΩ (typical) at V_{IN} = 5 V
  - R_{ON} = 53 mΩ (typical) at V_{IN} = 3.3 V
- 2-A Maximum Continuous Switch Current
- Low Quiescent Current
  - 8.3 µA (typical) at V_{IN} = 3.3 V
- Low-Control Input-Threshold Enables Use of 1 V or Higher GPIO
- Adjustable Quick-Output Discharge (QOD)
- Configurable Rise Time With CT Pin
- Small SOT23-6 Package (DBV)
  - 2.90-mm × 2.80-mm, 0.95-mm Pitch, 1.45 mm Height (with leads)
- ESD Performance Tested per JESD 22
  - ±2-kV HBM and ±1-kV CDM

2 Applications

- Industrial Systems
- Set Top Box
- Blood Glucose Meters
- Electronic Point of Sale

3 Description

The TPS22918 is a single-channel load switch with configurable rise time and configurable quick output discharge. The device contains an N-channel MOSFET that can operate over an input voltage range of 1 V to 5.5 V and can support a maximum continuous current of 2 A. The switch is controlled by an on and off input, which is capable of interfacing directly with low-voltage control signals.

The configurable rise time of the device greatly reduces inrush current caused by large bulk load capacitances, thereby reducing or eliminating power supply droop. The TPS22918 features a configurable quick output discharge (QOD) pin, which controls the fall time of the device to allow design flexibility for power down and sequencing.

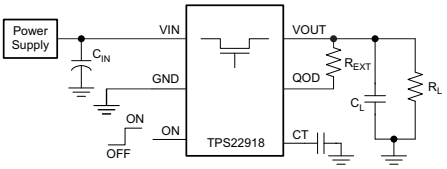
The TPS22918 is available in a small, leaded SOT-23 package (DBV) which allows visual inspection of solder joints. The device is characterized for operation over the free-air temperature range of –40°C to +105°C.

Device Information (1)

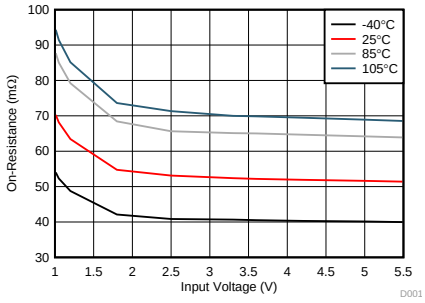
PART NUMBER	PACKAGE	BODY SIZE (NOM)
TPS22918	SOT-23 (6)	2.90 mm × 1.60 mm

(1) For all available packages, see the orderable addendum at the end of the datasheet.

Simplified Schematic



On-Resistance vs Input Voltage Typical Values



I_{OUT} = –200 mA

An IMPORTANT NOTICE at the end of this data sheet addresses availability, warranty, changes, use in safety-critical applications, intellectual property matters and other important disclaimers. PRODUCTION DATA.

9 Application and Implementation

NOTE

Information in the following applications sections is not part of the TI component specification, and TI does not warrant its accuracy or completeness. TI's customers are responsible for determining suitability of components for their purposes. Customers should validate and test their design implementation to confirm system functionality.

9.1 Application Information

This section highlights some of the design considerations when implementing this device in various applications. A PSPICE model for this device is also available in the product page of this device on www.ti.com (See the *Device Support* section for more information).

9.2 Typical Application

This typical application demonstrates how the TPS22918 can be used to power downstream modules.

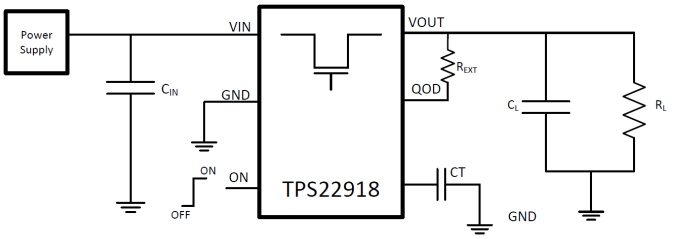


Figure 23. Typical Application Schematic

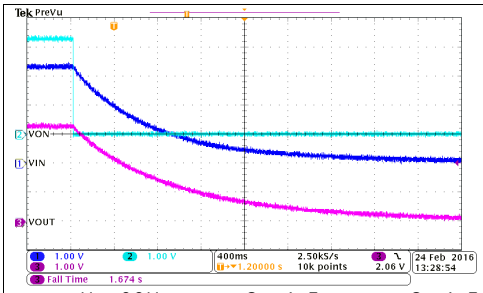
9.2.1 Design Requirements

For this design example, use the values listed in Table 5 as the design parameters:

Table 5. Design Parameters

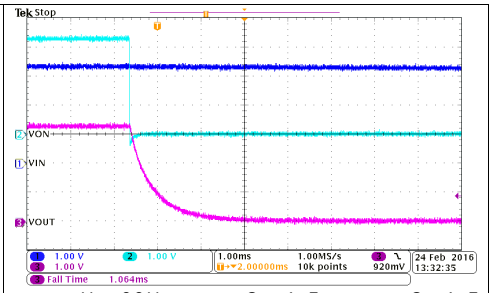
DESIGN PARAMETER	EXAMPLE VALUE
V_{IN}	5 V
Load Current	2 A
C_L	22 µF
Desired Fall Time	4 ms
Maximum Acceptable Inrush Current	400 mA

Copyright © 2016–2017, Texas Instruments Incorporated



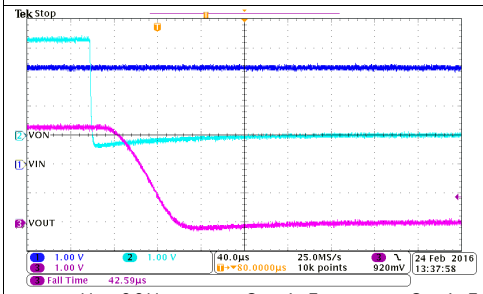
V_{IN} = 3.3 V C_{IN} = 1 µF C_L = 1 µF
QOD = Open

Figure 24. Fall Time (t_F) at V_{IN} = 3.3 V



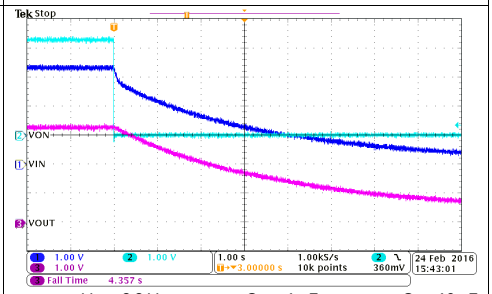
V_{IN} = 3.3 V C_{IN} = 1 µF C_L = 1 µF
QOD = 500 Ω

Figure 25. Fall Time (t_F) at V_{IN} = 3.3 V



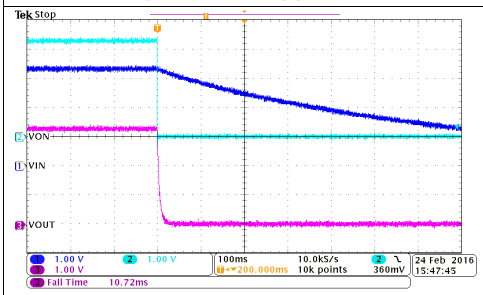
V_{IN} = 3.3 V C_{IN} = 1 µF C_L = 1 µF
QOD = V_{OUT}

Figure 26. Fall Time (t_F) at V_{IN} = 3.3 V



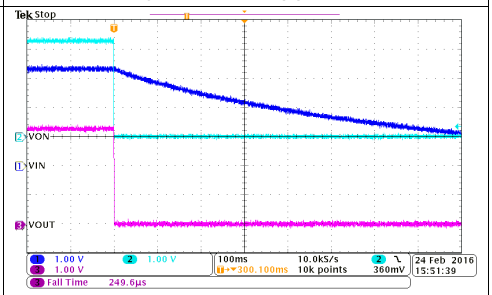
V_{IN} = 3.3 V C_{IN} = 1 µF C_L = 10 µF
QOD = Open

Figure 27. Fall Time (t_F) at V_{IN} = 3.3 V



V_{IN} = 3.3 V C_{IN} = 1 µF C_L = 10 µF
QOD = 500 Ω

Figure 28. Fall Time (t_F) at V_{IN} = 3.3 V



V_{IN} = 3.3 V C_{IN} = 1 µF C_L = 10 µF
QOD = VOUT

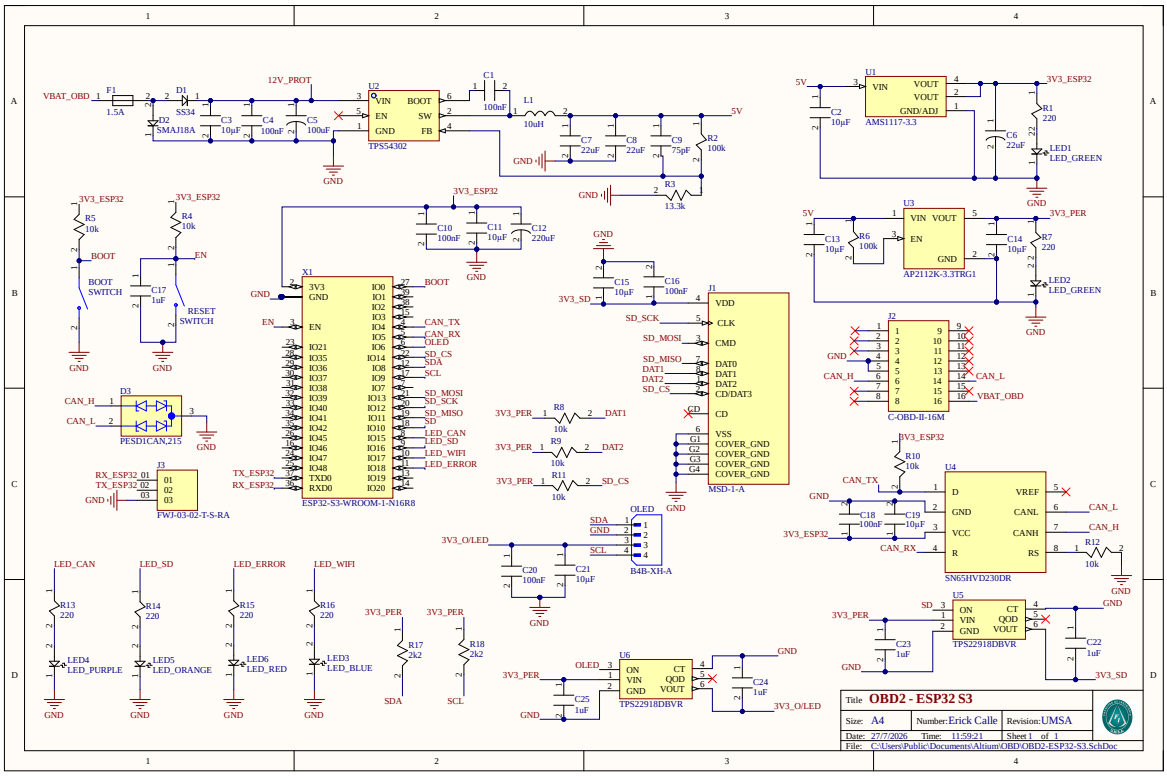
Figure 29. Fall Time (t_F) at V_{IN} = 3.3 V

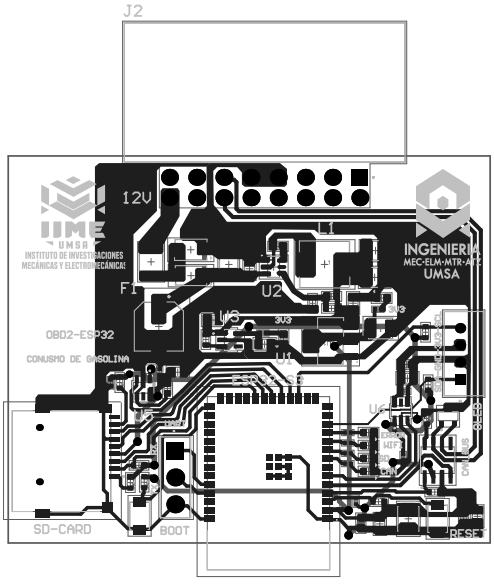
Copyright © 2016–2017, Texas Instruments Incorporated

Apéndice M

Esquemático eléctrico completo y diseño de la PCB

Esquemático eléctrico completo del sistema (diseñado en Altium Designer), disposición física de la PCB (vista de capas y ruteado) y reporte de la pila de capas (*board stack*), referenciados desde la Sección de diseño de hardware del Capítulo 3.





Apéndice N
Lista de materiales (BOM) — detalle de fabricación

A diferencia de la Tabla 3.15 del Capítulo 3 —que agrupa los componentes por bloque funcional con fines explicativos—, este apéndice reproduce la lista de materiales tal como se exportó para la fabricación y el ensamblaje de la PCB.

Name	Designator	Footprint	LCSC	Quantity
LED_GREEN	3V3	DIOC1608X28N	C7371905	2
SWITCH	BOOT, RESET	SW_TS-1088-AR02016	C720477	2
100nF	C1, C10, C16, C18, C2	CAPC1608X80N	C20539422	5
10µF	C11, C13, C14, C15, C1	CAPC2012X125N	C2182156	7
10µF	C3	CAPC3325X220N	C386166	1
100nF	C4	CAPC2012X85N	C282732	1
100uF	C5	CAPAE660X770M	C3151829	1
22uF	C6	CAPMP3228X210N	C1953638	1
22uF	C7, C8	CAPC3225X280N	C55530	2
75pF	C9	CAPC1608X80NS	C282074	1
220uF	C12	CAPMP3228X210NS	C2887903	1
1uF	C17, C22, C23, C24, C2	CAPC1005X50N	C5137473	5
LED_PURPLE	CAN	DIOC1608X28N	C7496831	1
SN65HVD230DR	CAN_BUS	SOIC127P599X175-8N	C12084	1
SS34	D1	DIOM4226X242N	C52023881	1
SMAJ18A	D2	DIOM4428X244N	C50087694	1
PESD1CAN,215	D3	DIO_PESD1CAN,215	C15771	1
LED_RED	ERROR	DIOC1608X28N	C965799	1
32-S3-WROOM-1-N1	ESP32-S3	ESP32S3WROOM1N8F	C2913202	1
1.5A	F1	CAPC4632X150N	C18198333	1
C-OBD-II-16M	J2	OMTECH_C-OBD-II-16	C9900147921	1
10uH	L1	INDP7066X220N	C524608	1
B4B-XH-A	OLED	JST_B4B-XH-A	C144395	1
220	, R7, R13, R14, R15, R	RESC1005X30N	C140180	6
100k	R2, R6	RESC1608X80N	C2906980	2
13.3k	R3	RESC1608X80N	C2933140	1
10k	R5, R8, R9, R10, R11,	RESC1608X80N	C2906982	7
2k2	R17, R18	RESC1608X80N	C2907005	2
LED_ORANGE	SD	DIOC1608X28N	C965800	1
MSD-1-A	SD-CARD	CUI_MSD-1-A	C393941	1
AMS1117-3.3	U1	SOT230P700X180-4N	C2992570	1
TPS54302	U2	SOT95P280X110-6N	C311983	1
AP2112K-3.3TRG1	U3	SOT95P285X140-5N	C51118	1
TPS22918DBVR	U5, U6	SOT95P280X145-6N	C131941	2
FWJ-03-02-T-S-RA	UART	MTEC_FWJ-03-02-T-S-	C2937625	1
LED_BLUE	WIFI	DIOC1608X28N	C965807	1

Apéndice O

PIDs OBD-II soportados por los vehículos de prueba: Changan Honor y Nissan Vanette

Listado completo y decodificado de los PIDs OBD-II (Modo 01) soportados por cada vehículo de validación, obtenido mediante el escaneo activo del Capítulo 3. Changan Honor: 34 PIDs, sin el PID 0x10 (MAF). Nissan Vanette: 33 PIDs, incluyendo el PID 0x10.

PIDS DISPONIBLES EN MODO 1 DEL CAHNGAN HONOR

CAN OBD-II supported pids	INFORMACION WIKIPEDIA	
0x01	Monitor	
0x02	Almacena imagen congelada	
0x03	Estado del sistema de combustible	
0x04	Carga del motor calculada	%
0x05	Temperatura del refrigerante del motor	°C
0x06	Ajuste de combustible a corto plazo (STFT): banco 1	%
0x07	Ajuste de combustible a largo plazo (LTFT): banco 1	%
0x0B	Presión absoluta del colector de admisión	kPa
0x0C	Velocidad del motor	rpm
0x0D	Velocidad del vehículo	km/h
0x0E	Avance de tiempo	
0x0F	temperatura del aire de toma	°C
0x11	Posición del acelerador	%
0x13	Sensores de oxígeno presentes (en 2 bancos)	
0x14	Sensor de oxígeno 1 R: voltaje B: Ajuste de combustible a corto plazo	V%
0x15	Sensor de oxígeno 2 R: voltaje B: Ajuste de combustible a corto plazo	V%
0x1C	Estándares OBD que este vehículo cumple	
0x1F	Tiempo de funcionamiento desde el arranque del motor	s
0x21	Distancia recorrida con la lámpara indicadora de mal funcionamiento (MIL) encendida	km
0x2E	Purga evaporativa comandada	%
0x2F	Entrada de nivel del tanque de combustible	%
0x30	Calentamientos desde que se borraron los códigos	
0x31	Distancia recorrida desde que se borraron los códigos	km
0x33	Presión barométrica absoluta	kPa
0x3C	Temperatura del catalizador: Banco 1, Sensor 1	°C
0x41	Supervisar el estado de este ciclo de conducción	
0x42	Voltaje del módulo de control	V
0x43	Valor de carga absoluto	%
0x44	Relación de equivalencia aire-combustible comandada (lambda,λ)	
0x45	Posición relativa del acelerador	%
0x47	Posición absoluta del acelerador B	%
0x49	Posición del pedal del acelerador D	%
0x4A	Posición del pedal del acelerador E	%
0x4C	Actuador de aceleración comandado	%
0x4D	Tiempo transcurrido con MIL encendido	min

PIDS DISPONIBLES EN MODO1 DEL NISSAN VANETTE

CAN OBD-II supported pids	Informacion wikipedia
0x01	Monitor Status
0x03	Estado del sistema de combustible
0x04	Carga del motor calculada
0x05	Temperatura del refrigerante del motor
0x06	Ajuste de combustible a corto plazo (STFT): banco 1
0x07	Ajuste de combustible a largo plazo (LTFT): banco 1
0x0B	Presión absoluta del colector de admisión (MAP)
0x0C	Velocidad del motor
0x0D	Velocidad del vehículo
0x0E	Avance de tiempo
0x0F	Temperatura del aire de admisión
0x10	Caudal de aire del sensor de flujo de aire masivo (MAF)
0x11	Posición del acelerador
0x13	Sensores de oxígeno presentes (en 2 bancos)
0x15	Sensor de oxígeno 2
0x1C	Estándares OBD que este vehículo cumple
0x1F	Tiempo de funcionamiento desde el arranque del motor
0x21	Distancia recorrida con la lámpara indicadora de mal funcionamiento (MIL) encendida
0x2C	EGR comandado
0x2E	Purga evaporativa comandada
0x30	Calentamientos desde que se borraron los códigos
0x31	Distancia recorrida desde que se borraron los códigos
0x34	Sensor de oxígeno 1 Relación de equivalencia aire-combustible (lambda,λ)
0x3C	Temperatura del catalizador: Banco 1, Sensor 1
0x41	Supervisar el estado de este ciclo de conducción
0x42	Voltaje del módulo de control
0x43	Valor de carga absoluto
0x44	Relación de equivalencia aire-combustible comandada (lambda,λ)
0x45	Posición relativa del acelerador
0x47	Posición absoluta del acelerador B
0x49	Posición del pedal del acelerador D
0x4A	Posición del pedal del acelerador E
0x4C	Actuador de aceleración comandado

Apéndice P

Hoja de datos técnicos del controlador CAN MCP2515 (arquitectura descartada)

Controlador CAN independiente con interfaz SPI (Microchip), evaluado en la etapa inicial del proyecto y descartado en favor del periférico TWAI integrado en el ESP32-S3-WROOM-1-N16R8 (Apéndice D). Se documenta únicamente como referencia del proceso de decisión de diseño, no como componente del sistema final.

MICROCHIP

MCP2515

Stand-Alone CAN Controller with SPI Interface

Features

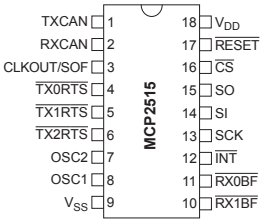
- Implements CAN V2.0B at 1 Mb/s:
  - 0 to 8-byte length in the data field
  - Standard and extended data and remote frames
- Receive Buffers, Masks and Filters:
  - Two receive buffers with prioritized message storage
  - Six 29-bit filters
  - Two 29-bit masks
- Data Byte Filtering on the First Two Data Bytes (applies to standard data frames)
- Three Transmit Buffers with Prioritization and Abort Features
- High-Speed SPI Interface (10 MHz):
  - SPI modes 0,0 and 1,1
- One-Shot mode Ensures Message Transmission is Attempted Only One Time
- Clock Out Pin with Programmable Prescaler:
  - Can be used as a clock source for other device(s)
- Start-of-Frame (SOF) Signal is Available for Monitoring the SOF Signal:
  - Can be used for time slot-based protocols and/or bus diagnostics to detect early bus degradation
- Interrupt Output Pin with Selectable Enables
- Buffer Full Output Pins Configurable as:
  - Interrupt output for each receive buffer
  - General purpose output
- Request-to-Send (RTS) Input Pins Individually Configurable as:
  - Control pins to request transmission for each transmit buffer
  - General purpose inputs
- Low-Power CMOS Technology:
  - Operates from 2.7V-5.5V
  - 5 mA active current (typical)
  - 1 µA standby current (typical) (Sleep mode)
- Temperature Ranges Supported:
  - Industrial (I): -40°C to +85°C
  - Extended (E): -40°C to +125°C

Description

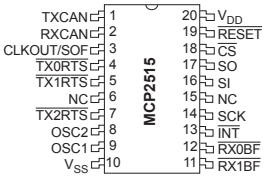
Microchip Technology's MCP2515 is a stand-alone Controller Area Network (CAN) controller that implements the CAN specification, Version 2.0B. It is capable of transmitting and receiving both standard and extended data and remote frames. The MCP2515 has two acceptance masks and six acceptance filters that are used to filter out unwanted messages, thereby reducing the host MCU's overhead. The MCP2515 interfaces with microcontrollers (MCUs) via an industry standard Serial Peripheral Interface (SPI).

Package Types

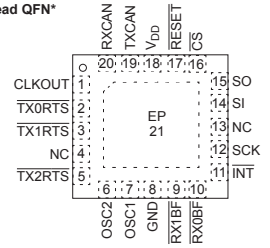
18-Lead PDIP/SOIC



20-Lead TSSOP



20-Lead QFN*



* Includes Exposed Thermal Pad (EP); see Table 1-1.

© 2003-2019 Microchip Technology Inc.

Apéndice Q
Registro de un viaje (trip_003.csv)

Este apéndice reproduce, a modo de ejemplo, el archivo real generado por el sistema en la tarjeta microSD con el formato descrito en la Sección 3.6.6 del Capítulo 3 (`/logs/trip_003.csv`, vehículo Changan Honor). El mismo archivo se emplea como fuente de datos en la prueba de integridad de registro en microSD (Tabla 4.12, Sección 4.3.6 del Capítulo 4): se trata de una sesión de **60,8 minutos con el motor en ralentí** (el vehículo permaneció detenido, `speed_kmh` = 0 durante toda la sesión, por lo que `trip_km` y `trip_avg_L100km` se mantienen en cero), cortada de forma abrupta al final para validar el comportamiento del sistema ante un corte de alimentación durante la escritura.

Q.1 Estructura del registro

La Tabla 3.30 del Capítulo 3 describe 16 de las 18 columnas escritas por `sdLoggerTask()`; las dos restantes, agregadas junto con la corrección por altitud (Sección 3.4.3), se detallan en la Tabla 1.

Tabla 1
Columnas adicionales de `trip_NNN.csv` no incluidas en la Tabla 3.30.

Columna	Descripción
`baro_kpa`	Presión barométrica, kPa: lectura real del PID 0x33 o, si el vehículo no lo soporta, estimación por el modelo ISA (Ecuación 3.17) evaluado en `SITE_ALTITUDE_M`.
`baro_is_estimated`	Bandera (0/1): 1 si `baro_kpa` proviene del modelo ISA en vez de una lectura real del PID 0x33.

Nota. En `trip_003.csv`, `baro_is_estimated` = 0 en todas las filas: el Changan Honor sí soporta el PID 0x33, por lo que `baro_kpa` es una lectura real y no una estimación.

Q.2 Muestra del registro real

Las Tablas 2, 3 y 4 reproducen seis filas representativas de las 3 649 que contiene el archivo completo (inicio, dos puntos intermedios y el cierre de la sesión), divididas en tres tablas por razones de ancho de página; las filas se corresponden una a una por `uptime_ms`.

Tabla 2

trip_003.csv — muestra real, parámetros del motor (columnas 1–9).

uptime_ms	rpm	speed_kmh	map_kpa	iat_c	coolant_c	ve_pct	maf_gs	fuel_gs
2 491	0	0,0	63,0	8,0	7,0	60,0	0,00	0,000
3 491	0	0,0	63,0	8,0	7,0	60,0	0,00	0,000
602 491	891	0,0	20,0	14,0	71,0	63,3	1,48	0,105
1 802 491	893	0,0	19,0	30,0	88,0	63,4	1,34	0,090
3 002 491	896	0,0	19,0	49,0	87,0	63,4	1,26	0,088
3 650 491	0	0,0	49,0	55,0	92,0	60,0	0,00	0,000

Tabla 3

trip_003.csv — muestra real, consumo instantáneo y acumulado del viaje (columnas 10–14).

instant_Lh	instant_L100km	trip_km	trip_fuel_L	trip_avg_L100km
0,00	0,00	0,000	0,0000	0,00
0,00	0,00	0,000	0,0000	0,00
0,51	0,00	0,000	0,1147	0,00
0,44	0,00	0,000	0,2765	0,00
0,43	0,00	0,000	0,4214	0,00
0,00	0,00	0,000	0,4982	0,00

Tabla 4

trip_003.csv — muestra real, estado CAN, costo y presión barométrica (columnas 15–18).

can_status	trip_cost_bs	baro_kpa	baro_is_estimated
1	0,00	63,0	0
1	0,00	63,0	0
1	0,80	63,0	0
1	1,92	63,0	0
1	2,93	63,0	0
1	3,47	63,0	0

Nota. Datos reales extraídos de `reportes/trip_003.csv`, mismo orden de filas que las Tablas 2 y 3. El refrigerante sube de 7 °C a 92 °C conforme el motor se calienta en ralentí; `trip_fuel_L` y `trip_cost_bs` crecen de forma monótona durante toda la sesión (0,4982 L, Bs 3,47 al cierre, con `FUEL_PRICE_BS_PER_L` = 6,96), mientras que `trip_km` y `trip_avg_L100km` permanecen en cero por tratarse de una prueba estática, no de un trayecto con desplazamiento.